

Sistemi Operativi

Simone Petta

Primo Semestre a.a. 2021/2022

Contents

1	Introduzione	9
1.1	Introduzione	10
1.1.1	Dove si collocano?	10
1.1.2	SO come astrazione dell'hardware	11
1.1.3	So come gestore delle risorse	12
1.1.4	Evoluzione sistemi calcolo (hardware)	12
1.1.5	Evoluzione dell'hardware	12
1.1.6	Sistemi batch	12
1.1.7	Os dos	13
1.1.8	Sistemi batch multiprogrammati	13
1.1.9	Sistemi di time-sharing	14
1.1.10	Sistemi per le masse (microcomputer etc...	14
1.1.11	Hardware	14
1.1.12	Processori	16
1.1.13	Memoria	17
1.1.14	Dispositivi di I/O	17
1.1.15	BUS	18
2	Componenti Essenziali	19
2.1	Processi (e thread)	20
2.1.1	Organizzazione processi	21
2.2	Spazio di indirizzamento	21
2.3	File System	22
2.3.1	Directory	23
2.4	System Call	23
2.4.1	Cosa succede nel kernel quando chiamo system call	24
2.5	Sistemi monolitici	25
2.5.1	Sistemi stratificati	26
2.6	Sistemi a microkernel	26
2.7	Sistemi distribuiti	26
2.8	Virtualizzazione	27
2.9	Capitoli	28

3	Processi	29
3.1	Modello concettuale	30
3.2	Creazione dei processi	30
3.3	Terminazione di processi	31
3.4	Ciclo di vita dei processi	31
3.4.1	Ciclo di vita (un po' meglio)	32
3.4.2	Ciclo di vita (quasi vero)	34
3.5	Implementazione dei processi	34
3.6	Multiprogrammazione	35
3.7	Thread	38
3.7.1	Modello classico	39
3.7.2	Thread hanno informazioni locali	39
3.8	Capitoli	41
4	Thred in Go	43
4.1	Gestione thread Go	47
4.1.1	Thread facili da gestibili	48
5	IPC: Inter-Process Communication	51
5.1	Competizione per le risorse	52
5.2	Regioni critiche	52
5.2.1	Accesso a regione critica	53
5.2.2	Mutua esclusione con Busy Waiting	54
5.3	Sleep and Wait	56
5.3.1	Produttore e consumatore	56
5.4	Strutture dati complesse	57
5.4.1	Soluzione con semafori	58
5.5	Barriere (pattern)	59
5.6	Problema dei 5 filosofi	59
5.7	IPC in Go	60
5.7.1	Mutex	61
5.7.2	Semafori	63
5.7.3	Produttore-Consumatore	64
5.7.4	Problema dei 5 filosofi	66
5.8	Capitoli	68
6	Scheduling	71
6.1	Comportamento dei processi	72
6.2	Categorie dei sistemi di scheduling	72
6.3	Obiettivi	73
6.4	FCFS (FIFO)	74
6.5	SJF (shortest job first)	75
6.5.1	SJF con prelazione (SRTF shortage remaning time first)	76
6.6	Round Robin	77
6.7	Scheduling con priorit�	78
6.8	Thread Scheduling	79

6.9	Esercizi di scheduling	80
7	Gestione della memoria	81
7.1	Nessuna astrazione	82
7.2	Spazi di indirizzamento separati	83
7.3	Swapping	84
7.3.1	Allocazione dello spazio	85
7.3.2	Pezzi di dimensione variabile	86
7.3.3	Strategie di allocazione dello spazio libero	87
7.4	Memoria virtuale	88
7.5	Memoria Paginata	89
7.5.1	Page table	90
7.5.2	Elementi page table	91
7.5.3	Page table su piu' livelli	92
7.5.4	Page table invertita	93
8	Paginazione	95
8.1	Paginazione su richiesta (demand paging)	96
8.2	Algoritmi presenti in letteratura	96
8.2.1	Algoritmo ottimo	97
8.2.2	FIFO (First in First out)	97
8.2.3	Seconda Chance	98
8.2.4	Algoritmo dell'orologio	99
8.2.5	LRU	100
8.2.6	Algoritmi basati su working set	102
9	Implementazione della memoria virtuale	105
9.1	Allocazione locale e globale	106
9.2	PFF (Page Fault Frequency)	107
9.3	Dimensione ottimale pagine	108
9.4	Separazione di spazio dati e testo	109
9.4.1	Pagine condivise	110
9.4.2	Shared library	110
9.4.3	Memory-mapped file	111
9.5	Gestione dei page fault	112
9.5.1	Backing store	113
9.5.2	Separazione tra politiche e meccanismi	114
9.6	Memoria segmentata	115
9.7	Confronto tra politiche	116
10	File System	119
10.1	File	120
10.1.1	Struttura interna	121
10.1.2	Tipi di file	121
10.2	Attributi	123
10.3	Operazioni possibili	124

10.4 Directory	124
10.4.1 Gerarchica	125
10.4.2 Operazioni possibili	126
11 Implementazione di un file system	129
11.1 Disk Layout	130
11.1.1 Sistemi industriale	131
11.2 File con allocazione continua	132
11.3 Allocazione a lista	133
11.4 FAT(file allocation table) 16	134
11.5 File con allocazione ad albero (i-node)	134
11.6 UNIX file system	136
11.7 Implementazione directory	137
11.8 UFS (UNIX file system) entry	139
11.9 File condivisi tra directory	140
11.10 Esempio di UFS	140
11.11 Varianti possibili	141
11.11.1 Virtual File System	142
12 Gestione ed ottimizzazione del file system	145
12.1 Logical Volume Manager	146
12.2 LVM e RAID	148
12.3 LVM - struttura generale	150
12.4 LVM - snapshot	150
12.5 LVM - striping e concatenazione di volumi	151
12.6 LVM - mappatura	151
12.7 Logical Volume Manager in NAS	152
12.8 Dimensione dei blocchi di dati	152
12.9 Gestione blocchi liberi	153
12.9.1 Gestione della free list in memoria	154
12.10 Analisi della consistenza	154
12.11 Disk caching	155
13 Input/Output	157
13.1 I/O Hardware	158
13.2 IO programmato	158
13.3 Memory Mapped I/O	159
13.4 VS prestazioni	162
13.5 Direct Memory Access (DMA)	163
13.6 Interrupt Preciso	165
13.6.1 Interrupt precisi o meno	166
13.7 I/O Software	167
13.8 Stratificazione IO	168
13.8.1 Gestione Interrupt	168
13.9 Device driver	169
13.10 Sistema di Buffering	170

14 Deadlock	173
14.1 Risorse	174
14.1.1 Request e release	175
14.2 Deadlock	176
14.2.1 Modello per il deadlock	176
14.2.2 Evitare un deadlock tramite scheduling	177
14.3 Cosa fare	178
14.3.1 Come trovare un deadlock	179
14.4 Come trovare un deadlock (risorse multiple) (Implementazione Dijkstra)	180
14.5 Risoluzione di un deadlock	182
14.6 Evitare un deadlock	183
14.6.1 Algoritmo banchiere	184
14.6.2 Verifica se uno stato e' sicuro o meno	185
14.7 Prevenire un deadlock	185
14.7.1 Mutua esclusione	186
14.7.2 Hold and wait	186
14.7.3 No prelazionabili	186
14.7.4 Attesa circolare	186
14.8 Varianti sul tema deadlock	186
15 Cloud	189
15.1 Virtualizzazione	190
15.1.1 Caratteristiche della virtualizzazione	190
15.1.2 Dinamica temporale di un servizio	191
15.2 Passaggio al cloud	192
15.3 Definizione di cloud secondo il NIST	193
15.4 Google docs	193
15.5 As A Service	193
15.5.1 Prima e dopo	194
15.6 Utility computing	195
15.7 Caratteristiche del cloud	195
15.8 Suddivisione per come rendo accessibile il cloud	196
15.9 Il lato oscuro del cloud	196
16 Multimedia	199
16.1 Dato multimediale	200
16.2 Caratteristiche delle informazioni multimediali (all'interno di un OS)	201
16.3 Codifica	202
16.3.1 motivi per codificare	203
16.3.2 dumb coding	203
16.3.3 Ottimizzazione dello spazio	203
16.3.4 Ridondanza	203
16.4 Compressione	204
16.4.1 Mezzo di fruizione	205

16.4.2	Parametri fisiologici	205
16.4.3	Parametri psicologici	205
16.5	Audio e video	205
16.6	MPEG	206
16.6.1	Vari tipi di MPEG	206
16.6.2	Codifica MPEG	206
16.6.3	Ridondanza spaziale	207
16.6.4	Ridondanza temporale	209
16.6.5	B-Frame	210
16.6.6	GOP - Group Of Pictures	210
16.6.7	Per non confondersi	210
16.7	Streaming	211
16.7.1	Buffering	213
16.7.2	Buffer underrun	213
16.7.3	Buffer overrun	213
16.7.4	E se perdiamo dei dati?	214
16.7.5	Le funzionalita' VCR	214
16.8	Processi multimediali	215
16.8.1	Processi periodici	215
16.8.2	RMS (Rate Monotonic Scheduling)	215
16.8.3	EDS (Earliest Deadline First Scheduling)	217
16.9	File System con supporto multimediale	218
16.9.1	Minimo tempo di accesso	218
16.9.2	Manipolazione della riproduzione	219

Chapter 1

Introduzione

1.1 Introduzione

Libro: Tanenbaum "I moderni sistemi operativi" 4^a edizione. Un sistema operativo e' generico e' un unione di software che mette a disposizione un hardware ad un applicativo (applicazione). Non sono solo nei computer, nei cellulari (ios, android), smart tv, sistemi embedded viene usato linux ad esempio nelle macchine o nei satelliti. Un sistema operativo esiste solo in funzione dell'hardware, esistono sistemi operativi per i server che non hanno monitor (non c'e' uscita video)

1.1.1 Dove si collocano?

Dobbiamo prendere la struttura software delle cose che utilizziamo e tagliarla a fette. L'abitudine e' quella di cominciare dalla parte piu' semplice a salire:

- Hardware: quello che devo gestire con sistema operativo
- Sistema operativo: E' un pezzo di software attraverso il quale sono obbligato a passare per poter accedere all'hardware, solo lui sa come si usa hardware. Il programma deve sapere come si usa il sistema operativo. Quando scarichiamo un'applicazione lei sa usare il sistema operativo ma non glie ne frega niente dell'hardware su cui gira. Il kernel di os e' implementazione dello strato software che si frappone tra hardware e lo strato utente (user mode) che non puo' arrivare ad ardware se non passare da kernel.
- Librerie di servizio (user mode): che parlano con il kernel, ad esempio l'uso della rete: abbiamo una libreria chiamata socket a cui diamo i dati e lei costruisce il pacchetto da dare al kernel che attiva la scheda di rete.
- Programma: ad esempio web browser, riproduzione multimediale...

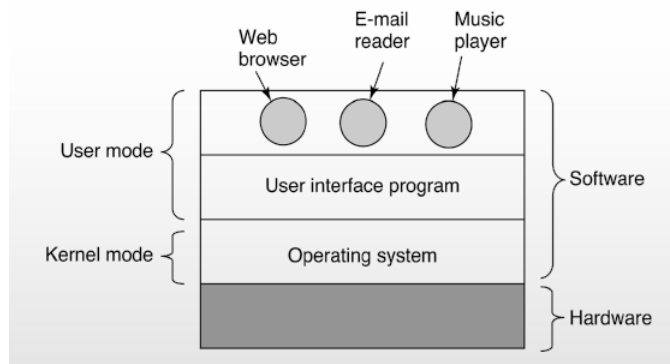


Figure 1.1: Dove si collocano

Kernel mode e User mode sono due modalita' diverse per usare il sistema. Una CPU moderna posso usarla in modalita' non protetta (kernel mode) in cui posso fare tutto quello che mi pare, tutte le funzionalita' possibili a disposizione; e in modalita' protetta (user mode) in cui ho a disposizione solo una limitata quantita' di funzioni. Perche' un software non deve andare ad interferire con tutta una serie di cose che gestisce il kernel. Quindi c'e' bisogno di un calice per passare alla kernel mode che per un attimo ci permette di modificare impostazioni di sistema, dopo di che torniamo nelle user mode. Questo ci permette di mantenere il sistema sicuro, al contrario quando permetto ad applicazioni di mettere le mani nello spazio di memoria succedono cose strane, l'os se ne rende conto e ci dice che le strutture dati non sono piu' gestibili ed entra in kernel panic (schermata blu di windows) questo significa che comunque abbiamo dei bug all'interno del kernel perche' comunque e' un software.

1.1.2 SO come astrazione dell'hardware

L'os si mette tra software e hardware. I problemi sono che: usare hardware e' una rogn, perche' programmare una scheda del pc e' difficoltoso (timer, temporizzazioni, informazione costruita come vogliono) o semplicemente non possiamo farlo, ad esempio nelle schede Nvidia dobbiamo usare il software che ci danno loro per far funzionare le schede in modo corretto. Solo Nvidia puo' fornirci i driver (che sono pezzi di sistema operativo) che espone ad os le interfacce. C'e' un doppio passaggio per passare le chiamate da belle a rognose quindi il 25% della capacita' computazionale e' perso nell'operazione di utilizzo dei device driver. A volte le aziende che forniscono i device driver non li forniscono per determinati os (linux) quindi in quel caso non si possono usare.

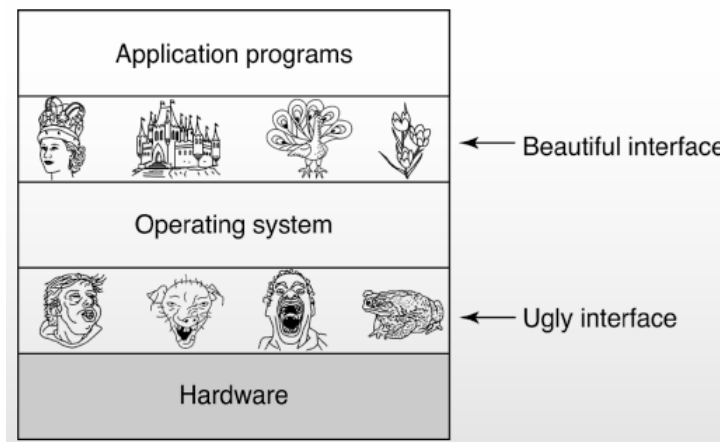


Figure 1.2: Astrazione

1.1.3 So come gestore delle risorse

Ha tanti compiti, deve gestire le risorse sul nostro calcolatore che sono tante, ad esempio il processore, la memoria (piu' complicata di un array), interfaccia di rete, il disco, risorse che funzionano in una direzione sola: monitor, tastiera, stampante. Ci sono risorse che non fanno riferimento a cose fisiche ma vanno comunque gestite, ad esempio cosa dobbiamo fare quando parte un'applicazione (rilevare il doppio click e trasformare quell'azione in un processo: che e' un programma con un contesto esecutivo che interagisce).

1.1.4 Evoluzione sistemi calcolo (hardware)

Non sono sempre stati cosi' ma c'e' stata un'evoluzione, 50 anni fa un pc era enorme costava tantissimo e faceva calcoli relativamente semplici. 20 anni fa avevano ridotto di molto le dimensioni dei sistemi di calcolo, costavano tanto ma non cosi' tanto e cominciavano a fare operazioni piuttosto complesse, quindi cominciavano ad apparire in universita' nelle banche. Oggi abbiamo sistemi di calcolo microscopici (telefono) e che costano molto poco (rete di sensori, che e' un agglomerato di oggetti con un piccolo sistema operativo ed un'antenna che possono comunicare via rete finche' dura la batteria, costano cosi' poco che conviene buttarli piuttosto che ricaricarli).

1.1.5 Evoluzione dell'hardware

50 anni fa costruivamo sistemi con delle valvole (sono versioni arcaiche dei transistor). Oggi infiliamo tutto il sistema di calcolo in un cip, che non lo rende semplice o difficili, lo rende diverso e ce lo fa utilizzare in ambiti piu' complessi, perche' ad esempio fare un os per un telefono e' piu' difficile che farlo per un pc.

Anche gli utenti sono cambiato, 50 anni fa eri un guru, 15 anni fa eri un esperto, oggi chiunque ha un sistema operativo. Gli esperti dei sistemi operativi hanno avuto una specie di declino, perche' chi gestisce i sistemi operativi ormai e' un operaio del terzo millennio.

1.1.6 Sistemi batch

Erano sistemi di elaborazione offline, scrivere un programma significava usare delle schede perforate che venivano caricate all'interno del vettore, poi stampavano. Per il calcolo di interessi etc andavano benissimo. Avevo tre stadi dell'elaborazione:

1. L'operatore scrive il programma con schede perforate e lo mette in un vettore
2. Il vettore trasforma il contenuto delle schede perforate in nastro magnetico che viene trasportato ad un CPU che caricava il contenuto in memoria eseguiva il programma e scriveva sempre su nastro il risultato

3. Un operatore portava il nastro in un altro sistema che stampava il risultato

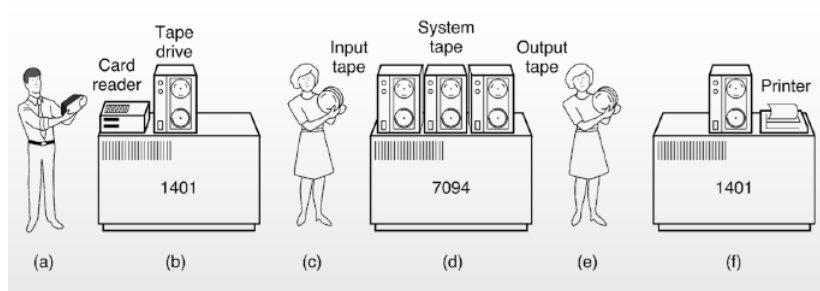


Figure 1.3: Sistemi Batch

Con questo sistema siamo partiti per innescare una serie di cose:

Memoria sistema batch

C'è un pezzo di memoria (resident monitor) dedicata ad un software che gestisce tutto quanto ed un pezzo di memoria che rimane libera nella quale il resident monitor veniva caricato il programma contenuto sul nastro e faceva una jump alla prima istruzione.

1.1.7 Os dos

Era un resident monitor, veniva caricato il kernel codificato in un programma command.com e mi dava un prompt lampeggiante che si istruiva su cosa volevamo fare e fintanto che il programma non aveva finito di eseguire non c'era nulla da fare se non spegnere e riaccendere.

1.1.8 Sistemi batch multiprogrammati

La memoria comincia a costare non tanto e ce n'era abbastanza (64k) perché devo avere un solo programma di memoria? tagliamo delle fette all'interno della memoria e ci mettiamo tanti software uno dietro l'altro, mi riservo un po' di spazio per i loro dati e facendo quest'operazione comincio a parlare di job e li carico ed eseguo in sequenza. Cominciano a sorgere i problemi, perché ad esempio se nel job1 c'è un bug potrebbe scrivere nella zona del job3 corrompendolo, quindi si è cominciato a parlare di protezione obbligando il sistema operativo ad incastrare il job1 nella sua zona di memoria. Un'altro problema: i job arrivano vengono eseguiti tolti dalla memoria ed io ho altri job che arrivano e succede la frammentazione perché non tutti i job hanno la stessa dimensione.

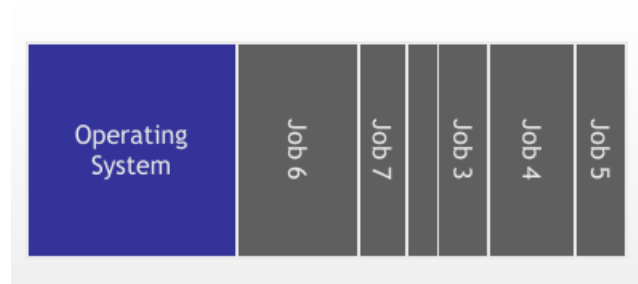


Figure 1.4: Sistemi batch multiprogrammati

1.1.9 Sistemi di time-sharing

Se abbiamo piu' programmi contemporaneamente all'interno della macchina perche' sono obbligato a farne andare uno solo per volta, perche' non posso far finta che ci siano 4 programmi in esecuzione contemporaneamente in una solo CPU. Oggi funziona cosi' tutti i programmi che funzionano insieme non potrebbero funzionare insieme perche' c'e' un solo CPU, esiste un Time-sharing nell'os che e' una specie di metronomo, batte il tempo dando dei segnali (interact) e seguendo i segnali sposta l'attenzione della CPU da un programma all'altro. Quindi si comincia a parlare di processi perche' le attivita' si alternano, quindi ho la sensazione che le app funzionino insieme ma il passaggio e' cosi' veloce che noi non possiamo rendercene conto. Non necessariamente se ho 4 processi eseguirli 1 2 3 4, dipende da cosa e' meglio, e per capirlo abbiamo costruito gli algoritmi di scheduling, qual'e' la politica con cui scelgo qual'e' il prossimo processo da eseguire?

1.1.10 Sistemi per le masse (microcomputer etc...

In tempi recenti si e' passati da computer a disposizioni da grandi aziende a sistemi che comprano le persone per usarle a casa. Cominciano con il CPM (prima MSDOS) ribattezzato da Microsoft dopo, era il primo che considerava il disco come periferica dell'utente. Con l'arrivo del time-sharing abbiamo avuto tanti sistemi operativi diversi (piu' o meno belli). Oggi questi generazioni di sistemi operativi viene convogliata in sistemi embedded e nei telefoni in quanto nei sistemi operativi dei telefoni ci sono sistemi operativi dei computer customizzati. Nei computer siamo ancora fermi ad un passo precedente.

1.1.11 Hardware

Non consideriamo nulla di quello che ci fa vedere come piovuto dal cielo, tutto quello che vedremo ha sempre una motivazione. Per capire l'evoluzione dei sistemi operativi e' conveniente vedere l'evoluzione dell'hardware dei sistemi di calcolo. Siamo abituati ad un hardware fatto cosi': c'e' un bus che si occupa

di portare in giro le informazioni (in realta' ne abbiamo sempre 2, data bus e system bus), raccoglie intorno a se alcuni elementi:

- CPU
- Memoria
- Controller: e' un dispositivo hardware pensato per tradurre le informazioni che passano sul bus in un sistema di scambio dati compatibile con un dispositivo che c'e' dall'altra parte.:
 - Video controller: scheda video, ha il compito di portare le informazioni del bus ad un altro bus (hdmi)
 - Keyboard controller: oggi in realta' quello che facciamo e' attaccare un controller USB (universal serial bus) che parla con tutte le periferiche che ragionano un byte alla volta. (quindi tastiera, mouse, stampante etc)
 - USB controller
 - Hard disk controller: il disco ha un suo bus chiamato SATA

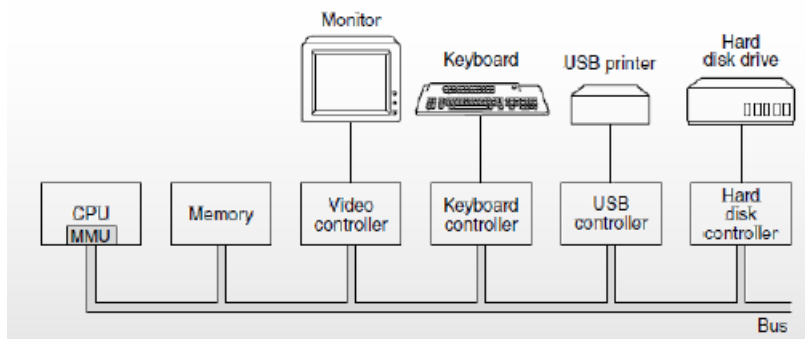


Figure 1.5: Hardware

La cosa interessante e' che il sistema operativo e' che deve essere in grado di parlare con i controller, e basta, perche' e' poi il controller che deve parlare con la periferica. Questo mi permette di semplificare molto la progettazione. IN quanto posso fare un sistema molto semplice, ci installo i device driver e poi non devo fare piu' nulla.

E' un po' piu' complicato

Ci sono tanti BUS:

- System bus: uso per gestire le periferiche, I/O bus che e' lento, perche' le periferiche sono lente, quindi lo teniamo lento per abbassarne i costi.

- Memory BUS, Bridge: e' uno dei pezzi piu' importanti in una scheda madre, perche' ha in mano tutti i dati della scheda madre, se si intoppa abbiamo tutto l'hardware da buttare. Fa da ponte tra CPU e Memoria Centrale

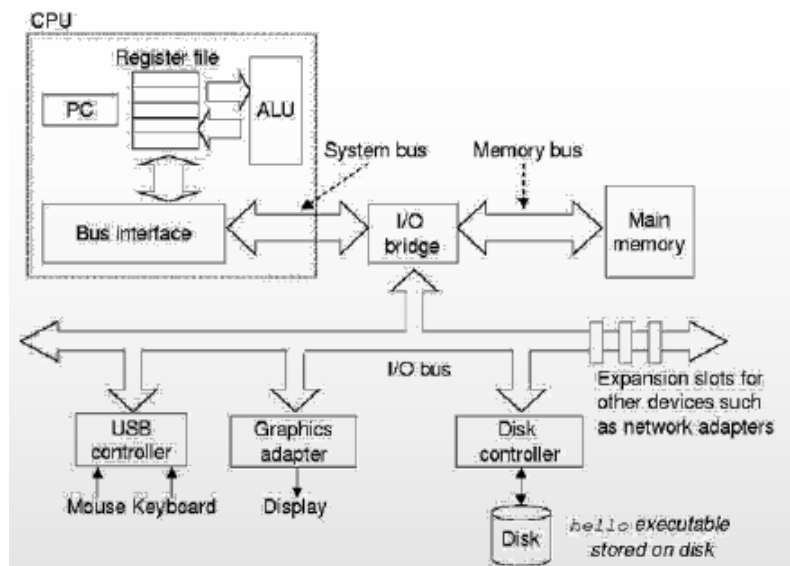


Figure 1.6: Bus

1.1.12 Processori

I processori si sono evoluti, siamo passati da processori semplici (a) dove c'era un modello di esecuzione in cui c'era: fetch, decode, execute. Oggi noi abbiamo dei sistemi complicati detti multi pipeline (b) porta anche dei problemi di gestione del sistema operativo.

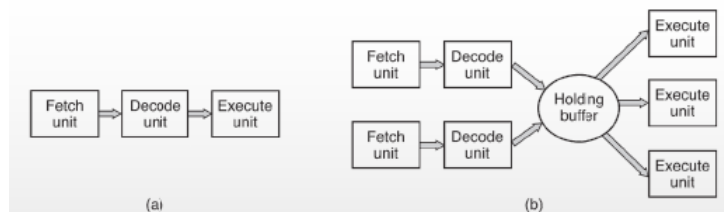


Figure 1.7: Evoluzione del processore

Il fatto che sono multipipeline porta ad avere CPU multicore, dal punto di sistema operativo non e' molto diverso da avere un sistema multiprocessore.

1.1.13 Memoria

Una volta i sistemi erano lenti, con i sistemi multiprogrammati e time sharing ci siamo posti il problema delle prestazioni. Posso aumentarle in maniera devastante usando sistemi di caching (nell'immagine non cache ma caching associativa). Cache e' un processo in cui tengo in una zona di memoria ad alte prestazioni i dati che mi servono piu' spesso. Io organizzo la memoria in tipologie diverse, c'e' un fenomeno per cui memorie di prestazioni altissime sono di dimensioni molto ridotte e costano una follia:

- testa metto i registri di un processore, il dato di un registro e' disponibile in un nanosecondo, peccato che ci sia poco spazio
- cache associativa: non vediamo nel sistema operativo, e' una zona che si occupa di mantenere in transito le informazioni della memoria centrale, di fianco alla memoria centrale, raggiungo queste informazioni in 3/4 milisecondi
- RAM: e' tanta e costa anche relativamente poco, ci arrivo in 10 nanosecondi. Se faccio uso di un'informazione in una certa posizione e' molto probabile che faccia uso di un'informazione limitrofa, quindi non devo sempre andare in memoria centrale ma mi fermo alla cache associativa.
- Storage esterno: tutto quello che non ci sta in RAM, all'interno della memoria principale ci metto anche una cache (tampone del disco). Quando scriviamo un'informazione sul disco non ci va direttamente, la parcheggio in memoria principale gestita dal sistema operativo e periodicamente prendo tutte le modifiche e le scrivo, e' il motivo per cui a volte il disco frulla lo stesso anche se non lo stiamo utilizzando

Ci sono cache tra tutti i livelli in realta'. Anche internet ha una cache, perche' e' piu' lento del disco, nella directory del browser c'e' la cache. Nel bios c'e' un'opzione per disabilitare la cache. La cache e' la base su cui costruiamo il sistema operativo, perche' abbiamo bisogno di prestazioni.

1.1.14 Dispositivi di I/O

I controller sono tanti, in realta' ci sono dei controller che sono cablati sulla scheda madre e sono strategici per tutto quello che stiamo facendo: interrupt controller, e' un controller che si intrapone tra disco e dispositivi lenti, quando faccio andare il sistema non ha senso tenere ferma la CPU nel periodo in cui una periferica lenta sta andando a prendere i dati (mentre il disco frulla posso eseguire un altro processo? SI) a patto che quando il disco ha finito di prendere i dati mi mandi un segnale (attenzione i dati sono pronti) questo segnale prende il nome di interrupt: e' un segnale asincrono che arriva su un piedino della CPU ogni volta che una periferica ha finito il suo compito. L'interrupt controller media questi segnali verso la CPU, se comincio ad avere dozzine di periferiche ho bisogno di essere sicuro che i segnali che mi arrivano dalle periferiche non

si accavallino, che non si perdano. Quindi l'interrupt controller li raccoglie e li notifica uno alla volta alla CPU.

1.1.15 BUS

I BUS sono di diversa natura, oggi ne abbiamo tanti e diventano specializzati:

- Per la scheda di rete
- PCIe per la grafica
- Sata: per la memoria

E tutto questo concorre per la gestione del nostro sistema. Una volta non era così semplice e c'erano dei BUS messi in cascata (anni 90) avevamo dei livelli di bus diversi e li distinguiamo fisicamente, su certi server c'erano schede madri fatte a moduli.

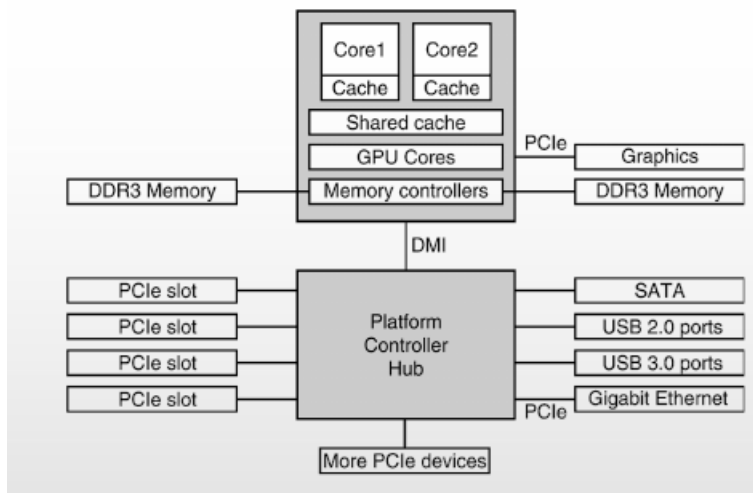


Figure 1.8: Tanti Bus

Sul libro capitolo 1: 1.1 1.2 1.3 (tranne 1.3.3 e 1.3.6)

Chapter 2

Componenti Essenziali

Dobbiamo cominciare a fare un elenco di tutte le cose che vivono in un sistema operativo. Con questo modulo si fa una rassegna di tutti i macro argomenti che andremo a trattare.

2.1 Processi (e thread)

Un processo non e' un programma, un programma e' un insieme di byte che possono essere eseguiti da CPU. Un processo e' uno stato esecutivo associato ad un programma. E' un programma che sta facendo qualcosa. Storicamente abbiamo costruito i sistemi operativi basati sui processi, UNIX e tutto quello arrivato dopo di lui ha lavorato sui processi, ad un certo punto ci si e' accorti che i processi non erano abbastanza flessibili, se devo far lavorare tra loro processi e' un'operazione abbastanza rognoso, per questo ad un certo punto si e' fatto finta che il processo non sia piu' qualcosa che fa un lavoro ma sia un contenitore, all'interno del contenitore (chiuso) possiamo mettere dei flussi esecutivi concorrenti (thread) che fanno qualcosa. Ogni thread e' indipendente, ma puo' accedere ed influenzare le risorse del processo. Condividono ad esempio la memoria del processo. Ha portato non pochi problemi perche' i sistemi operativi vecchi (UNIX) hanno fatto fatica ad adeguarsi, perche' i thread sono comodi, in quanto un linguaggio moderno prevede i thread. Anche word prevede thread, e' un processo ma prevede molti flussi esecutivi diversi. Windows e' stato costruito basandosi sul processo di thread, ha un sistema di gestione dei flussi di esecuzione piu' avanzato di UNIX. Un processo (processo come un solo flusso) ha un suo ciclo di vita, nasce, fa delle cose, si mette in attesa e poi termina. Gli piace paragonare a quello che succede quando andiamo in un ufficio postale: entriamo nell'ufficio per richiedere un servizio, ci mettiamo in coda per parlare con la persona allo sportello (CPU), quando tocca a noi accediamo allo sportello (il processo va in running), in questo momento possono succedere tante cose, se dovevamo fare una cosa veloce ci mettiamo poco e ce ne andiamo, o potrebbe succedere che torniamo in coda (La CPU ci dice che deve prendere i dati dal disco, quindi bloccati fintanto che i dati sono pronti), se no puo' anche capitare (sistemi time sharing) che ci chiedano di rimetterci in coda per fare quello dopo di noi per aumentare le prestazioni del sistema. Tre stadi per la gestione del processo:

- Ready (il processo e' in coda ed e' pronto ad essere eseguito)
- Running (il processo viene eseguito)
- Blocked (il processo e' bloccato e deve rimettersi in coda per ottimizzare il sistema)

Passeremo il 90% del tempo a discutere del sistema di avvicendamento (sistema di scheduling) e a parlare di cosa succede quando entra in gioco l'interact.

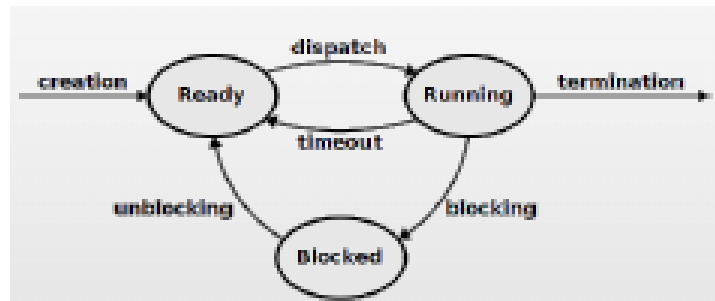


Figure 2.1: Stadi per la gestione del processo

2.1.1 Organizzazione processi

Si organizzano in maniera gerarchica, un processo genera processi figli che a loro volta generano i nipoti e via andare. Tutti i sistemi operativi nascono con un processo dentro. Quando compilo un kernel imposto delle strutture dati, quando viene caricato in memoria mi trovo quelle strutture dati che hanno già un processo e sono pronte ad eseguire, e' il problema di init (e' il processo che si occupa di far partire tutto). In sistema UNIX posso chiedere l'elenco dei processi tramite il comando **ps** (nel terminale ovviamente). **bash** e' il processo che mi esegue il terminale, **ps** e' il processo che ho generato quando ho scritto **ps**. Posso chiedere di dirmi tutti i processi che ha all'interno con il comando **ps aux**. I processi appartengono ad utenti diversi, sia a me sia a gestore del sistema (**root**) che e' quello che puo' fare tutto. Questi processi sono organizzati in una gerarchia, perche' con **ps alx** posso richiedere anche delle informazioni aggiuntive tipo parent process id (di chi e' figlio questo processo). Esistono tool comodi per farci vedere la gerarchia **pstree** che ci fa vedere la gestione di processi ad albero. In UNIX quando c'e' un processo che finisce con **d** (**daemon**) e' un processo che si occupa di gestire il sistema operativo. Windows associa i processi alle finestre, tipo **word** quando parte genera diversi thread per tutto quello che deve fare internamente.

2.2 Spazio di indirizzamento

Una cosa e' la memoria (RAM) e una cosa e' lo spazio di indirizzamento: e' l'insieme delle localita' di memoria che il processore e' in grado di andare a gestire, questo dipende dall'architettura del processore, **intel64** ha un sistema di indirizzamento a 64 bit, significa che al suo interno riesce a gestire bit di 2^{64} . Siamo partiti con processori a 16 bit che avevano 64 KB di spazio di indirizzamento. Poi c'e' stata una seconda generazione di sistemi con sistemi di indirizzamento a 20 bit e li' 1 MB di spazio di indirizzamento. Poi ad esempio con il 386 si poteva arrivare fino a 4GB.

Se io ho tutto questo spazio di indirizzamento ed i processi non sanno nulla

Spazio di indirizzamento	Dimensione RAM
16 bit	1MB
Seconda generazione 20 bit	1 MB
Intel 386, 32 bit	4 GB
core i7, 40 bit	1TB
48bit	256TB

Table 2.1: Spazio di indirizzamento

della RAM succede che abbiamo inventato dei meccanismi chiamati memoria virtuale. Mettendo insieme un po' di RAM un po' di disco un po' di meccanismi posso ingannare il processo per fare finta che abbia a disposizione tutto lo spazio in RAM possibile. Ma non li usera' mai tutti, e' un processo che si autoregola in quanto un processo che usa troppa memoria viene killato.

2.3 File System

La struttura dati che organizzo all'interno di un disco per poter permettere al mio sistema operativo di piazzarmi li' un archivio per le mie informazioni. Siamo arrivati quasi ad uno standard, esistono dei contenitori chiamati directory ed esiste una possibilita' di mettere all'interno di una directory altre directory o file (non chiamiamole cartelle o diventa antipatico), la directory e' la directory che abbiamo in centro commerciale dove c'e' un cartello in cui ci sono indicazioni per i negozi piazzati in ordine alfabetico. Nel file system ci dicono le informazioni dove sono e come fare per andarle a recuperare. Non diamo per scontato che un file system stia su un disco (potrebbe stare sul nastro) e che abbia per forza delle directory (introdotte negli anni 80).



Figure 2.2: File System

2.3.1 Directory

Anche le directory si organizzano in un sistema gerarchico, la directory principale si chiama root (/), definisce a partire da lei un albero (senza cicli) in cui ho sotto-directory o file. Esiste una differenza fondamentale tra il file system fatto alla unix e quello alla windows. Quello windows e' arrivato 20 anni dopo unix ed e' piu' moderno, pero' si porta dietro anche un sacco di disgrazie di retro-compatibilita' ad esempio C: e D: quando inseriamo le chiavette USB, perche' per windows ogni dispositivo fisico e' un albero a se stante. Per Unix invece l'albero e' uno solo ed e' l'albero che ha la radice all'interno del disco in cui ho fatto partire il sistema operativo (sempre), se ho altri dischi e' vero che hanno delle strutture dati separate ma fanno riferimento al sistema operativo. Esistono delle operazioni di mount, nel momento in cui ho un altro disco prendo il file system del disco e lo aggancio in una directory del mio file system principale (posso agganciare anche file system visti via rete).

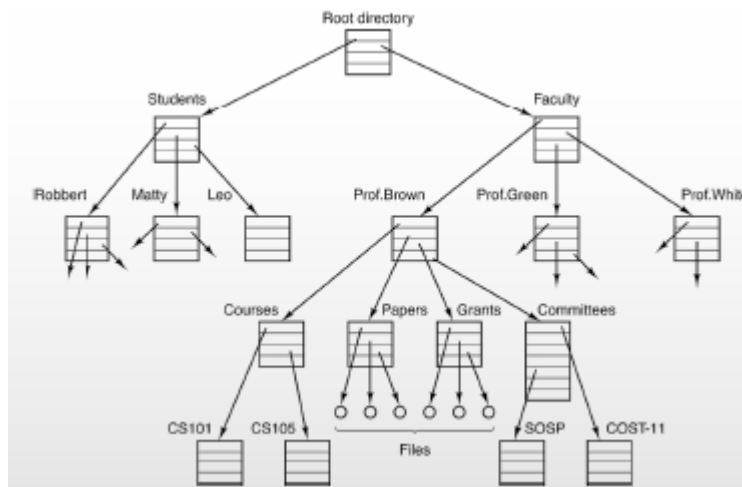


Figure 2.3: Directory

2.4 System Call

E' quel meccanismo per cui qualunque sistema operativo permette ad un processo di chiedere al kernel di fare del lavoro per lui. Io processo appartengo ad un utente e quell'utente deve fare il suo lavoro e basta, vuol dire semplicemente avere accesso alle risorse dell'utente che appartengono all'utente che ha fatto partire il processo. Pero' esistono situazioni in cui ho bisogno di interagire con le strutture dati del kernel, pero' non posso farlo io come utente per non distruggere tutto. L'os mi mette a disposizione delle system call, loro sanno come alterare le strutture interne del kernel. Si dividono in famiglie. Le aree principali sono la gestione dei file, memoria e processi. Posix e' uno standard che ha

detto che per poter essere un sistema standard il nostro sistema operativo deve fornire minimo queste system call (sono solo 40), poi ognuno ci mette le sue. Windows ne ha circa 14k. Non ce le chiede a memoria ma sappiamo come si suddividono, per tipologia, gestione memoria, processi, file system e poi tutte le altre. Questo meccanismo e' comune a tutti i sistemi operativi.

Process management	
Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &status, options)</code>	Wait for a child to terminate
<code>s = execve(name, args, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status
File management	
Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing, or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &stat)</code>	Get a file's status information
Directory- and file-system management	
Call	Description
<code>s = mkdir(name, mode)</code>	Create a new directory
<code>s = rmdir(name)</code>	Remove an empty directory
<code>s = link(name1, name2)</code>	Create a new entry, name2, pointing to name1
<code>s = unlink(name)</code>	Remove a directory entry
<code>s = mount(special, name, flag)</code>	Mount a file system
<code>s = umount(special)</code>	Unmount a file system
Miscellaneous	
Call	Description
<code>s = chdir(dirname)</code>	Change the working directory
<code>s = chmod(name, mode)</code>	Change a file's protection bits
<code>s = kill(pid, signal)</code>	Send a signal to a process
<code>seconds = time(&seconds)</code>	Get the elapsed time since Jan. 1, 1970

Figure 2.4: System Call

2.4.1 Cosa succede nel kernel quando chiamo system call

Si scatenano una serie di operazioni. Chiamata di funzione deposito i parametri sullo stack, push di tutti i parametri, non mi interessa sapere che cosa rappresentano i parametri, dopo di che faccio una jump alla prima istruzione che mi chiede l'istruzione. Nel caso del system call e' diverso perche' non posso fare una jump al codice del system call perche' e' nascosto nel kernel. Quindi devo chiamare un intermediario. Ogni system call ha un equivalente accessibile al codice utente con lo stesso nome: **stup**. L'esempio e' la chiamata read, leggo i dati da un canale di comunicazione:

1. push dei tre parametri della read, quanti byte voglio, dove devono andare a finire una volta letti, e gli dico il canale da cui vanno letti.
2. faccio una jump ad una stup della system call. Una stup e' un pezzo di

codice breve che capisce che system call mi serve, identifica il numero di system call, e solleva una trap fa si che l'esecuzione del processo si blocca ed il kernel vista la segnalazione dice fermi tutti tocca a me, si trova nell'indirizzo della CPU il codice della system call da eseguire e capisce che system call e' stata chiamata (dal numero della system call), da questo momento l'operazione passa in modalita' kernel, faccio tutto quello che mi serve, chiamo l'equivalente in kernel mode della mia system call, uso il numero che c'era nel mio indirizzo da eseguire, il kernel fa una jump a quell'indirizzo. A quel punto termina, ovvero restituisce il controllo al chiamante ma prima tramite un'istruzione della CPU torno in modalita' utente

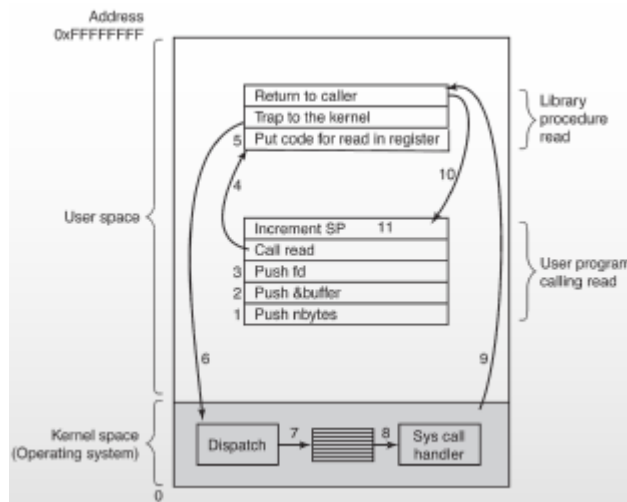


Figure 2.5: Gestione System Call

2.5 Sistemi monolitici

Esistono i cosiddetti sistemi monolitici, tutto il sistema operativo e' concentrato in un solo punto (linux, windows). Vuol dire che tutto il codice e' raggrumato in un singolo file o in tanti file ma su uno stesso disco. Tre livelli:

- Kernel
- System Call
- Funzioni dell'utente: posso costruire qui tutte le funzioni per usare il sistema operativo

2.5.1 Sistemi stratificati

Tutti i sistemi operativi monolitici sono stratificati, e' relativamente recente: anni 80/90. Con l'arrivo di UNIX e WINDOWS forse non e' corretto che il codice e' completamente destrutturato e tutto vale. I sistemi operativi oggi sono fatti a buccia di cipolla, il kernel al centro (livello 0) ed una serie di strati attorno con la parte piu' esterna che mi aiuta a fare i programmi utente. Non posso arrivare al kernel senza passare dalle system call e cosi' via, devo per forza passare dagli strati piu' esterni.

2.6 Sistemi a microkernel

Sono sistemi operativi specializzati, nati per scopi particolari o nati per esperimenti. I micrikernel sono nati per risolvere problematiche a sistemi operativi real time. Il concetto e' che voglio fare un kernel piu' piccolo possibile, perche' deve darmi velocissimo le risposte. Il microkernel ha solo un sistema di gestione dei processi ed un sistema di comunicazione tra gli elementi che vivono nell'os, dopo di che qualsiasi altra cosa sono processi. Perche' nel momento in cui non mi serve lo escludo.

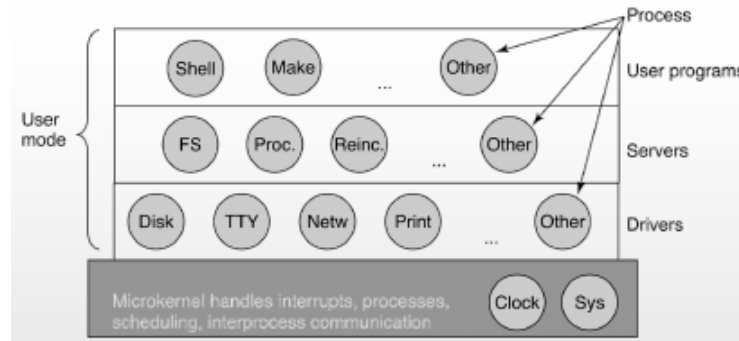


Figure 2.6: Microkernel

2.7 Sistemi distribuiti

Se ho una rete abbastanza veloce posso piazzare i pezzi dell'os in computer diversi, collaborando a creare un sistema operativo distribuito. Il problema era la rete ci sono delle situazioni in cui il collo di bottiglia non e' piu' la rete ma altre cose (hard disk...). Un sistema cloud e' un sistema in cui abbiamo una grossa mesh di sistemi di calcolo che condividono risorse e contribuiscono a creare un sistema di calcolo distribuito, ognuno di loro ha un pezzo di kernel i quali interagiscono tra di loro. I sistemi client-server non sono sistemi distribuiti, e'

un paradigma di fruizione di servizi, non parliamo di sistemi operativi in questo caso.

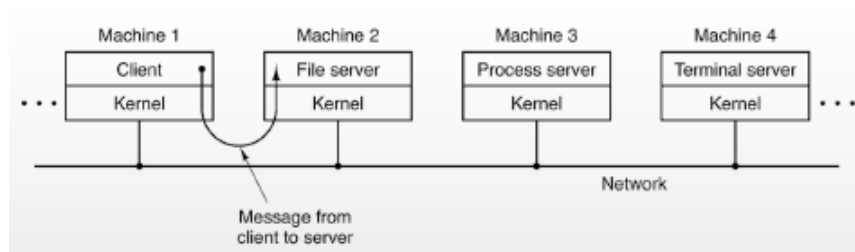


Figure 2.7: Sistemi Distribuiti

2.8 Virtualizzazione

E' il cuore del cloud oggi, perche' sul cloud abbiamo un sistema distribuito in cui ci sono all'interno dei sistemi di calcolo che collaborano per darci la virtualizzazione di hardware e software. Il problema e' che uso un sistema di calcolo per simulare un altro sistema di calcolo, lo facevamo fino dagli anni 70, si erano fatti un virtualizzatore di tipo a, dove un hypervisor si occupa di estrarre l'hardware e di dividerlo tra vari sistemi operativi che vengono eseguiti contemporaneamente, fa passare le risorse da un sistema all'altro, e' un sistema di time sharing sul sistema operativo. Dopo si e' passati ad un sistema b, s400 era una roba server venduta da IBM per cui il sistema operativo e' costruito su API, quindi per fare aggiornamenti c'e' bisogno di pochissimo tempo di fermo macchina. Oggi per qualunque azienda non avere un server acceso significa perdere fatturato. Ad un certo punto, 15 anni fa, abbiamo cominciato a parlare di simulatori, quindi niente piu' hyper visor, avevamo un sistema operativo ospite gia' installato sul ferro (windows) poteva eseguire un software (virtualBox) che al suo interno andava a simulare un sistema operativo ospitato, questo simulatore era un processo come tutti gli altri quindi andava in competizione con tutti gli altri, funzionava bene ma avevo perso un sacco di prestazioni quindi non potevo assumere che il sistema operativo fosse costante nel tempo. Dopo abbiamo introdotto gli hypervisor di tipo 2, ho sempre il concetto di sistema operativo ospite e di software che mi crea un ambiente simulato, solo che questo software non fa una simulazione, tutto il lavoro sporco chiede al kernel del sistema operativo ospitante di farlo: ho bisogno di accedere a servizi? pezzi di codice di cui sono sicuro? lo fa direttamente il sistema operativo ospitante. Quindi questi programmi ci installano pezzi di kernel per far vedere il programma direttamente al sistema (c). Quando compro una macchina in cloud compro una virtualizzazione di tipo c, in realta' vedo solo il sistema operativo ospite.

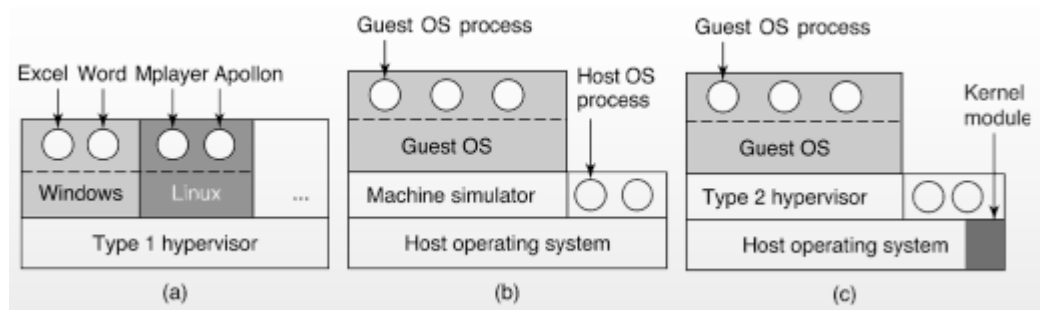


Figure 2.8: Virtualizzazione

2.9 Capitoli

- Capitolo 1:
 - 1.5: fino a 1.5.3 incluso
 - 1.6: tranne 1.6.5
 - 1.7: tranne 1.7.6

Chapter 3

Processi

Il processo e' un'unita' esecutiva che svolge del lavoro, un thread e' una visione piu' moderna in cui un processo viene visto come un contenitore all'interno del quale ho dei flussi esecutivi paralleli o semi-paralleli. Il programmatore crea quanti thread servono in base a cosa deve fare il software.

3.1 Modello concettuale

Dal punto di vista del processore la situazione e' quella del caso (a): c'e' un program counter del processore che punta alla prossima istruzione utile, come processore vedo che ci sono 4 job ed eseguo un pezzo di ognuno, mentre dal punto di vista dell'utente abbiamo la situazione (b), perche' abbiamo tanti programmi (processi) che a noi risultano aperti contemporaneamente, ma e' un'illusione perche' c'e' solo un program counter, non 4 che vengono fatti andare in parallelo per i processi a, b, c, d. Il modello (c) e' quello visto dal punto di vista del sistema operativo (time sharing), vede sull'asse temporale dei processi che vengono eseguiti a turno. La difficolta' e' implementare in un kernel il software che realizza il modello (c), dobbiamo fare una serie di assunzioni.

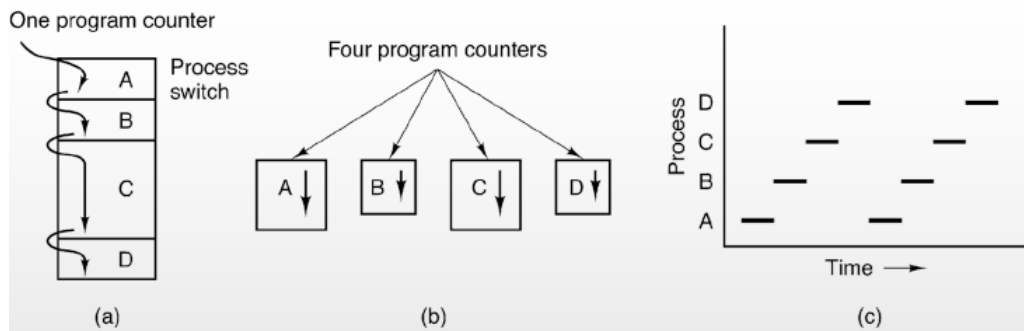


Figure 3.1: Modello Concettuale

3.2 Creazione dei processi

I processi si organizzano ad albero ed io creo dei processi lungo la strada.

1. Inizializzazione del sistema: nel momento in cui il kernel viene caricato in memoria in boot le sue istruzioni sono gia' inizializzate per cui viene un processo viene eseguito
2. Chiamata di una system call da parte di un processo in esecuzione: creazione di un figlio per creare l'albero
3. Un utente chiede di creare un nuovo processo: a chi lo chiede? ad un processo esistente. Il quale usera' una system call, quindi di fatto siamo ancora nel caso 2

4. Esecuzione di un job in un sistema batch. Però i batch di sistemi lo eseguono un processo, quindi siamo ancora nel caso 2.

Secondo lui abbiamo solo 2 sistemi per generare un processo, al boot, o tramite un processo tramite una system call

3.3 Terminazione di processi

1. **Volontaria:** volontariamente un processo termina solo tramite chiamata di una system call "exit", che viene sempre chiamata, anche se non lo inseriamo in fondo ad un programma. Il valore di ritorno usato sulla system call è la differenza tra normal ed error exit. Il valore di ritorno della exit viene dato al kernel il quale manda un segnale al processo padre del processo che è terminato dicendogli che il figlio è morto e questo è il valore di ritorno. Quindi sappi che il processo che hai generato può essere andato a buon fine oppure no. In UNIX il kernel mantiene le risorse del processo congelate in uno stato di zombie e non libera la memoria ed i file finché il processo padre non dice "ok messaggio ricevuto"

- Normal exit: Il valore di ritorno della exit è 0
- Error exit: Il valore di ritorno è inferiore a 0 ed il valore dipende dal tipo dell'errore.

2. **Non volontaria:**

- Fatal error: posso aver fatto un errore, tentato di usare una zona di memoria vietata, per cui appena ho tentato di fare questo il sistema operativo ha generato un'altra trap per cui termina il processo
- Terminazione da parte di un altro processo: è possibile che un altro processo che appartiene a qualcun'altro chiede al kernel di uccidere qualcun'altro. Si fa tramite system call kill, non serve solo a terminare i processi, uno di questi è appunto kill del processo.

Durante le uscite volontarie sono nella fase di running, perché se chiamo la system call exit sto usando la CPU. Se invece è non volontaria il processo non è mai nello stato di running, perché qualcun'altro sta usando la CPU per ucciderci.

3.4 Ciclo di vita dei processi

I processi sono soggetti ad un ciclo di vita. Un processo nasce e gli viene attribuito uno stato. Ho una specie di automa ed il mio processo sta in uno di quei tre stati (è una bugia ma si approfondirà dopo):

1. Stato di Ready: stato in cui è pronto in coda a ricevere la CPU. Quando nasce il processo è in stato di Ready.

2. Stato di Running: il processore decide che e' il turno del processo ad essere eseguito, per fare questo passaggio (3) si usa la parte di kernel "". Il pezzo di kernel che si occupa di fare questo e' il dispatcher. A questo punto posso usare la CPU per tutto il tempo che mi e' concesso e la CPU mi rimette in coda e torno in Ready, se ho l'abitudine di usare tutto il tempo della CPU prendo il nome di CPU bound che tipicamente ha una priorita' bassa
3. Stato di Blocked: potrebbe capitare che mentre sono in running potrei chiedere di fare un system call per lettura o scrittura in periferiche esterne, allora il processo passa in uno stato di bloccato e questa transizione avviene sempre per via di una system call. Quando sono in questo stato rimango li' e non vengo neanche considerato dal kernel, fino a che i dati che stavo aspettando non saranno disponibili. Quando la periferica avra' finito di fare lettura o scrittura mandera' segnale di Interrupt e dira' che ci sono dati disponibili, il kernel manda i dati in memoria e riprende il processo che li doveva usare che si trovava in stato bloccato e lo rimette in corsa (ready). Quindi la transizione (4) avviene quando il contenuto della periferica e' disponibile in concomitanza di un segnale di interrupt. Ci sono system-call: sleep, che richiede al'os di metterci in uno stato di sospensione fino a che non scatta un timer, ad esempio dobbiamo aspettare che finisca l'interazione con l'utente, sono delle eccezioni da system-call per leggere o scrivere dati.

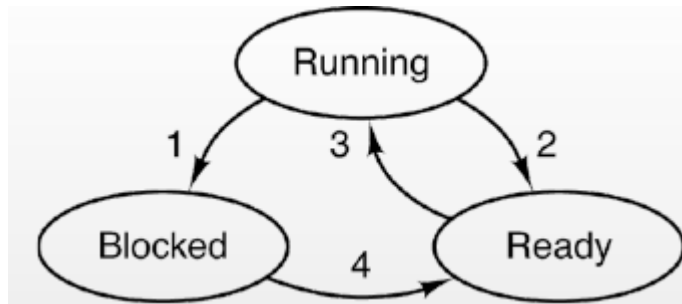


Figure 3.2: Ciclo di Vita (semplificato)

Non puo' capitare di passare da bloccato allo stato di esecuzione (passo prima da ready) non puo' passare da ready a bloccato, perche' in ready non ho a disposizione la CPU. La exit volontaria puo' essere chiamata solo in running, se l'uscita e' involontaria puo' essere fatta in blocked o ready, non mentre sono in running perche' la CPU e' mia e nessuno puo' uccidermi.

3.4.1 Ciclo di vita (un po' meglio)

In realta' gli stati sono molti di piu', ci sono sempre Blocker, Running e Ready, ma ce ne sono altri:

- Running in kernel mode: ho due modi per usare la CPU, in modalita' utente (dove non posso interagire con il kerne) e la kernel mode (dove posso fare tutto), quanto vengo schedulato per essere eseguito (stato 3), vengo messo in running ma in user mode, dopo di che da questa modalitaa' posso richiedere una system call, pero' nel richiederla prima passo ad eseguire codice in modalita' kernel, quindi mi porto nello stato 2, se quella era una system call che prevede un'operazione di IO passo a Blocked direttamente da stato running in modalita' kernel. Non e' detto che vada sempre in sleep, potrei passare da running in kernel mode a ready to run, poteri chiamare una system call che come effetto ha quello di dire "basta cosi'", rimettermi in coda, cerco di essere educato" (nice). Se invece ho sempre il dispatcher mi riporta in ready c'e' comunque una freccia tra stato 1 e stato 3.
- Processo appena creato (5): quando il processo e' stato creato ma il kernel non puo' ancora metterlo in ready perche' sta facendo altro, ad esempio se dovesse arrivare un interrupt.
- Zombie (6): quello stato in cui io vado quando termino come processo (terminazione avviene sempre da kernel mode, sia perche' ho chiamato la exit che e' una system call, o perche' c'e' stata una trap, oppure perche' qualcun'altro ha usato una system call per uccidermi)

Purtroppo non e' ancora lo schema vero, c'e' un operazione che si chiama swap out, il sistema operativo ha deciso che un programma occupa un sacco di spazio e decide di parcheggiarlo sul disco per portarmi avanti con il lavoro, questo succede quando un programma non lo usiamo per tanto tempo. Quindi esiste un altro modo di eseguire un processo, ready da disco (dopo operazioe di swap out)

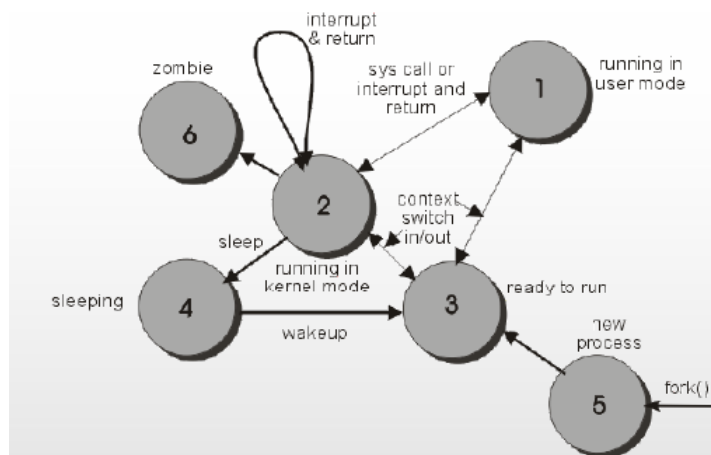


Figure 3.3: Ciclo di vita gia' un po' meglio

3.4.2 Ciclo di vita (quasi vero)

La parte destra del disegno e' la stessa immagine di prima, la parte sinistra e' la parte in cui considero che un processo puo' stare nello stato di sleep (bloccato) su disco e nello stato di ready su disco. Se da stato di sleep su disco vengo richiamato in running deve accadere uno swap in per riportare il programma da disco a memoria. L'area di swap non puo' essere usata come storage per preservare i dati salvati.

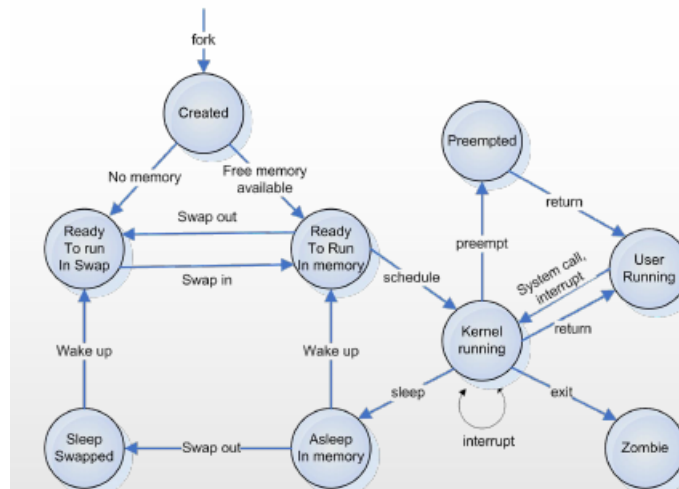


Figure 3.4: Ciclo di vita quasi vero

3.5 Implementazione dei processi

Ad un certo punto dobbiamo capire come implementare i processi. Il processo e' una struttura dati all'intero del disco, esiste una process table che e' una tabella all'interno del kernel in cui ci sono strutture dati che contengono tutti i dati del processo, dev'essere veloce da utilizzare. Quando dico che un processo ha process ID ha 1024 vuol dire che in questa tabella alla posizione 1024 ho la struttura dati che mi descrive il processo, che si chiama process control block, la quale al suo interno ha tutti gli stati del processo e dati del processo (dati, program counter, indirizzi ...). Il processo control block ognuno ha il suo, ogni os lo implementa a modo suo. Noi dobbiamo ricordarci che i dati al suo interno si dividono in 3 categorie:

- Gestione del processo: qui dentro ci metto tutte le statistiche del processo. Ad esempio e' interessante sapere quanta CPU a usato, i processi che la usano tutta si chiamano process bound. Infatti spesso catalogo i processi secondo le statistiche che raccolgo nel process control block. La cosa migliore e' dare priorit' che fanno molto I/O.

- Gestione della memoria: dove sono i dati di questo processo? quante zone di memoria sono state allocate? come le sta usando? quante ho parcheggiato sul disco per fare swap out?
- Gestione dei file: che file ha aperti? con quali permessi? sono condivisi con altri processi? a che punto sono arrivato a leggere?(molto importante perche' devo ricominciare a leggere dallo stesso punto)

Process management	Memory management	File management
Register	Pointer to text segment info	Root directory
Program counter	Pointer to data segment info	Working directory
Program status word	Pointer to stack segment info	File descriptors
Stack pointer		User ID
Process state		Group ID
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

Table 3.1: Implementazione dei processi

Rispecchiamo: processi, memoria e file system. Gia' sapere le tre categorie del processo control block lo rendono felice.

3.6 Multiprogrammazione

Di fatto su un sistema abbiamo tanti processi, pero' l'os non e' un equazione ma e' un oggetto con le sue problematiche, ad esempio interrogare il kernel tantissime mi fa erodere la possibilita' di usare la CPU al 100% per i processi. Se ho tanti processi l'os ha bisogno di usare una percentuale di potenzialita' di calcolo della CPU per gestire le sue tabelle interni. In realta' i processi che usano tutto il carico di CPU sono molto pochi (htop per vedere i processi attivi con percentuale di utilizzo CPU). Quando il sistema va piano o e' perche' c'e' un processo anomalo o per errori delle periferiche. Il livello di multiprogrammazione sono i processi che ci eseguo sopra, le prestazioni dipendono dall'interazione con i dispositivi esterni. Piu' tempo i processi aspettano dati dall'I/O piu' processi posso buttarci dentro. In un sistema reale ho un I/O wait del 99%, quindi difficilmente arrivero' a saturare la CPU. Saturando la CPU il clock si alza, quindi le ventole partono. Non e' piu' la CPU il problema. Quindi avendo piu' processi in parallelo mi da un carico in piu', devo implementare il dispatcher.

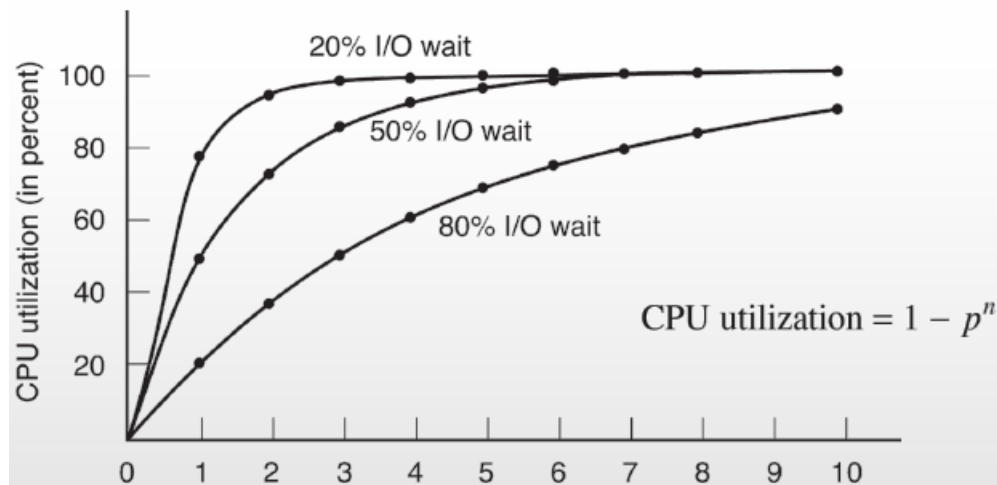


Figure 3.5: Multiprogrammazione

Vediamo cosa succedere nella gestione dei passaggi 3 e 2 contemporaneamente:

1. La CPU usando dei registri non utilizzabili in modalita' utente prende tutta la configurazione della CPU (tutto quello che c'e' nei registri a partire dal program counter) e la mette nello stack del processo. Viene generato l'interrupt di clock che da il via a questo processo, la CPU non esegue neanche una micro istruzione.
2. La CPU capisce qual'e' il pezzo di codice da eseguire appoggiandosi su quello che si chiama vettore di interrupt. Arriva un interrupt per capire che interrupt e' succede che la periferica che lo genera mette sul bus dati un informazione che dice "guarda che io sono tizio". Quando devo fare avvicendamento di processi e' sempre un interrupt che li genera, e' un interrupt interno alla CPU pero' il meccanismo non e' cambiato, perche' c'e' sempre il generato dell'interrupt che va a scrivere sul bus dati il codice dell'interrupt che ha mandato. Quindi il sistema identifica il tipo di interrupt e poi va a prendersi l'interrupt vector, usa il numero che ha trovato sul bus come indice dell'interrupt vector, e li' c'e' un riferimento ad un pezzo di codice che dev'essere eseguito per fare quell'interrupt. Fa la jump al pezzo di codice puntato dall'interrupt vector.
3. Una procedura scritta in assembly salva i registri: e' una specificazioe per dire che questo pezzo di kernel non puo' essere scritto da un linguaggio ad alto livello, perche' ho bisogno di avere un controllo nel dettaglio di quello che fanno i registri. Perche' senza compromettere quello che c'e' in CPU devo prendere lo stack e metterlo nel processo control block. Una volta

che ho aggiornato il process control block so che posso sempre recuperare quel processo.

4. Scritto in assembly cambia lo stack: perche' adesso dovrò eseguire del codice come kernel, non posso usare lo stack del processo perche' ci sono tutti i dati e parametri del processo (non posso compromettere il kernel), quindi mi procuro un mio stack. Faccio puntare lo stack ad un'altra zona di memoria e posso fare le chiamate nel kernel senza preoccuparmi.
5. Posso smettere di usare l'assembly e posso usare un linguaggio ad alto livello ad esempio: eseguo una routine di gestione dell'interrupt che è scritta in C.
6. Sempre in C decido qual'è la prossima istruzione che dev'essere eseguita. Devo fare passare una da ready a running. Scelta basata su priorità, da quanto era in coda, su cosa stava facendo. Siccome sono tanti e parecchi sono in coda quest'operazione dev'essere fatta bene.
7. Torno a fare un pezzo di codice scritto in assembly e mi prende i dati dal process control block, li copia nella CPU senza sovrascrivere registri e copia il program counter appena dopo aver riportato la CPU in modalità utente, da lì il nuovo processo ricomincia a fare quello che stava facendo.

Traduzione in inglese di quello scritto prima, preso dal libro:

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector
3. Assembly-language procedure saves registers
4. Assembly-language procedure sets up new stack
5. C interrupt service runs (typically reads and buffers input)
6. Scheduler decides which process is to run next
7. C procedure returns to the assembly code
8. Assembly-language procedure starts up new current process

Il clock della CPU va velocissimo, non è corretto dire che noi abbiamo un cambio di processo ogni volta che il clock ci dà un segnale. Quando arriva il segnale muove una serie di cose nella parte digitale della CPU, quello che viene fatto dal kernel è aggiornare un contatore (+1), c'è un numero che dipende dal sistema operativo che dice di aver accumulato abbastanza tick che il quanto di tempo è finito, è un parametro che vogliamo configurare, ad esempio nel server microsoft ci dicono che per agevolare le operazioni di servizio allora il quanto di tempo è più alto. Noi non dobbiamo dire che ogni volta che il clock batte noi cambiano processo, in ciclo di clock possiamo fare un'operazione, quindi non ha senso che un processo abbia solo un ciclo di clock.

3.7 Thread

I thread sono dei flussi esecutivi paralleli che stanno all'interno di un processo. All'interno di un processo non viene toccato il kernel, ad esempio nell'editori di testo d'esempio abbiamo tre thread, uno gestisce la tastiera, uno l'impaginazione ed uno il salvataggio periodico sul disco.

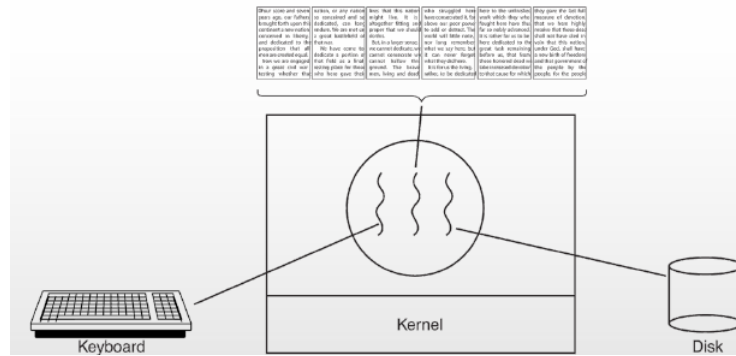


Figure 3.6: Esempio1

I thread vengono anche utilizzati nei sistemi distribuiti per fornire servizi ad utenti remoti. E' esattamente lo stesso esempio di prima, e' un modello di servizio, quando abbiamo una fruizione remota di un servizio abbiamo modello client-server, esiste un concetto di processo che e' quello di web server che detiene le pagine che andiamo a consultare, per ogni browser che si collega viene creato un thread creato a lui che vive di vita propria che gli fornisce le pagine, pescando dallo stesso database del servizio.

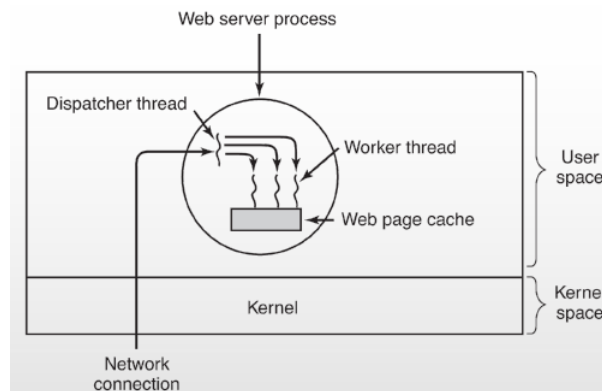


Figure 3.7: Esempio2

3.7.1 Modello classico

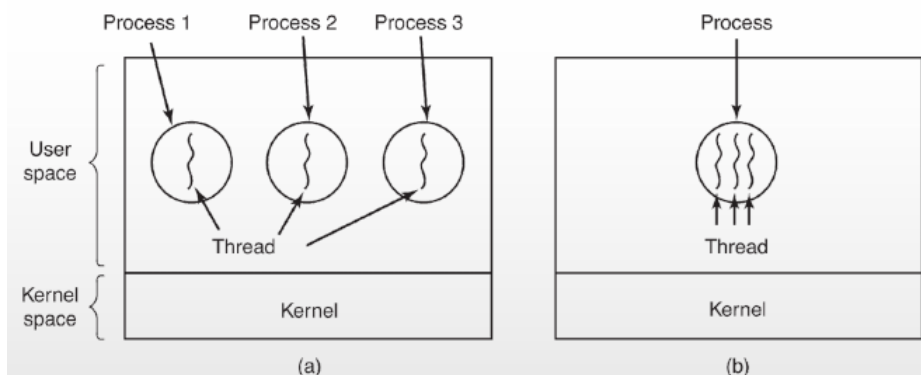


Figure 3.8: Il modello "classico"

Mi trovo nella posizione (a) in cui ho un processo che e' detentore di un flusso esecutivo, in questo caso si parla di un processo mono thread. Agli inizi degli anni '90 si e' introdotto il multi-thread, java ci ha dato un meccanismo per poter gestire i thread in modo semplice ed efficace, quindi abbiamo cominciato a chiamare i processi come contenitori di thread. Dobbiamo gestire delle strutture dati per coordinare i thread peche' i kernel non sanno cosa sono i thread e di conseguenza non sono in grado di gestirli.

Implicazioni del modello classico

Ho una serie di informazioni per processo, condivise da tutti i thread, ed informazioni per thread in private dei thread. Le prime sono gestite dal sistema operativo che guarda solo il contenitore, ad esempio la memoria del processo (poi come i thread la gestiscono non e' problema dell'os), le variabili globali, i file aperti, processi figli, allarmi segnali, informazioni di accounting (le statistiche del processo sono le statistiche comuni a tutti i thread). Quelle specifiche sul thread sono poche e solo quelle che servono per gestire il thread: il program counter, i registri, lo stack, lo stato.

3.7.2 Thread hanno informazioni locali

Ho dei dati globalmente disponibili e dei dati locali, se non c'e' anche il kernel, qualcuno deve implementare questi meccanismi. Il kernel ha due opzioni:

- A: i kernel vecchio stampo (UNIX) continuano a fare tutti i calcoli in termini di processo, quando decido a chi va la CPU decido qual'e' il prossimo processo do la CPU a lui e lui internamente (dotato di libreria) decide che thread eseguire ogni volta che prende la CPU. Devo farlo all'interno di ogni processo, a rigor di logica non e' obbligatorio che le librerie siano

uguali. La Java VM e' un esempio di libreria che si smazza i thread in UNIX.

- B: i kernel piu' moderni (windows) gestiscono meglio i thread, perche' all'interno del kernel mettono l'informazione che esistono i kernel, quindi ho sia la process table sia la thread table. E' la thread table quella che uso per fare lo scheduling. Quando decido qual'e' il prossimo thread ad usare la CPU scelgo direttamente il thread e non il processo. In questo caso posso definire i processi come una famiglia di thread.

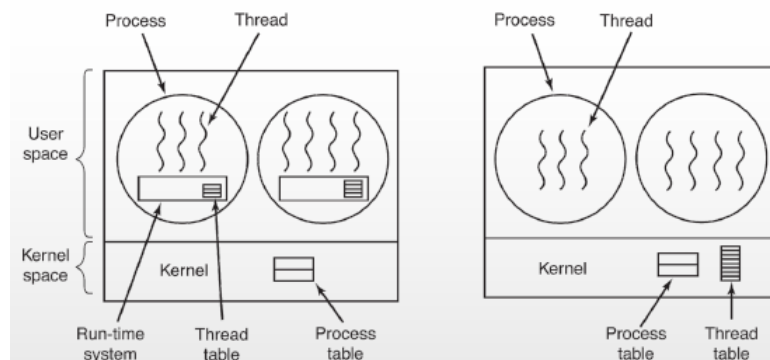


Figure 3.9: Spazio utente vs kernel

Soluzione ibrida

E' la cosa piu' complessa che possiamo fare per gestire i thread, faccio i thread anche all'interno del kernel. E quindi posso associare gruppi di thread a livello utente a gestori implementati a livello kernel. Si e' rivelato troppo complicato da gestire.

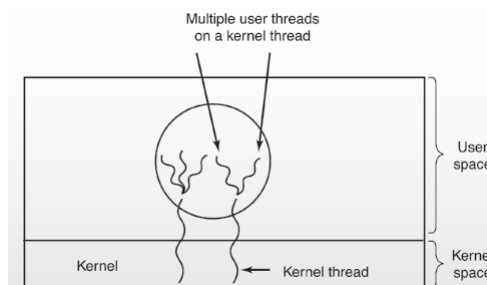


Figure 3.10: Soluzione ibrida

3.8 Capitoli

Capitolo 2:

- 2.1
- 2.2:
 - tranne 2.2.3
 - Fino a 2.2.6 (implementazione ibrida) incluso

Chapter 4

Thred in Go

Come faccio a gestire i thread con un linguaggio di programmazione ad alto livello (GO). Il modo poi' comodo e veloce per sperimentare con flussi esecutivi concorrenti:

- Facile da usare
- Con poche chiamate possibili: esiste una keyword che fa diventare un blocco di codice un thread, in pratica fa diventare una funzione un thread, nel momento in cui termina il thread muore.
- Facile implementare lo scambio di messaggi e la coordinazione per l'accesso a una risorsa. Immaginiamo di aver un processo che gestisce qualche struttura multimediale, c'e' un thread che prende campioni da un microfono, un'altro li trasforma in dati digitali e li manda in play out. Esiste un rapporto produttore consumatore all'interno dei sistemi.

Voglio fare un processo che usa piu' thread, ne voglio uno che scandisce il tempo:

```
package main

import "fmt"
import "time"

func sleep(delay int) {time.Sleep(time.Duration(5) * time.Millisecond)}

func main() {
    go bump("*", 300)
    go bump("_", 245)
    go bump("$", 199)
    go bump(".", 211)
    fmt.Println("I hate this keyboard")
    sleep(5000)
}

func bump(mark string, delay int) {
    for {
        fmt.Println(mark)
        sleep(delay)
    }
}
```

Se chiamo la funzione bump cosi' non mi stampa mai la seconda cosa, usando go bump dico al bump che diventa un flusso esecutivo parallelo, in questo caso non stampa punti. Nei thread c'e' lo stesso tipo di rapporto tra processi. Ho il thread principale che ha eseguito il main, ha creato un thread secondario che associa come genitorialita' al main, che stampa "I hate this keyboard" e poi termina. Questo solo in go, perche' dipende dalla libreria che abbiamo

sottomano. Per stampare qualche punto dobbiamo dire al thread principale di aspettare del tempo prima di levarsi di torno, si fa importando la libreria `time`. Complichiamo un po' modificando la funzione `bump`, dico che voglio stampare un marker con un delay di tot millisecondi. Faccio partire quindi 4 thread figli del main (i go bumpo). Non esiste un modo per fermare i thread, per questo ho bisogno di dire ai thread che non mi servono piu', quindi una cosa che posso fare e' definire una variabile `run` booleana e nella funzione `bump` devo dire che il for continua finche' il valore e' true, non funziona perche' gli sto passando un valore, non una risorsa condivisa, quindi il valore non cambiera' mai ed il thread non si ferma:

```
package main

import "fmt"
import "time"

func sleep(delay int) {time.Sleep(time.Duration(5) * time.Millisecond)}

func main() {
    run := true
    go bump("*", 300, run)
    go bump("_", 245, run)
    go bump("$", 199, run)
    go bump(".", 211, run)
    fmt.Println("I hate this keyboard")
    run = false
    sleep(5000)
}

func bump(mark string, delay int, run bool) {
    for run == true {
        fmt.Println(mark)
        sleep(delay)
    }
}
```

Una soluzione sarebbe di mettere `run` come variabile globale:

```
package main

import "fmt"
import "time"

globalrun := true
```

```

func sleep(delay int) {time.Sleep(time.Duration(5) * time.Millisecond)}

func main() {
    go bump("*", 300, globalrun)
    go bump("_", 245, globalrun)
    go bump("$", 199, globalrun)
    go bump(".", 211, globalrun)
    fmt.Println("I hate this keyboard")
    globalrun = false
    sleep(5000)
}

func bump(mark string, delay int, run bool) {
    for globalrun == true {
        fmt.Println(mark)
        sleep(delay)
    }
}

```

Ma e' una pessima tecnica di programmazione, perche' cosi' la funzione non e' riutilizzabile, esiste un altro modo piu' pulito che e' il passaggio per riferimento, cosi' facendo passo un riferimento alla variabile e non la passo per copia: (stampa i simboli in base a come gira allo scheduling del kernel)

```

package main

import "fmt"
import "time"

run := true

func sleep(delay int) {time.Sleep(time.Duration(5) * time.Millisecond)}

func main() {
    go bump("*", 300, &run)
    go bump("_", 245, &run)
    go bump("$", 199, &run)
    go bump(".", 211, &run)
    fmt.Println("I hate this keyboard")
    run = false
    sleep(5000)
}

func bump(mark string, delay int, run *bool) {
    for *run == true {
        fmt.Println(mark)
        sleep(delay)
    }
}

```

```
    }
}
```

4.1 Gestione thread Go

I thread in go prendono il nome di goroutine, possono essere creati dinamicamente ma non ho una corrispondente direttiva per farli terminare. La direttiva `go` non mi dà un output di alcun tipo. Sono funzioni che prendono vita in maniera indipendente tramite la direttiva `go`. Anche la funzione `main` è una goroutine. Quando `main` termina tutti gli altri thread vengono spazzati via, non è vero il contrario. Non posso manipolare lo stato esecutivo di un thread, posso manipolare il suo comportamento facendo leva su un set di risorse condivise, ad esempio il `run` poteva essere un intero che dava un'indicazione sul tempo di attesa, ad esempio se era 1 il tempo era quello, 2 lo raddoppiavo etc. L'unica cosa che posso fare è mandare questi input come variabili comuni, risorsa condivisa.

Voglio ricreare una situazione in cui ho un conto corrente su cui faccio operare due thread, uno che va a vedere periodicamente quanti soldi ci sono sopra, ed uno che periodicamente tenta di fare un investimento: prende un euro, se lo tiene per un po' e poi lo rideposita:

```
package main
```

```
import "fmt"
import "time"
```

```
//questa è la risorsa condivisa in cui investitore e broker mettono mani
type Account struct {cash int}
```

```
func sleep(delay int) {time.Sleep(time.Duration(5) * time.Millisecond)}
```

```
func main() {
    acc := Account{1000}
    thinkdelay := 100
    investdelay := 100
    go observer(&acc, 1000)
    // pensa per 100 millisecondi fai le tue cose per altri 100 millisecondi e rideposita
    go broker(&acc, 1, thinkdelay, investdelay)
    go broker(&acc, 1, thinkdelay, investdelay)
    go broker(&acc, 1, thinkdelay, investdelay)
    go broker(&acc, 1, thinkdelay, investdelay)
    // vedo cosa succede nel sistema per 10000 millisecondi
    sleep(10000)
}
```

```
func observer(acc *Account, delay int) {
```

```

    for {
        fmt.Println(acc.cash)
        sleep(delay)
    }
}

// tolgo un'euro e lo rimetto facendo una pausa
func broker (acc *Account, amount int, pause int, invest int) {
    for {
        sleep(pause)
        acc.cash -= amount
        sleep(invest)
        acc.cash += amount
    }
}

```

Aggiungo tanti broker e vedo che ci sono momenti in cui piu' di loro sono in sleep quindi risulta dalla stampa dell'observer che ho meno soldi in banca. Provo a ridurre il tempo in cui i broker pensano prima di prendere i soldi e fare gli investimenti, provo addirittura a togliere le sleep e sovraccarico la CPU. Provo a tenere le sleep e chiamo la sleep con 0 millisecondi con questa cosa vengono fuori numeri diversi (sia in salita che in discesa verso lo zero), perche' la sleep del go e' una front end di una system call, per qualche motivo oscuro la sleep con un parametro 0 mi fa trattare il thread in modo strano, pero' questo trattamento produce un risultato brutto, capita che il thread venga interrotto a meta' dell'esecuzione del += o del -=, perche' account.cash viene tradotto come: guardo cosa c'e' nel conto sottraggo 1, copio la quantita' nel conto. Siccome ho questa situazione posso essere interrotto dall'interupt a meta' di questa operazione in cui sto aggiornando il mio conto in banca, quindi mi trovo in una situazione in cui un broker usa il riferimento al conto che usava un'altro. Quindi possono generarsi o perdersi euri. In banca un'operazione su una risorsa condivisa (tipo un conto corrente) deve essere transazionale, evo proteggerla in modo tale che guardare quanto c'e' tirare fuori un euro e riaggiornare devono essere condivise. Esistono dei meccanismi per rendere la transazione di prendo un euro e metto un euro atomica, anche se vengo tolto dalla schedulazione nessuno mettera' le mani sulla risorsa condivisa fintanto che lo diro' io.

4.1.1 Thread facili da gestibili

Si, ma solo se implementati con criterio. Nel codice di prima non c'erano bug, ma ciononostante generava codice sui dati nel momento in cui veniva gestito in un contesto di multiprocesso. Quando due o piu' thread accedono alla stessa risorsa nessuno lo sa, perche' dipende tutto dalla schedulazione. Questo apre una serie di problematiche relative all'uso delle periferiche, o della memoria. Se risolvo il problema della transazionalita' delle operazioni creo altri problemi. Nella nostra vita' un'operazione del genere e' una telefonata (telefono e metto

giu), puo' capitare il deadlock si aspetta che le risorse siano disponibili e tutti sono bloccati. Come quando due persone si aspettano che sia l'altro a chiamarli.

Chapter 5

IPC: Inter-Process Communication

5.1 Competizione per le risorse

I processi devono coordinarsi per usare le risorse (la CPU). L'esempio tipico e' quello fatto con la stampante, perche' all'epoca della scrittura del Tanenbaum non c'era il concetto di programmazione concorrente. Ho due flussi esecutivi che devono coordinarsi per occupare uno slot di un sistema di spooling, che e' una directory in cui metto in coda i file da stampare. Nella posizione 7 ho due processi, entrambi devono stampare un documento, se vengo interrotto dal' sistema di time sharing tra verifico se e' libera e ci metto qualcosa ho un problema di sovrapposizione.

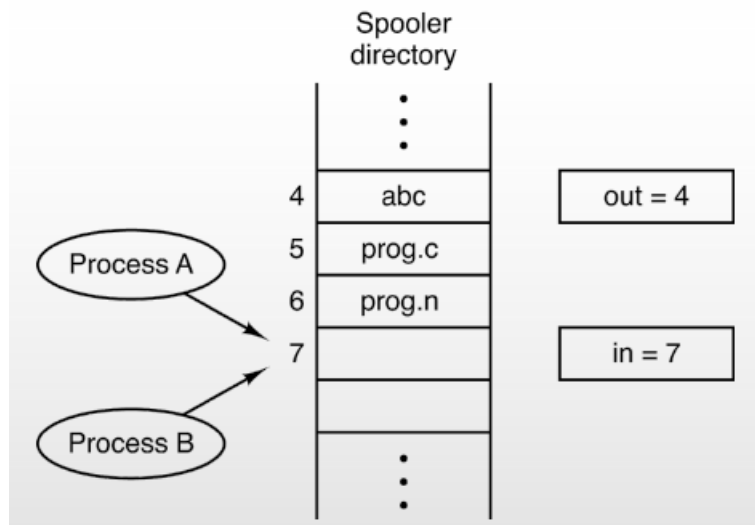


Figure 5.1: Competizione

5.2 Regioni critiche

La regione critica e' una parte del codice di un processo all'interno della quale faccio accesso ad una risorsa condivisa. Definite a livello di azioni che fanno un processo. Vedremo tramite esempio dei processi con accesso a regioni critiche e non critiche. Ci sono 4 regole fondamentali:

1. Non ci possono essere contemporaneamente due processi all'interno di regioni critiche. Non sto neanche considerando che due processi eseguono lo stesso programma.
2. Non posso fare assunzioni sulla base della velocita' della CPU ed il numero di CPU. Perche' questo va soltanto a complicarmi la vita, non posso assumere che l'operazione sia finita prima dell'intervento dello scheduling. Posso essere interrotto in qualsiasi momento.

3. Se un processo sta eseguendo codice al di fuori della regione critica (risorsa condivisa), non deve dare fastidio al processo che la sta usando.
4. Nessun processo dovrebbe aspettare un tempo indefinito per entrare all'interno della sua regione critica. Si introduce il concetto di attesa: devo entrare nella zona critica, c'è già qualcuno dentro ed aspetto. Il non aspettare per sempre dipende dalla buona intenzione del programmatore.

5.2.1 Accesso a regione critica

Ho due processi A e B, il tempo scorre da sinistra verso destra, i due processi vengono eseguiti in time sharing, quindi abbiamo esecuzione pseudo-parallela. All'istante t_1 A entra in una delle sue regioni critiche (perché la regione critica di un processo non dev'essere per forza continua). Nel momento in cui B raggiunge t_2 anche lui vuole eseguire delle istruzioni che sono nella sua regione critica, che fanno riferimento alla stessa risorsa condivisa di A, entra in gioco la regola 1 quindi B non può continuare la sua esecuzione ed entra in stato di bloccato. Tornerà in ready nell'istante t_3 in cui A esce volontariamente dalla sua regione critica. A questo punto il kernel segnala a B che può continuare, appena la schedulazione sarà a suo favore potrà accedere alla regione critica fino al tempo t_4 . Sto facendo l'assunto che ci siano solo 2 processi per quell'unica risorsa condivisa. Se ci sono più processi il primo che arriva entra, c'è quindi una competizione per le risorse. Il problema è l'attesa, come faccio a garantirmi che il processo B non entri nella sua regione critica mentre c'è A.

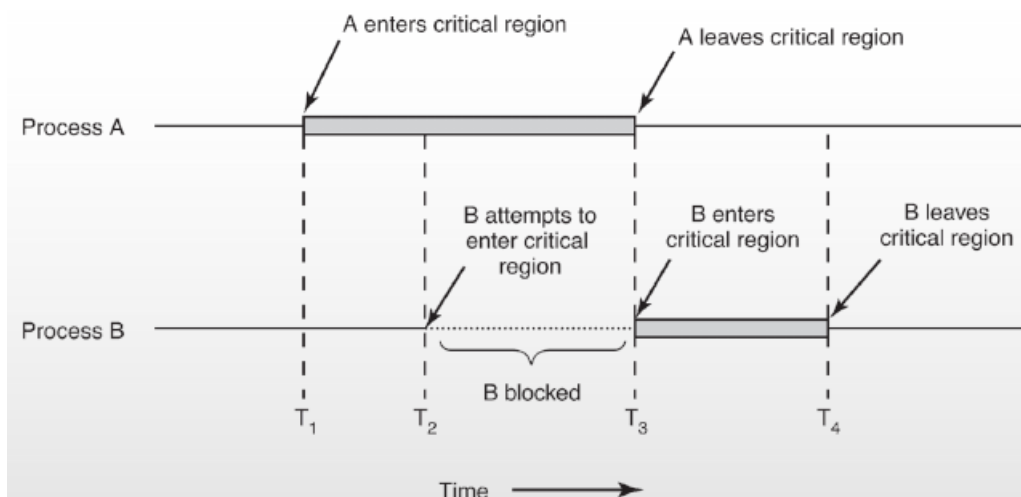


Figure 5.2: Accesso a regione critica

5.2.2 Mutua esclusione con Busy Waiting

Inizialmente per risolvere il problema si e' provato con un sistema basato sulla **disabilitazione degli interrupt**. Il processo dice al kernel di disabilitare tutti gli interrupt, incluso quello di clock. Disabilito lo scheduling fino a che sono nella zona critica. Per fare questo devo assumere che il codice sia esente da bug, nel caso in cui si impiantasse l'unico modo che ho e' quello di resettare la macchina. Poi abbiamo provato a usare delle **variabili di lock**, sono delle variabili che faccio usare dai processi per dire se una risorsa e' libera oppure no, il problema di questa cosa e' che anche le variabili di lock sono delle risorse condivise tra processi, come faccio ad assumere che l'accesso alle variabili di lock sia atomico? Successivamente si e' detto: portiamo l'esecuzione a livello di microprocessore, sono nati una serie di processori con una microistruzione (TSL: test and set lock) della CPU che io uso su una locazione di memoria se il contenuto della locazione lo mezzo a 0 e vado avanti, se e' gia' zero vado ad eseguire l'istruzione, il problema e' che essendo a livello del processore non sa dell'esistenza dei processi, per cui non puo' mettere in stato di bloccato che e' chiamato dalla TSL, potrei dover richiamare la TSL subito dopo averla bloccata perche' potrebbe essersi liberata la risorsa, facendo cosi' si crea uno stato di busy waiting, uso la CPU per capire se e' libera (non entro in stato di waiting). Quando ci siamo resi conto che queste soluzioni avevano problemi, sempre seguendo una filosofia di busy waiting ma piu' strutturate. Il busy waiting potrebbe essere la cosa migliore in alcuni contesti, quindi per risolvere almeno questo tipo di problematiche abbiamo creato prima l'alternanza stretta: definisco una variabile turno che e' condivisa tra due processi e la uso per far accedere alla risorsa in maniera furba alternando tra processo 0 e processo 1. Va bene se ad esempio ho un sistema di campionamento audio

```
while (TRUE) {
    while (turn !=0) //loop
        critical_region();
    turn=1;
    noncritical_region();
}

while (TRUE) {
    while (turn !=0) //loop
        critical_region();
    turn=0;
    noncritical_region();
}
```

Fintanto che turn e' diverso da 0 continuo a controllare il valore della variabile e quindi uso la CPU finche' non interviene lo scheduling. Nel momento in cui turno e' diverso da 0 allora il while ha una condizoina falsa ed eseguo il codice entrando nella regione critica, dopo di che dico che turno =1, dall'altra parte il processo 1 potra' entrare nella sua regione critica ed intanto continuo ad eseguire

la mia regione non critica. Questa cosa funziona ma ha un problema, mi forza a fare alternatamente i processi. Che cosa succede se ad esempio il processo 0 per attraversare la regione critica impiega meno del processo 1? Potrebbe essere piu' comodo eseguire prima i processi veloci. Algoritmo di Peterson: qui ho sostituito la variabile del turno con chi sta usando la regione critica ed un'array che dice chi sono i processi interessati ad entrare nella regione critica (e' la variante semplice in cui si assume che ci siano solo 2 processi), definisco una variabile `other` che indica l'altro processo (se `processo = 1` `other = 0`). Per entrare nella regione critica metto valido il flag che se c'e' diponibila' io sono in coda, dopo di che provo a dire che il turno e' mio (sono interessato e provo a dichiarare il mio turno). Dopo di che mi blocco su `while` e faccio `busy waiting` fintanto che e' il mio turno e l'altro processo e' interessato ad entrare nella regione critica, perche' se e' il mio turno sono io l'ultimo in coda, pero' se c'e' qualcun'altro che ha gia' passato nella regione critica allora l'interesse di `other` e' `true` (vale 1). Significa che c'e' l'altro che e' gia' dentro. Come faccio a far diventare false la condizione nel `while`. Puo' darsi che l'altro processo abbia cambiato il valore di `turn` oppure che l'interesse di `other` sia `true` (il che significa che l'altro processo ha lasciato la regione critica).

```
#define FALSE 0
#define TRUE 1
#define N 2

int  turn;
int  interested[N];

void enter_region(int process)
{
    int other;

    other = 1 - process;
    interested[process] = TRUE;
    turn = process;
    while (turn == process && interested[other] == TRUE)
}

void leave_region(int process)
{
    interested[process] = FALSE;
}
```

Noi dobbiamo passare dall'alternanza stretta (da turno come testimone) alle region. Tutto si gioca su quelle righe di codice mettendo `interested(process)` a `true` e `turn = process`, e poi nel `while` controllo l'interesse dell'altro processo. Quindi potrei rimanere in questo loop per sempre se non interviene lo scheduling. In un sistema da uso generico vorrei usufruire del multi thread.

5.3 Sleep and Wait

Metto il processo in fase di sleep (bloccato), risolve una serie di problemi:

- Uso inutile della CPU: e' chiaro perche' devo continuamente monitorare lo stato della CPU
- Priority inversion: immaginiamo di avere due processi che usano alternanza stretta, uno a priorita' altissima ed uno a priorita' molto bassa, il processo a priorita' altissima deve comunque aspettare quello a priorita' bassa. Non importa la priorita' dei processi, perche' alla fine il lavoro lo giuda quello con priorita' bassa, quindi inverte la priorita' dei processi.
- Faccio assunzioni sulla struttura del sistema, il busy waiting funziona bene nel momento in cui ho risorse in piu'.

E' molto meglio chiedere al sistema operativo che ci faccia aspettare che una risorsa sia disponibile. Quando si parla di sleep and wait mediata dal sistema operativo ci sono dei pattern (problemi ad esempio utili per capire se la soluzione e' giusta)

5.3.1 Produttore e consumatore

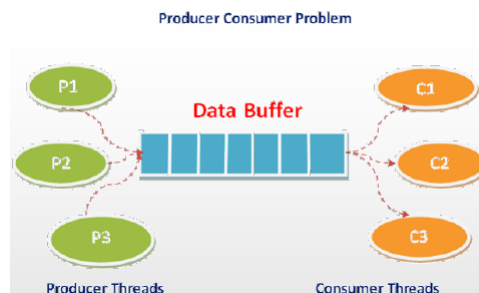


Figure 5.3: Produttore-Consumatore

Il problema produttore e consumatore e': ho un buffer di capacita' limitata, ho una popolazione di thread che producono informazioni e le infilano nel buffer. E dall'altra parte ho dei consumatori che devono prendere le informazioni dal buffer ed utilizzarle. non posso fare assunzioni sulla velocita con cui i produttori creano informazioni e sulla velocita con cui i consumatori le tirano fuori. Inoltre non le faccio neanche sulla numerosita' dei miei gruppi. I problemi: il buffer potrebbe essere pieno o vuoto, ho il problema che i produttori e consumatori accedano al buffer senza pestarsi i piedi (come nell'esempio del conto in banca dell'altra volta). Proviamo a risolverlo utilizzando una soluzione sbagliata: ho un buffer da 100 posizioni e poi una serie di produttori/consumatori di cui non specifico il codice. Produttore ciclo infinito, produce qualcosa che va a finire in una variabile di tipo item, se il buffer e' pieno mi metto in sleep, quando

termino la sleep e' perche' qualcuno mi ha svegliato (verosimilmente perche' hanno svuotato il buffer).

```
#define N 100
int count = 0;

void producer(void) {
    int item;

    while (TRUE){
        item = produce_item();
        if(count==N) sleep();
        insert_item(item);
        count = count + 1;
        if(count==1) wakeup(consumer);
    }
}

void consumer(void) {
    int item;

    while (TRUE) {
        if(count==0) sleep();
        item = remove_item();
        count = count -1;
        if(count == N-1) wakeup(producer);
        consume_item(item);
    }
}
```

Ho ancora il problema che il `count = count + 1` o `-1` potrebbero essere soggetti ad interruzioni, quindi andrei a compromettere il valore di `count`. Anche se dovessi avere il buffer pieno il produttore va in sleep, ed appena prima di entrare in sleep vengo sospeso, la wakeup parte comunque ma non sveglia nessuno, quindi se lo scheduler mifrega in questo momento mi trovo ad avere un produttore bloccato che non si sveglia mai ed un consumatore che mi svuota il buffer. Dobbiamo rendere atomico l'accesso al `count` e rendere in un colpo solo il controllo del buffer e il mettersi in sleep.

5.4 Strutture dati complesse

Semafori(mutex) e' una struttura dati con due operazioni possibile (somiglia ad un oggetto), posso fare down e up. Se devo entrare in una stanza devo accendere la luce (down, la accendo mentre entro), quando qualcuno esce e tira su l'interruttore, quando qualcuno esce noi possiamo fare la nostra operazione atomica di uscire e accendere la luce (down). Quando faccio la down se il se-

maforo aveva il valore 1 diventa di valore 0 ed io funziono, se il semaforo aveva valore 0 facendo la down io mi blocco (entro in bloccato), quando usciro' dal bloccato vuol dire che saro' riuscito a fare l'operazione di down. Non tutti i semafori funzionano cosi', dobbiamo immaginarci un locale con capienza massima come un buffer, si puo' entrare quando qualcuno esce, c'e' un contatore nel buffer che quando e' a 0 si blocca ed aspetta che qualcuno esca dal buffer. A livello basso e' implementato da TSL a livello implementativo non posso assumere che quando torno in running quando tocca a me entrare ma chi e' il piu' veloce entra. In go si fa importando la libreria MUTEX. Sono in meccanismo di basso livello, non ci implementa un semaforo con n posizioni ma solo up e down. I monitor sono un'altro approccio, chiedono l'aiuto di un linguaggio di programmazione, ad esempio Java usa solo i monitor. Tu scrivi il tuo codice come ti pare poi i pezzi di codice fanno parte della regione critica me li decori con SINCRONIZE una volta che abbiamo dichiarato quali sono i pezzi di codice che fanno parte della regione critica da li in poi ci pensa il linguaggio.

5.4.1 Soluzione con semafori

Definisco tre semafori: 1 e' il mutex (mi assicura che i processi non vadano a pestarsi i piedi quando mettono le mani nel buffer, quando tolgo ed inserisco oggetti nel buffer proteggero l'operazione facendo down ed up del mutex), 1 serve a contare il numero delle posizioni libere nel buffer (empty che inizialmente vale n, il buffer e' di 100 posizioni e' empty vale 100, arrivera' a 0 nel momento in cui il buffer e' pieno), 1 full che conta il numero di posizioni occupate all'interno del buffer, vuol dire che un produttore fara' up aumentando il valore della variabile ed il consumatore fara' down quindi arriva a 0 quando il buffer e' vuoto. Mi garantiscono la mutua esclusione, di bloccare i produttore quando il buffer e' pieno e di bloccare i consumatori quando e' vuoto. Devo prima di andare ad operare sul buffer assicurarmi nel caso del produttore che ci sia spazio vuoto, se c'e' e' sicuramente il mio, dopo che ho finito di occuparlo segnalo un up che il full ha un elemento in piu', percui se c'erano dei consumatore che erano bloccati su una down di full in quel momento si svegliano, il primo che riesce ad arrivare alla CPU consuma e continua. Dall'altra parte il consumatore fara' down di full (si prenota uno slot che dev'essere pieno) e dopo di che segnalera' che uno slot e' vuoto facendo una up di empty.

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void procedure(void) {
    int item;

    while (TRUE) {
```

```

        item = produce_item();
        down(&empty);
        down(&mutex);
        intert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void) {
    int item;

    while(TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}

```

5.5 Barriere (pattern)

Ho 4 processi che si devono sincronizzare, questi arrivano ad un certo stadio di elaborazione, si devono bloccare e poi c'e' un'agente esterno (tendenzialmente un altro processo) che dice che sono arrivati tutti alla barriera e si riparte tutti insieme. Ad esempio nella riproduzione multimediale abbiamo tanti processi da sincronizzare.

5.6 Problema dei 5 filosofi

E' un pattern che va a dimostrare il problema dello stallo (deadlock). Abbiamo 5 filosofi, sono seduti ad un tavolo ed alternano tra il mangiare ed il pensare per mangiare hanno bisogno di 2 forchette, devono prendere quella dx e es del piatto, se i filosofi sono fatti male tutti partono per mangiare contemporaneamente tutti prendono contemporaneamente la forchetta dx e sono in attesa di prendere la sx che ce l'ha in mano quello di sx che non l'appoggera' mai, quindi nessuno mangia. Significa che il sistema e' in stallo, la mutua esclusione delle forchette e' fatta bene (nessuno rubera' la mia forchetta), ma abbiamo incastrato tutti i sistemi. Chi si occupa del deadlock usa questo problema come beanchmark.

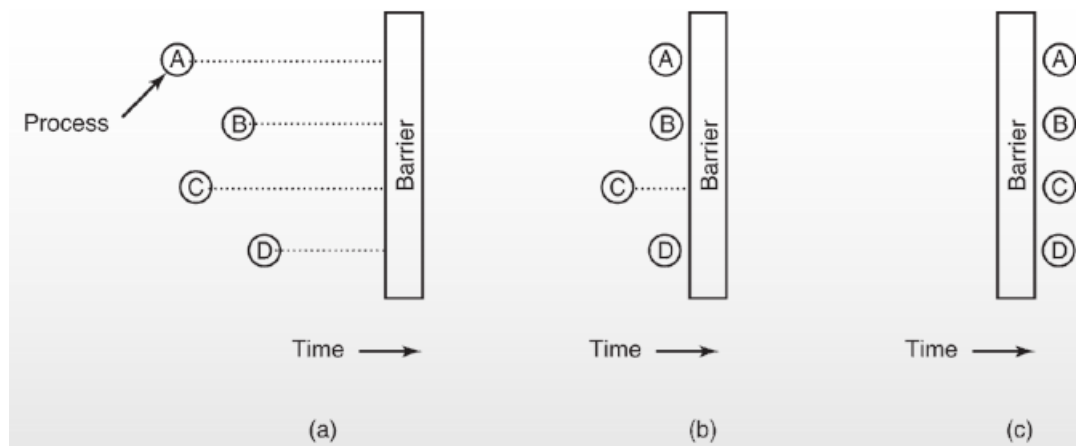


Figure 5.4: Barriere

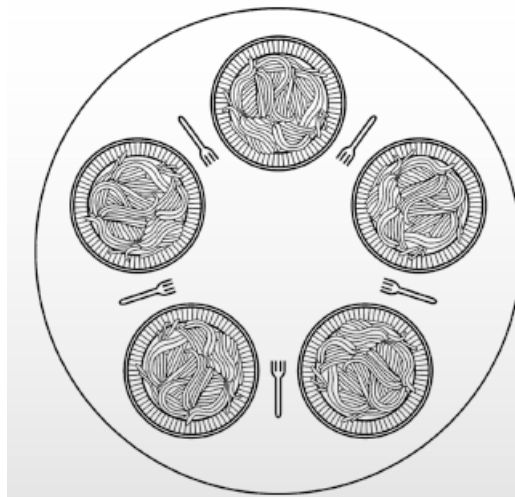


Figure 5.5: 5 filosofi

5.7 IPC in Go

Dobbiamo implementare:

- Mutex
- Semafori
- Produttori-consumatori
- 5 filosofi

5.7.1 Mutex

Sono già disponibili nella libreria sync:

- `sync.Mutex.Lock()` = Down, porto il semaforo da 1 a 0
- `sync.Mutex.Unlock()` = Up, porto il semaforo da 0 ad 1 cercando di proteggere la risorsa condivisa che stavo utilizzando.

```
package main
```

```
import "fmt"
import "time"
import "sync"
```

```
type Account struct {cash int}
```

```
func sleep(delay int) {time.Sleep(time.Duration(5) * time.Millisecond)}
```

```
func main() {
    acc := Account{1000}
    thinkdelay := 100
    investdelay := 100
    go observer(&acc, 1000)
    // pensa per 100 millisecondi fai le tue cose per altri 100 millisecondi e ridepo

    \\ creo un mutex che mi sincronizza le operazioni
    mux := sync.Mutex{}
    go broker(&acc, invest, thinktime, &mux)
    go broker(&acc, invest, thinktime, &mux)
    go broker(&acc, invest, thinktime, &mux)
    go broker(&acc, invest, thinktime, &mux)
    // vedo cosa succede nel sistema per 10000 millisecondi
    sleep(10000)
}
```

```
func observer(acc *Account, delay int) {
    for {
        fmt.Println(acc.cash)
        sleep(delay)
    }
}
```

```
// tolgo un'euro e lo rimetto facendo una pausa
func broker (acc *Account, amount int, pause int, invest int, mux *sync.Mutex) {
    for {
        sleep(pause)
```

```

        mux.Lock()
        acc.cash -= amount
        mux.Unlock()
        sleep(invest)
        mux.Lock()
        acc.cash += amount
        mux.Unlock()
    }
}

```

Un modo migliore per scrivere codice che poi possa essere cambiato e' quella di legare in modo funzionale il mutex al conto corrente. Questo conto corrente ha un bilancio e per accedere al bilancio bisogna usare un mutex che e' proprio del conto corrente, quindi il mutex lo si appiccica in modo indissolubile ai dati. Tutte le informazioni che sono legate ad un thread le metto nello stesso oggetto, nei linguaggi evoluti si preferisce dare la funzione ma non fornire il modo in cui e' implementata. Creo un oggetto ci metto un dato, e dico che per accedere a quel dato ci posso passare solo tramite le funzioni dell'oggetto.

```
package main
```

```
import "fmt"
import "time"
import "sync"

```

```
type Account struct {cash int, mux sync.Mutex}
```

```
func sleep(delay int) {time.Sleep(time.Duration(5) * time.Millisecond)}
```

```
func main() {
    acc := Account{1000, sync.Mutex} // lego il mutex all'account
    thinkdelay := 100
    investdelay := 100
    thinktime := 100
    go observer(&acc, 1000)
    // pensa per 100 millisecondi fai le tue cose per altri 100 millisecondi

    \\ creo un mutex che mi sincronizza le operazioni
    //mux := sync.Mutex{}
    go broker(&acc, invest, thinktime)
    go broker(&acc, invest, thinktime)
    go broker(&acc, invest, thinktime)
    go broker(&acc, invest, thinktime)
    // vedo cosa succede nel sistema per 10000 millisecondi
    sleep(10000)
}
// Il fatto che questa funzione non sia sincronizzata non importa molto

```

```
// se e' usata solo per visualizzare quello che succede.
// non posso poter accedere alla regione critica senza sincronizzazione
// perche' potrei avere due thread contemporaneamente nella regione critica
func observer(acc *Account, delay int) {
    for {
        fmt.Println(acc.cash)
        sleep(delay)
    }
}

// tolgo un'euro e lo rimetto facendo una pausa
func broker(acc *Account, amount int, pause int, invest int, mux *sync.Mutex) {
    for {
        sleep(pause)
        acc.mux.Lock()
        acc.cash -= amount
        acc.mux.Unlock()
        sleep(invest)
        acc.mux.Lock()
        acc.cash += amount
        acc.mux.Unlock()
    }
}
```

5.7.2 Semafori

Questi invece non esistono all'interno del linguaggio, anzi in generale e' difficile trovare linguaggi che ci danno il semaforo generico. Possiamo simularli con un canale, ricreando i semafori, un canale e' una rappresentazione a livello di linguaggio di una pipe (nel senso di tubo). E' un meccanismo di scambio dati tra due processi, uno scrive da una parte e uno legge dall'altra.

```
package main

type empty struct{}
\\c'e' una struttura vuota, non me ne frega niente
// Il semaforo porta avanti ed indietro strutture vuote, mi serve come contatore
type Semaphore chan empty

func CreateSem (size int) Semaphore {
    return make(Semaphore, size)
}
```

Creo un canale di n posizioni che sposta strutture vuote, mi interessa solo la quantita' di posizioni libere, se ho un canale con 10 posizioni libere e' come avere un semaforo con 10 posizioni libere, inserire un oggetto nel canale significa fare

una down (riduco le posizioni disponibili), togliendo faccio una up (aumentano le posizioni disponibili). Fatto questo devo implementare down e up, su un semaforo generico la metafora up e down cambia, ho un semaforo in cui potrei voler fare down di tre volte in un colpo solo. Dal punto di vista funzionale il semaforo mi deve dare la possibilita' di fare tre down in un colpo solo, se non ci sono 3 slot disponibili ne prendo 2 e aspetto il terzo. Ci viene comodo se abbiamo risorse con numerosita' multipla, ad esempio ho bisogno di 4 slot di memoria nel sistema operativo, ne voglio 4 insieme o nessuno. Posso mettere un parametro opzionale con l'array di interi passato come parametro, se l'ho usato n e' il primo valore del array.

```
func (s Semaphore) Down(qtt []int) {
    n := 1
    if len(qtt) > 0 {
        n = qtt[0]
    }
    for i := 0; i < n; i += 1 {
        s <- empty{} // creo una struttura vuota in s
    }
}

func (s Semaphore) Down(qtt []int) {
    n := 1
    if len(qtt) > 0 {
        n = qtt[0]
    }
    for i := 0; i < n; i += 1 {
        if len(s) > 0 {
            // se il canale e' vuoto lo metto in bloccato
            <- s // toglì una struttura da s, non mi interessa
                // quale ne dove la metti
        }
    }
}
```

5.7.3 Produttore-Consumatore

Riprendiamo le stesse cose che facevamo con la banca, definiamo una struttura e dato che non ho informazioni sul conto corrente, ci metto solo i semafori che mi servono per gestire la situazione.

```
type Buffer struct{
    empty Semaphore
    full Semaphore
    mux sync.Mutex
}
```

```

func main() {
    var size int = 50

    b := Buffer{CreateSem(size), CreateSem(size), sync.Mutex{}}
    b.full.Down(size)

    go oserver(&b, 500)

    go producer(&b, 10, 50)
    go producer(&b, 10, 50)
    go producer(&b, 10, 50)

    go consumer(&b, 5, 70)
    go consumer(&b, 5, 70)
    go consumer(&b, 5, 70)

    sleep(10000)

}

```

Il produttore agisce su un buffer, ha un tempo di elaborazione per fare quello che deve, ha un tempo di ritardo dove poi andrà a provare a creare un nuovo elemento. Il tempo di lavoro lo proteggo con i mutex, e poi siccome è un produttore diminuisce il numero degli spazi vuoti e dopo che ha fatto il suo lavoro aumenta il numero degli spazi pieni.

```

func producer(b *Buffer, jobtime int, delay int) {
    for {
        b.empty.Down()
        b.mux.Lock()
        sleep(jobtime)
        b.mux.Unlock()
        b.full.Up()
        sleep(delay)
    }
}

```

Il consumatore aumenta il numero degli slot liberi ed aumenta il numero degli slot occupati.

```

func consumer(b *Buffer, jobtime int, delay int) {
    for {
        b.full.Down()
        b.mux.Lock()
        sleep(jobtime)
        b.mux.Unlock()
        b.empty.Up()
    }
}

```

```

        sleep(delay)
    }
}

```

L'osservatore stampa degli asterischi per la parte di buffer pieno e degli `_` per la parte di buffer vuoto. In go posso usufruire di `capacity` (`cap`) e `len` del canale.

```

func observer(b *Buffer, delay int) {
    for {
        f := cap(b.full) - len(b.full)
        i := 0
        for ; i < f; i += 1 {
            fmt.Print("*")
        }

        for ; i < cap(b.full); i += 1 {
            fmt.Print("_")
        }

        fmt.Println()
        sleep(delay)
    }
}

```

E' chiaro che i produttori vanno piu' veloce, quindi non si svuota mai completamente, ma il buffer risulta quasi sempre pieno, se modifichiamo i tempi si riempie piu' lentamente. Puo' capitare a causa dello scheduler che due o piu' consumatori tolgano oggetti dal buffer e quindi si decrementa.

Di fatto implementiamo i produttore-consumatore con un altro produttore-consumatore (il canale). La scelta di GO non e' darci il semaforo ma direttamente la soluzione per il problema del produttore-consumatore.

5.7.4 Problema dei 5 filosofi

Creo una struttura per la forchetta (la zona critica) e ci appiccico un mutex, solo un filosofo alla volta puo' accedere alla forchetta. Chi prende la forchetta deve anche verificare se puo' prenderla

```

type Fork struct {
    up bool
    holder int
    mux sync.Mutex
}

func main() {
    // creo il tavolo
    var table [10]Fork

```

```

    for i := 0; i < len(table); i++ {
        table[i] = Fork{false, -1, sync.Mutex{}}
    }

    thinktime := 10

    //ogni 5 sec ci stampa la situazione della tavola
    go observer(table[:], 500)
    for i := 0; i < len(table); i++ {
        //creo un thread filosofo e gli dico che ha una forchetta a sx e de
        // il % e' per fare il giro del tavolo
        go philosopher(i, &table[i], &table[(i+1)%len(table)], thinktime)
    }

    //l'ultimo filosofo ha una situazione particolare e me lo gestisco a parte
    go philosopher(len(table)-1, &table[0], &table[len(table)-1], thinktime)

    // per 10 secondi sto a guardare
}

    Il filosofo pensa per un tempo a caso fino a 10 millisecondi (rand.Int(delay)),
    faccio un lock sul mutex della forchetta di dx. Nel momento in cui vado oltre il
    mutex ce l'ho io quindi dico che la forchetta di dx ce l'ho io (older = id). Poi
    mentre ho in mano quella di destra cerco di prendere quella di sx, faccio un lock
    di quella di sx, quando la prendo dico che e' impegnata e quello che c'e' l'ha
    sono io, poi un latro sleep casuale, metto giu' quella di sx e poi quella di dx,
    dopo di che il ciclo ricomincia.

func philosopher(id int, right *Fork, left *Fork, delay int) {
    for {
        sleep(rand.Intn(delay))
        right.mux.Lock()
        right.up = true
        right.holder = id
        left.mux.Lock()
        left.up = true
        left.holder = id
        sleep(rand.Intn(delay))
        left.up = false
        left.mux.Unlock()
        right.up = false
        right.mux.Unlock()
    }
}

```

Chiudo definendo un'osservatore che fa un for sul tavolo e stampa lo stato

delle forchette dei filosofi, la o rappresenta la testa di un filosofo e poi ho uno punto o un | se la forchetta e' abbassata o alzata.

```
func observer(table []Fork, delay int) {
    for {
        sleep(delay)
        for i := 0; i < len(table); i++ {
            printFork(&table[i], i)
            fmt.Print("o")
            printFork(&table[(i+1)%len(table)], i)
            fmt.Print(" ")
        }
        fmt.Println()
    }
}

func printFork(f *Fork, r int) {
    if f.up && f.holder == r {
        fmt.Print("|")
    } else {
        fmt.Print(".")
    }
}
```

E' possibile trovare due filosofi accanto con le forchette alzate, questo perche' l'osservatore non e' sincronizzato quindi potrebbe aver guardato nel momento esatto in cui uno la metteva giu' ed uno la prendeva, con l'intervento dello scheduler. Sincronizzare l'osservatore significherebbe avere un calo di prestazioni, perche' usa un po' di tempo. Abbassando i tempi delle schedulazioni i primi n filosofi si incartano con la forchetta destra in mano e gli ultimi se la giocano. Siamo in questa situazione per lo scheduler, perche' fa un discorso di primo arrivato primo servito (i thread), per qualche motivo quello scheduler da priorita' ai filosofi che stanno in cima al tavolo, gli ultimi decide che non hanno priorita' e quindi quelli devono contendersi le forchette mentre gli altri aspettano. Il sistema evolve, ma in una maniera cosi' difficoltosa da predire che con la frequenza delle fotografie che abbiamo impostato non lo notiamo.

5.8 Capitoli

Capitolo 2

- 2.3

- Della TSL sappiate che cose e', ma ignorate gli esempi in pseudo-assembler
- Tranne la parte di implementazione dei Mutex

- Tranne "Mutexes in Pthreads"
- Tranne 2.3.8 e 2.3.10

Chapter 6

Scheduling

Di tutto quello che dira' non e' mai una soluzione perfetta, quello che va bene per una politica va male per un'altra.

6.1 Comportamento dei processi

Abbiamo gia' discusso, e' relativamente difficile in un os trovare un processo che fa qualcosa, nel 90% della sua vita la CPU aspetta e non fa niente, la maggior parte dei processi e' in attesa dei dati che devono arrivare (processo I/O bunding). La vita di un processo si divide in due:

- Dove attende dei dati: Waiting for I/O: il processo si trova in bloccato
- Dove usa la cpu: CPU burst: sono in uno stato di ready.

Posso cominciare a classificare i processi in base a quanto usano la CPU:

- CPU bound: burst lunghi
- Processi IO bound: burst corti

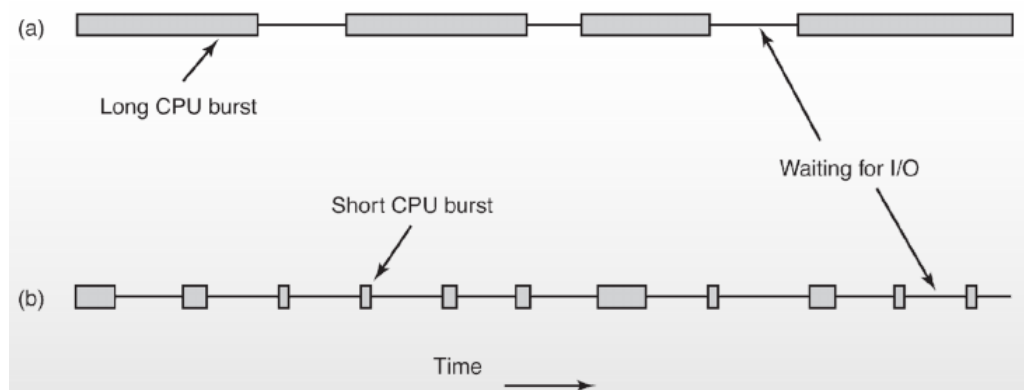


Figure 6.1: Long vs Short CPU burst

La differenza e' solo del burst la lunghezza degli intervalli di waiting e' uguale. E' l'unica distinzione a livello di scheduling che ci conviene fare. Quest'informazione l'abbiamo trovata scritta anche nel process control block.

6.2 Categorie dei sistemi di scheduling

I sistemi di scheduling si dividono in:

- **Batch**: sistemi in cui do in carico il mio job, carico il mio programma che diventa un processo e verra' eseguito fino alla fine. Non ho un sistema di scheduling nel senso in cui ci si aspetta, ha solo il compito di caricare il

processo ed eseguirlo fino alla fine e passare al prossimo. In quel caso parlo di run to completion (eseguo fino a completamento). Ad esempio il sistema di fatturazione di una banca funziona in batch, si fa partire il sistema e lui per ore va avanti a fare fatture, una alla volta, fino a che non ha finito. Sono sistemi molto vecchi, quindi guai se il computer stava fermo.

- Interattivi: sono i sistemi moderni, l'obiettivo e' quello di rispondere all'utente nella maniera piu' veloce possibile. Introducono il time sharing, do' l'impressione all'utente di fare piu' cose contemporaneamente chiedendo l'aiuto alla CPU che con un metronomo interno interrompe l'esecuzione di un processo. Decisione di scheduling una volta per quanto di tempo. Si dividono in base all'obiettivo del sistema di calcolo.
 - soft real time: ad esempio nella realta' tento di azzeccare tutti gli esami ma non muore nessuno se non ci riesco.
 - mission critical devo garantire che una certa percentuale di attivita' deve essere completata entro il tempo richiesto, o sistemi hard real time (quando freno l'azione deve verificarsi istantaneamente affinche' il treno non deragli)

6.3 Obiettivi

- **All system**, sono obiettivi comuni a tutti i tipi di sistemi:
 - Fairness: bisogna dare a tutti i processi la possibilita' di accedere alla risorsa di calcolo (CPU) in maniera ferrea, chi piu' chi meno si vedra', ma tutti quanti devono accederci.
 - Enforcement: una volta che decido qual'e' la logica con cui do la CPU quella regola deve essere rispettata. Non devo dare la possibilita' all'utente di modificare lo scheduler.
 - Balance: Tutti i sistemi di scheduling lavorano con l'obiettivo di tenere il sistema di calcolo con un carico uniforme su tutte le sue parti.
- **Batch system**, stiamo cambiando completamente discorso, nel caso dei singoli approcci ho delle cose numerabili, ho un numero e l'obiettivo e' tenerlo il piu' alto o basso possibile:
 - Throughput: massimizzare il numero di job nell'unita' di tempo

- Turnaround time: minimizzare il tempo che intercorre tra l'istante in cui un processo viene creato ed il momento in cui termino la sua esecuzione
- CPU utilization: la CPU deve sempre essere utilizzata al massimo
- Interactive system:
 - Tempo di risposta: e' il tempo che intercorre tra quando un processo comincia ad esistere all'interno del sistema e quando questo mi da il primo segnale di vita (doppio click sull'icona a quando posso cominciare a scrivere). Bisogna rispondere velocemente.
 - Proportionality: bisogna combaciare con le aspettative dell'utente, e' simile al rispondere velocemente
- Real-Time, misurano quante cose sono effettivamente riuscito a completare rispettando le scadenze:
 - Meeting deadlines: evitare la perdita dei dati
 - Predictability: Devo garantire un buon funzionamento anche all'aumentare del carico e del lavoro che gli chiedo di fare. Sono sicuro di smaltire tutto il lavoro fintanto che ti limiti a tot tracce audio e video, oltre non garantisco.

6.4 FCFS (FIFO)

Primo arrivato primo servito, first in first out.

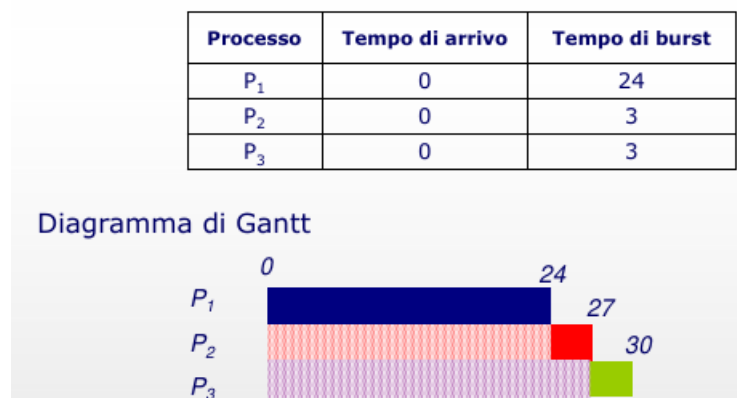


Figure 6.2: FCFS

Il diagramma di GANTT illustra l'evoluzione di un sistema nel tempo. Ho la politica di gestione dei processi (FCFS first come first served) i dati sono riassunti nella tabella, per ogni processo indico un tempo di accesso e un tempo

di burst (le unita' di tempo della CPU che questo processo ha bisogno per arrivare a completamento). All'istante 0 scelgo il processo 1 perche' e' il numero minore e lo eseguo per tutto il suo burst dopo di che riapplico la regola del numero piu' basso. Posso cominciare a tirare fuori degli indici di prestazione:

- Tr (tempo di risposta, quando un processo si presenta all'interno del sistema a quando si impossessa della CPU): nel nostro caso: $P_1 = 0$; $P_2 = 24$; $P_3 = 27$. Se facciamo la media troviamo il Tr medio: $(0 + 24 + 27)/3 = 17$
- Tta (Tempo di turn around, quando il processo termina): nel nostro caso: $P_1 = 24$; $P_2 = 27$; $P_3 = 30$. Se facciamo la media troviamo il Tta medio: $(24 + 27 + 30)/3 = 27$

Il problema di questo scheduler e' che e' fortemente dipendente dalla policy della scelta dell'algoritmo da eseguire. Cambiando l'ordine con cui metto in esecuzione cambiano drasticamente i tempi, pero' non ho ancora predicibilita'.

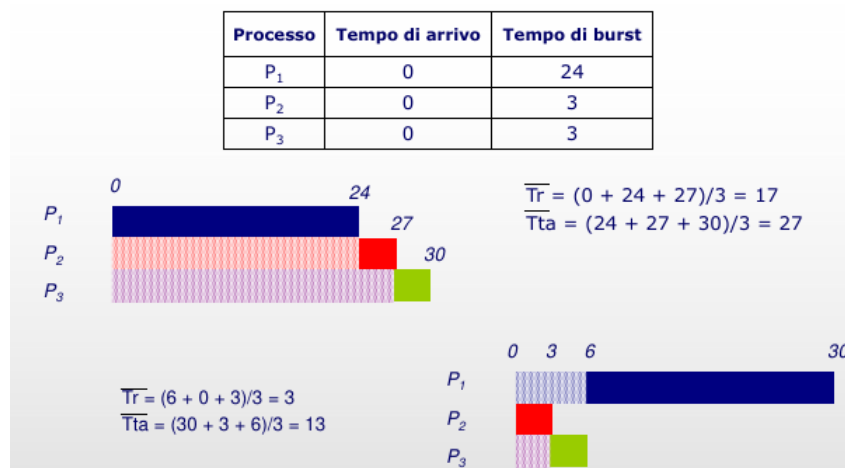


Figure 6.3: Con variazioni del tempo di arrivo

6.5 SJF (shortest job first)

L'approccio migliore e' quello di mettere per primo il processo che ha il burst piu' corto. Nell'immagine essendo sempre in un sistema Batch il processo P_1 deve terminare, dopo decido di dare la CPU a P_3 con l'obiettivo di massimizzare le prestazioni del sistema. Tra P_2 e P_4 dato che hanno lo stesso burst prendo il 2 perche' e' un numero piu' piccolo. Così facendo ottengo:

- Tr (medio): $(0 + 6 + 3 + 7)/4 = 4$
- Tta (medio): $(7 + 10 + 4 + 11)/4 = 8$

Introducendo il concetto della prelazione (non sono piu' obbligato a completare tutto il processo in esecuzione) posso mandare via il processo se nel momento in cui arriva gli darei la CPU, ma e' una cosa che non posso piu' fare in sistemi batch.

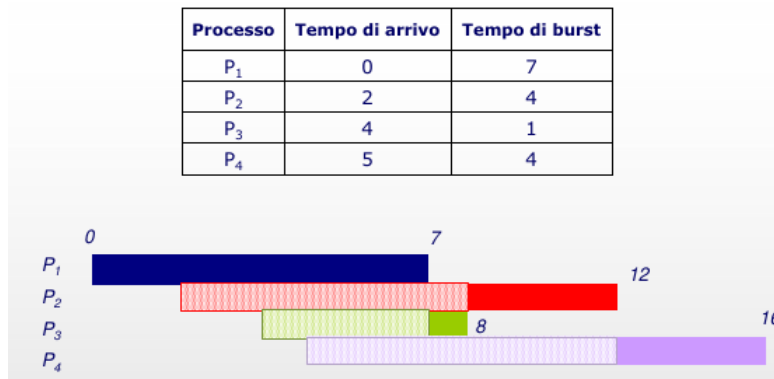


Figure 6.4: SJF

6.5.1 SJF con prelazione (SRTF shortage remaning time first)

Passa davanti il processo con il tempo minore per arrivare infondo.

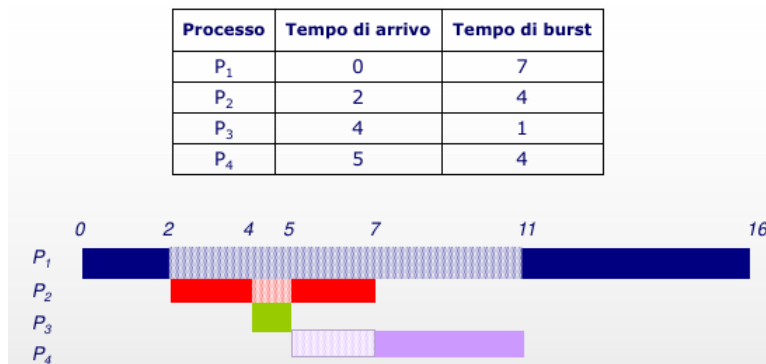


Figure 6.5: SRTF

Inizialmente e' arrivato solo P1 quindi comincio ad eseguirlo, P2 e' arrivato, guardando lo stato dei processi noto che P2 mi chiede 4 unita' di tempo mentre a P1 ne mancano ancora 5, quindi accantonio P1 e comincio P2, stessa cosa con P2, terminato P3 e' arrivato anche P4 ma P2 ha meno unita' di tempo residue quindi ricomincio ad eseguire P2. Utilizzando questo sistema di scheduling ottengo:

- T_r (medio): $(0 + 0 + 0 + 2)/4 = 0.4$
- T_{ta} (medio): $(16 + 5 + 1 + 6)/4 = 7$

E' stato dimostrato che meno di cosi' con questi processi non posso fare. Questo sistema si basa sull'assunto che sappiamo quanto il processo deve stare attivo, ma in un sistema reale non lo sappiamo, quindi l'SJF con prelazione e' un benchmark con cui far scontrare l'algoritmo di scheduling che abbiamo creato. Nel mondo reale non eseguiamo mai due volte gli stessi processi, facciamo una stima della lunghezza del burst di un processo, si basano sul round clock (un approccio in cui implemento il time sharing).

6.6 Round Robin

Interrompo un processo secondo un tempo stabilito, non scelgo i processi, semplicemente vado in ordine.

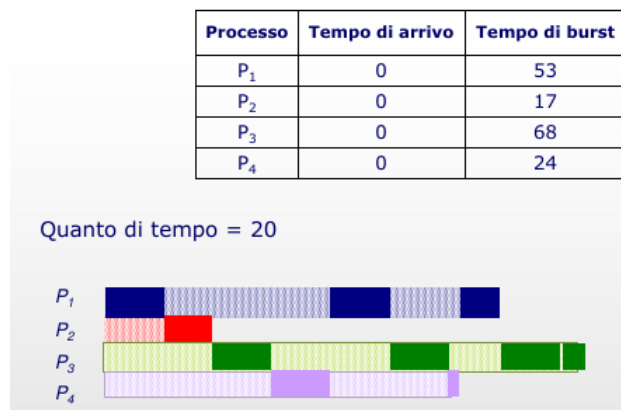


Figure 6.6: Round Robin

Non mi interessa che un processo termini in fretta, ma devo ridurre il tempo di attesa, potrei farlo riducendo il quanto di tempo (guarda il triangolo in basso a destra del grafico, deve ridursi), se lo diminuisco diminuisco anche il livello di utilizzo della CPU perche' continuando a prendere decisione di scheduling lavora piu' per il kernel e meno per i processi.

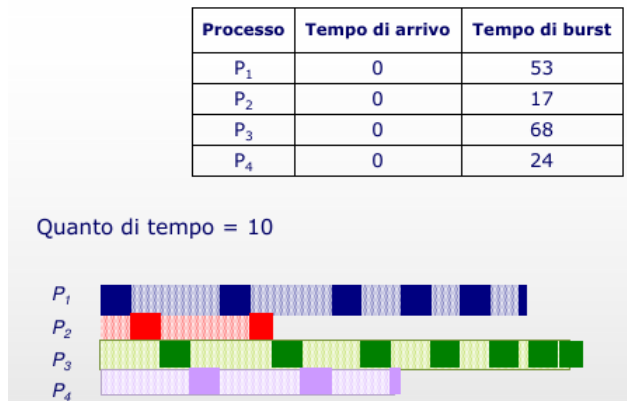


Figure 6.7: Round Robin

6.7 Scheduling con priorit 

Parlando di SJF abbiamo introdotto il concetto che i processi non sono tutti uguali, ci sono processi che potremmo essere interessati a trattare in maniera privilegiata, c'  un concetto di priorit  che   inversamente proporzionale alla dimensione del burst. La priorit  pu  essere statica nel momento in cui la appiccico al processo quando comincia ad esistere e sara' cos  per tutta la loro durata. Pu  anche essere dinamica, pu  avere priorit  diversa in base a come si sta comportando. In UNIX un processo ha priorit  alta quando fa tanto uso IO e bassa quando usa la CPU per tutto il quanto di tempo. Gli os moderni usano uno scheduler a coda multipla, abbiamo una coda per ogni livello di priorit :

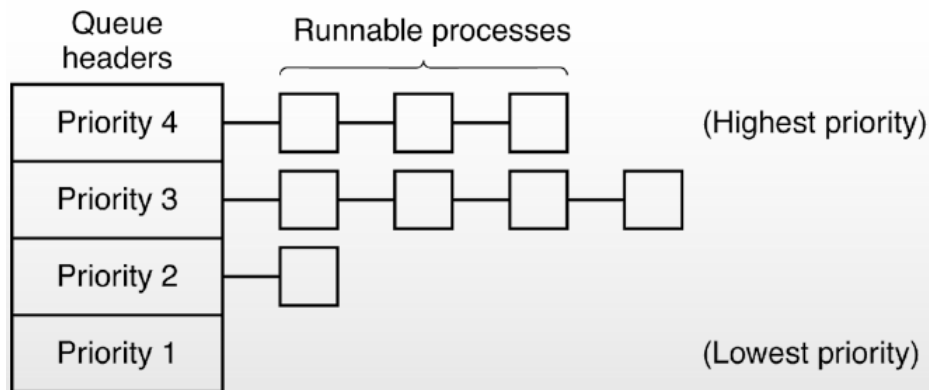


Figure 6.8: Scheduling con priorit 

in questo caso dividiamo in 3/4 categorie. Una volta che   chiaro ad il processo qual'  la sua priorit  questo si mette in coda nella coda giusta per

la sua priorit , io metto in esecuzione sempre e solo il processo primo in coda nella coda a priorit  piu' alta non vuota (4 ha priorit  piu' alta), quando un processo finisce il suo quanto di tempo si rimette in coda nella sua coda e tocca al successivo. I processi nelle code piu' basse vengono comunque fatte perche' nelle priorit  piu' alte ho processi che sono quasi sempre in waiting (corsia di emergenza in autostrada, e' sempre vuota ma quando deve passare un'ambulanza e' giusto che ci vada perche' deve metterci poco tempo) perche' usano solo I/O. Se ho un processo con priorit  statica lo metto in una coda e li' rimane per sempre, se ha priorit  dinamica ogni volta si sposta, per ricalcolare la priorit  del processo non devo per forza eseguirlo, potrei spostarlo in base a quanto tempo e' rimasto in coda (spostamento da code in base alla necessit ). Lo scheduling piu' incasinato e' quello di windows, ha code multiple con 128 livelli di priorit , ogni thread ha associato a se un livello di interattivit  (quando scegliamo che tipo di processo e': real time, alta operativit  etc) ed operativit . Associa ad ogni processo un range ammissibile ed ogni processo si muove nel sistema di scheduling. Abbiamo altre opzioni per sviluppare la priorit , che non si usano nei sistemi operativo moderni:

- **Guaranteed Scheduling:** decide qual'e' il prossimo processo da eseguire tenendo conto che ogni processo deve avere egual quantita' di calcolo, uso le statistiche che raccolgo per ogni singolo processo.
- **Lottery:** vengono creati dei biglietti della lotteria e distribuiti a tutti i processi, il biglietto che estraggo fa vincere il processo e questo usa la CPU per un tempo fisso, una volta che il numero e' estratto e' bruciato, vado avanti finche' non uso tutti i biglietti. E' guaranteed scheduling perche' do a tutti i processi tot tempo di CPU, potrei inquinare con un sistema di priorit , alcuni processi ritenuti piu' importanti hanno piu' biglietti, avere piu' biglietti non mi garantisce di avere piu' probabilit  di arrivare alla CPU, mi garantisce che arrivero' ad avere la CPU piu' volte concretizzandosi in un quantitativo maggiore di risorse e non in un effettivo vantaggio di tempo per arrivare alla risorsa
- **Fair sharing:** puo' capitare che all'interno del sistema operativo ci siano piu' utenti ed ogni utente ha i suoi processi, raggruppa i processi in base agli utenti e da' ad ogni singolo utente una porzione della CPU (se siamo in 10 ogni utente avra' il 10% della CPU)
- **Real Time**

6.8 Thread Scheduling

Lo scheduling process scheduling e' UNIX like, lo scheduling sceglie un processo ed il processo se ha thread si aggiusta da solo per scegliere quali sono da eseguire prima. Al contrario nei sistemi con i thread nel sistema, il sistema stesso sa dell'esistenza dei thread quindi e' lo scheduling che decide quali sono da eseguire prima.

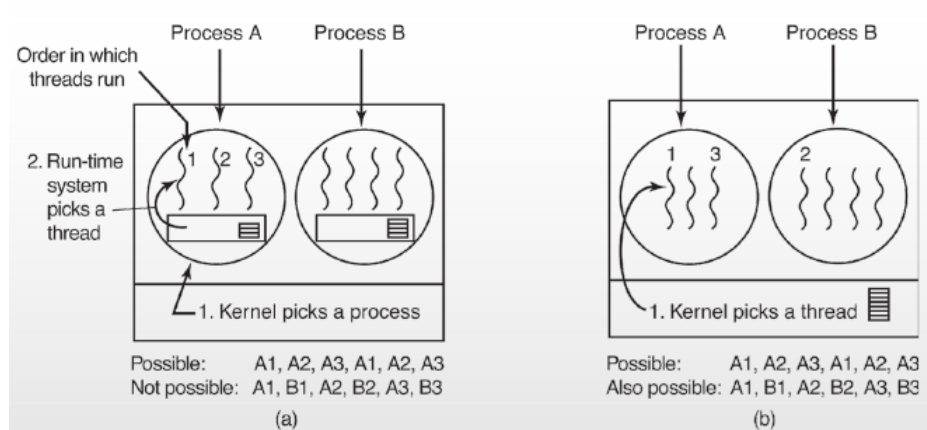


Figure 6.9: Thread Scheduling

6.9 Esercizi di scheduling

E' stata fatta una pagine web apposta per fare esercizi: <https://pong.di.unimi.it/ex/>.
 Il tempo di attesa e' il tempo che passa all'interno del sistema senza usare la CPU e' la parte tratteggiata

Chapter 7

Gestione della memoria

7.1 Nessuna astrazione

La divisione della memoria puo' essere semplice, e nonostante questo ci troviamo davanti a molte opzioni, se un pc e' un po' vecchio potremmo avere un sistema operativo che parte dalla locazione 0 (il kernel sta nella memoria partendo dalla locazione 0) riservo tot spazio al kernel ed il resto ai programmi, se abbiamo una situazione diversa, tipo il telefono o alexa, in questo caso la scelta e' inversa, tanto l'os e' cablato nella rom e nessuno lo puo' toccare, quindi lo piazzo alla fine della memoria tanto so gia' quanto e' la memoria quindi lo caccio in fondo in maniera tale da poter scrivere i programmi utente a partire dalla locazione 0 (semplificando il lavoro del programmatore, scelta strategica). Abbiamo la situazione in cui abbiamo un pezzo dell'os in rom ed un in ram, tendenzialmente il telefono, o una serie di device driver che mi sono dati dal costruttore e non posso modificarli, lui li mette in zone della rom ed io non posso modificarle, tra rom e ram vivono le nostre applicazioni, qui abbiamo un problema, abbiamo in qualche modo delle scelte obbligate, la locazione di memoria 0 e' una sola ed e' cosi' per tutti, se io non posso andare a dare fastidio alla zona del sistema operativo e' perche' l'os stesso che me lo impedisce (con l'aiuto del processore).

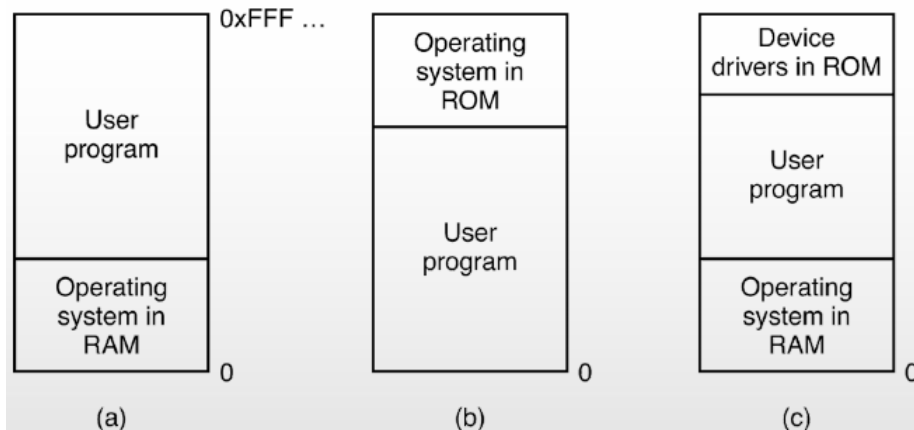


Figure 7.1: Nessuna astrazione

Quando non ho astrazioni ho, ad esempio, un processo A nell'allocazione 0 che seguendo le istruzioni deve fare una jump all'allocazione 24, perche' quando ho compilato il programma del processo A cosi' era il codice. Inoltre ho il processo B che come istruzione fa una jump alla locazione 28. Nel momento in cui ho un sistema multiprogrammato e la memoria e' una sola li metto uno in coda all'altro, quando eseguo il processo B la prima istruzione continuera' ad essere jump alla posizione 28, attenzione che la memoria e' una e la posizione 28 e' quella in basso ed e' in comune ad entrambi i processi, quindi il processo B comincia a saltare nelle posizioni che appartengono al processo A, e questo non va bene. Noi di conseguenza abbiamo come minimo la necessita'

di implementare meccanismi di protezione della memoria. Non e' strettamente necessario, perche' questi meccanismi costano da implementare, perche' il kernel diventa piu' complicato ed io rischio anche di fare errori.

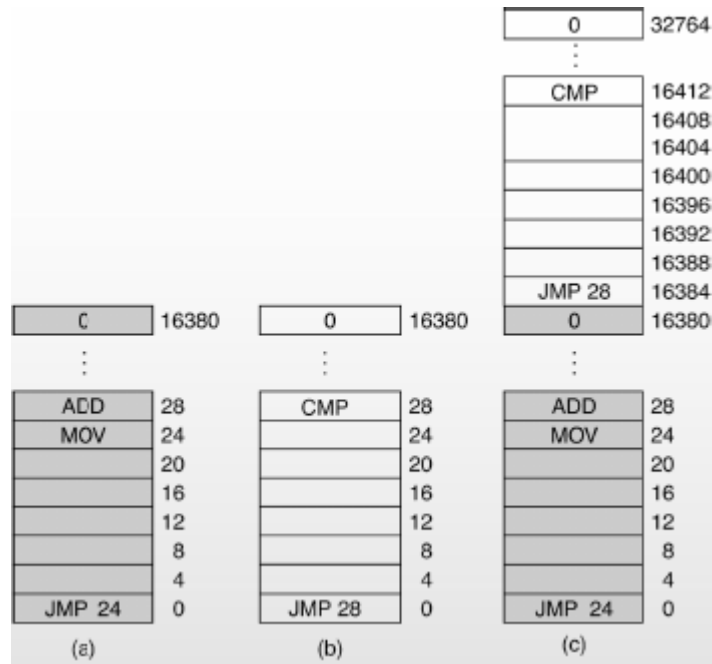


Figure 7.2: Nessuna astrazione

7.2 Spazi di indirizzamento separati

Una possibile soluzione al problema sopracitato e' utilizzare la PIC (position independent code), ovvero posso chiedere a chi compila di usare la PIC (fai una jump di +100 posizioni), ma non posso mai assumere che il sistema funziona perche' gli utenti si comportano bene, quindi non funziona quasi mai questo meccanismo. L'os da quindi uno spazio di indirizzamento separato e dice al processo A: "puoi ragionare come la memoria sia tutta tua, inoltre fai finta che puoi avere 2^{32} posizioni della memoria disponibili, poi come queste posizioni le traduco nella memoria fisica e' un problema mio". Un primo modo di tradurre la memoria fisica e' quello di fornire alla CPU due registri:

- Base: dice alla CPU qual'e' la posizione in memoria fisica nella quale comincia il processo, quindi questo e' l'indirizzo della locazione 0 dello spazio di indirizzamento di un processo
- Limite: indica la dimensione di memoria che e' stata destinata a quel processo.

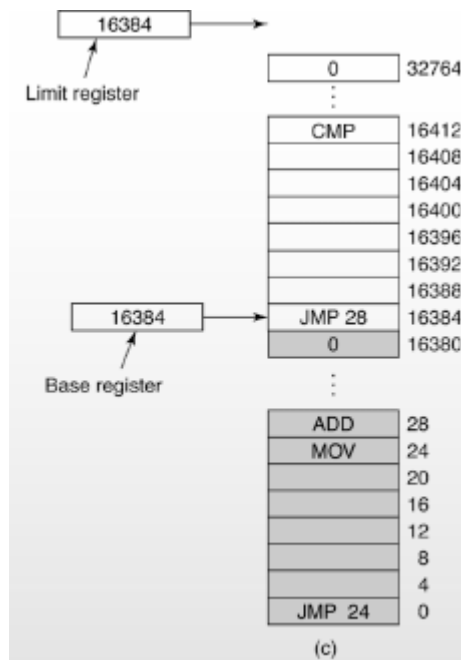


Figure 7.3: Esempio

Nell'esempio comincia a $16\text{kb} + 1$ e lo spazio è di 16 kb , quindi la CPU ogni volta che deve interpretare una locazione di memoria deve:

- Controllare che l'indirizzo non sia superiore all'indirizzo limite e se per caso succede il processo va abbattuto, questo si fa sollevando una trap di segmentation fault.
- Sommare il valore dell'indirizzo di memoria all'indirizzo di base, perché se quello indica la posizione vero faccio uno shift della memoria a quell'indirizzo.

Con questo semplice meccanismo ci siamo assicurati di non far andare oltre la sua zona di competenza un processo. Bisogna ricordarsi che l'indirizzo limite è la dimensione dell'area di memoria allocata al processo, non l'indirizzo in cui finisce il processo. Purtroppo sono contenti di questa soluzione perché se il processo ha bisogno di più memoria potremmo aumentare la dimensione del registro limite, non è semplice farlo per il processo A perché ha il B in coda, il problema arriva dal fatto che l'os è vivo, i processi partono muoiono, chiedono memoria.

7.3 Swapping

Mi trovo con una memoria all'interno della quale ho processi di dimensione variabile che vanno e vengono, e generalmente i processi che abbiamo occupano più

spazio di quello disponibile in RAM, quindi una delle soluzioni piu' semplici e' quella di swapping, che consiste nel mettere sul disco un processo che non stavamo usando da un po' per liberare spazio in RAM. Nell'esempio in figura vediamo che il processo A esce e rientra nella RAM, questo comporta un cambiamento dei registri di quel processo, quindi questi devono essere riallocati. La riallocazione dei registri interni al processo puo' essere fatta tramite software durante lo swap oppure, preferibilmente, dall'hardware durante l'esecuzione del processo.

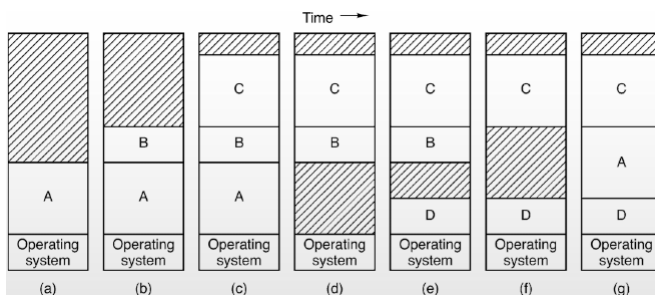


Figure 7.4: Esempio Swapping

Notiamo che quando lo swapping crea tanti "buchi" all'interno della memoria sarebbe possibile crearne uno solo "spingendo" il piu' possibile in basso i processi, questa operazione si chiama memory compaction.

7.3.1 Allocazione dello spazio

Non e' vero il modello per cui un processo e' un unico blocco di memoria, un processo e' costituito da tanti blocchi di memoria che devono contendersi uno spazio. Quando programmiamo dobbiamo tenere conto che esistono tanti tipi di memoria (stack ed heap), lo stack all'interno di un processo cresce dall'alto verso il basso e lo heap, che e' chiamato Data, cresce dal basso verso l'alto, mano a mano che faccio allocazioni di memoria fuori dallo scope questo sale, il problema e' quando si toccano, anche in questo caso sollevo un eccezione. In realta' i processi sono fatti di 3 segmenti:

- Heap
- Stack
- Text segment: (program)

Il problema e' che non tutti i processi hanno la stessa dimensione, poi ad un certo punto devo gestire una situazione con variabili all'interno della memoria. Non alloco mai spazio ad un processo ma ad un sottoinsieme e poi questi sottoinsiemi sono diversi, la gestione della memoria puo' essere fatta in due modi:

- A segmenti: i cui ho locazioni ce si basano su pezzi variabili ma ognuno indipendentemente, quindi spazi di locazione piccoli che devo gestirmi in memoria
- Pagine: posso gestire la memoria utilizzando locazioni a dimensione fissa che pero' non sono indipendenti, un' insieme di pagine collabora a costruire lo spazio di indirizzamento di un processo

La distinzione e' che esistono due casistiche:

- Gestione della memoria con pezzi di dimensione arbitraria
- Gestione della memoria con pezzi di dimensione fissa

poi l'uso di uno o l'altro lo vedremo dopo.

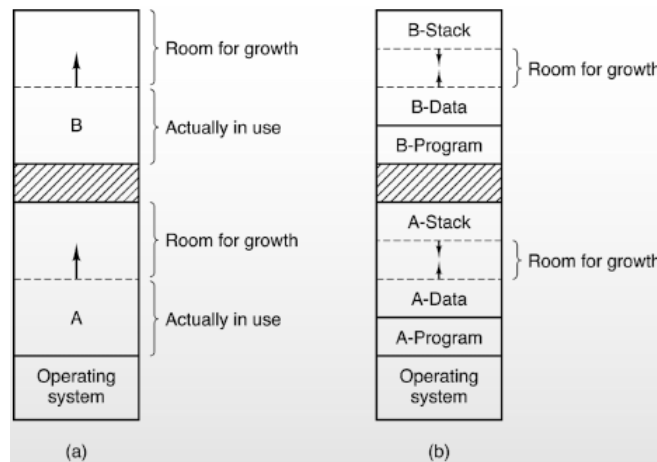


Figure 7.5: Esempio allocazione di spazio

7.3.2 Pezzi di dimensione variabile

Se la memoria e' la parte A del disegno, dentro ho processi che vanno e vengono e mi devo ricordare che cosa e' libero e che cosa e' occupato. Il fatto che in una sequenza di byte ci sia scritto 0 non vuol dire che siano liberi, ho allocato un'array di tante posizioni con tutti 0 quello spazio e' comunque occupato, non posso basarmi sul contenuto. Posso utilizzare un approccio basato su bitmap (la parte B del disegno), individuo un'unita' minima 1 byte o qualcosa di piu' 10,12 byte e metto il bit a 1 se quel pezzettino di memoria piccola e' occupato e 0 se quel pezzettino e' libero, il problema e' quando lo devo utilizzare. Il problema e' quando un processo viene creato, ho bisogno di allocare qualcosa (tipo 15 posizioni disponibili), la struttura quindi e' un disastro, perche' anche se trovassi 15 bit a 0 tutti vicini non significa che siano tutti nello stesso byte,

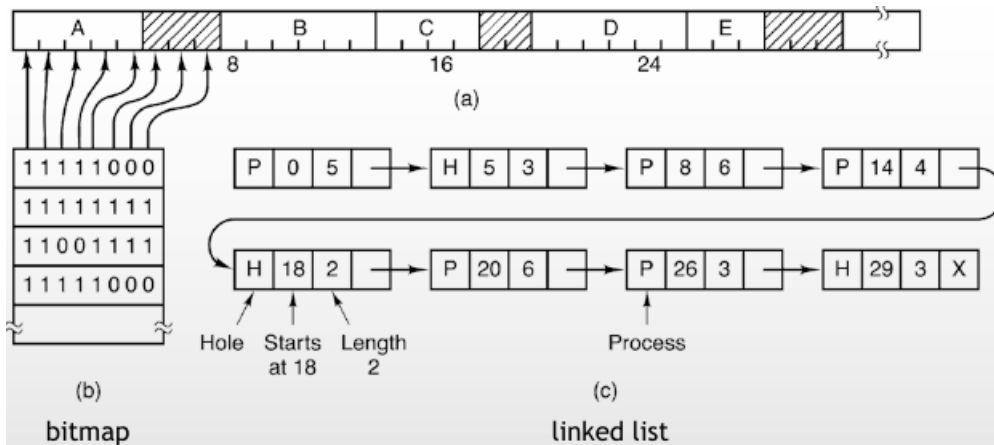


Figure 7.6: Tenere traccia della memoria disponibile

gestire i processi così è un problema. Può andare bene ma è totalmente disastrosa come prestazioni. Preferiamo usare un sistema complesso che è quello indicato con C che è una lista collegata. Generalmente ho una o più zone allocate (P) una in coda all'altra intervallate da una zona libera e la indico con H, nella prima sequenza sto dicendo che presa l'unità di allocazione minima ho un processo che parte da 0 e dura 5 unità di allocazione poi ho un buco che parte da 5 e lungo tre unità di allocazione e così via. Nel momento in cui devo allocare un nuovo processo devo trovare un'elemento H tale per cui la sua dimensione (il secondo numero) sia sufficiente a contenere la richiesta di allocazione che sto facendo. Gli piace fare un paragone automobilistico, è un'operazione dalla quale viene determinate le prestazioni del sistema operativo e possiamo immaginare questa situazione un po' come quando torniamo a casa il sabato sera in macchina (tutte di dimensione variabile) e quella è la via dove abitiamo, molto spesso lo spazio per parcheggiare c'è solo che lo spazio è frammentato in pezzi di 40cm, l'operazione di compattazione è spingere le macchine all'inizio della via per compattarle, ma dobbiamo anche parcheggiare in fretta, quindi posso parcheggiare bene se tutti si comportano bene, ho bisogno di un algoritmo che mi permetta di gestire i parcheggi in modo globale (aiuto tutti quanti).

7.3.3 Strategie di allocazione dello spazio libero

- First FIT: scandisco la lista e il primo buco disponibile è il mio, tengo basso il tempo di allocazione. Se i frammenti sono troppo piccoli si chiama frammentazione esterna. Il First Fit tiene il tempo più basso possibile però crea frammentazione. Non sempre può essere la soluzione migliore.
- Next Fit: parto a rifare il controllo dal punto in cui avevo parcheggiato sabato scorso, mi ricordo ed assumo che se parto a guardare da lì riesco a

trovare posto piu' velocemente, assumo che quando rallento poi ci metterò poco, riesco a spargere le macchine parcheggiate piu' uniformemente nella via, anche il next fit punta a tenere basso il tempo, pero' tenere basso il tempo di allocazione genera continuamente frammentazione

- **Best Fit:** per me avere una memoria in ordine e' prioritario rispetto al tempo che ci metto a parcheggiare. Guardo tutta la via e parcheggio nel posto piu' piccolo che pero' e' in grado di contenere la mia macchina, alla la zona di memoria libera in grado di soddisfare la richiesta con la dimensione minore, in questo modo spreco il minor spazio disponibile tra due allocazioni contigue. Tiene piu' basso possibile la frammentazione ma ci fa perdere un sacco di tempo, dipende (l'os si fa delle statistiche) se mi conviene questo tempo o fare first o next e fare compattazione della memoria
- **Worst Fit:** e' un'alternativa al best fit, semplicemente io parcheggio nel posto piu' grande possibile, uso per l'allocazione il buco piu' grosso che trovo, anche in questo caso faccio due volte il giro della strada, questo approccio rispetto al best fit e' vero che genera piu' frammentazione ma quello che otteniamo e' un'insieme di spazi disponibili che sono mediamente piu' grandi, perche' se prima avevo l'idea di lasciare poco spazio tra una macchina e l'altra, adesso con uno spazio di metri magari qualcuno una smart riesce a parcheggiarla.
- **Quick Fit:** e' un approccio che ha un'altra logica, il os tiene traccia tramite una struttura ad albero di zone di memoria (potenza di 2) che vengono allocate, quando un processo chiede uno spazio il sistema operativo gli chiede quanto serve, ad esempio 15, l'os glie ne da 32, cosi' il processo ha spazio per crescere e l'os si risolve un problema di trovare degli spazi vuoti. Perche' invece che scandire una lista scandisco un'albero. Funziona ma perche' c'e' un piccolo trucco, ne riparlamo quando parleremo di pagine.

7.4 Memoria virtuale

Smettiamo di parlare di allocazione di memoria a segmenti di dimensione arbitraria e cominciamo a parlare di pezzi a dimensione fissa, le pagine. Non ho tutto lo spazio di indirizzamento su un unico segmento, ma lo spazio lo frammento in pagine di memoria, non avendo dimensioni fisse, dentro il package della CPU (quello con i piedini che escono) ci sono vari pezzi, la CPU che fa i calcoli e poi di fianco c'e' un'altro dispositivo memory management unit che si frappone tra la CPU ed il bus indirizzi, tutte le volte che la CPU tenta di mettere un valore sul bus indirizzi la MMU intercetta la richiesta e traduce il numero che la CPU voleva scrivere sul bus indirizzi che e' un indirizzo di memoria nello spazio di indirizzamento del processo e lo traduce nell'equivalente locazione all'interno della memoria fisica, quindi ad esempio alla CPU sembra di utilizzare l'indirizzo 28, la

MMU traduce questo 28 nel numero reale. La MMU basa il suo funzionamento su un sistema di memoria paginato.

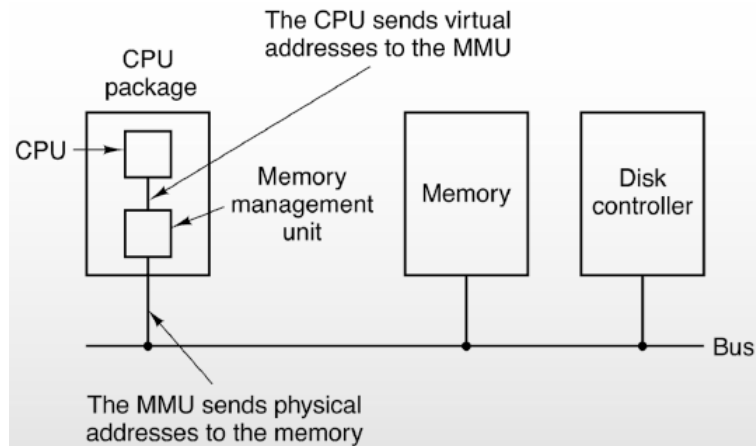


Figure 7.7: Memoria Virtuale

7.5 Memoria Paginata

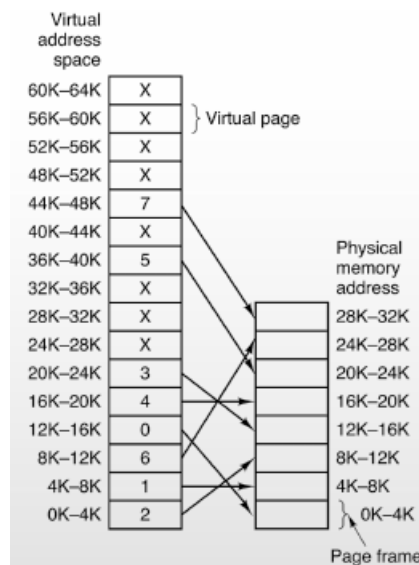


Figure 7.8: Memoria Paginata

Se considero lo spazio di indirizzamento del processo come la colonna di

sinistra lo divido in pagine, una pagina e' di 4k. Abbiamo un sistema di indirizzamento di 16 bit quindi volendo un processo puo' usare 16^{64} kB della memoria, ed ogni processo effettivamente pensa di poterli usare tutti. Pero' in realta' il processo viene fregato, perche' c'e' una page table, la quale mette in corrispondenza una pagina con un equivalente di eguale dimensione nella memoria fisica (chiamato frame), quindi abbiamo una tabella che mi dice che una certa pagina nella suddivisione virtuale del processo equivale ad un determinato frame in memoria fisica. Nell'esempio abbiamo che tra 0 e 4 k la prima pagina equivale al frame n'2. Stiamo fregando il processo perche' dove lui non scrive noi non allochiamo pagine, perche' e' vero che la tabella esiste per tutto lo spazio virtuale, pero' dove ci sono le x significa che il processo non ha usato quell'allocazione, quindi non l'alloca neanche nella memoria fisica, significa che al processo servono meno risorse dello spazio di indirizzamento, al processo sembra di avere tantissimo ma dal punto di vista delle risorse l'os usa molto meno.

7.5.1 Page table

Tramite la page table avviene la mappatura tra page e frame. Nel momento in cui ho un riferimento alla memoria ed il riferimento alla memoria e' la stringa di 0 e 1 in basso, se la guardo come sequenza di bit posso tagliarlo in due, la parte dei bit piu' significativi e' un identificativo della pagina, quelli meno significativi sono un offset, che posizione ha il byte nella pagina. Dato che la Pagina e' il frame se sostituisco il numero di pagina con qualcosa che e' il numero del frame sono a posto. Quindi spezzo in due la pagina, l'offset lo tengo cosi' com'e', dopo di che uso l'identificativo della pagina come indice della page table, nell'esempio i bit piu' significativi sono 110 guardando in posizione 2 c'e' 110, questo ci indica che il frame e' gia' stato allocato, tiro fuori 110 e concateno questo numero con l'offset, trovando cosi' l'indirizzo fisico. Attenzione che non sta scritto da nessuna parte che il nostro sistema di indirizzamento virtuale debba avere la stessa capacita' del sistema di indirizzamento fisico, perche' qui passo da un sistema virtuale di 16 bit ad un sistema fisico di 15 bit, perche' la page table mi permette di fare anche questo, per questo ci sono processi che vanno in emulazione a 32 o 64 bit, perche' la page table trasforma i processi a 32 emulando un processo a 48 bit.

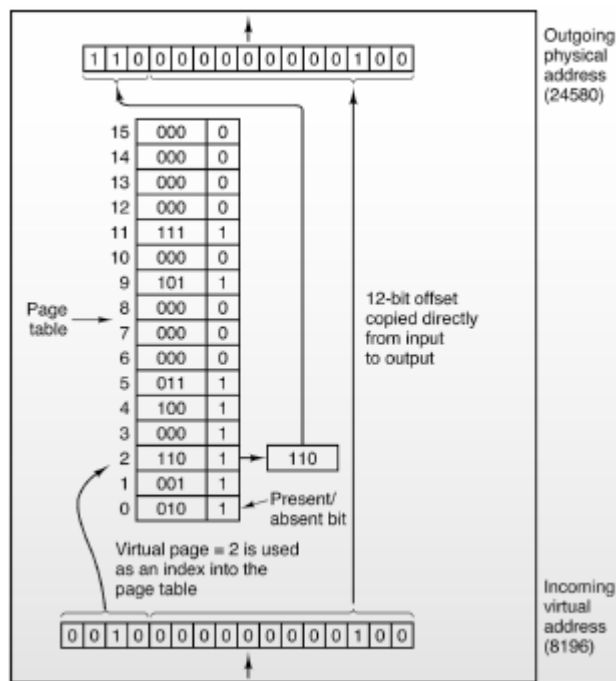


Figure 7.9: Page Table

7.5.2 Elementi page table

Non tutti gli elementi all'interno della page table sono uguali, c'è un entry e poi ci sono una serie di informazioni, ad esempio una pagina non può essere messa in cache. Può essere referenziata e modificata (quando è stata creata e poi messa in swap l'ho messa su disco e poi ricaricata, da quando è in memoria e' cambiato il contenuto? si è modificata?) Ci sono informazioni riguardanti:

- bit di protezione
- quali sono i processi che hanno diritto ad accedere a questa pagina
- a pagina c'è o non c'è, cioè il numero corrisponde veramente ad un frame o non è ancora stata allocata

Il punto difficile è un'altro, la MMU converte un indirizzo logico in un indirizzo fisico NON usa la page table, perché la MMU è un dispositivo fisico cosa ne può sapere delle strutture dati nel kernel, la MMU non sa dell'esistenza della page table ed utilizza una sua struttura dati interna (cache associativa) chiamata TLB, ha al suo interno una serie di informazioni riguardo alle traduzioni pagine-frame più utilizzate. Ho questa tabella di dimensioni limitate (TLB), nel momento in cui la MMU deve tradurre un indirizzo logico in fisico guarda

nella sua TLB, se ce l'ha usa l'informazione che ha nella sua TLB (stiamo accelerando il sistema di calcolo con sistema di caching), se non ce l'ha la MMU si frega di tutto e genera una trap di page fault, ovvero dice al kernel che non e' in grado di fare la traduzione ed ora e' un problema del kernel, il quale intercetta la trap e guardando nella tabella delle pagine infila l'informazione nella TLB e ripassa il controllo alla MMU. Quindi la MMU non usa la page table, ma una cache della stessa. Il meccanismo della page table funziona. Se abbiamo troppi processi solo per gestire la page table di ognuno occupo troppa ram, devo quindi trovare un meccanismo efficiente.

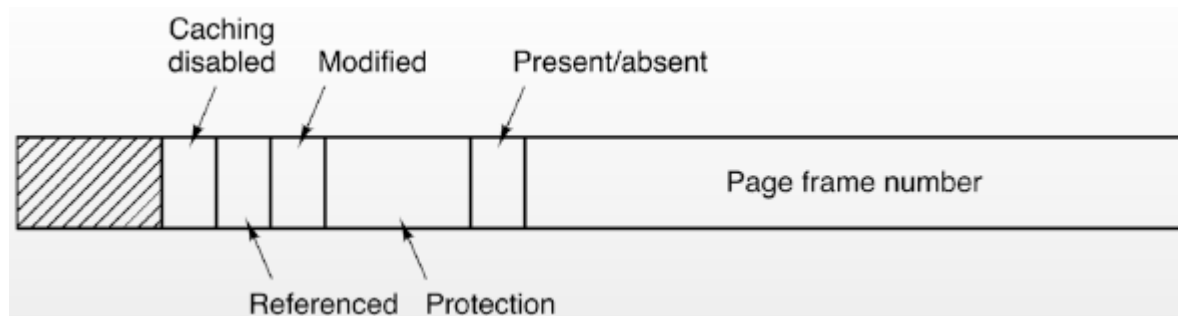


Figure 7.10: Elementi page table

7.5.3 Page table su piu' livelli

Anziche' spezzare l'indirizzo virtuale in due lo spezzo in 3, ottengo cosi':

- identificatore primo livello
- identificatore di secondo livello
- un offset

Ho 32 bit, i primi 10 sono un primo identificatore di pagina, dall'11 al 20 ho un secondo identificatore di pagina, dopo ho 12 bit di offset (con 12 bit una pagina e' grande 4k), quindi ho pagine di 4k organizzate in tabelle di 1024 elementi. Avro' una prima tabella che uso mettendo come indice il primo gruppo di 10 bit, da li' mi salta fuori un'informazione relativa ad un puntatore di un'altra tabella, quella di secondo livello, uso il secondo gruppo di 10 bit come indice della tabella di secondo livello, dopo di che faccio lo stesso gioco con l'offset, lo prendo e lo concateno al numero di frame. Così facendo risparmio spazio per il concetto di localita', quando utilizziamo una zona di memoria tendiamo ad allocare i byte uno di fianco all'altro, succede che di questa page table di primo livello pochi saranno i puntatori validi, per molte pagine non ho neanche un riferimento, non alloco neanche la tabella di secondo livello, quelle che mi serviranno verranno allocate e da li' avro' i riferimenti ai frame. Nel momento in cui ho un'informazione nella TLB ho la traduzione automatica e non faccio accesso

in memoria, quando non ce l'ho uso la page table ed in questo caso faccio almeno due accessi a memoria, ma nel momento in cui i miei livelli diventano molti di più, ad esempio 6 io faccio 6 accessi in memoria (con i processori a 64bit), quindi statisticamente per avere un guadagno devo avere una percentuale di successo sulla TLB mostruosa.

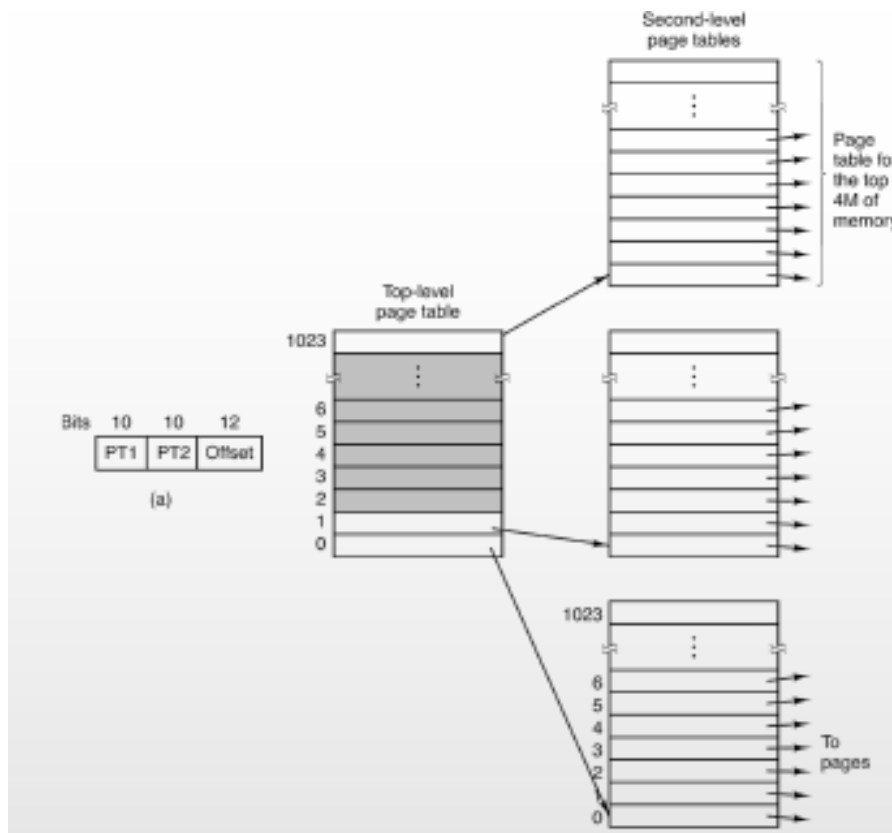


Figure 7.11: Page Table su più livelli

7.5.4 Page table invertita

E' la cosa su cui ci incagliamo. Funziona su un paradigma diverso, continuo ad avere le mie pagine in memoria, per rimapparle su un frame devo utilizzare una funzione di hashing (e' una funzione che dato un input da un dominio di dimensione arbitraria produce in output un valore in un range limitato, nel nostro caso preso in considerazione un numero di una pagina ci restituisce un numero che va da 0 a 255). Se io uso una funzione di hashing ho tante pagine ed un insieme limitato, vuol dire che ci saranno tante pagine che collassano sullo stesso valore di hash, mi serve per dividere le pagine in famiglie. La hash table

e' una tabella che usa come indice un'informazione suddividendo in famiglie in modo da aumentare le prestazioni, applico la funzione di hash al valore e quello che salta fuori lo uso come indice della tabella. Applico la funzione di hash sulla pagina ed il valore risultante lo uso come indice della tabella di hash, se quella pagina ha un frame lo trovo nel sottoinsieme di frame referenziati in quella tabella di hash. Nella tabella trovo informazioni pagina-frame. Se la funzione di hash e' fatta in modo furbo e la tabella di hash ha la giusta dimensione allora posso garantire che le liste che attacco dietro alla tabella di hashing hanno una lunghezza limitata, quindi saranno tutte molto brevi ed alla fine mi conviene andare a scansionare questa lista. La cosa e' un po' piu' complicata, di table invertita non c'e' ne' una per processo, ma ce ne' una per sistema, quindi tutti i processi pescano dalla stessa tabella, quindi non posso avere pagina-frame ma devo avere delle terne pagina-processo-frame, quindi scorro la lista e trovo il frame per quel processo in quella pagina. La seconda cosa e' che non conviene avere una lista, perche' in certe situazioni potrei avere una lista lunga, molto spesso i sistemi operativi si preferisce implementare un'albero binario, la struttura dati a cui punta la hash table non e' una lista linkata ma un albero binario (passo da complessita' di $O(n)$ a $\log O(n)$), un albero abbassa il tempo di accesso

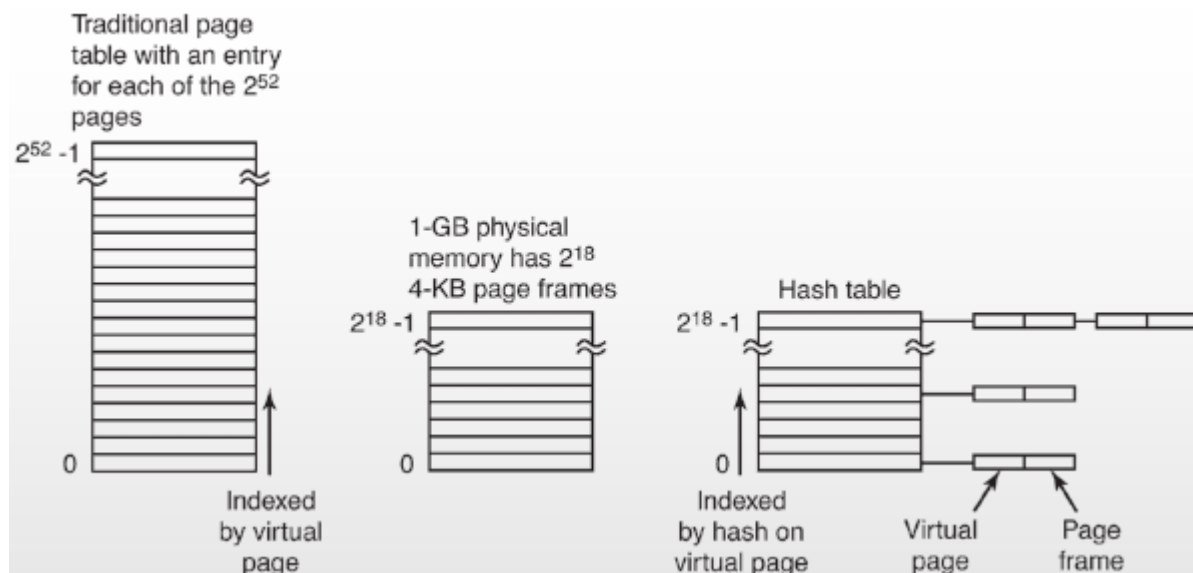


Figure 7.12: Page Table invertita

Chapter 8

Paginazione

Fisicamente non abbiamo 2^{64} byte per ogni processo, quindi ci sarà obbligatoriamente un avvicendamento delle pagine. Ma se ad un certo punto la memoria finisce? se non ho più frame liberi ne devo liberare uno, i frame che devo togliere il kernel li parcheggia sul disco, il problema è la scelta, come faccio a scegliere il frame giusto da togliere dalla memoria per ottimizzare l'utilizzo della memoria virtuale.

8.1 Paginazione su richiesta (demand paging)

- Viene generato un page fault: la mmu si rende conto di non avere il processo paginato
- Il processo viene interrotto: la trap lo fa interrompere
- Il memory manager identifica la pagina mancante e le fa spazio in memoria
- Se lo spazio non è disponibile devo liberare un frame con una politica di rimpiazzamento, mettendo il frame che mi interessa meno nel disco, un po' come quello che succedeva con lo swap ma con una pagina di memoria:
 - Solo se non ce ne sono già liberi
 - Occorre adottare una politica di rimpiazzamento
- La pagina viene caricata nel frame libero
- La page table viene aggiornata + aggiornare la TLB della MMU
- Il processo riprende, quindi la MMU trova l'informazione del processo nella TLB quindi tutto riprende come deve.

Il problema è qual'è l'algoritmo che uso per capire qual'è la pagina vittima, quella che deve uscire dalla memoria, dalla memoria centrale vogliamo mandare via il frame che verrà utilizzato di meno. Su questa fase ci sono da fare degli esercizi, per questi faremo sempre questi procedimenti:

- Dichiariamo una stringa di riferimento: una sequenza di numeri, non c'è un riferimento temporale, sono solo pagine in sequenza dove va il processo
- Sotto la stringa di riferimento riportiamo una tabella con la configurazione della memoria istante persistente
- Contiamo il numero di page fault (pagine rimpiazzate/allocate)

8.2 Algoritmi presenti in letteratura

ottimo, fifo, seconda chance e lru (chiesti all'esame)

- Algoritmo ottimo

- FIFO
- Seconda chance
- Orologio
- Not Recently Used (NRU)
- Last Recently Used (Not Frequently Used)
- Working set
- WSClock

8.2.1 Algoritmo ottimo

Non e' implementabile nella realta'. Quando andremo a fare l'esercizio avremo una stringa di riferimento (R) e poi la configurazione cache da implementare, ci mettiamo delle spunte per dire dove ci sono i page fault, possiamo tenere i numeri nella TLB come ci pare, non ci fanno le pulci su cosa ci scriviamo dentro, ma e' chiaro che se facciamo qualcosa che ci permette di fare poca fatica riusciamo a fare pochi errori. Lui vede una di queste colonne come un tubo, le pagine che entrano nella TLB vengono spinte dal basso e slittano in avanti man mano che le pagine vengono aggiunte, quella in ultima posizione quando viene spinta se ne va. All'inizio avro' una parte di transitorio in cui ho tutti page fault, quando riempio la TLB devo effettivamente applicare l'algoritmo, in questo caso 1 non l'ho in memoria quindi sicuramente c'e' page fault, secondo ottimo devo buttare via pagina che referenziero' piu' in futuro (come mettere da parte l'esame che e' piu' lontano) cosi' dobbiamo gia' sapere quand'e' la prossima volta che la pagina verra' referenziata (non e' replicabile nella realta'). Nel nostro caso ci conviene buttare via la pagina 3, perche' e' quella che verra' referenziata dopo tre istanze. Quando trovo 2 che gia' ce l'ho in memoria la configurazione rimane uguale e vado avanti. Arrivati al 6 dato che 1 3 0 non verranno piu' referenziati non importa piu' quale pagina butto via. Quindi in totale ho 10 Page Fault, si dimostra che non e' possibile essere piu' bravi di cosi', non c'e' un'algoritmo che riesce a fare meno page fault con questa configurazione in memoria.

8.2.2 FIFO (First in First out)

Butto via la pagina che e' all'interno della mia memoria da piu' tempo. Quindi l'abitudine di dire che le pagine piu' nuove entrano dal basso mi permette di fare l'esercizio senza neanche pensare. Con il fifo in questa configurazione ottego 16 page fault. Possiamo migliorare la situazione aumentando la ram, nel nostro caso aggiungo una posizione nella tlb cosi' facendo avro' 8 page fault. Purtroppo la vita non e' sempre cosi' semplice. Soffre dell'anomalia di belady, aumentando le dimensioni della memoria con alcune stringhe peggioro i risultati.

Stack Algorithms

Alcuni algoritmi si comportano meglio di altri. Sappiamo che gli algoritmi che soddisfano la proprietà di inclusione non sono soggetti all'anomalia di Belady. La proprietà di inclusione dice che l'insieme delle pagine presenti in memoria caricate all'istante t con m frame è sempre un sottoinsieme di quelle caricata all'istante t con $m+1$ frame.

0	1	2	3	0	1	4	0	1	2	3	4
0	0	0	1	2	3	0	0	0	1	4	4
	1	1	2	3	0	1	1	1	4	2	2
		2	3	0	1	4	4	4	2	3	3
0	0	0	0	0	0	1	2	3	4	0	1
	1	1	1	1	1	2	3	4	0	1	2
		2	2	2	2	3	4	0	1	2	3
			3	3	3	4	0	1	2	3	4

Figure 8.1: FIFO e anomalia di Belady

8.2.3 Seconda Chance

Parte sempre dalla pagina che è in memoria da più tempo (come se un buttafuori chiedesse se una persona ha già consumato, quindi se entriamo per primi di solito usciamo per primi, un buttafuori in un locale parte dalla persona che è entrata per prima chiedendogli se ha consumato, se lo ha fatto gli prende lo scontrino e passa oltre, se non ha consumato dall'ultima volta che è passato lo butta fuori). Arrivati al 3, partiamo il 2 non ha avuto una 0 canche, no e lo segno con il puntino, scendo fino al 2, arrivato alla fine ricomincio, trovo di nuovo lo zero che però ha la 2 canche, lo butto via page fault ed entra il 3. Occhio che 1 e 2 si portano avanti la 2 chance. Quando incontro lo zero ad un certo punto avendolo già in memoria equivale a consumare di nuovo, quindi la configurazione rimane la stessa ma vado avanti togliendo la seconda chance. In pratica quando ho una configurazione senza seconde chance la metto a tutti e tolgo la pagina in memoria da più tempo. Rimane brutto perché mi intestardisco a mettere le pagine in ordine da più vecchia a più recente. La cosa migliore che posso fare è usare una struttura dati circolare. (non soffre dell'anomalia di bellady)

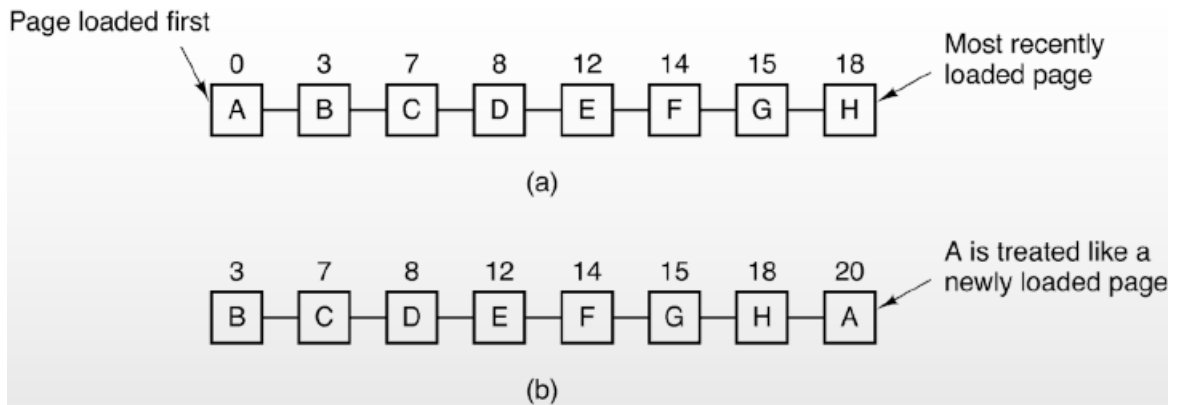


Figure 8.2: Seconda Chance

8.2.4 Algoritmo dell'orologio

Non e' richiesto negli esercizi. Metto la lista in tondo come un quadrante di un orologio con una lancetta che gira, riprendo a controllare la seconda chance partendo dall'ultima pagina che ho selezionato come vittima + 1. In questo modo riusciamo a migliorare in modo efficace le prestazioni di un algoritmo.

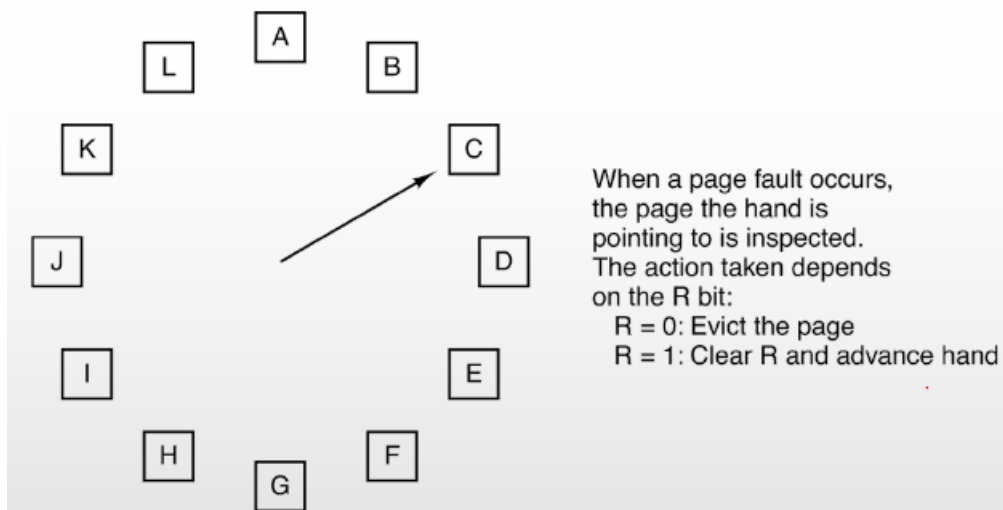


Figure 8.3: Orologio

8.2.5 LRU

Funziona bene, implementabile e facile da fare. Last Recently Used, butto via la pagina che ho usato recentemente. Arrivati all'1 guardo quand'e' l'ultima volta che ho referenziato le cose in memoria, quello che ho referenziato piu' in passato e' il 2, quindi con questo algoritmo e' giusto che venga buttato fuori. Facendo tutta questa manfrina nel primo esempio e' che mi ritrovo a fare 16 page fault. Fortunatamente LRU e' stack, quindi siamo sicuri che aumentando la memoria faremo meno page fault, pero' se ho una stringa di riferimento particolarmente rognosa va a degenerare sul FIFO (la turnazione e' dello stesso ordine di grandezza della memoria). Aumentando di una posizione la TLB dimezzo le PF.

2	0	3	1	2	0	3	1	2	0	3	1	6	4	5	7
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2	2	2	0	3	1	2	0	3	1	2	0	3	1	6	4
	0	0	3	1	2	0	3	1	2	0	3	1	6	4	5
		3	1	2	0	3	1	2	0	3	1	6	4	5	7

2	2	2	2	0	3	1	2	0	3	1	2	0	3	1	6
	0	0	0	3	1	2	0	3	1	2	0	3	1	6	4
		3	3	1	2	0	3	1	2	0	3	1	6	4	5
			1	2	0	3	1	2	0	3	1	6	4	5	7

Figure 8.4: LRU e anomalia di Belady

Posso quando reincontro le page metto quella che ho chiamato in fondo e tengo in testa quella che ho chiamato piu' lontano cosi' sono piu' comodo. Il problema e' che ogni volta che incontro una pagina che gia' ho in TLB devo riordinare le pagine (quindi e' lunghissimo con milioni di pagine). I sistemi reali approssimano il comportamento di LRU:

- NRU: ogni pagina ha due bit uno ci dice se la pagina e' stata referenziata ed uno che dice se e' stata modificata, in questo modo ho 4 combinazioni:
 - Classe 0: non referenziate e non modificate
 - Classe 1: non referenziate e modificate
 - Classe 2: referenziate e non modificate
 - Classe 3: referenziate e modificate

Viene buttata via una pagina pescata a caso nella classe non vuota di valore piu' basso. Com'e' possibile avere in memoria una pagina che e' stata modifi-

cata ma mai referenziata? le pagine mano a mano che il sistema porta avanti l'evoluzione del processo entrano e sono tutte in classe 2 o 3, classe 0 e 1 sono vuote inizialmente, il sistema di sua iniziativa con delle costanti di tempo (2 sec.) prende e mette tutti i bit di referenziazione a 0, quindi tutte la pagine da classe 2 vanno in classe 0 e tutte la pagine da pagine 3 in classe 1. Poi ci saranno delle pagine che verranno ancora referenziate e modificate, passando di nuovo in classe 2 e 3, ci saranno poi delle pagine che non mi serviranno piu' rimanendo in classi 0 e 1, quelle pagine rimaste indietro sono quelle che il mio algoritmo puo' buttare via, perche' tanto hanno un'utilita' molto bassa, quindi posso anche buttarne via una a caso. Preferisco anche buttare via da classe 0 perche' se eliminassi da classe 1 dovrei oltre a buttarla via aggiornarla sul disco.

Not Frequently Used (NFU)

E' un approssimazione di LRU che sfrutta un altro tipi di approccio, l'invecchiamento. NRU divideva in 4 categoria, qualcuno ha pensato che non sono abbastanza, per fare un lavoro raffinato ne vorremmo di piu'. Quindi attacco ad ogni pagina un'etichetta di tipo numerico, con quest'etichetta numerica faccio una suddivisione in tante famiglie, le pagine stano nel sistema (A) e raccolgo delle statistiche nell'informazione che ho in alto, ed ad un certo punto fermo tutto e guardo all'interno del periodo quali sono le pagine che sono state referenziate, metto tutto a 0, prendo tutte le etichette e faccio a uno shift a sinistra di un bit, poi di tutte le pagine referenziate metto un uno al bit piu' significativo (faccio lo shift a destra (divisione per 2)), in pratica divido per 2 ed aggiungo 128. Al prossimo periodo una pagina referenziata di nuovo avra' $64 + 128$ come etichetta, una che non e' stata referenziata passera' da 128 a 64 e cosi' via. Quando devo selezionare una vittima guado in queste famiglie divise per etichette (256 famiglie) e butto via una pagina a caso tra quello con etichetta di valore piu' basso, se un etichetta mi va a 0 vuol dire che quella pagina e' stata referenziata piu' di 8 cicli fa. Questo e' un po' piu' laborioso perche' devo tenere le pagine in 256 famiglie ma da ottimi risultati.

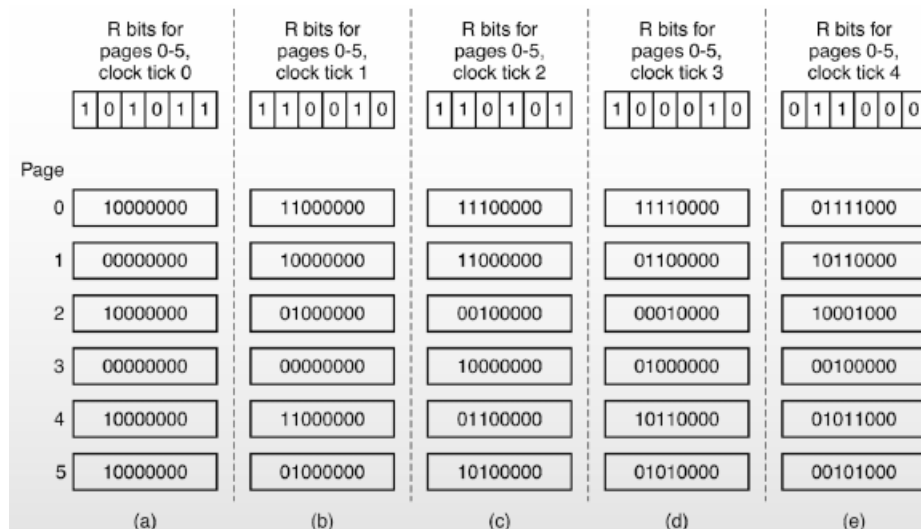


Figure 8.5: NFU

8.2.6 Algoritmi basati su working set

Il tanenbaum dice che esistono delle classi di algoritmi che cercano di discriminare pagina che sta nel working set di un processo e pagina che non ci sta. Working set sono l'insieme delle pagine che il sistema ha bisogno che rimangano in memoria e pagine no. Quando il processo va in fase di running si aspetta che il working set sia pronto, secondo lui tutti gli algoritmi sono di working set, la discriminante è come capiamo se una pagina è o no parte del working set. Il carico di lavoro dato ad un sistema non è linearmente dipendente dal working set, più aumento questo nel tempo poi le prestazioni degradano.

Algoritmo working set

Hanno fatto un mischione di tutti gli algoritmi visti, mi tengo una lista delle mie pagine, per ogni pagina mi segno all'interno della PAGE table se è stata referenziata o no dall'ultima volta che sono passato ad esaminarla (seconda chance) definisco in più un tempo virtuale tau che il sistema si calcola in base a delle sue statistiche per cui se una pagina è in memoria per più tempo che non tau cicli (le unità di misura sono i cicli della cpu) allora è molto probabile che la pagina non mi serve più, però la scansione non la faccio partendo dalla pagina che è stata referenziata più tempo addietro, la faccio dalla pagina che è entrata per prima. Ho mischiato la seconda chance, LRU (perché mi segno l'ultima volta che ho usato la pagina) in più faccio una scansione lineare che parte dalla base quindi è una specie di fifo. L'idea è partire dalla pagina che entrata per prima all'interno del sistema gli do in caos una seconda chance ma se dall'ultima volta che l'ho usata è passato più di tau tempo allora la butto via.

In pratica e' un riassunto delle cose che abbiamo visto prima. Quell'algoritmo scritto a destra e' quello che usano i sistemi operativi oggi, e' una media di tutte le tecniche che abbiamo visto prima, ovviamente qui continuiamo ad avere il problema di aver ed il buttafuori che si intestardisce a partire dalla prima persona che e' entrata nel locale.

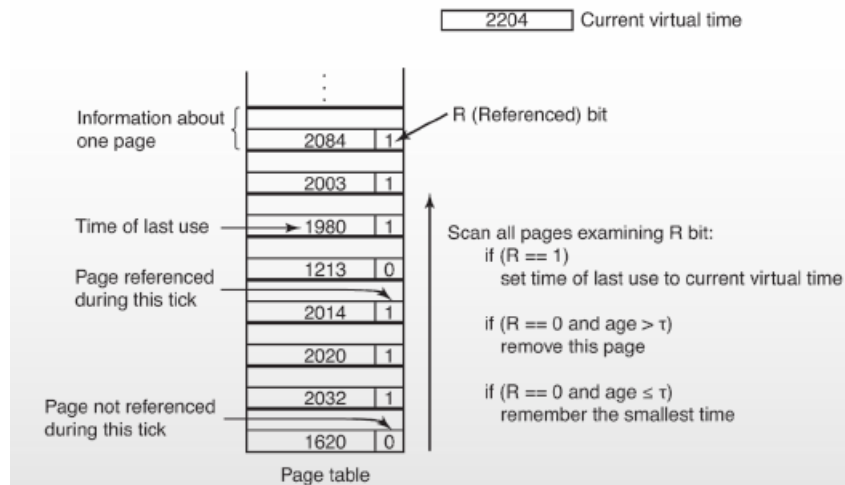


Figure 8.6: Working Set

WSClock

Esiste un algoritmo per cui invece che partire dalla prima pagina entrata ho una struttura ad orologio come visto prima.

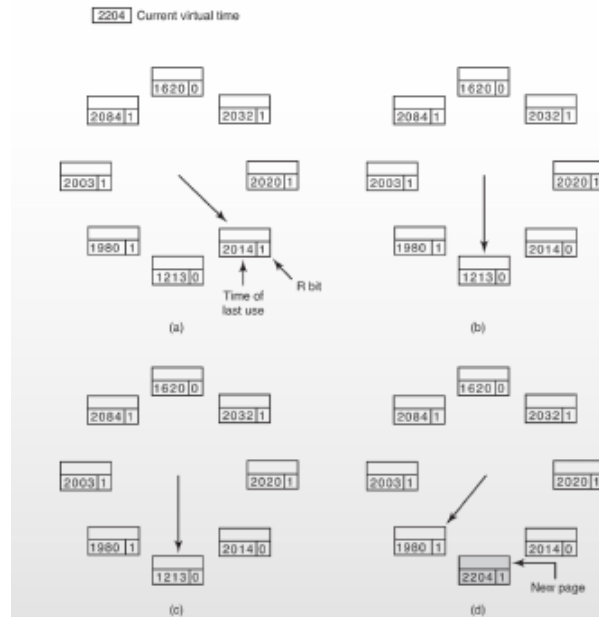


Figure 8.7: WSClock

Chapter 9

Implementazione della memoria virtuale

9.1 Allocazione locale e globale

Non abbiamo mai parlato di dove vado a prendere una pagina vittima quando devo rimpiazzarla nella page table, infatti quando ho a che fare con un sistema vero devo capire se la vittima la scelgo tra le altre pagine del processo che ha bisogno di allocare un'altra pagina, questo vuol dire che un processo con poche pagine fa tanti page fault, oppure prendo la pagina vittima tra tutte la pagine che ho nel sistema anche tra quelle degli altri processi.

	Age		
A0	10	A0	A0
A1	7	A1	A1
A2	5	A2	A2
A3	4	A3	A3
A4	6	A4	A4
A5	3	A6	A5
B0	9	B0	B0
B1	4	B1	B1
B2	6	B2	B2
B3	2	B3	A6
B4	5	B4	B4
B5	6	B5	B5
B6	12	B6	B6
C1	3	C1	C1
C2	5	C2	C2
C3	6	C3	C3
(a)		(b)	(c)

Figure 9.1: Esempio allocazione locale o globale

Nell'esempio abbiamo una politica basata su LRU (colonna A), ogni pagina ha un tag temporale cioe' quand'e' l'ultima volta che ho referenziato quella pagina, nel momento in cui ho una locazione locale le pagine con cui faccio la scelta sono solo quelle di A, al contrario con la politica globale le guardo tutte (nell'esempio la B3 e' quella referenziata piu' tempo fa). Come faccio la scelta non e' indifferente, in un caso garantisco che un processo ha la stessa dimensione del working set (insieme di pagine che un processo ha bisogno per essere eseguito senza page fault), do una dimensione di working set al processo e lui nasce con quel numero di pagine, dopo di che un processo evolve, potrebbe averne bisogno di piu' o meno, se la posizione del working set la creo giusta mi sta bene usare politica locale, se la dimensione del working set e' troppo piccola rispetto a quello che il processo ha bisogno di piu' spazio ruba il working set agli altri nel sistema. Va bene ma non puo' succedere sempre, perche' se il processo ruba spazio ad un processo che a sua volta ha bisogno di molte pagine fara' lui le page fault. In realta' uso un po' di allocazione locale e un po' globale, non per tutto il sistema ma per ogni singolo processo, per ogni processo associo nel PCB un flag che mi dice se sono allocato localmente o globalmente perche' uso un indice prestazionale.

9.2 PFF (Page Fault Frequency)

E' la frequenza con cui un processo genera page fault, se un processo me ne fa tanti vuol dire che il suo working set e' piccolo, in questo caso lo passo a fare allocazioni globale, se i page fault sono pochi mi trovo in una situazione in cui il processo ha un sacco di pagine ma ne sta usando poche, quindi lo riporto in allocazione locale, in modo che non solo non mi ruba pagine da altri processi ma e' in condizione di farsele rubare.

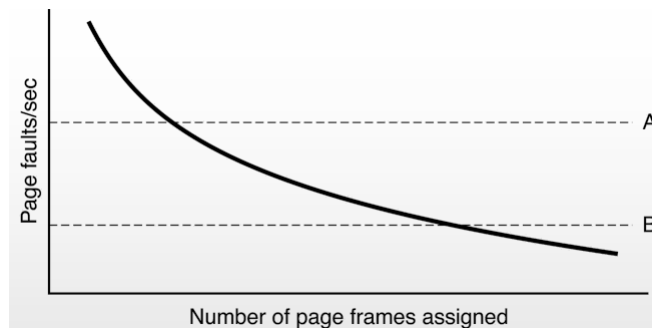


Figure 9.2: PFF

A grandi linee se ho poche pagine faccio tanti page fault (Parte sx del grafico), al contrario se ho tante pagine avro' pochi page fault (parte dx del grafico). Pero' non stabilisco una soglia e dico che sopra vado in globale e sotto in locale (il contrario), il sistema lavora con due soglie:

- Alta: A
- Bassa: B

Il mio processo parte e comincia a fare page fault, nel tempo salgono perche' questo all'inizio alloca un sacco di spazio, aumentando i page fault ad un certo punto viene sforata la soglia A e' il segnale che l'allocazione locale non funziona piu' quindi si passa a quella globale, il PFF inizialmente si assesta e poi comincia a diminuire perche' il working set diventa abbastanza grande, continuando a scendere ad un certo punto mi trovo a tornare sotto la soglia alta ma non posso cambiare la politica subito, continuo a diminuire fintanto che il processo passa sotto la soglia inferiore, quindi quando scendo sotto B reimposto il processo con la politica locale (niente piu' pagine in piu') quindi questo si riassistera' e ricomincerà a crescere. Quindi in generale quando il PFF e' sopra A uso l'allocazione globale, mentre quando e' sotto B uso la locale, in mezzo tra A e B continuo a fare quello che sto facendo, perche' se avessimo una sola soglia di riferimento anziche' avere un respiro ampio del grafico il processo oscillerebbe sulla soglia, questa cosa lo farebbero tutti i processi, quindi tutti i processi continuerebbero ad oscillare sulla soglia cercando di allocarsi il massimo numero di pagine ed il sistema si incarta ed entrerebbe in trashing. Il fenomeno di

trashing si ha quando la dimensione totale dei working set necessari a tutti i processi supera la capacita' fisica del sistema, quindi tutti hanno un working set piccolo quindi qualunque in esecuzione genera una raffica di page fault, il tempo che impiego per gestire il page fault viene contabilizzato nel quanto di tempo disponibile per il processo, quindi non e' da escludere che finisca il quanto di tempo facendo solo IO sul disco (gestendo i page fault), anche gli altri processi usera' tutto il quanto di tempo per fare page fault e poi quanto tocchera' di nuovo a me il processo si ritrova che tutte le pagine sono fuori dalla memoria, quindi abbiamo una macchina che lavoro all'impazzata il sistema va al 100 % pero' di fatto e' tutta paginazione e per gli utenti non stanno facendo nulla.

Questo meccanismo A e B permette di evitare il trashing.

9.3 Dimensione ottimale pagine

Qual'e' la dimensione ottimale che posso dare ad una pagina in memoria? C'e' scritto da qualche parte che 4k come abbiamo visto finora e' la dimensione ottimale? Ogni sistema ha la sua e ci sono dei metodi per stimarla. Cerchiamo di capire quant'e' la dimensione totale della memoria che uso di servizio (butto via, il kernel impegna) per implementare la memoria virtuale, dopo di che scrivo questa stima sulla base della dimensione della pagina e trovo la dimensione della pagine che minimizza il mio overhead (lo spazio sprecato). I dati sono:

- p: dimensione della pagina
- e: dimensione in byte di un elemento della page table. Il motivo per cui le tabelle sono multilivello e' che diventano troppo grandi.
- s: dimensione media in byte di un processo
- n: il numero di processo

Avendo questi dati comincio a costruirmi la stima, uno stimatore e' un'ipotesi su come si comporta una variabile aleatoria (e' una variabile che cambia valore secondo probabilita' ad esempio il lancio di un dado). Ipotizziamo quindi che ci sia una prima parte di memoria sprecata dovuta a frammentazione interna, se ogni processo (n processi) utilizza un certo numero di pagina allora l'ultima pagina probabilmente sara' usata a meta' :

- Frammentazione interna:
- Frammentazione esterna

Poi c'e' una questione di overhead per processo, quanto mi occupa un processo inevitabilmente all'interno del sistema, per ogni processo butto via la mezza pagina che occupa a meta' + il numero di pagine che usa per la dimensione nella tabella delle pagine, questo e' uno spazio che il sistema operativo deve comunque allocare ma va a finire nella tabella delle pagine. Quindi ogni processo spreca questo spazio, dopo di che se all'overhead e' lo spazio sprecato da un processo

posso esprimerlo in funzione di p e poi scrivo la derivata dell'overhead, devo trovare il valore minimo dell'overhead in funzione di p , lo esprimo in p e poi faccio la derivata prima rispetto a p , devo trovare il valore di p per cui la derivata prima dell'overhead vale 0 e questo sarà il punto dove il valore di p mi dà le prestazioni migliori dal punto di vista dell'occupazione della memoria, salta fuori che $p = \sqrt{2se}$. Salta fuori p dando dei valori arbitrari ad s ed e . Cosa vuole sapere quando ci fa l'esame, interessa capire che calcolo l'overhead e lo minimizzo con il valore di p attraverso la derivata prima della formula che salta fuori, se non ricordiamo la formula non gliene frega niente.

9.4 Separazione di spazio dati e testo

Il concetto di avere una memoria sola in cui butto dentro sia dati e codice è un concetto saltato fuori dopo una lunga diatriba, c'era qualcuno che all'epoca vedeva più funzionale due memorie disgiunte, una per i dati ed una per il codice da eseguire.

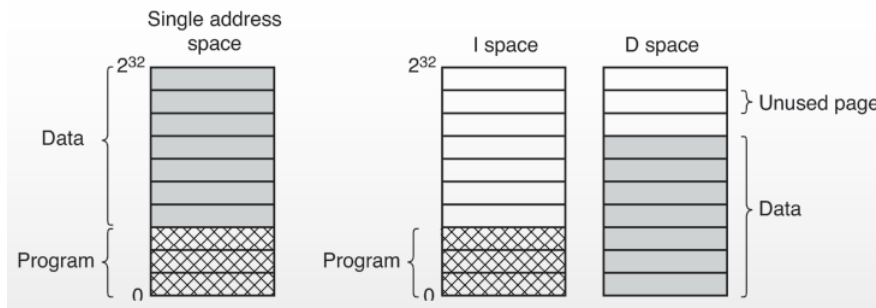


Figure 9.3: separazione

All'interno dello spazio di indirizzamento in cui ho un solo numero per indicare un indirizzo mi ritrovo con un pezzo occupato dal programma ed uno occupato dai dati, potremmo costruirci due spazi di indirizzamento per mettere in uno i programmi ed in un altro i dati (la colonna centrale), l'idea è che questo instruction space può essere condiviso da tanti processi perché è abbastanza comune che più processi eseguono lo stesso programma. Poi ho il data space che è uno spazio di indirizzamento che delego ai dati specifici di un singolo processo. Abbiamo detto che è possibile gestire la memoria in segmenti ed ogni segmento ha il suo indirizzo 0 e questo è il motivo per cui parlando di processo control block abbiamo detto che la parte di gestione della memoria definisce almeno 3 aree, un segmento per i dati, uno per lo stack e uno chiamato text segment che è l' instruction space quindi questo io già lo faccio. Mi conviene perché voglio ottimizzare il sistema facendo condividere delle zone di memoria a quanti più processi possibile. Quindi sarebbe bello se possiamo condividere dei segmenti andare ad una granularità più fine.

9.4.1 Pagine condivise

Potrei decidere che dei processi condividono delle pagine (delle singole pagine) e magari il sistema operativo sa quali sono le pagine riservate che vengono condivise tra alcuni processi, ad esempio i primi 4k di memoria vengono contraddistinti come red page, e' una pagina condivisa tra tutti i processi dove l'os piazza dei dati che tutti devono leggere ma nessuno puo' scriverci dentro, ad esempio mette dentro i dati relativi ai semafori. Per cui posso avere dei dati condivisi semplicemente delegandoli ad una pagina e facendola condividere, quindi ho anche questa complicazione. Attenzione che quando devo scegliere una pagina vittima in questo caso dev'essere buona per tutti i processi che la gestiscono.

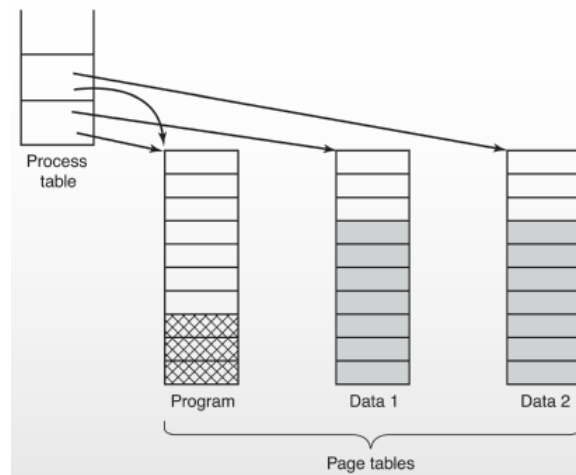


Figure 9.4: Pagine condivise

9.4.2 Shared library

In Windows ci sono librerie con pezzi di codice che l'os mette a disposizione, cose che servono a tutti i processi, ad esempio stampare una stringa o stampare una finestra sul monitor, si implementa una volta e la usano tutti i processi. Quindi crea una dll (dynamic link library) e quando serve mette quel pezzo di codice nello spazio di indirizzamento del processo. Quando serve tutti quanti possono andare a pescare quel pezzo di codice, UNIX e' SO (shered object). L'idea e' che uso il meccanismo di pagine condivise tra processi, carico la dll in memoria la quale avra' un set di pagine dedicato ed anziche' duplicarla per ogni processo il processo prende le pagine della page table e faccio puntare i frame dove la dll e' stata gia' caricata.

Ad esempio abbiamo i due processi, i quali fanno uso della stessa libreria condivisa, la quale occupa 5 pagine, fisicamente l'os la prima volta che e' stata usata se l'e' caricata in RAM, quindi i processo 1 fa puntare la pagine a quel frame che gia' ci sono, la stessa cosa fara' il processo 2.

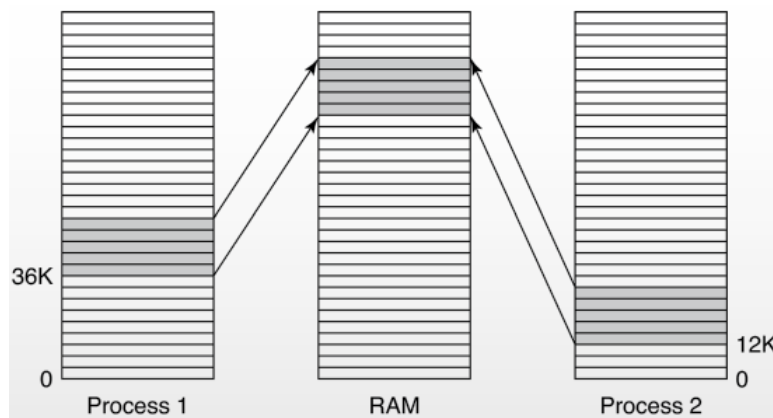


Figure 9.5: Shared library

Esempio su quanto e' utile avere librerie condivise

Si crea un programma stupidino al volo, il programma compilato e' di 98 byte, per trasformare il sorgente in un eseguibile ho bisogno di un compilatore, compilato ho un eseguibile nella stessa directory il quale ha dimensione 50k byte, il punto e' che quell'eseguibile da 50K e' il risultato che il compilatore usa le librerie dinamiche di default, ogni volta che c'e' un riferimento ad una libreria esterna tira dentro una libreria dinamica. Proviamo a fare adesso la compilazione statica, l'eseguibile compilato in maniera statica ha sicuramente dimensione maggiore, e' grosso quasi 1 Mb. Quindi se compilo in maniera dinamica per tutti gli eseguibili ho una forte ottimizzazione, pero' se spostiamo questo file dinamico in un altro sistema puo' essere che non funzioni piu' perche' nell'altro mancano le librerie, al contrario quello statico posso spostarlo (intel con intel) avendo tutto dentro posso eseguirlo.

9.4.3 Memory-mapped file

Uso sempre un trucco legato alla page table, se io potessi dire che una sequenza di indirizzi (colonna di sx) all'interno dello spazio di indirizzamento corrisponde ad un pezzo di disco, quindi quando vado a scrivere e leggere in quella zona di memoria i byte vengono scritti e letti direttamente in un file, e' una facility che UNIX ci da con system call "memory mapped". Ci risolve un sacco di problemi perche' uso direttamente la memoria e quando il programma termina i dati sono gia' sul disco, alcuni software (cifatura avanzata, o quelli che hanno sistemi avanzati di anti tempering) aprono il loro eseguibile in questa modalita' e poi fanno dei controlli se per caso sono stati modificati, senza doversi autocaricare da disco, semplicemente guardando in memoria.

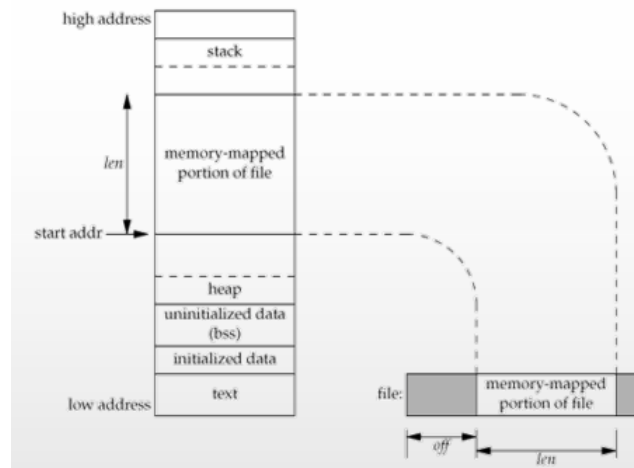


Figure 9.6: Memory mapped file

9.5 Gestione dei page fault

Ricordiamo che il messaggio che la MMU manda al kernel (trap) nel momento in cui non trova qualcosa all'interno della TLB perché la MMU non è in grado di mettere mani nella page table. Questa è la procedura che assomiglia molto alla trap della system call, perché passa dallo stesso meccanismo, l'unica cosa che deve fare l'os è distinguere la trap di page fault dalle altre trap :

- Hardware (MMU) manda una trap al kernel
- Il kernel ferma tutto, non è una chiamata di system call, quindi devo fare delle operazioni senza che il processo che ha chiamato se ne renda conto, quindi metto in sicurezza lo stato della CPU. Siccome il processo non verrà tolto dallo stato di running lo metto da parte. Di solito per rendere l'operazione prestante non si va neanche ad interessare il process control block
- L'os capisce qual'è la pagina virtuale che ha generato il page fault e capisce che dev'essere presente,
- ma la pagina ci viene data se quella è una pagina dove possiamo accedere, ci sono zone di memoria dove un processo non può mettere le mani, quindi l'os controlla se può accedere
- Decido che c'è un frame da liberare, se per caso il frame è occupato da una pagina che è stata modificata allora quella pagina modificata che ha corrispettivo sul disco in swap deve avere una copia nuova sul disco, se non è modificata la copia sul disco è già buona

- L'os fa apparire un frame la pagina richiesta, se la pagina non e' mai stata utilizzata invece che caricarla dal disco la crea ex novo,
- Aggiornata la tabella della pagine, Il frame viene contrassegnato come nuovo, referenziato ma non modificato, aggiorno la TLB
- Il processo che ha generato il page fault Ricominciamo dall'inizio l'esecuzione
- Il processo che ha generato il page fault viene rimesso in esecuzione
- Ricarico i registri di stato sulla CPU, ricarico anche il program counter e per la CPU e' come se non fosse successo nulla
- Ricomincia l'esecuzione

Quando ce lo chiede non ha bisogno di avere tutti i passaggi, salvare lo stato del processo, pag forse creata o generata e se e' modificata va salvata sul disco, ricomincio il processo dalla prima istruzione.

Instruction backup

Il punto 8 e' rognoso, perche' se abbiamo pagine che contengono sia dati che codice potrei fare page fault sia quando c'e' un dato sia quando c'e' dell'eseguibie, cosa succede se faccio una page fault a meta' del fetch del comando da eseguire. Non ho neanche tutta la move caricata nel decodificatore della CPU, dobbiamo sapere che c'e' il problema e non come si risolve.

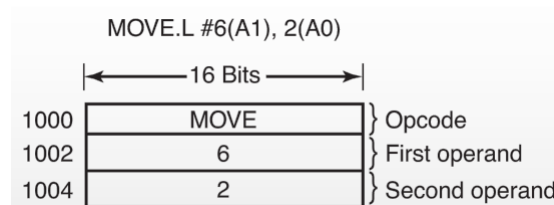


Figure 9.7: Instruction backup

9.5.1 Backing store

Un'altra scelta da fare e' quello di gestire il backing store (area di swap), qui ci finiscono i processi che vanno in swap out e ci vanno a finire le singole pagine che dalla page table vengono migrate sul disco perche' c'e' bisogno di spazio, due scelte:

- All'interno del disco c'e' una zona di memoria che ricalca la page table e metto le pagine come se fossero sulla page table ma sul disco, rende il codice semplice ma mi fa fare un sacco di overhead sul disco (A) le zone grigie sono spazio che ho allocato ma non sto utilizzando

- Disk map, ho una mappatura del disco che mi fa perdere prestazioni ma risparmiare spazio, dalla page table ho dei riferimenti delle pagine mancanti all'interno della tabel e questa tabella mi manda all'area del disco corretta (B)

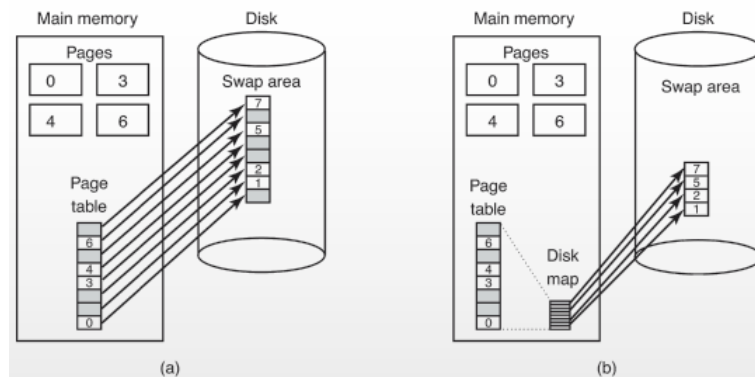


Figure 9.8: Backing store

I os moderni optano tutti per l'opzione B

9.5.2 Separazione tra politiche e meccanismi

Abbiamo sempre parlato di quello che va fatto ma mai di come va fatto, in tutte queste cose soprattutto perche' abbiamo bisogno di prestazioni definiamo una definizione tra politiche (cosa va fatto) e meccanismi (implementa una specifica scelta). Attenzione di non mischiare le due cose, da una parte kernel che gestisce il fault ed ha contatto con MMU, dall'altra parte ci sono i processi che usano il meccanismo della memoria virtuale ma non sa com'e' implementata.

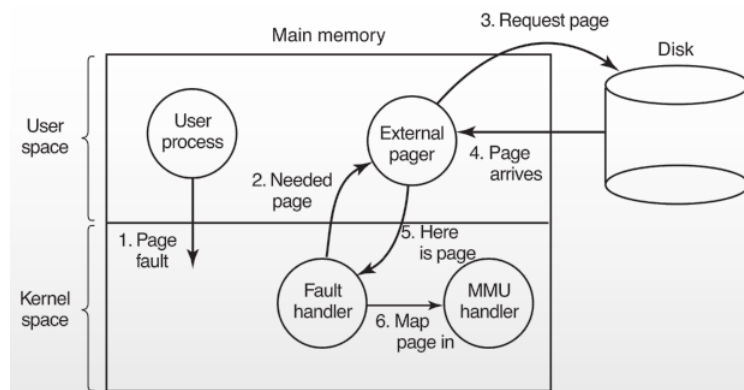


Figure 9.9: Politiche e meccanismi

9.6 Memoria segmentata

Il processo possiede una tabella dei simboli (metadati con comando mn) ed una serie di segmenti che si collocano all'interno dello spazio di indirizzamento ed ogni segmento e' specializzato nel contenere un certo tipo di dato. Questo disegno dice che c'e' un processo con il suo spazio di indirizzamento ed all'interno ci sono i segmenti, questo perche' nei processori di ultima generazione la memoria e' gestita in segmenti paginati, il processo dichiara dei segmenti ed i segmenti dividono in pagine, nonostante eviti la frammentazione esterna mi trovo a gestire la frammentazione interna nello spazio di indirizzamento virtuale. Ha i suoi

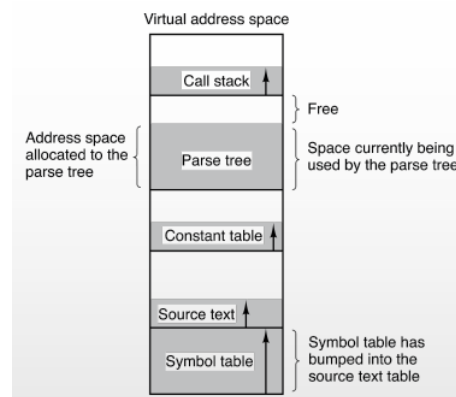


Figure 9.10: Memoria segmentata

vantaggi, ovvero che ogni segmento ha una sua locazione 0, se uso dei segmenti invece che un unico spazio di indirizzamento ho delle differenze, devo indicare le locazioni di memoria in maniera diversa, in questo caso il processo dice che una posizione di memoria = identificativo del segmento + spaziamento nel segmento, il processore deve dire che la CPU usa un indirizzo identificativo segmento : spaziamento, prima passo da registro base e limite e lo faccio diventare indirizzo fisico poi dovrò fare lo stesso discorso per le pagine e farle diventare indirizzi in memoria fisica. Per il programmatore e' comodo definire segmenti con proprieta' diverse : tabella dei simboli, l'eseguibile, le costanti, struttura ad albero, gestione delle chiamate. Sono sicuro che quando programmo e' difficile fare un bug che corrompera' il sistema.

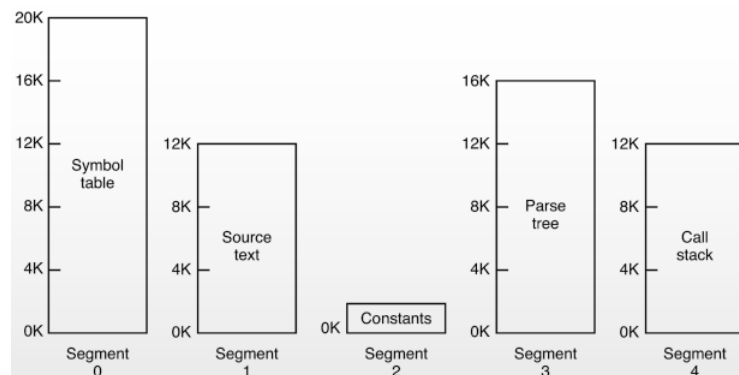


Figure 9.11: Segmenti

Pero' ritorno ad avere il problema della frammentazione esterna che risolvo con la memoria segmentata a pagine.

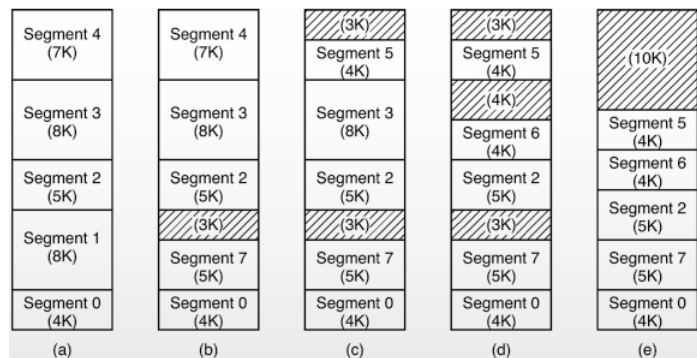


Figure 9.12: Frammentazione esterna

9.7 Confronto tra politiche

Ricordiamo che la realta' e' diversa da quella che racconta il libro. (che differenza tra memoria a pagine e segmenti: la pagina usata ed il programmatore non lo sa ma sa i segmenti, etc etc secondo la tabella della slide)

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

Figure 9.13: Confronto tra politiche

Chapter 10

File System

E' una di quelle cose particolarmente delicate dell'os ma non possiamo neanche farne a meno, perche' la ram ha un difetto, e' volatile e questo vuol dire che quando manca l'alimentazione tutti i dati che avevo nel sistema li ho persi, in piu' costa troppo per byte, di conseguenza e' necessario implementare un sistema di storage. Un sistema di storage e' un altro tipo di hardware che ci permette di non essere volatile e costa di meno, pero' mentre la memoria (ram) e' un bidone di dimensione variabili comincia da 0 e va fintanto che ho spazio, lo storage non possiamo usarlo allo stesso modo, ha bisogno di avere appoggiato sopra una struttura dati, perche' non possiamo usarlo come memoria dove la struttura la decidiamo noi, abbiamo bisogno di una struttura impostata dal sistema operativo che si gestisce il kernel e le sue cose, e questa struttura dati ci aiuta ad archiviare le informazioni (non chiamiamo cartelle le directory). Per costruire questa struttura dati che ci aiuta ad archiviare le informazioni secondo una logica di contenitori e sequenze di byte devo operare:

- A livello software definendo le strutture dati che il kernel va ad utilizzare
- A livello hardware perche' ho periferica che dev'essere gestita fisicamente e separatamente

Le due cose devono andare di pari passo, perche' voglio sia le massime prestazioni software sia hardware. Sganciare la nozione di file system da quella di disco, il disco e' una periferica, il file system e' una struttura per la memorizzazione di dati che metto su una periferica, che come nel cellulare non e' un disco.

10.1 File

Abbiamo bisogno di memorizzare le informazioni. Siamo abituati ad avere delle directory e dei raggruppamenti di file che chiamiamo directory, il concetto di directory e' relativamente recente, negli anni '70 di solito si costruivano dei file system che mettevano tutti i file in un unico spazio. E' chiaro che al giorno d'oggi sarebbe molto scomodo, ma all'epoca ce n'erano un centinaio. Oggi siamo abituati a parlare di file con una serie di attributi:

- Nome: abituati a raggruppare i file dandogli un nome, e' un informazione tipicamente esterna al file (nella lista della spesa non scrivo lista della spesa)
- Struttura interna:
- Tipi di file: hanno bisogno dell'estensione (windows) altri non se ne fanno nulla.
- Modalita' di accesso: definisce chi puo' fare operazioni su un dato file e quali operazioni puo' fare, sono tutte informazioni esterne
- Attributi: quando creato, modificato quanto grande

- Operazioni possibili

Inizialmente abbiamo saltato struttura e tipo di file, perche' sono punti in cui ci confondiamo.

10.1.1 Struttura interna

Una volta un file poteva essere letto un blocco alla volta o un byte alla volta, oggi possiamo farlo ma e' un problema del programma, posso leggere un byte o scrivere una sequenza di byte e dargli il significato che voglio, comunque per un os un file e' comunque una sequenza di byte, se poi a quella sequenza do un particolare significato e' un problema del programmatore. Nell'immagine la parte B non e' una cosa modera.

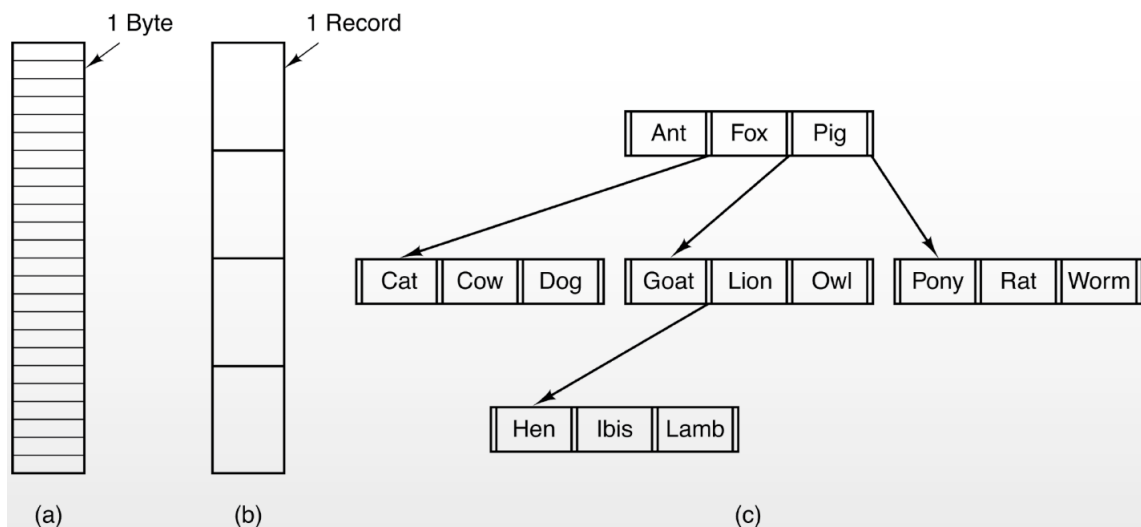


Figure 10.1: Struttura interna

10.1.2 Tipi di file

Il libro parla di tipo di archivio, che e' una cosa che interessa il kernel, non e' l'estensione del file. Per capire il tipo del file l'os sbircia i primi byte dello stesso e capisce che tipo di file ha davanti, in UNIX con il comando `file` ci dice che tipo di file e', questo tool ci trova il tipo del file anche senza estensione perche' si appoggia ad un manuale con tutte le definizioni del tipo di file in base alla prima manciata di byte. Non confondiamo mai il tipo di file con la sua estensione. Il libro fa un'assunzione, ci fa vedere un esempio basato unicamente su un eseguibile (colonna A), il quale ha in testa un magic number (4 byte), i quali dichiarano come va trattato l'eseguibile e poi seguono una serie di informazioni, poi ci sono le varie cose da caricare, l'header impacchettato e'

quello che chiameremmo .exe (in windows non sono così'). Per gli eseguibili e' così' per gli altri file ci siamo dovuti accrocchiare.

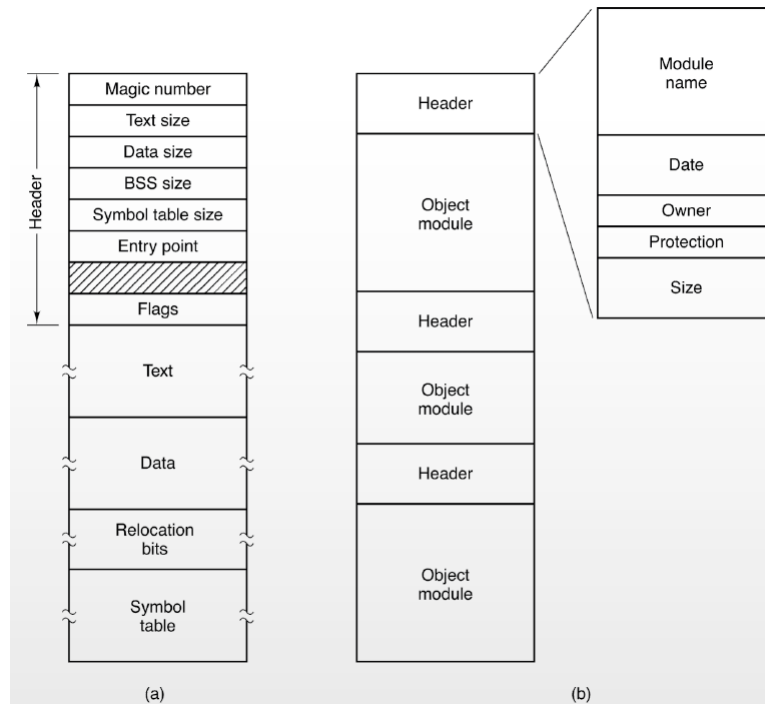


Figure 10.2: Tipi di file

Dopo di che un file e' dotato di attributi, appiccico ad ogni file una serie di informazioni, in un file in UNIX ci sono una sequenza di caratteri con i privilegi di accesso, lui distingue i permessi in 3 categorie, usando un byte per ogni categoria, dove trovo il meno il permesso non c'e':

- r si puo' leggere
- w si puo' scrivere
- x si puo' eseguire

Di questa terna ne faccio 3 copie:

- Proprietario del file (simi o dario)
- Diritti che verranno applicati al gruppo a cui fa parte il proprietario del file
- Permesso concesso a tutti gli altri utenti del sistema

Rispetto a windows questa roba e' ferma all'eta' della pietra, assume che faccio parte di un solo gruppo, le operazioni possibili sono solo scrittura, lettura esecuzione etc.

Dopo i permessi ho informazioni sulla dimensione, quando e' stato modificato l'ultima volta e poi il nome, tutte queste cose sono scritte nella directory in cui sta il file. Esistono piu' informazioni di queste, in UNIX c'e' il comando stat che ci ripropone le stesse informazioni facendo la richiesta ma anche informazioni in piu': i blocchi (usa una serie di unita' discrete e per essere allocato sul data storage ne sta usando 24), link, permessi di accesso, nome proprietario e gruppo scritti anche in forma numerica, time step (acceduto ultima volta, modificato ultima volta, cambiati gli attributi ultima volta).

10.2 Attributi

Sono tanti e variano anche da file system a file system, non solo tra UNIX e windows, ma anche tra UNIX e UNIX.

Attribute	Meaning	
Protection	Who can access the file and in what way	 
Password	Password needed to access the file	
Creator	ID of the person who created the file	
Owner	Current owner	 
Read-only flag	0 for read/write; 1 for read only	
Hidden flag	0 for normal; 1 for do not display in listings	
System flag	0 for normal files; 1 for system file	
Archive flag	0 for has been backed up; 1 for needs to be backed up	
ASCII/binary flag	0 for ASCII file; 1 for binary file	
Random access flag	0 for sequential access only; 1 for random access	
Temporary flag	0 for normal; 1 for delete file on process exit	
Lock flags	0 for unlocked; nonzero for locked	 
Record length	Number of bytes in a record	
Key position	Offset of the key within each record	
Key length	Number of bytes in the key field	
Creation time	Date and time the file was created	 
Time of last access	Date and time the file was last accessed	
Time of last change	Date and time the file was last changed	 
Current size	Number of bytes in the file	
Maximum size	Number of bytes the file may grow to	

 Unix
 Windows

Figure 10.3: Attributi

10.3 Operazioni possibili

Il libro ci da una lista piuttosto lunga su cui fa un taglio, non le chiede mai perche' gli sembra una cosa di buon senso

- Create: sulla directory
- Dlete: sulla directory
- Open: sul file
- Close: sul file
- Read: sul file
- Write: sul file
- Append: e' una open + posizionamento + scrittura
- Seek: posizionamento
- Get Attribute: sulla directory
- Set Attribute: sulla directory
- Rename: sulla directory

E' stato fatto un mischione perche' alcune operazioni le faccio sul file altre sulla directory (come scritto nell'elenco), ad esempio cancellare un file non e' aprire un file e farci un operazione ma e' aprire una directory e fare qualcosa con il contenuto della directory. Anche cambiare gli attributi, dato che sono esterni al file, lo faccio tramite directory ma non tramite file, anche il nome appartiene alla directory e non al file.

Perche' non posso avere strutture in cui il kenel mette dentro struttura date ed associa nome a file e la chiamo directory. Sul file system ho file di archivio ed un po' di tipo directory i quali vengono gestiti direttamente dal kernel.

10.4 Directory

Sono dei file di tipo speciale che si gestisce il os. Costruisco con questi file speciali una struttura dati ad albero e quello che succede e' che aggancio all'interno di una directory una serie di record (nome : dov'e' sul sistema di storage, dice che e' l'unico punto in cui gli sembra sensato parlare di record). Nel nostro esempio la directory usa tot unita' di allocazione e dentro ci sono 6 elementi e quindi 6 riferimenti:

- . : e' la directory stessa, prima dei permessi se c'e' una directory c'e' la d.
- .. : e' una directory contenuta in un' altra directory

- demo: e' un record da cui posso accedere da qui che si trova in una posizione della memoria

Con un meccanismo del genere posso costruire una struttura dati gerarchica per distribuire i file in modo comprensibile per un umano. Inizialmente la struttura della directory era a singolo livello, oggi non sono piu' a singolo livello ma hanno una conformazione gerarchica dato che mettiamo la directory nella directory. Un path ci dice la sequenza di directory ce dobbiamo attraversare per arrivare ad un file. Path assoluta se partiamo dalla principale "/" root in UNIX, esiste anche il path relativo esprimendo la posizione partendo dalla directory in cui mi trovo. Anche per le directory ci sono una serie di operazioni possibili, ma se la directory e' solo un file speciali le operazioni sono le stesse viste per i file ma con nomi diversi.

10.4.1 Gerarchica

Si parte da una root ("/" in UNIX, non esiste una distinzione di file system, esiste una sys call di mount e me l'aggancia alla root) , ho una struttura ad albero.

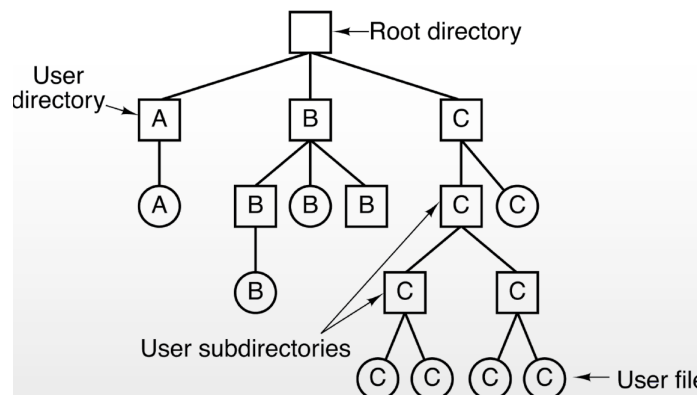


Figure 10.4: Organizzazione gerarchica ad albero

Se non mi piace posso vederlo come root directory, sequenza di record, ogni record mi punta ad un file

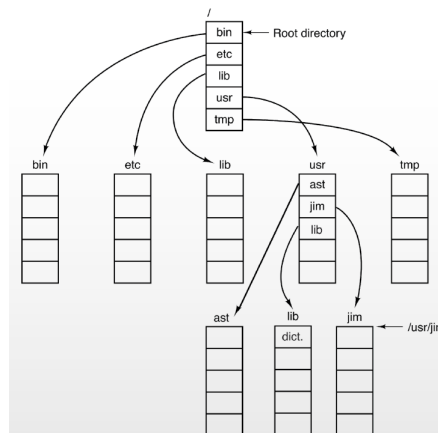


Figure 10.5: Organizzazione gerarchica 2

10.4.2 Operazioni possibili

Ha ripreso la lista fatta per i file e li ha fusi insieme:

- Create:
- Delete:
- Open: OpenDir e' una sys call diversa da quella per i file
- Close: CloseDir e' una sys diversa da quella per il file
- Read: Dimmi quali riferimenti hai al tuo interno
- Write: link/unlink aggiungi dei riferimenti o togli dei riferimenti
- Append: e' una open + posizionamento + scrittura
- Seek: posizionamento
- Get Attribute: anche la directory ha i suoi attributi che vengono attribuiti dalla directory che la contiene, posso quindi usare il comando stat sulla directory corrente
- Set Attribute
- Rename: sulla directory

Nonostante la struttura ad albero i file sono dei blocchi di dati che galleggiano nel sistema di storage, una cosa e' la struttura poi c'e' il dato che posso agganciare dove mi pare, quel numero che compare dopo i permessi e' il numero di posizioni nel file system da dove posso raggiungere la sequenza di byte (il file), in UNIX posso linkare un file in un'altra directory :

```
ln ../demo.c pippo
```

Creo un file pippo che e' uguale a demo.c e se modifico pippo cambia anche demo.c ma non perche' la modifica sia stata propagata ma perche' sto usando gli stessi byte, il file e' lo stesso puntato sia da pippo che da demo.c. Non duplico i dati ma aggiungo due entry nel file system che puntano alla stessa sequenza di file, quando cancello uno di questi punti accesso decremento il contatore delle posizioni da cui e' referenziato, quando questo contatore arriva a 0 butto via anche il file. Posso avere un altro riferimento al file con un link simbolico utilizzando il parametro -s, la differenza e' che non aumenta il contatore dei riferimenti, un processo che va ad usare un riferimento relativo viene ingannato dal file system perche' viene spostato. Gli attributi sembrano non avere senso perche' sembra che tutti possono fare tutto, ma non e' cosi' perche' quando uso paperino vengo rimbalzato sul file originale. Posso anche eliminare il file eliminare il file originale ma il link simbolico continua ad esistere.

Chapter 11

Implementazione di un file system

Abbiamo capito cos'è un file system, a cosa serve e come si usa. Attenzione che il file system vede solo dei file e le directory sono solo file speciali che l'os si gestisce in autonomia. Ora cominciamo ad addentrarci in una parte del libro che è trattata in maniera vintage, ci sono dei pezzi di libro che lui si rifiuta di farci (ad esempio di una tecnica di gestione di storage che sul libro non è menzionata ma lui ci fa perché è importante).

11.1 Disk Layout

Il file system è una struttura dati che tendenzialmente dev'essere un albero. Facciamo conto che prendo il sistema di storage e lo faccio diventare un bidone in cui metto i file, alcuni di questi mi fanno venire la struttura ad albero. Come faccio a convincere l'os ad usare il sistema di storage in modo giusto. Lo schema in immagine è giusto, però presuppone che il disco sia un'unica periferica fisica, il disco si compone di una serie di elementi che vanno in sequenza, dal primo byte mi trovo con degli elementi fissi. Un disco che fa partire il os si compone di :

- MBR (master boot record): una sequenza di pochi kbit che il firmware del sistema di calcolo (motherboard) è in grado di riconoscere e dice che quello è il sistema di boot, copia quei byte in memoria e dalla jump di prima istruzione deve continuare l'utente. Oggi si chiama EFI, ad oggi la parte di boot la gestiamo nel bios, fuori dall'os.
- Partition Table: ogni sistema operativo ha la sua, perché ogni MBR capisce una sua partition table (grub), è un pezzo molto piccolo di dimensioni variabili che ci dice che il resto del disco è suddiviso in pezzi (partizioni). Una volta si facevano le partizioni perché i dischi erano piccoli ed è chiaro che questo poco spazio dovevo dividerlo per metterci i vari dati, perché se si comprometteva una partizione non mi rovinava tutto il sistema. In Linux ci fanno una partizione di swap. Il motivo per fare più partizioni è che il file system in quanto software aveva dei bug, una volta era facile trovarsi in una condizione in cui l'os aveva dei problemi e scriveva cose a caso sul disco ed avevamo perso tutti i dati. In una partizione c'è (solo in UNIX):
 - Boot block: sequenza di byte che servono al os per partire (ci mette il primo pezzo di kernel). L'MBR capisce come sono fatte le partizioni e trova il boot block che carica il resto del kernel che sta sul file system, quindi devo almeno metterci il codice che interpreta le strutture dati del file system. Il boot manager parte dal boot block.
 - Superblock (struttura dati a dimensione fissa): come è fatto il file system, ho tutte le informazioni e sono organizzate in blocchi grossi tot. Dopo questo ho una serie di informazioni che ci danno informazioni sul file system.
 - Free space management

- I-nodes
- Root dir
- File and directories: come sono organizzati i file e directory

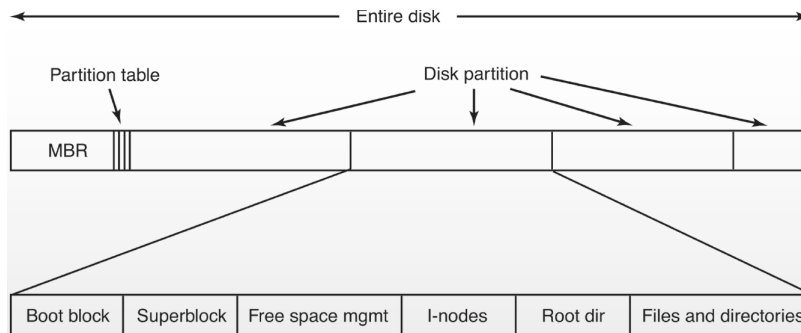


Figure 11.1: Disk Layout

Abbiamo tante sequenze nel disk partition perche' quando i dischi erano meccanici erano fatti da tanti dischi di silicio, un cylinder group era fatto da tutte le tracce incolonnate sui cilindri, era importante perche' se avevo una rottura meccanica mi rompeva tutta la colonna, se riuscivo ad isolare la rottura anche nelle strutture dati riuscivo a salvare piu' dati.

11.1.1 Sistemi industriale

Non esisteva il concetto di partizione a nostra scelta. C'era un file di testo che si trovava sul file system che ci diceva che avevamo tot partizioni sul disco, non puoi toccare niente e ci dice quali erano le partizioni e dove stavano.

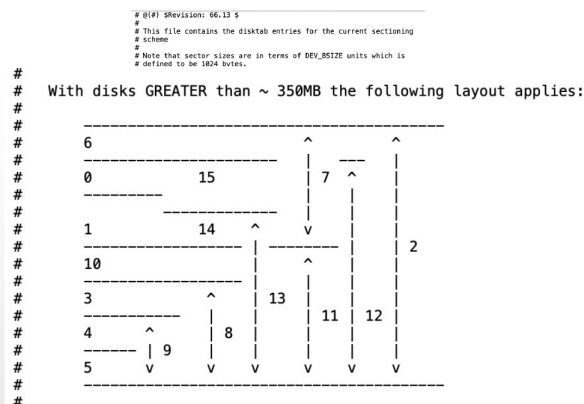


Figure 11.2: Vecchio Disk Layout

L'idea era che sapendo la dimensione del disco si usava la partizione 8 per il boot, la 10 per lo swap e la 7 per i file dell'utente (a spanne). Quando si usavano questi tipi di tecnologie i dischi erano grandi come lavatrici, il sistema UNIX e' nato prendendo come riferimento dei dischi a cuscino ad aria (winchester e' la stessa azienda dei fucili) e quindi tutte le scelte fatte sono state fatte per evitare di perdere dati con quella tecnologia hardware. Windows ha un file system piu' articolato perche' essendo arrivato dopo si e' potuto basare su un hardware diverso.

11.2 File con allocazione continua

Posso fare uso di tante politiche per mettere i byte all'interno di una partizione, assumiamo che una partizione sia nient'altro che una sequenza di unita' allocabili, chiamiamoli datablock (blocchi). Ho all'interno di una partizione n data block in cui mettere file, ho una serie di file e cosa faccio? La prima soluzione e': ho una sequenza di file (file system), metto un file in una serie di blocchi uno dopo l'altro e quando finisce un file inizio con il prossimo (e' sempre il problema dell'allocazione della memoria), perche' continuo a creare e distruggere file e quindi ho il problema della frammentazione esterna. Un meccanismo di questo tipo contiguo e' molto conveniente se faccio tanti accessi casuali perche' sono velocissimi, l'accesso sequenziale anche ma c'e' il problema della frammentazione. E' utilizzatissimo in un contesto in cui abbiamo un hardware molto lento che quindi fa fatica ad andare avanti ed indietro (quindi leggiamo sequenzialmente) e noi non cancelliamo i file. Ad esempio lo usiamo nei DVD, quando faccio la masterizzazione di un DVD ho un software che crea un'immagine che e' esattamente cosi', ora non lo usiamo piu' perche' non abbiamo piu' i lettori.

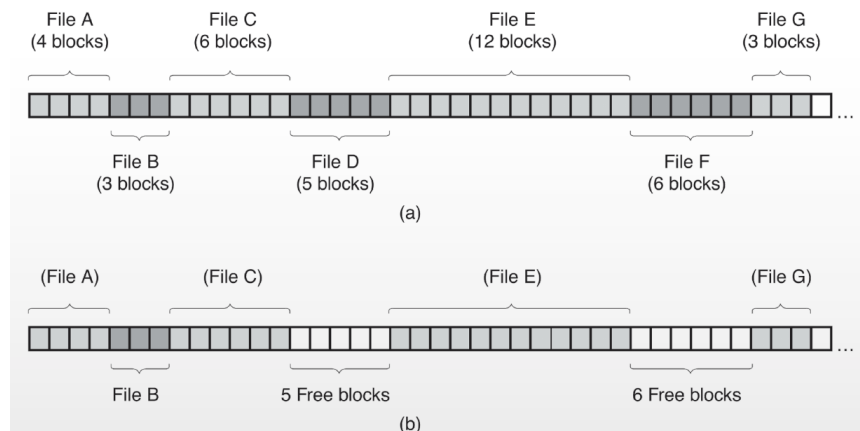


Figure 11.3: File con allocazione continua

11.3 Allocazione a lista

Ho dei dati che sono dinamici e quindi ho bisogno di allocare i file dove mi servono, e può essere che i file cambino di dimensione, quindi ho bisogno di un meccanismo per saltare da un blocco all'altro. Non ci siamo inventati niente, ci sono le liste linkate nella gestione memoria in cui metto i dati, usiamola anche su disco. Mi segno da qualche parte nella directory il datablock in cui il file comincia (0 nella posizione 4, e mi segno solo 4), quindi quando devo leggere quel file leggo il datablock 4. Dentro ci sono i dati e poi c'è un preambolo in cui c'è scritto che il prossimo datablock è il numero 7 (in generale qual'è il prossimo datablock). Nell'ultimo datablock c'è scritto 0 quindi vuol dire che è finito il file. Se si allunga il file prendo il primo datablock libero e metto in coda il file. Mi dà della flessibilità ed è un'evoluzione rispetto a prima, ma ha due difetti:

- L'accesso casuale diventa un disastro, perché se devo accedere al byte 1000 so dove comincia ma devo leggerlo tutto finché arrivo al datablock che mi interessa.
- Ho messo un'informazione aggiuntiva all'interno del datablock (è un metadato), per motivi di prestazione ho la dimensione di un datablock che coincide con la dimensione del buffer del disco, se ho un disco che legge e scrive 2 kb per volta, quello che mi conviene fare è un file system che legge e scrive 2 kb per volta per avere una sola operazione di lettura. Qui non faccio questa cosa perché se il processo vuole 2kb di dati ed un datablock = 2kb comunque devo fare due accessi di disco, perché devo prendere dei dati nel datablock successivo, perché ho l'header che mi ruba spazio.

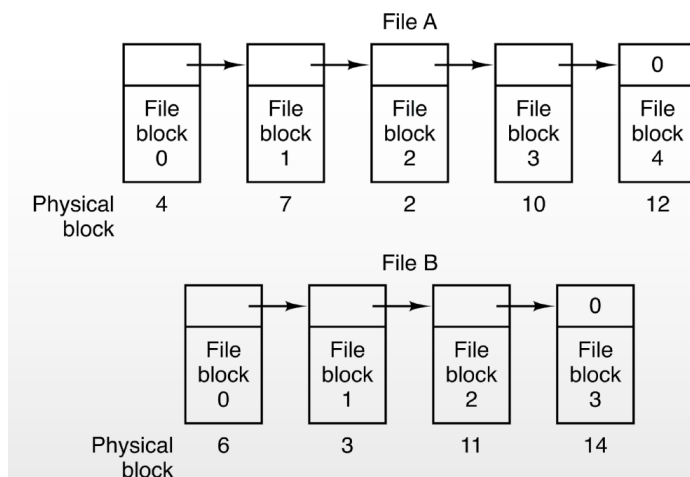


Figure 11.4: File con allocazione a lista

11.4 FAT(file allocation table) 16

Ho dei sistemi di storage molto piccoli, l'MSDOS ha tolto i puntatori dei datablock. Prendo l'elenco dei puntatori e li metto tutti nella prima traccia del disco, in posizione 4 della FAT c'è scritto 7, il file comincia dal blocco 4. Guardando la FAT dico che il secondo blocco è il 7 (perché nella FAT in posizione 7 c'è il 4) e così via, quindi io ho messo tutti i puntatori nella prima traccia del disco e faccio un parallelo tra la posizione del datablock nel disco e la posizione dell'elemento all'interno della FAT. Va benissimo e lo usiamo anche adesso. 16 perché sono i byte presenti nella entry di un'allocation table, non tutti venivano usati e venivano inserite delle informazioni, quindi con 16 bit siamo in grado combinandoli con datablock di avere file grandi fino a 2Gb. Crescendo la dimensione dei file siamo passati da FAT16 a FAT32 per avere più spazio nell'allocation table. Domanda tipica all'esame è dove la colloco la FAT, la FAT viene portata in memoria per questione di prestazioni, il problema di averla in memoria è che se il sistema si spegne mi fumo tutto il file system, quindi la risposta è che la FAT sta nella prima traccia del disco e poi ne faccio una copia in memoria per aumentare le prestazioni del sistema.

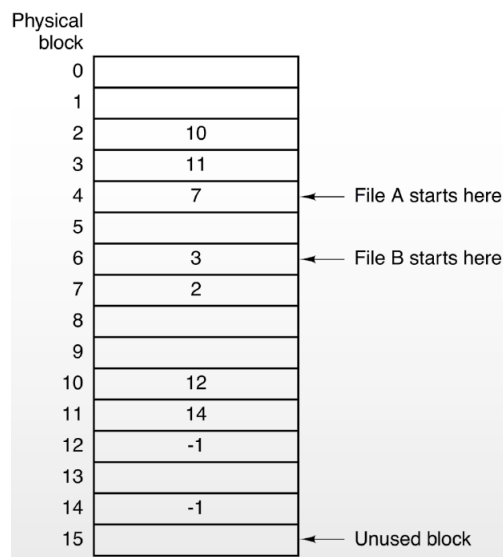


Figure 11.5: FAT 16

11.5 File con allocazione ad albero (i-node)

Ad UNIX non andava bene la FAT, perché secondo loro era più di una sequenza di byte. Dichiaro nel file system che ci sono strutture dati apposite per gestire i file, ho un i-node per ogni file ed all'interno ci metto un insieme di informazioni

(metadati del file, non contengono il nome del file) e poi una serie di puntatori ai datablock che contengono le informazioni, se ci sono anche info relative alla dimensione so esattamente quanti datablock vado ad utilizzare. Ci ha messo l'immagine usata sul libro, ma attenzione che UNIX non funziona così. Quali sono attributi all'interno di un i-node, sono tanti:

- Dimensione del file
- Permessi di accesso
- Marcature temporali di quando è stato creato modificato acceduto ultima volta
- Informazioni riguardo al proprietario
- Informazioni riguardo al tipo, non concepisce il tipo come l'estensione, ma intende file che è una sequenza di byte, piuttosto che directory, piuttosto che altri tipi di file che servono al os (l tipi i link simbolici, b sono file che rappresentano un punto di ingresso per la periferica che utilizza un buffer quando ho un disco ho un file blockdevice che mi va a pescare sul disco e leggo e scrivo un blocco di dati alla volta, tipo c dove ho una periferica che ragiona un carattere alla volta (stampante), p punti di scambio sul disco dove ho due processi (produttore consumatore) dove uno ci scrive e l'altro ci legge) in pratica tutte le informazioni che saltano fuori dal comando stat.

Ora i datablock sono di circa 2/4 kb (dipende dal sistema), nell'esempio abbiamo 8 puntatori (quindi il file è di 16 kb).

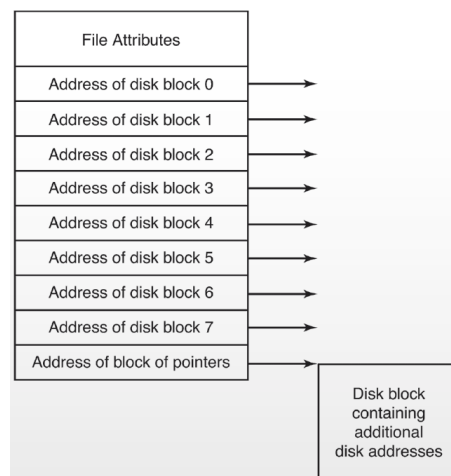


Figure 11.6: File con allocazione i-node

11.6 UNIX file system

I puntatori ai datablock non sono 8, nell'implementazione standard sono 13, i primi 10 vengono detti indirizzamenti diretti (puntano a dei datablock in cui l'os mette direttamente dei dati) quindi arrivo a 20 kb di file (e' poco). L'undicesimo e' un puntatore che mi punta ad un datablock ma senza dati, ci metto dentro una sequenza di riferimenti ad altri datablock e quindi la chiamo referenziazione indiretta (indirezione singola). Se li uso tutti (file piu' lungo di 1034 datablock) ho bisogno di usare il dodicesimo, pero' occhio che scatta campanello d'allarme (perche' chi la fatto ha visto per statistiche che l'os e' fatto di tanti file molto piccoli ed alcuni molto grossi), quindi uso indirezione doppia, punta ad un datablock in cui metto riferimenti ai datablock con i dati. Il 13 ho un riferimento a tripla indirezione, perche' punta a datablock che puntano a datablock che puntano a datablock che puntano a dati. Quindi anche se il datablock e' piccolo riesco ad avere un file grosso 2.5 Tb. Ma se questo meccanismo funziona cosi' bene perche' non posso gestire tutto con tripla indirezione? per le prestazioni. Perche' se ho tutti file system ad indirezione tripla mi dice che bisogno di accedere al disco 4 volte e la capacita' di throughput e' del 20%. Quindi per i file piccoli uso indirezione diretta (addirittura se ce l'ho in cache e' ancora piu' veloce) per essere veloci, e se i file sono grossi pace ci mettero' tanto, comunque l'utente sa che sono grossi e quindi si accolla che vada lento. Questa schifezza (poco intuitiva) e' la struttura dati di uno dei file system piu' utilizzati al mondo.

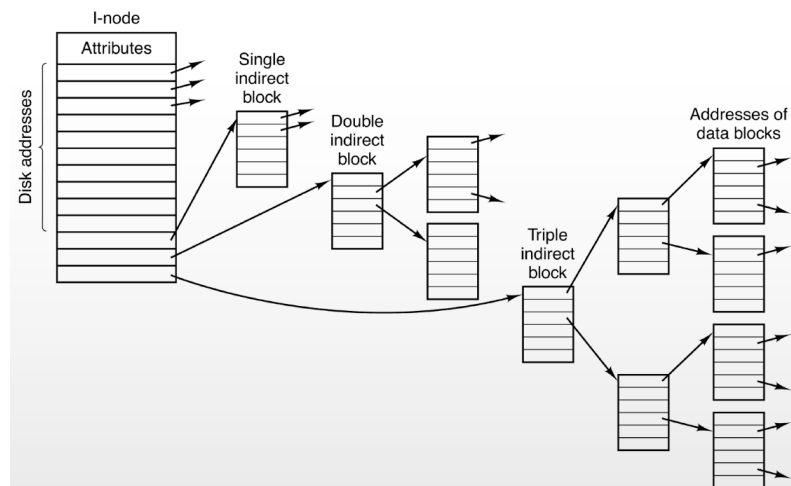


Figure 11.7: Unix File System

Nella seconda immagine ritroviamo delle informazioni che abbiamo appena visto in nomenclature UNIX:

- Un hard disk e' dotato di piu' partizioni

- Ogni partizione e' all'interno del' file system: boot blocks, super block, Cylinder group etc. Ogni file system e' diviso in settori.
- Ogni cylinder block contiene una copia del superblock, perche' se si rovina quello di sistema quelle sono le posizioni in cui recuperare quello di backup, ci sono delle mappe degli i-node liberi, diversamente da memoria mi sta bene usare una mappa perche' tanto vado a prendere un i-node per volta ed un byte per volta da allocare. All'interno degli inode ci sono una sequenza di informazioni (uno per file). Ho anche la bitmap block e i datablock.
- dentro i-node ho metadati e poi una sequenza di riferimenti al data block

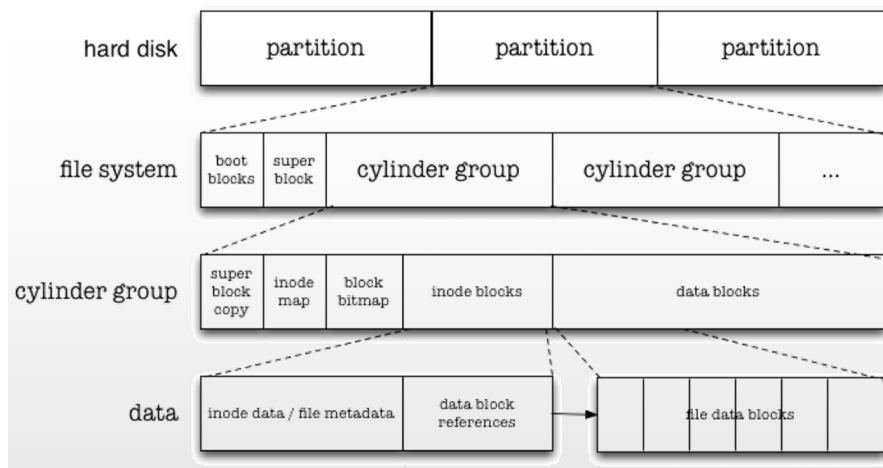


Figure 11.8: Unix File System

11.7 Implementazione directory

Ho il nome del file e degli attributi tra cui il puntatore al primo blocco utilizzabile, per cui tengo questa tabellina e per attributi c'era il bit che diceva che tipi era (file di sistema, directory, file da nascondere). La limitazione di utilizzare questo approccio e che avevo delle forzature sulla dimensione massima dei file, perche' se un record ha una dimensione fissa mi do un numero massimo di byte per immagazzinare il nome del file, in questo caso poteva essere 8 caratteri piu' 3 caratteri di estensione. Questo spiega anche perche' sui sistemi moderni abbiamo un'estensione di file sempre su 3 caratteri. Ovviamente non e' l'unico modo di farlo.

Posso mettere nella directory i record e tentare di fare qualcosa di raffinato, se andiamo su sistemi che supportano nomi piu' lunghi di 8 ci sono due possibilita':

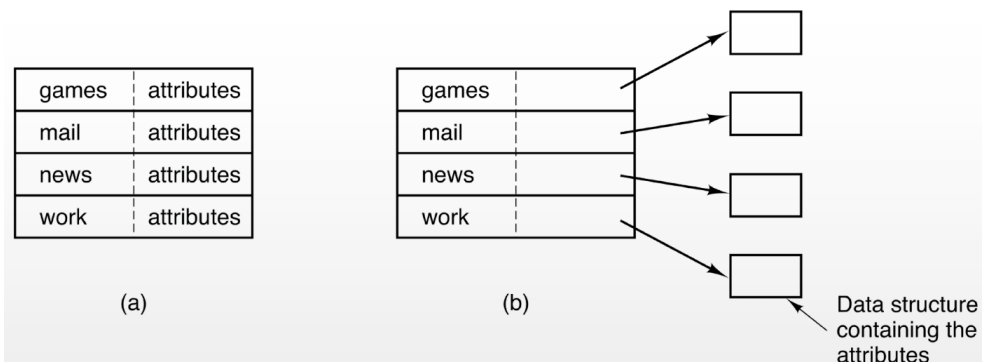


Figure 11.9: Implementazione di directory

- A: entry successive che non sono dei record, in cui ho un preambolo di dimensione nota con tutti gli attributi e tra questi ci scrivo anche quanto e' lungo il nome del file, quindi finito il preambolo so quanti byte sono per il nome del file. I rettangoli grigi perche' non fare uso di esattamente quanti byte voglio, dipende dall'implementazione del sistema a 32 bit devo allocare sul disco 4 byte alla volta e gli ultimi li spreco. Il problema e' quando devo rinominare un file, devo allungare la parte di dati che ho destinato al nome del file e potrei non poterlo fare, quindi dovrei spostare tutto in coda creare un buco ricompattare, per ovviare a questo abbiamo l'opzione B.
- B: mi tengo dei record a dimensione fissa come una specie di tack della directory, e tengo dentro gli attributi ed invece di dire quanto e' lungo il nome tengo un heap (piu' in la') e dico dove comincia il nome del file. In questo caso il file 1 che aveva nome project punta ad una zona dello heap dove c'e' scritto sempre project. Quindi se devo cambiare il nome non tocco la entry ma sposto solo la stringa, e l'operazione di compattazione mi pesa molto di meno.

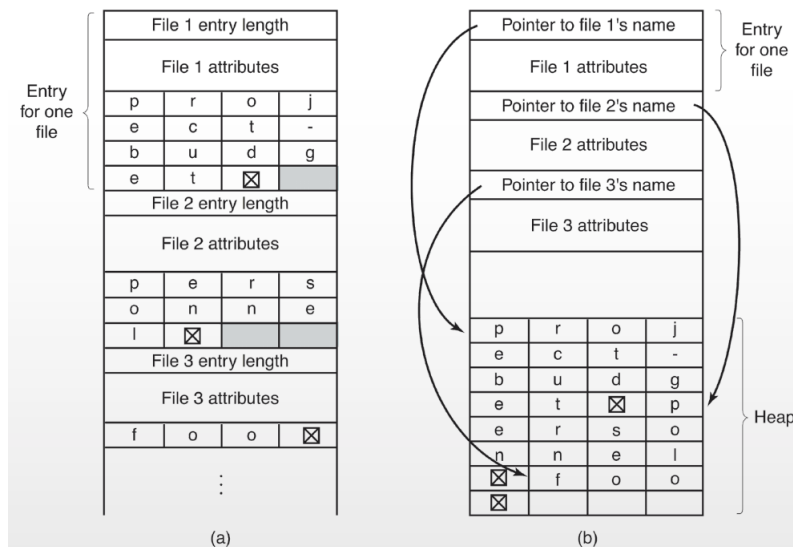


Figure 11.10: Implementazione directory

11.8 UFS (UNIX file system) entry

Vede all'interno delle directory un record di 2 byte per il numero dell'i-node e 14 byte per il nome del file, quest'informazione lascia il tempo che trova perche' e' una cosa vintage, perche' sappiamo usando linux che i nomi possono essere piu' lunghi di 14 bytes. Quindi io ho sul mio file system una limitazione di avere solo 65636 byte tra file e directory che se andiamo a prendere un os moderno e' ridicolo, quindi oggi possiamo dire che la entry non assomiglia per niente a questa roba.

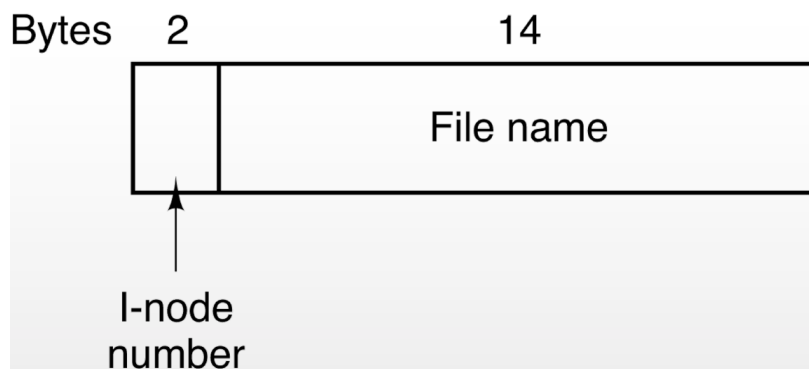


Figure 11.11: UFS

11.9 File condivisi tra directory

Riprende la cosa del link e link simbolico, posso linkare un file per avere piu' di una entry dello stesso, ricordiamo che se modifico il file linkato modifico anche l'originale.

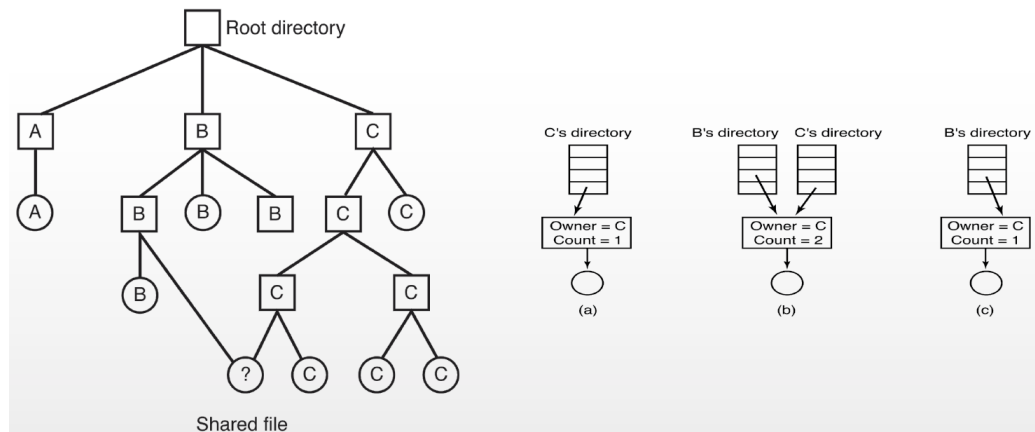


Figure 11.12: File condivisi tra directory

11.10 Esempio di UFS

E' un esempio riportato dal libro (gli 1 in alto a sinistra sono in realta' 2, perche' in tutti gli UNIX la root sta nell'i-node 2, nell'1 c'e' il backup). Se devo andare a prendere il file che /usr/ast/mbox, parto dalla directory vado a vedere nell'i-node 6, dentro il 6 ci sono dei metadati e poi c'e' il primo riferimento che dice che i dati della directory partono dal datablock 132, prendo il datablock 132 che e' la directory usr e qui cerco l'elemento ast e mi dice che da qui devo saltare all'i-node 26 il quale ci manda al datablock 406 dove c'e' scritto che mbox e' un file che posso cominciare a leggere prendendo l'i-node 60. Grossomodo i file system funzionano cosi', ci sono tante varianti.

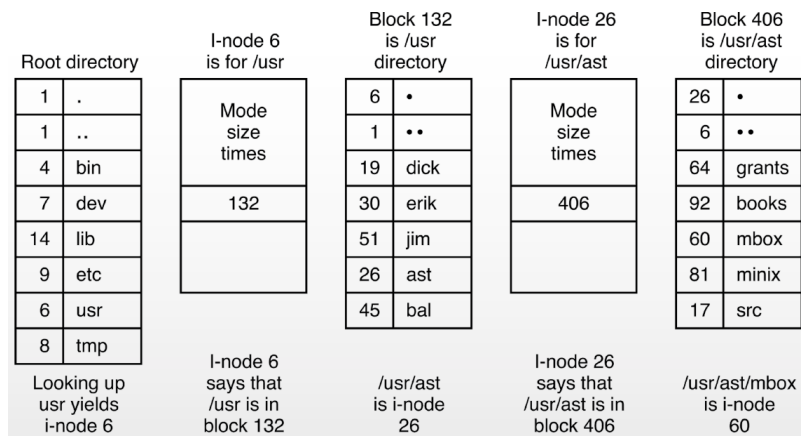


Figure 11.13: Esempio UFS

11.11 Varianti possibili

- **Log-Structured File system:** tiene una specie di diario di tutto quello che succede, perche' sappiamo che prima o poi il sistema crashera'. Un sistema di log aiuta perche' se segno tutte le cose che stano succedendo nel momento in cui devo ripartire vado a vedere nel log e cerco di rimettere insieme le cose prima di fare il controllo dell'integrita' delle strutture dati. Il file system lo vedo come struttura dati complessa e per questioni di prestazioni non faccio le operazioni richiesta dal file system sul disco, ma le faccio in memoria e periodicamente quando gira l'os le operazioni in memoria vengono aggiornate sul disco. Il problema di inconsistenza e' quando c'e' un crash quando ancora non ho aggiornato il disco. Se ho un log ho una partizione del disco e quando c'e' un operazione la faccio sulle strutture dati in memoria e le scrivo sul disco subito. Ci sono molti modo di gestire il log, quello piu' semplice e funzionale e' quello del Journaled File System.
- **Journaled file system:** applico il concetto di cache point, il log e' una partizione (non un file), mano mano che il file system evolve ci sono momenti in cui dico adesso ho sincronizzato tutto, quando dal punto di vista temporale quello che ho in memoria e quello che ho sul disco combaciano segno questo punto nel log come cechkpoint, dopo di che vado avanti. Quando ho un crash e quando la macchina riparte avendo un log prendo l'ultimo cechkpoint e leggendo tutte le operazioni le rifaccio sul disco pari pari. Funziona se e solo se le operazioni che ho scritto nel log sono operazioni idempotenti (non cambia il risultato anche se la rifaccio tante volte) sono tranquillo che queste non mi introdurranno inconsistenze tra i dati dopo di che il sistema non puo' comunque ripartire perche' devo comunque rifare i controlli di consistenza, ma molto spesso mi dira' che e' tutto a posto.

- Virtual File System: e' un middleware (libreria che hanno tutti i os)

11.11.1 Virtual File System

Abbiamo un disco formattato UTFS e potremmo avere tipi di file system diversi, pero' i processi che abbiamo all'interno del os usano sempre le solite system call, perche' tutti i processi fanno riferimento a system call posix che non vanno ad interagire direttamente con il file system ma con una libreria condivisa chiamata virtual file system che solo lei sa quanti e quali tipi di file system abbiamo agganciati al nostro sistema e traduce le chiamate nelle chiamate corrette dove vogliamo andare a fare l'operazione. Questo mi permette di avere piu' file system con implementazioni completamente diverse e tutti i file system condividono la stessa buffer cache.

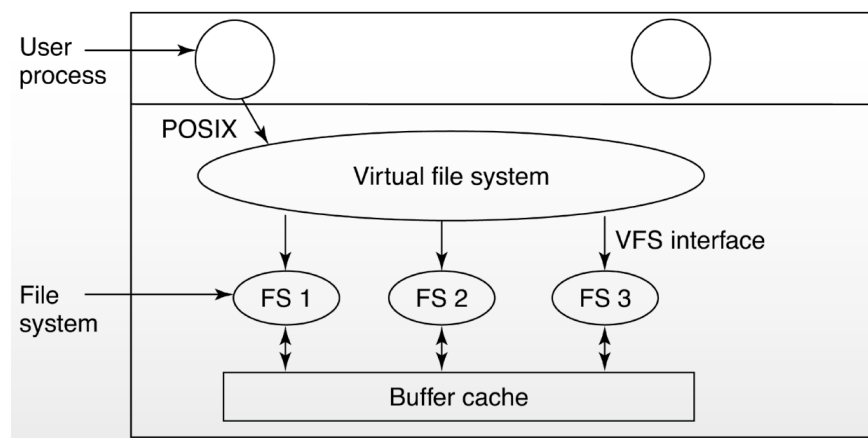


Figure 11.14: VFS

In pratica utilizza un meccanismo chiamato v-nodes che non hanno niente a che fare con i-node e fanno un accoppiamento tra le system call posix che chiama il processo con delle chiamate di librerie specifiche per il file system che dobbiamo implementare.

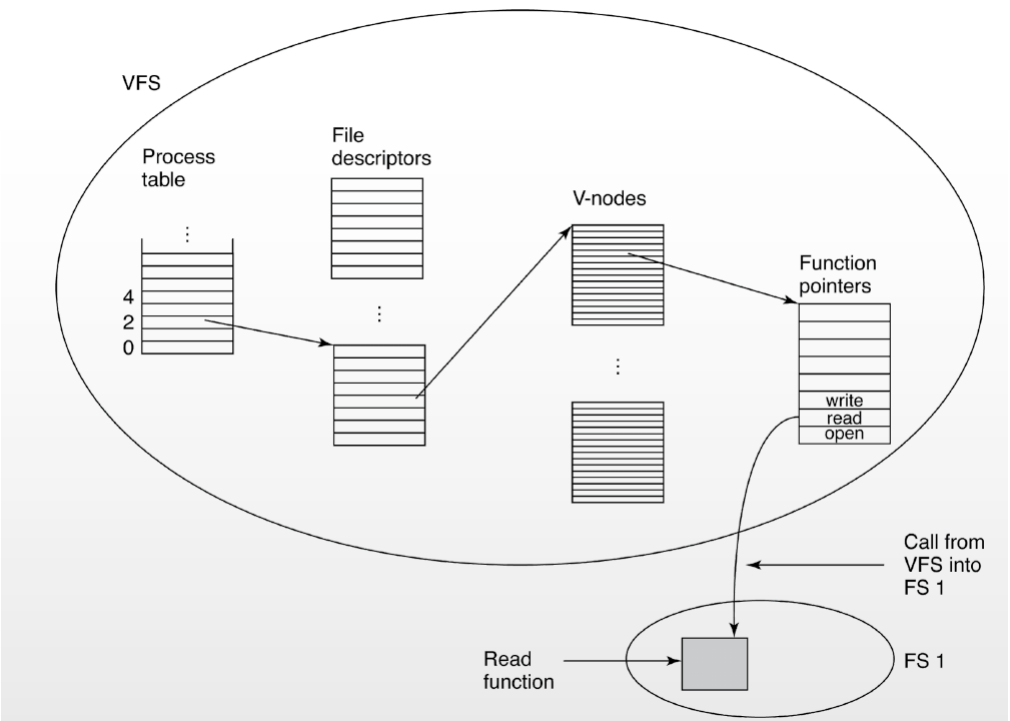


Figure 11.15: VFS2

Chapter 12

Gestione ed ottimizzazione del file system

12.1 Logical Volume Manager

Mediamente in un industria 1Tb di disco costa tra i 2k e 3k euro, quindi non stiamo parlando dei dischi che abbiamo noi a casa. In un azienda lo storage viene messo nel NAS (network access storage), l'idea e' che abbiamo questi armadi con un sistema operativo minimale dedicato alla gestione dello storage. Sopra questo problema se ne pone un'altro di tipo organizzativo, i database sono enormi e sono composti da piu' dischi. Ci siamo inventati il logical volume manager. Il disco diventa un phisical volume che e' una periferica fisica dove io ho costruito una specie di intelaiatura, gli ho attaccato un etichettina in cui metto delle informazioni sulle aree del disco e quanto e' grande l'entita' minima in cui ho diviso il disco, e questa tesserina minima prende il nome di phisical extend. Nel momento in cui ho il disco e ci costruisco questa roba qui che e' piu' piccola di una formattazione (e' un nome ed un id), non e' utilizzabile per metterci i dati. Quello che posso farci e' metterli insieme, costruisco aggregando i phisical volume il volume group, quindi il nas e' il risultato dell'aggregazione di tanti dischi fisici che concorrono a dire che abbiamo tot spazio di storage. Il volume group ancora non e' disponibile per farci il file system, aziendalmente conviene molto che da una parte c'e' chi compra i dischi e dopo di che il volume group e' una risorsa disponibile per tutti, ci sono i vari responsabili dei settori a cui serve tot spazio e noi tagliamo una fetta di questo spazio e glie la mettiamo a disposizione attraverso la rete. Quindi come amministratore dello storage nel momento in cui devo fare il mio lavoro taglio una fetta del volume group e dico che quella fetta diventa la virtualizzazione di un disco (non mi interessa come sono disposti e dove sono). La cosa interessante e' che questa memoria e' dinamica e disponibile via rete. Quindi ho un disco virtualizzato e tecnicamente qui sopra posso costruire un file system, come i phisical volume erano divisi in tesserine di grandezza fissa, anche il volume group e' diviso in unita' elementari che possiamo chiamare group extend quindi uso un group extend alla volta, devo ricordami quali sono i dischi che ne fanno parte, quanto e' grande la sua group extend e a quale phisical extend corrisponde una determinata group extend perche' potrebbe essere che parti di group extend siano su phisical volume diversi, quindi il dato e' ridondato e nel caso in cui un disco fisico si rompe tutto continua a funzionare ma dobbiamo cambiare quel disco. Come associo il group extend al phisical extend e' un problema mio e mi posso sbizzarrire (avere alte prestazioni o basse prestazioni). Il Logical Volume (e' il volume virtualizzato su cui costruiamo il file system) e' anch'esso diviso in tesserine e queste prendono il nome di logical extend, anche qui posso fare associazioni 1 a 1 o 1 a molti. Questo meccanismo che ormai usano tutti risolve tanti problemi:

- Posso gestire via software la questione della ridondanza, sicurezza, rottura di hardware
- Rendere dinamica la gestione, se devo usare altra memoria butto dentro altri dischi senza cambiare sistema
- La ridondanza tra Volume Group e Physical Volume e' l'equivalente soft-

ware di quello che prima si faceva con i RAID (ho un controller a cui attacco due dischi che li fa vedere alla motherboard come un disco unico) il problema del RIDE e' che se me ne rompe uno dei due devo sostituirlo con uno che e' identico a quello che si e' rotto perche' e' un collegamento fisico. Con il nas li abbiamo fregati perche' tanto i dischi possono anche essere diversi tra di loro.

- Abbiamo risolto il problema di backup, perche' e' vero che posso essere ridondante con i miei extend ma e' vero che i disastri succedono (incendi etc) se ho il volume pieno come faccio a fare il backup su un altro supporto che metto da parte con una mole di dati cosi' grande e importanti (tipo le banche o gli operatori telefonici), il sistema di LVM mi permette di fare uno snapshot, mentre il sistema va dall'esterno richiedo uno snapshot ferma tutto, ma non ferma di lavorare, congela i dati che ci sono sul volume group in questo momento, le operazioni vanno avanti ma io mi tengo una parte del volume group come una specie di cache, tutti i cambiamenti ai dati che avvengono nel momento in cui apro lo snapshot vengono parcheggiati in cache, come il journal del File System. Quando il backup e' finito chiudo lo snapshot e vado a riconciliare tutti i dati che avevo parcheggiato con il volume group con un operazioni di resync per mettersi in pari, il risultato e' che ho fatto il backup e non ho interrotto il servizio. Le aziende tendenzialmente fanno il backup su altri dischi, in modo che se perdo i dati attacco direttamente i dischi e non ripristino il backup (ci converrebbe farlo sullo spazio di archiviazione piu' economico il nastro, ma e' lento e brutto)

Prendo una dimensione per logical extend, group extend e phisical extend che sia la stessa (e la decido nel momento in cui creo il tutto), noi ragioniamo in termini di extend di 4k, tipicamente si sceglie la dimensione che sia un multiplo in potenza di 2 di quello che sia un buffer medio di un disco in commercio.

/dev/sda e' il file descriptor, poi linux dice che dobbiamo almeno fare una partizione per il phisical volume, su questa partizione costruisco il phisical volume, si creano altri device file e questi tutti quanti finiscono le volume group e posso tagliarne fette grosse quante voglio e addirittura metterci su sistemi operativi diversi.

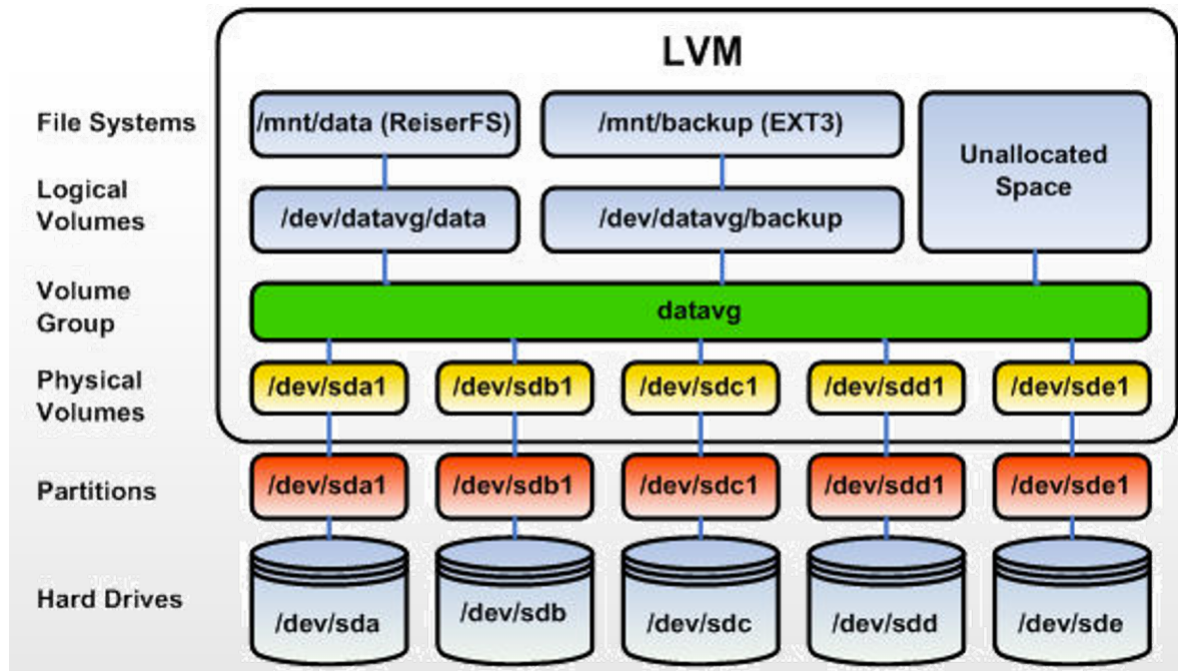


Figure 12.1: LVM

12.2 LVM e RAID

Una volta queste tipo di soluzioni venivano fatti dai RAID che semplicemente e' un controller che dice all os di essere un solo disco ma in realta' ne ho attaccati tanti. I RAID sono di vario tipi:

- RAID-0: striping, scrive prima sul primo disco intanto la meccanica del secondo disco e' partita e messo a posto la testina, dopo scrivo sul secondo disco e mentre scrivo la meccanica del primo disco si prepara a scrivere e poi scrive. Aumenta la velocita' di scrittura.
- RAID-1: mirroring, non aumento la velocita' ma aumento l'affidabilita' perche' i dati sono copiati su tutti i due dischi, anche qui i dischi devono essere assolutamente identici
- RAID-5: con parita', sto scrivendo in sequenza in cui costruisco delle informazioni di parita' (p(A)). Posso ricostruire l'informazione di uno di questi dischi partendo dall'informazioni di parita' e altre due informazioni sopravvissute (A1 e A2 con P(A) posso recuperare A3). La parita' viene fatta girare da un disco all'altro. Il RAID 10 non esiste. Con meno di 4 dischi il RAID 5 non ha senso.



Figure 12.2: RAID 0 e 1

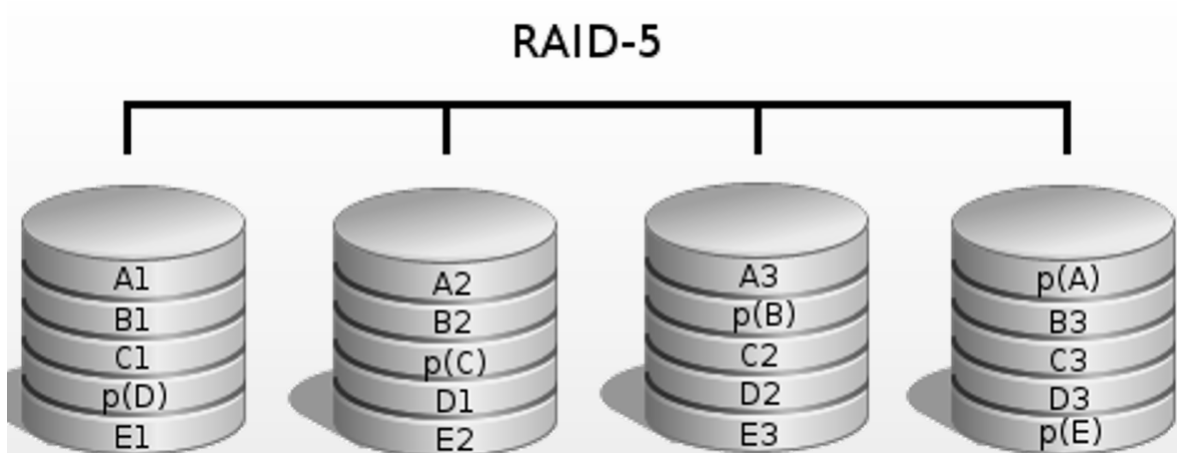


Figure 12.3: RAID 5

Esistono due concetti:

- Fault resilience: caratteristica per cui a fronte di un guasto il servizio non si interrompe ed e' un concetto di cui possiamo parlare con un tecnico
- Business continuity: tutte le cose che metto in piedi per assicurare che il flusso di soldi che arriva all'azienda non si interrompa, possiamo parlarne con un manager.

Questi giochi fatti hardware si possono fare con Logical Volume Manager, metto dei group extend consecutivi in dischi consecutivi (come nel RAID-0), prendo group extend e lo associo a dischi distinti (come nel RAID-1).

12.3 LVM - struttura generale

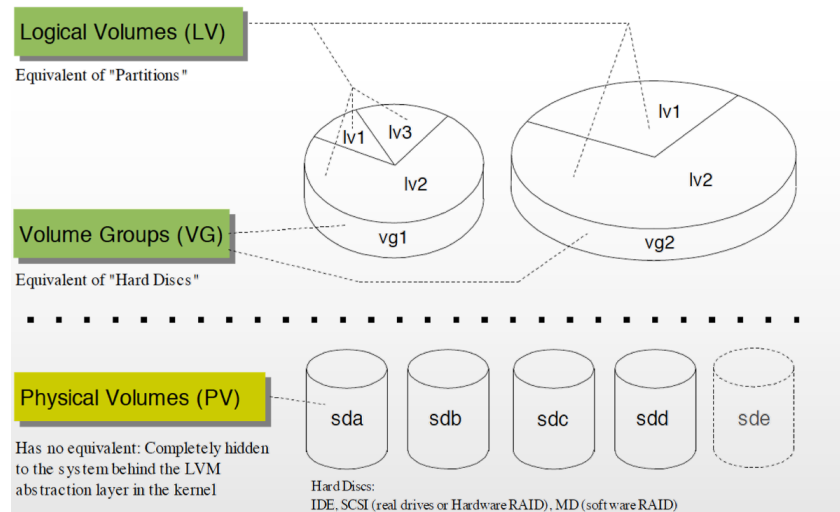


Figure 12.4: LVM - struttura generale

12.4 LVM - snapshot

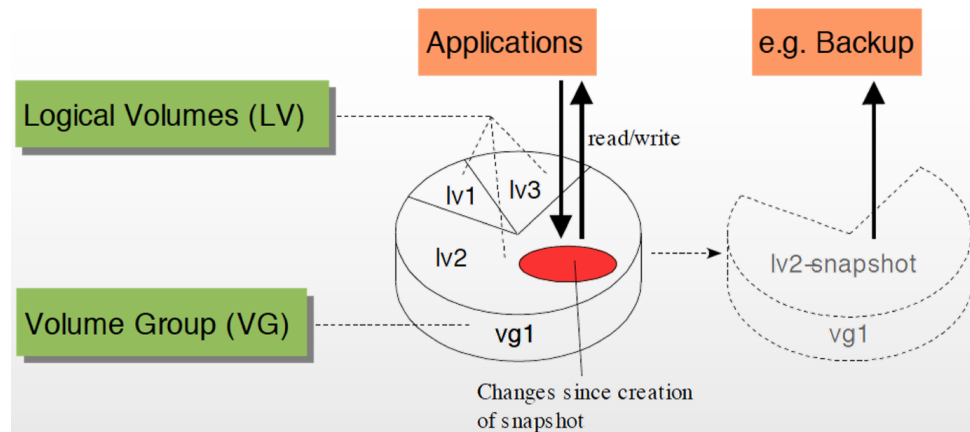


Figure 12.5: LVM - snapshot

12.5 LVM - striping e concatenazione di volumi

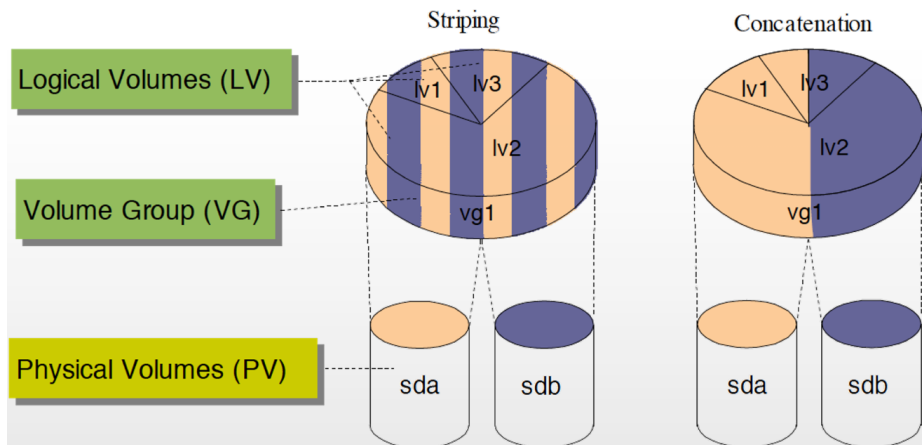


Figure 12.6: LVM - striping

12.6 LVM - mappatura

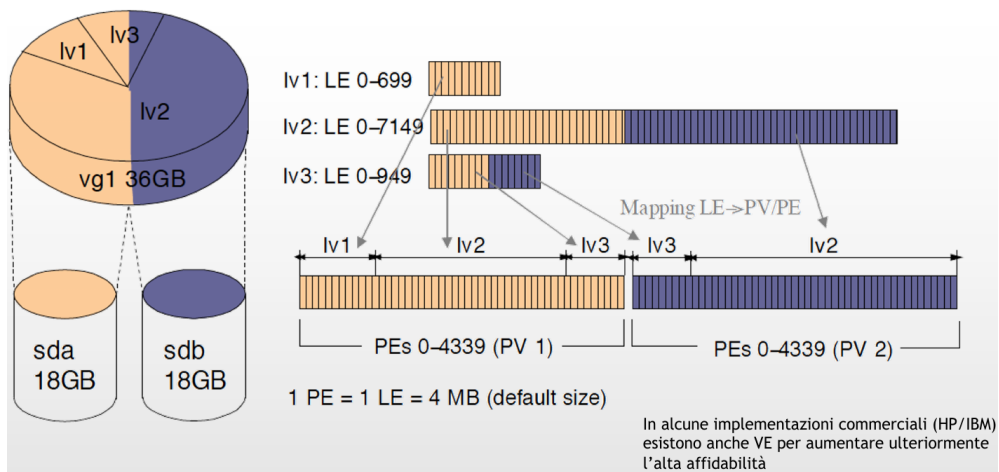


Figure 12.7: LVM - mappatura PE - LE

12.7 Logical Volume Manager in NAS

E' implementato nei nas, ed e' l'unico motivo dell'esistenza del os che gestisce i dischi ed esporta il logical volume manager in rete. I nas di casa sono dei giocattoli (soprattutto la QNAP) perche' ci crea un unico Logical Volume con un'unica partizione e forse ci permette di scegliere che tipo di RAID vogliamo.

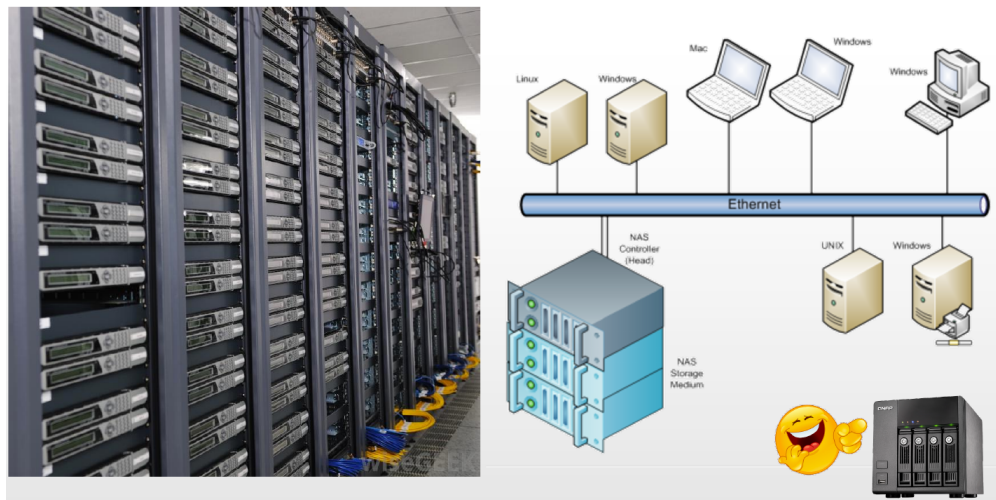


Figure 12.8: LVM in NAS

12.8 Dimensione dei blocchi di dati

Dobbiamo decidere qual'e' la dimensione ottimale da dare ai data block, ed anche qui si fa un lavoro spannometrico come nelle pagine di memoria. Abbiamo due trend, sull'asse delle x la dimensione del data block, da una parte abbiamo l'overhead (lo spazio che buttiamo via per frammentazione interna), dall'altra parte abbiamo il datarate, ovvero sia un throughput che possiamo associare al sistema. All'aumentare della dimensione del datablock ho una frammentazione interna sempre piu' forte (e voglio tenerla bassa), dall'altra parte ho le prestazioni perche' se ho un datablock piccolo per leggere i dati devo fare tanti accessi a disco quindi voglio tenere il datablock grande, ma non troppo grande. Quindi le prestazioni di accesso migliorano se il datablock e' grande e lo spazio sprecato migliora se e' piccolo. Dice che la cosa migliore e' beccare un datablock che sta nell'intersezione dei due grafici (64 Kb). Pero' attenzione ideale per lui perche' oggi usiamo datablock di 4k al massimo 16k.

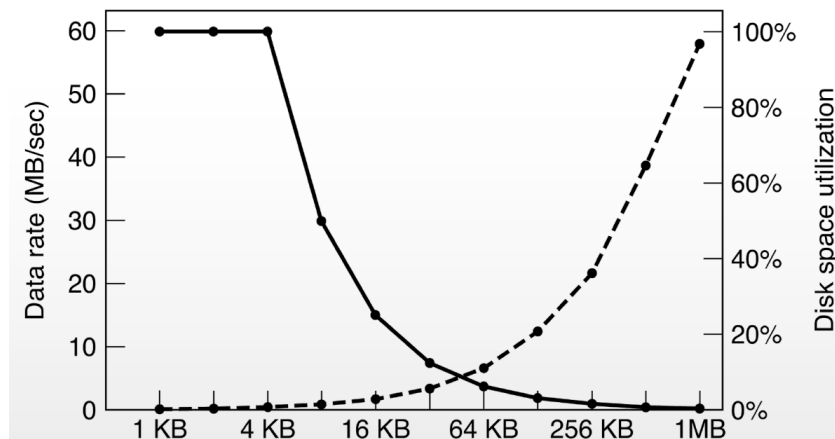


Figure 12.9: Dimensione dei blocchi di dati

12.9 Gestione blocchi liberi

Ho dei blocchi liberi ed occupati, ho bisogno di usare un datablock. Posso scandire come B la bitmap e beccare il primo libero. Se no per andare un po' piu' veloce costruisco delle liste concatenate di datablock (A), per gestire la lista di datablock liberi uso dei datablock a mia volta che risulteranno occupati, quindi quando ne cerco uno libero cerco il primo della lista. Ha dei forti effetti di bordo, se il primo datablock della lista lo sto usando ce lo avro' in memoria (non lo tengo su disco) se ce l'ho in memoria pero' a meta' strada (50 dalla lista e 50 libero) sono a posto, il problema arriva quando e' quasi vuoto o libero, perche' a volte aggiungo e tolgo tanti datablock a volte uno.

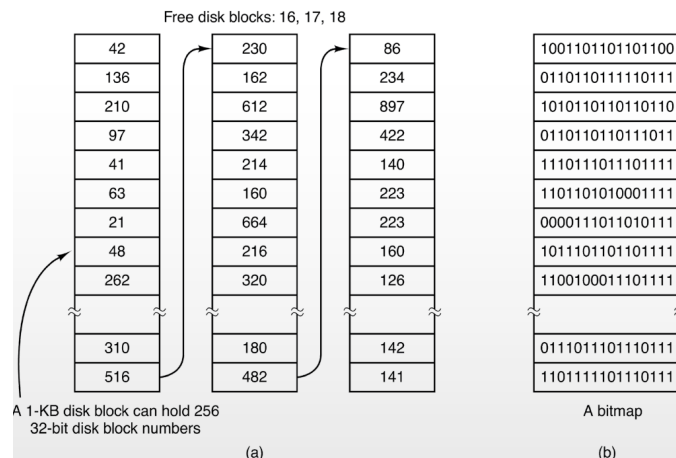


Figure 12.10: Gestione dei blocchi liberi

12.9.1 Gestione della free list in memoria

In A e B ho prestazioni deludenti, quello che si fa e' mettersi in C. Accetto di un datablock associato ad una lista a meta' ed uno uguale in memoria ed ogni tanto li pareggio per evitare il problema di bordo.

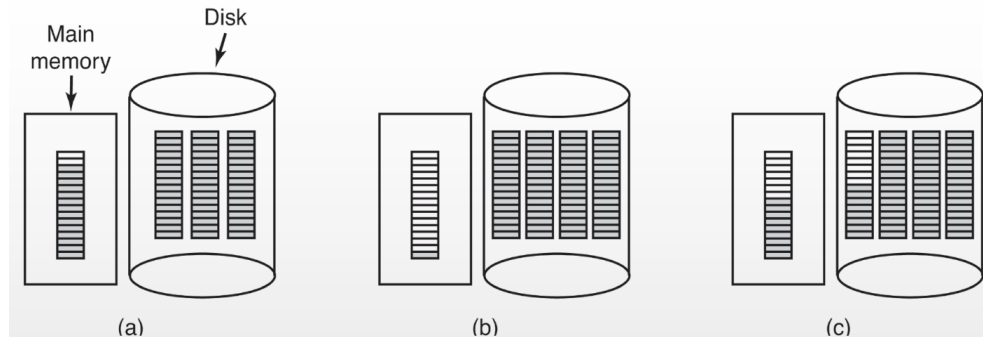


Figure 12.11: Gestione della free list in memoria

12.10 Analisi della consistenza

Il crash di un os puo' succedere e dicevamo che in pratica il problema sono i dati che erano stati applicati dal file system in memoria ma non applicati su disco, ho il dovere di controllare che tutte le strutture dati sul disco siano consistenti. Unix fa tante operazioni, sono almeno 7, ci concentriamo solo sull'analisi di consistenza dei datablock, voglio sapere se e' allocato se e' libero e se e' allocato che lo sia soltanto ad un file (perche' datablock puo' appartenere a solo 1 i-node). Costruisco due array, il primo con i datablock occupati e li tiro fuori facendo una scansione di tutti gli i-node (che fanno riferimento ad un file) e guardo a che datablock stanno puntando, tutti questi datablock almeno vengono spuntati come in uso. Dopo di che guardo la free list e costruisco un secondo array, guardo tutti i datablock elencati nella free list e li marco come liberi, alla fine se il mio file system e' consistente confronto questi due array e si devono incastrare come un pettine. Quindi mi trovo per ogni datablock che o e' occupato o libero (deve esserci 1 da una parte e 0 dall'altra o viceversa) se questo non succede ho un'inconsistenza, come nella situazione B nella posizione 2 in cui ho 0 e 0 si chiama blocco orfano, non so se e' libero o occupato, questa e' di facile soluzione perche' se non e' usato da nessun file era usato dal pezzo di free list in uso quando il sistema e' crashato, lo aggiungo ai blocchi liberi. Se fosse 1 e 1, se e' all'interno di un file vuol dire che l'ho usato ma non ho fatto in tempo a depennarlo, quindi lo tolgo dalla free list. Il caso noioso e' quello in cui ho trovato due volte lo stesso datablock all'interno della free list (perche' ho inconsistenza grave), devo andare a ritoccare le free list (modifico un informazione sul disco mentre lo sto riparando, mi piace poco) vuol dire pero' che era veramente libero quindi

lo rimetto a 1. Il caso quello piu' problematico e 'quello in cui l'ho trovato due volte referenziato da due i-node diversi (mi compare in due file) e siccome non ci possono essere due datablock condivisi tra file devo trovarmi un datablock libero davvero, come nel caso D in posizione 5 dell'array, quindi devo prendere il contenuto del datablock 5 e lo sdoppio, do una copia al primo file ed una al secondo file, pero' devo segnalare all'utente che questi due file potrebbero avere un contenuto inaspettato perche' potrei aver corrotto la semantica dei dati all'interno del file (per il file system sono a posto pero'), quindi devo costruire anche un sistema per gestire questi file che cambiano e dato che l'utente non e 'davanti alla console prendo i file e li sposto in una directory speciale lost and found, mettiamo i due file dentro li' e diciamo all'utente di guardarli perche' non siamo in grado di garantire che il contenuto sia corretto.

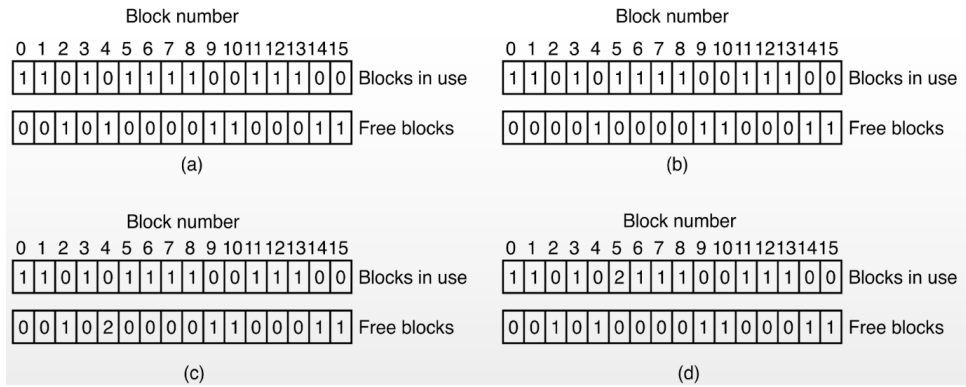


Figure 12.12: Analisi della consistenza

12.11 Disk caching

Ho anche la necessita' di gestire una cache del disco, ci sono dei datablock modificati questi rimangono in memoria fintanto che faccio il ceck point (journal file system

), devo gestire in maniera efficiente una struttura dati per prendere il datablock in memoria subito, la hash table gestisce una lista concatenata doppia ed ad ogni livello mi trovo a gestire dei blocchi e la lista concatenata doppia la gestisco secondo una politica LRU (come nelle pagine di memoria), perche' nel momento in cui modifico datablock mi devo ricordare che e' appena stato modificato e potrebbe essere modificato successivamente quindi lo metto in coda alla lista e sara' l'ultimo ad essere messo sul disco.

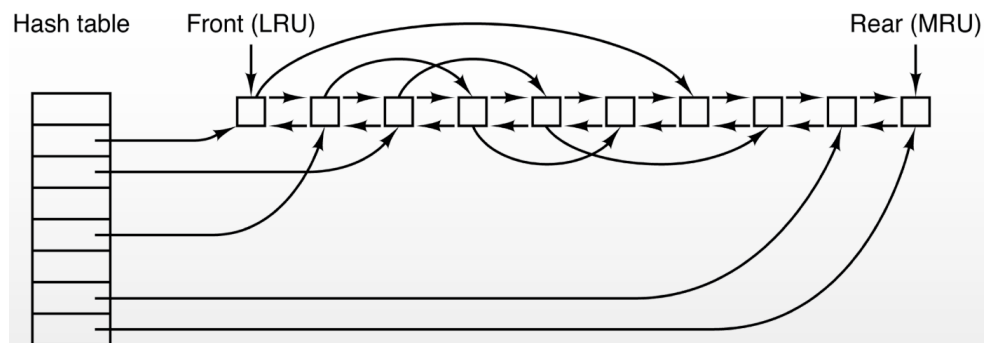


Figure 12.13: Disk caching

Chapter 13

Input/Output

Andiamo a capire tutto quello che succede nel kernel nel momento in cui devo fare transitare dei dati da memoria a periferica e viceversa, non e' particolarmente complicato perche' non e' cambiato tanto nel tempo.

13.1 I/O Hardware

Quando parliamo di hardware per IO usiamo delle keyword:

- Dispositivi: device controller e device driver. Si tende ad avere controlli piu' unificati possibili (USB universal serial bus ci attacchiamo tutto quello che funziona un carattere alla volta). Alcuni dispositivi hanno un buffer e quindi li utilizzo un blocco alla volta (block, device), come mai ho un USB e lo uso per un dispositivo a blocchi come puo' essere un disco? mi fa un adattamento ed il trasferimento di un carattere alla volta va cosi' tanto piu' veloce del disco che non mi rendo conto di avere un collo di bottiglia, e' nato inizialmente per le stampanti e poi a tutte le periferiche che ragionano un carattere alla volta
- Controller e Adapter che e' un dispositivo hardware che fa un collegamento elettrico tra due tipi di bus diversi (tra bus di memoria e bus di disco ad esempio)

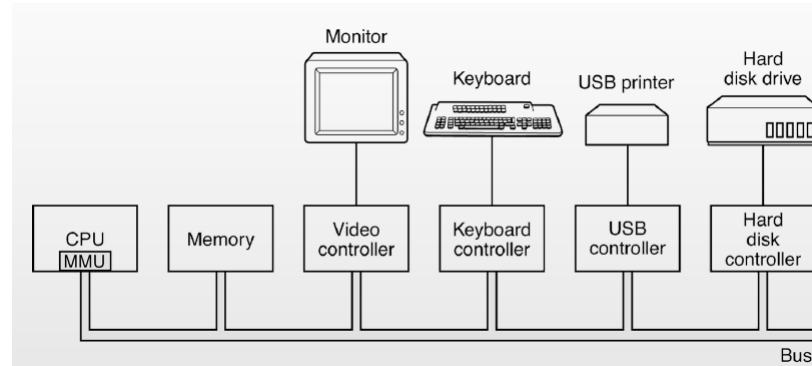


Figure 13.1: I/O Hardware

13.2 IO programmato

I/O e' prendere un informazione nella memoria e scriverla in una periferica: Inizialmente si faceva in modo programmato (molto grezzo), il problema che si verificava era ad esempio quando facevo un print di un file con msdos e fintanto che la stampante andava mi si bloccava il sistema e non potevo fare altro. Prevede che il processo sposti tutta l'informazione da stampare (blocco

di informazioni) dallo spazio utente all'interno del kernel (da A a B in figura). Il kernel manda il primo carattere alla stampante e poi chiede continuamente alla stampante se ha finito di stampare quel carattere, quindi si faceva busy waiting (che come visto per i thread concorrenti non vogliamo mai fare), ed andava avanti così fintanto che finiva il blocco di informazioni da trasferire. Il risultato è che il sistema è utilizzato male. Finito l'epoca dell'MSDOS ci siamo inventati qualcosa di meglio, ma serviva una struttura che nel sistema di calcolo ci desse una mano, quindi i processori hanno cominciato a fare discorso di IO mappato in memoria.

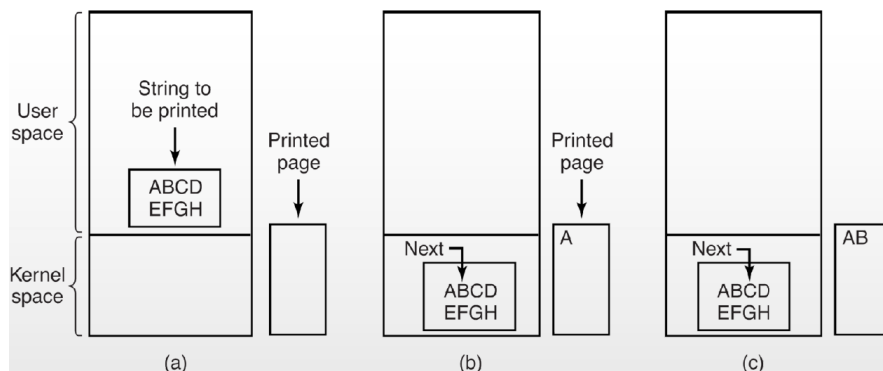


Figure 13.2: I/O Programmato

13.3 Memory Mapped I/O

Non richiedo più al kernel di interagire direttamente con la periferica (non deve sapere come si parla con la stampante, ci pensa un altro pezzo di software) ma il kernel deve depositare la roba da stampare nel punto giusto della memoria, da cui faccio partire una specie di trucco: prendo una zona della memoria e la faccio coincidere con il buffer e tutti i vari flag di funzionamento di un dispositivo, quindi scrivere in una determinata zona di memoria vuol dire scrivere all'interno del registro di un controllore di un dispositivo e la traduzione me la fa il device driver. Ovviamente non ci siamo messi d'accordo su come si fa questo controllore, qualcuno ha detto che è meglio utilizzare due spazi di indirizzamento separati (spazio dove metto i dati, spazio dove deposito i dati per le periferiche) i due non si vedono e non si toccano ed ho meno bug, come vediamo nell'immagine A. Il secondo spazio lo chiamo spazio delle porte, che mi serve per contattare tutte le periferiche, il problema è che la CPU deve avere a disposizione una microistruzione per andare a mettere e leggere dati sullo spazio delle porte. Intel ha deciso di fare la scelta scellerata e l'istruzione si chiama IN (come interrupt). Però questo approccio permetteva soltanto al kernel di interagire con lo spazio delle porte. Un'altra corrente era di tipo B, anche i processi devono essere in grado da soli di interagire con la periferica, per non

tirare in mezzo il kernel e migliorare le prestazioni (ad esempio un microfono). Allora la seconda possibilit  e' usare un solo spazio di indirizzamento e tenere (la parte alta) collegata alle periferiche, ogni periferica ha un suo sistema per farsi riconoscere (universal plug and play) e si fa riconoscere da sistema quando accendo la macchina e viene segnato un indirizzo. Ovviamente per questioni di retrocompatibilit  si usano tutti e due i metodi (C in immagine), perche' qualcuno preferisce accedere in maniera diretta alla periferica e qualcuno preferisce passare dal kernel, quindi quello che troviamo negli os moderni e' la modalit  C, in cui abbia la periferica mappata sia nello spazio del processo sia nello spazio separato.

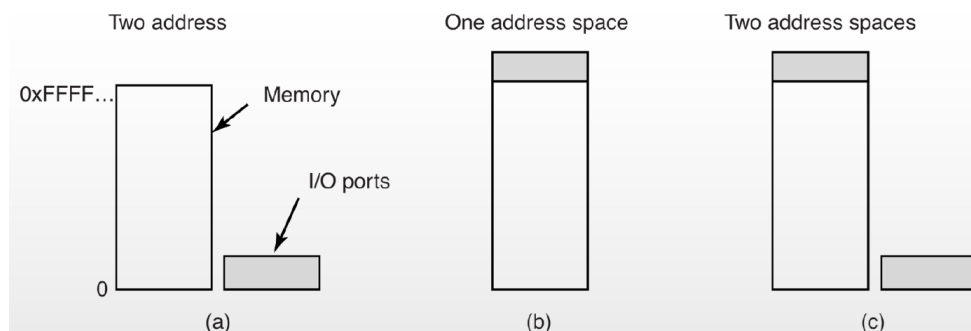


Figure 13.3: I/O memory mapped

Windows fa sta roba qui, ad esempio nell'immagine selezionando la scheda di rete mi da i valori di IO range (indirizzi occupati da quella periferica nello spazio di memoria) e memory range (l'equivalente dello spazio di indirizzamento separato), IRQ (Interrupt request, vuol dire che questa periferica nel momento in cui deve fare qualcosa lo dara' in autonomia perche' tanto i dati sono gia' dentro e quando avra' finito di lavorare solleva' un interrupt e comunichera' che sara' lei ad aver sollevato l'interrupt perche' mettera' 19 sul BUS dati, mette questo numero perche' ricordiamo che l'interrupt e' un bit quindi non sarebbe sufficiente per specificare da quale dispositivo e' partito.

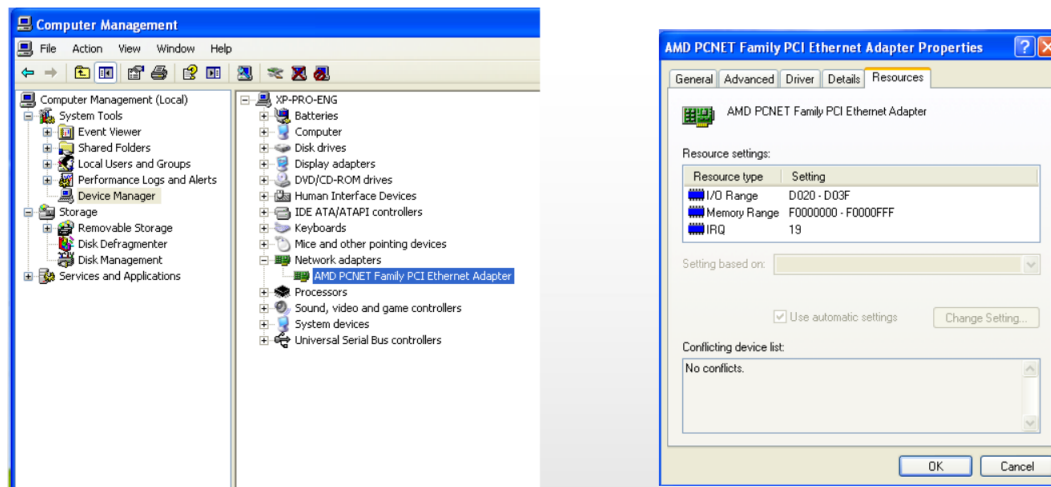


Figure 13.4: Esempio Windows XP

Su Windows 8.1 e' la stessa situazione solo che ho un os a 64 bit quindi avremo tutti i numeri codificati su piu' byte.

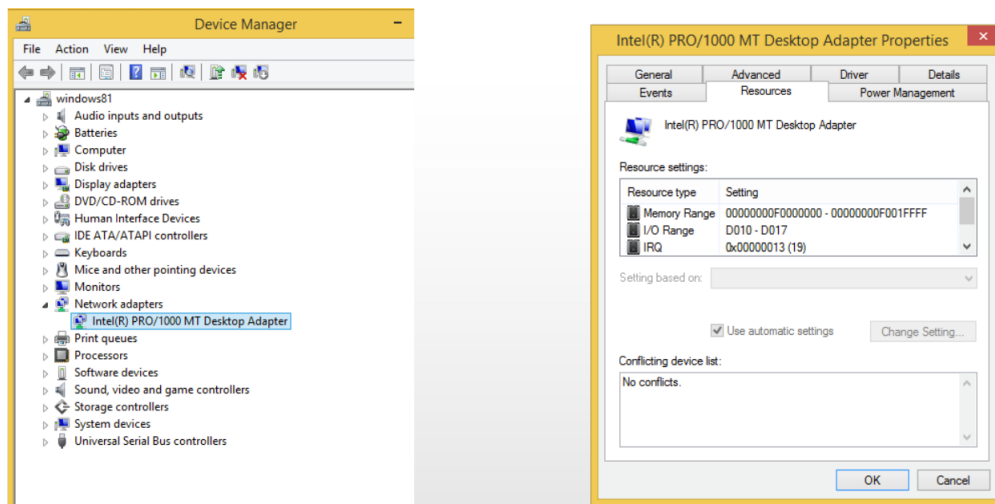


Figure 13.5: Esempio Windows 8.1

Se vogliamo andare su Windows 10 prendendo la scheda di rete aldila' della rienumerazione permette di piazzare le schede di rete in piu' buffer consecutivi all'interno della memoria.

L'hardware ci permette di fare questa roba qui, posso fare I/O programmato anche se ho I/O mappato in memoria, perche' l'I/O programmato e' un approc-

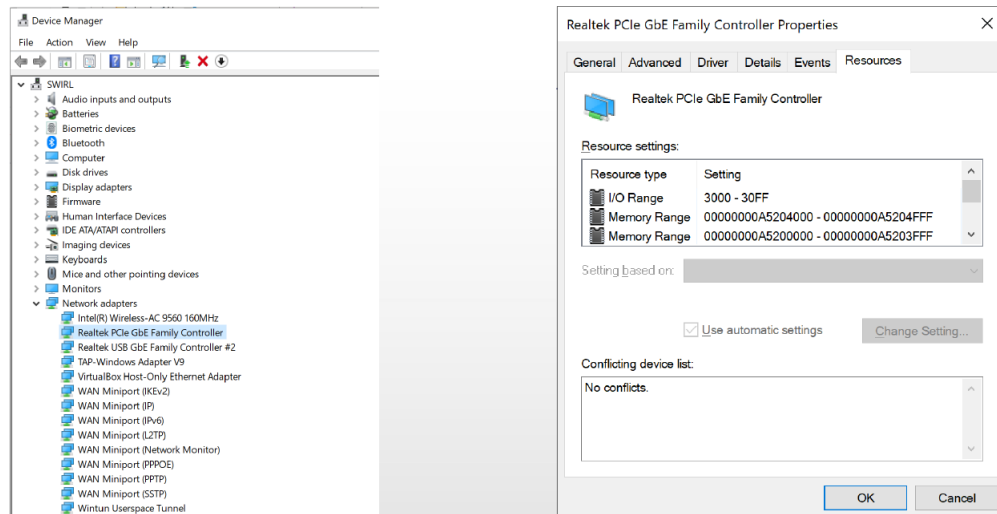


Figure 13.6: Windows 10

cio per l'uso della periferica, non diciamo che l'IO mappato in memoria e l'IO programmato sono opposti. I/O Programmato e' come lo faccio l'I/O con una periferica mentre l'I/O memory mapped e' come gestisco i dati all'interno del sistema di calcolo.

13.4 VS prestazioni

Una volta che mi permettono di fare I/O mappato in memoria posso fare delle operazioni che mi migliorano le prestazioni. Un Bus solo per tutto il sistema puo' essere restrittivo, perche' quando vado a preparare delle operazioni di I/O vorrei poter spostare tanti dati, e metterli tutti sul BUS me lo bloccherebbe, quindi di solito usiamo un memory bus che e' un bus molto corto dal punto di vista fisico (per cui ha un'elevata larghezza di banda) e uso questo per tutte le comunicazioni che vanno dalla CPU alla memoria mentre uso il bus di sistema per tutto il resto.

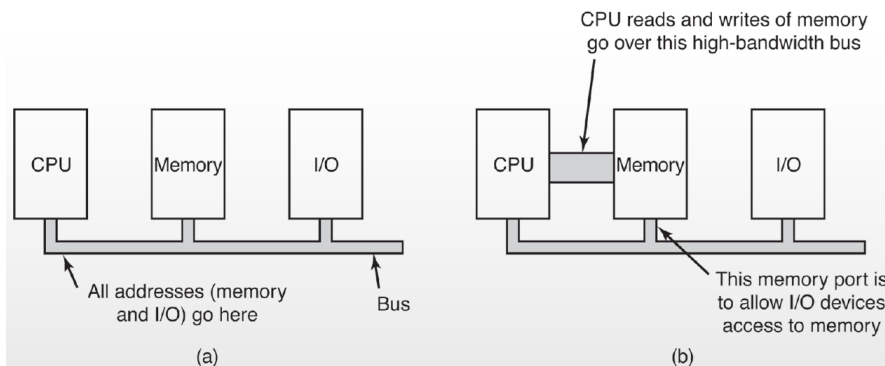


Figure 13.7: Memory Bus

Preso questa architettura posso cominciare a fare il discorso di I/O interrupt, e' quel segnale asincrono che mi arriva dalla periferica che ha bisogno della mia attenzione, e' diversa da trap. Le periferiche che terminano il processo di I/O generano un interrupt, il sistema capisce quale periferica l'ha e va a gestire i dati. Vuol dire che la periferica ha i dati e la mia procedura e': capito quale ha generato l'interrupt andare a prendere i dati e spostarli in memoria. Mi devo accollare che il trasferimento di dati in memoria centrale lo faccio usando il bus dati. Potrei tentare di migliorare le prestazioni di tutto il sistema tentando di non andare io (CPU) a prendere i dati della periferica, quindi basta uso dell'interrupt facciamo un passo oltre.

13.5 Direct Memory Access (DMA)

Ricapitolando: la prima generazione tentava di gestire l'I/O con I/O programmato, poi siamo passati ad I/O con interrupt (aggiunto il concetto di interrupt), ad oggi lo gestiamo con I/O tramite DMA che e' una cosa in piu' messo sopra l'I/O con interrupt (vogliamo evitare che la CPU vada a prendere i dati dalla periferica). Quando arriva l'interrupt i dati devono gia' essere in memoria, aggiungo quindi il DMA controller. La CPU deve fare I/O, ma non lo chiede alla periferica ma si rivolge al DMA controller e da in carico una richiesta di I/O (mi servirebbe un operazione di IO da questa periferica per tot byte e quando mi arriva l'interrupt i dati devono gia' essere nella porzione di memoria che voglio). Il DMA controller ha una serie di registri interni (per gestire piu' richieste contemporaneamente, buffer). Potrei chiedere un trasferimento dati molto grandi, ma il buffer del disco e' piccolo e quindi occorra' fare le operazioni piu' volte (fatto tramite contatore scalando, ovvero se devo trasferire file molto grandi devo spezzarli e richiedere n trasferimenti, dove n e' il numero di parti in cui ho diviso il file originale). Vediamo i passaggi dell'immagine:

1. La CPU si rivolge al DMA controller per fare I/O

2. Il DMA controller va ad istruire il controller della periferica e si dice al disco che e' abilitato DMA di fare operazioni di I/O, istruendolo subito in che posizione di memoria mettere i dati, fa quello che deve fare.
3. Il disco scrive in autonomia il RAM i dati nella posizione che gli aveva detto il DMA controller.
4. Il disco dice al DMA tramite ACK (Acknowledge) che il lavoro l'ha fatto, a questo punto sta al DMA capire se deve fare fare il disco altre operazioni (ad esempio se deve trasportare tanti dati). Quando il DMA controller vede che il suo lavoro per la periferica e' finito sara' il DMA controller a mandare l'interrupt alla CPU e con questo interrupt sa che i dati che ha richiesto sono tutti nella porzione di memoria in cui servono.

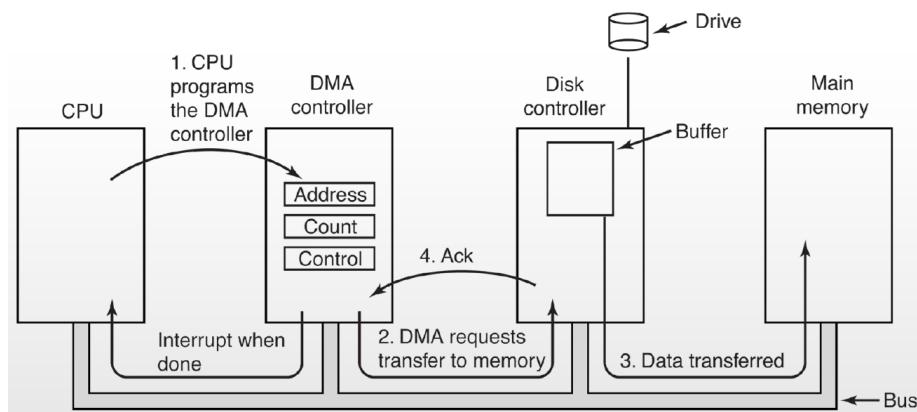


Figure 13.8: DMA

Ci sono due punti critici:

- Il passaggio 3 di trasferimento dati: Il disk controller come trasporta i dati in memoria? Se sono tanti dati passando dal Bus di sistema potrei intasarlo. Possiamo portare dell'ottimizzazione, ci sono due modalita' con cui il controller va a trasferire dati in memoria:
 - Burst: entra in competizione con le altre periferiche per usare il bus ed una volta che lo ha a disposizione fa una raffica di dati per trasferirli in memoria centrale. (li trasferisce tutti insieme)
 - Cycle Stealing: rubo un ciclo di utilizzo del bus, il controller guarda il bus ed anziche' prenotarlo cerca un periodo temporale e quando vede che c'e' uno slot libero ci infila dentro un po' di suoi dati, non tutti, e poi si rimette a guardare quando il bus e' ancora disponibile ed alla fine piano piano trasferisce tutto. Va bene, ha delle prestazioni inferiori dell'approccio a Burst ma mi permette di disturbare molto meno il meccanismo del BUS

- Interrupt: le periferiche all'interno del sistema di calcolo sono tante, ognuna di loro fa operazioni di I/O ed alcune funzionano in DMA (ovvero riescono a mettere da sole i dati in memoria senza chiedere al kernel) ed altre no, quando genero un interrupt in realta' la CPU non ne riceve uno per volta, c'e' 'una specie di tempesta ed attacco alla diligenza perche' sono segnali elettrici, non posso metterli in coda, quindi serve l'interrupt controller, che si frappone tra la CPU ed il resto del mondo, perche' ha un piccolo buffer in cui riceve gli interrupt da periferiche se li tiene al suo interno e li smista uno per uno alla CPU. Per cui si occupa del fatto che non si sovrappongano gli interrupt, se un interrupt e' in elaborazione quello dopo aspetta un attimo. Mi va a penalizzare per alcuni aspetti, se la periferica genera dati in tempo reale ed ho una grossa coda di interrupt qualcosa poteri perdere perche' il buffer del controller e' limitato. Aiuta la CPU a non perdere segnali elettrici, il problema e' che anche la CPU oggi e' complicata e con la sua complicazione abbiamo anche dei problemi sull'interrupt.

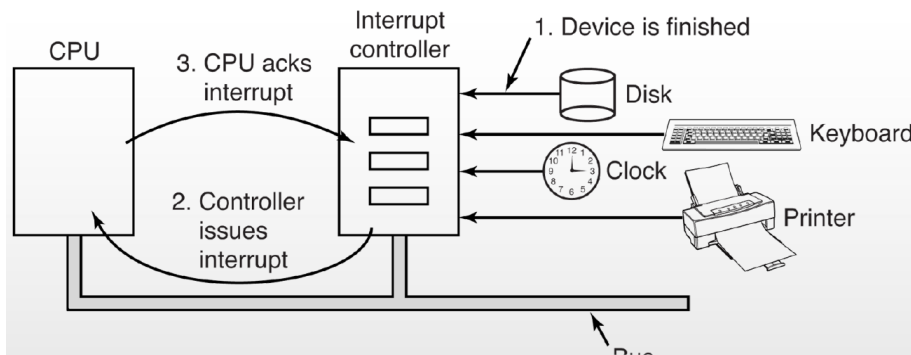


Figure 13.9: Interrupt Controller

13.6 Interrupt Preciso

Cos'e' successo alle CPU, sono passate da architettura a single pipeline a multi pipeline, c'e' una linea di lavoro che fa il fetch e comincia a fare il decode e c'e' un'altra pipeline che comincia a fare il fetch di quella dopo e quando una va in esecuzione ce n'e' un'altra che fa il decode. Riescono ad avere anche 16 pipe in parallelo, cosa succede quando ho un if? Di fatto non sappiamo piu' in che situazione e' la CPU, diventa un sistema logic fuzzy (ovvero un sistema in cui non ho piu' solo 1 e 0, ma gestisco la probabilita' che sia vero in percentuale, tipo e' vero all'80 % dei casi), non sappiamo cosa sta facendo, non sappiamo neanche quanto vale il program counter.

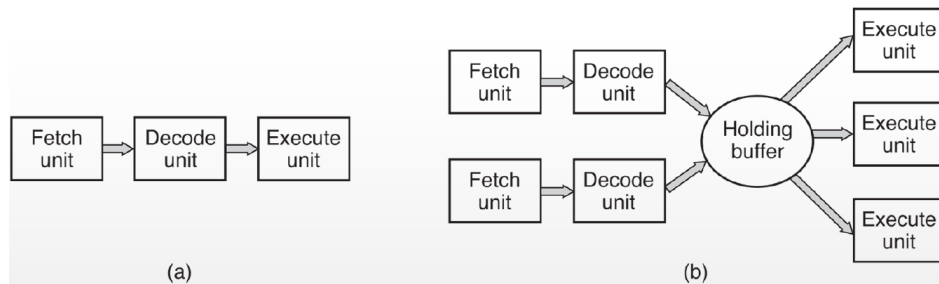


Figure 13.10: Pipeline

Quando siamo in multi pipeline l'arrivo di un interrupt e' un momento di indecisione, parliamo di **interrupt preciso** quando arriva e trova la CPU in uno stato ben determinato (una configurazione certa), in che condizione si riceve un interrupt preciso:

- quando arriva devo mettermi in condizione di precisione perche' il program counter so quanto vale
- Tutte le istruzioni che sono prima del program counter sono eseguite completamente (raggiunta e terminata la fase di execute)
- Tutte le microistruzioni che si trovano al dila' (in posizione successiva del program counter) non sono state eseguite per nulla (neanche il decode)
- Per quanto riguarda l'istruzione che sto puntando con il program counter e' in uno stato ben determinato (abbiamo problema di quote delle pagine, qual'e' il parametro che mi ha generato il page fault per fare roll back)

13.6.1 Interrupt precisi o meno

Se ho interrupt preciso mi trovo nella situazione A, perche' e' l'unica situazione in cui posso fare foto del os e metterla nel process control block. Se non e' preciso siamo in B ho il program counter che contiene sempre 316 e poi tante istruzioni eseguite un po' si un po' no, notiamo che non sono neanche state eseguite in ordine, capiamo che se la situazione e' la B l'interupt non riusciamo mai a gestirlo.

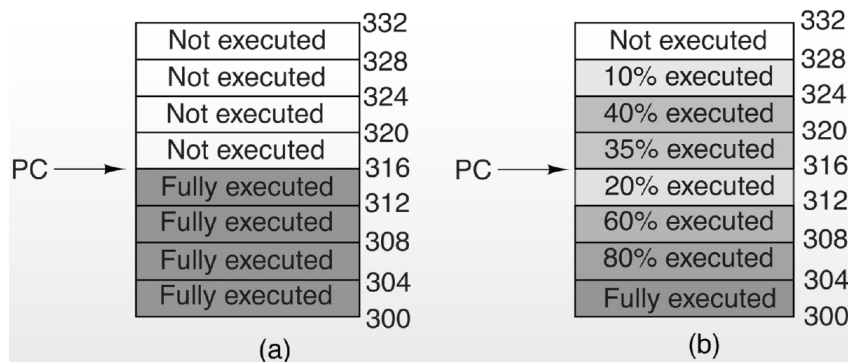


Figure 13.11: Interrupt precisi o meno

13.7 I/O Software

Fino a qui abbiamo parlato di problematiche hardware ma come al solito qualcuno deve scrivere i software per usare questo sistema. Abbiamo però dei pilastri su cui tutti quanti devono appoggiarsi;

- Indipendente dal dispositivo: il processo non vuole avere una procedura diversa di I/O per ogni dispositivo ma avere una system call alla quale al massimo passo dei parametri e fa tutto indipendentemente dal dispositivo (UNIX dice che abbiamo 4 system call: open, read, write e close, in UNIX inoltre qualunque dispositivo che noi abbiamo e' in realta' un file e viene gestito con queste system call). Qualunque dispositivo lo trovo nella directory /dev, e vale soprattutto per le I/O tant'e' vero che per avere un elenco di file aperti associati ad un processo posso usare il comando lsof (list of file). Facciamo I/O basato su file dopo di che tutto viene riportato su file. Windows fa una scelta diversa internamente il kernel di Windows ha una grossa parte progettata ad oggetti n(C++), prevede la creazione di un oggetto il quale diventa proprietario di una periferica, questa e' una dei motivi per cui a parita' di pc linux va piu' snello nonostante i dischi meccanici.
- Nomenclatura uniforme, non posso accedere a periferiche diverse con system call diverse
- IL software deve essere in grado di gestire errori, ma non e' come una libreria di un linguaggio, le chiamate sono tutti asincrone, faccio la chiamata e dopo un po' la periferica ma da la risposta, potrebbe dipendere da problematiche software sia da problematica hardware (zona del disco non leggibile), tipo quando ci si incartano le schede sd perche' una parte e' diventata elettricamente non affidabile e os non e' in grado di gestire correttamente l'errore e ci riprova finche non scade il time out.

- Sincrono/asincrono: puo' essere sincrono o asincrono. Sincrono: quando chiamo una funzionalita' e aspetto fino a quando ricevo la risposta. Asincrono: richiedo una funzionalita' faccio altro e poi ricevo la risposta. IO di os moderno e' asincrono ma il software del sistema IO e' sincrono.
- Un sistema di IO deve prevedere dei buffer perche' portano le prestazioni a livelli accettabili. Per la questione della gerarchia di memoria e' quello che ci fa guadagnare tempo piu' di tante altre cose.

13.8 Stratificazione IO

Non realizziamo mai i sistemi di I/O all'interno dell'os in un blocco solo, costruiamo delle librerie che si appoggiano una sull'altra e si specializzano in una tipologia di utenti:

- Software di IO che dialoga con gli utenti (shall)
- Scrive una libreria di os che e' indipendente dal dispositivo, ed e' tipicamente l'implementazione delle system call
- Ci sono i device driver all'interno del dispositivo, sono add on che mettiamo dentro, soltanto chi costruisce il dispositivo sa come funziona al suo interno e sa come implementare i device driver
- Device driver parlano con il dispositivo ed a meta' ci sono i gestori dell'interrupt

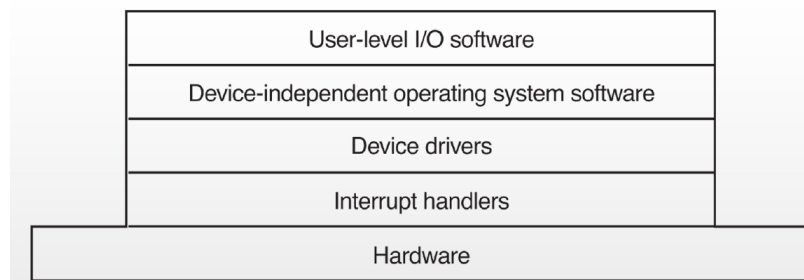


Figure 13.12: Stratificazione I/O

13.8.1 Gestione Interrupt

Abbiamo visto la gestione delle trap e di fatto questa e' soltanto una generalizzazione di quello che abbiamo visto per l'interrupt di clock, si parla di ISR (interrupt service routine):

- Metto in sicurezza la CPU, salvando ogni registro (include il PSW) che non era gia' stato salvato

- Preparo il contesto, metto in sicurezza il processo, perche' devo dare il contesto esecutivo all'interrupt che non mi vada a distruggere quello che stavo facendo (TLB, MMU, page table, etc)
- Arrivato l'interrupt lo gestisco (per mettere in coda gli altri interrupt, set up a stack)
- ACK the interrupt controller
- Copy saved registers to the process table
- Mi preparo a gestire l'interrupt ed eseguo la ISR (interrupt service routine): ho vettore di risposta per gli interrupt (in cui ci sono scritti i codici delle periferiche e l'indirizzo delle istruzioni a cui saltare per gestire l'interrupt), guardo sul bus dati e c'e' scritto qual'e' la periferica che mi ha generato l'interrupt e uso quello come indice nel mio vettore di interrupt per capire qual'e' il codice da eseguire per gestirlo. E' il motivo per cui e' importante non avere sovrapposizioni a livello di interrupt. Quando la ISR termina l'interrupt l'ho gestito.
- Faccio ripartire il processo che era in esecuzione prima dell'interrupt, ma per farlo devo scegliere quale processo mettere in run, perche' devo smettere di utilizzare la CPU se e' arrivato un interrupt di cui io non centro niente?. Quando termino l'ISR devo interrogare i sistemi di scheduling, perche' potrebbe essere che il processo che quell'interrupt ha fatto passare da bloccato a pronto ha una priorita' piu' alta della mia quindi io smetto di usare la CPU e quello mi va a rubare il mio quanto di tempo perche' quel dato potrebbe essere particolarmente importante per il sistema. Deciso se la CPU va al processo interrotto o a qualun altro
- Carico i nuovi registri del processo, compreso il PSW
- Run del nuovo processo

13.9 Device driver

Ne abbiamo gia' parlato, pezzi di software che ci forniscono i produttori dei device per gestire gli stessi (driver delle schede video Nvidia).

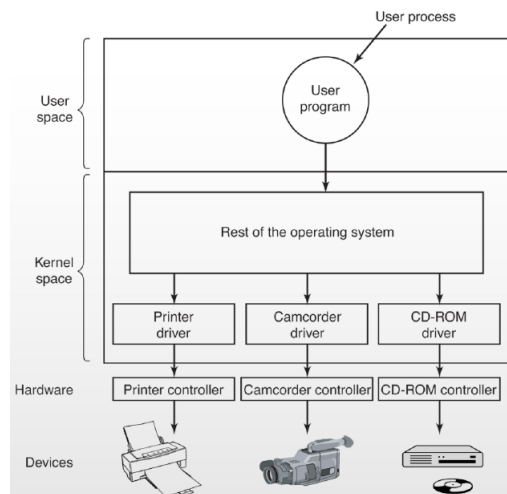


Figure 13.13: Device Driver

13.10 Sistema di Buffering

Area di parcheggio dove appoggio i dati e a parte gli aspetti prestazionali ho dei dati che devono essere consumati per tempo. Il libro fa degli esempi con degli apparati di rete, la scheda di rete va usata subito, perché i pacchetti della rete vengono messi in buffer e se il kernel non è veloce la scheda di rete prende il prossimo pacchetto e lo sovrascrive, quindi ho perso un pacchetto. L'altro problema è che quel pacchetto potrei utilizzarlo più di una volta mettendolo in un sistema di riciclo.

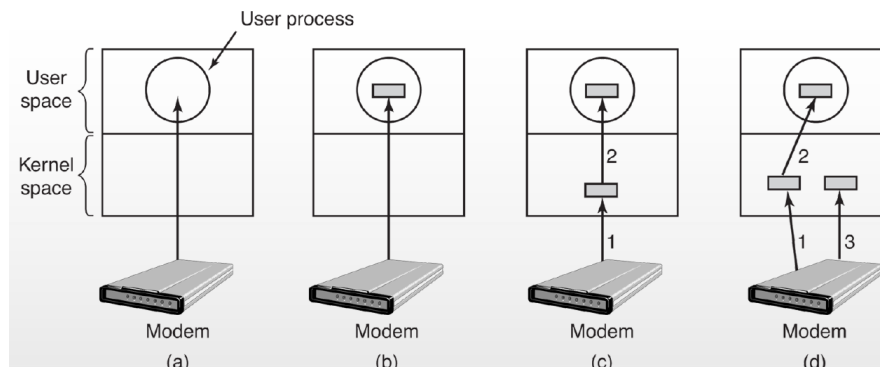


Figure 13.14: Buffering Modem

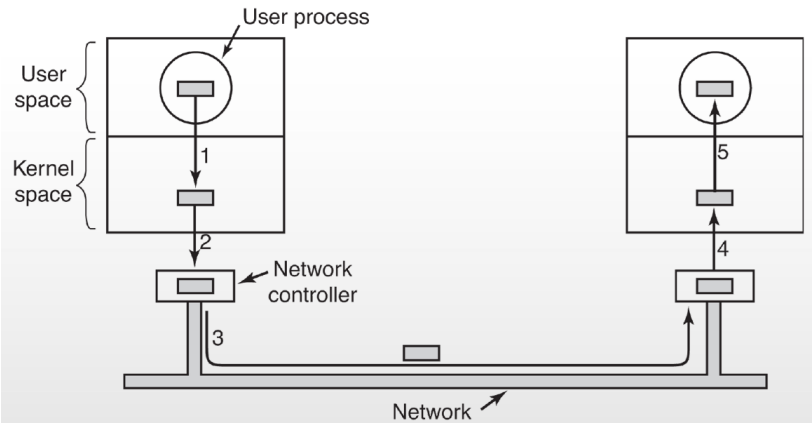


Figure 13.15: Buffering scheda di rete

Chapter 14

Deadlock

Abbiamo introdotto il concetto di semafori per garantire la mutua esclusione per una risorsa, ma attenzione perche' anche se uso un semaforo potrei avere il problema di portare il sistema in una fase di stallo, perche' ho scritto il codice senza pensare alle conseguenze, l'esempio che abbiamo fatto e' quello dei 5 filosofi quando per sfortuna tutti prendono allo stesso momento la forchetta destra e nessuno puo' mangiare. Nell'esempio il codice si riferisce al produttore consumatore senza l'uso dei semafori abbiamo visto che non funzionava, abbiamo introdotto i semafori e da li' abbiamo parlato dei 5 filosofi.

```
#define N 100
int count = 0;

void producer(void) {
    int item;

    while (TRUE) {
        item = produce_item();
        if(count==N) sleep();
        insert_item(item);
        count=count+1;
        if(count==1) wakeup(consumer);
    }
}

void consumer(void) {
    int item;

    while (TRUE) {
        item = remove_item();
        if(count==0) sleep();
        count = count - 1;
        if(count==N-1) wakeup(producer);
        consume_item(item);
    }
}
```

14.1 Risorse

Qualunque risorsa all'interno dell'os ha una caratterizzazione. Quando si parla di deadlock le risorse coinvolte sono prelazionabili (un processo puo' strappare la risorsa condivisa al processo che se n'e' impossessato), esistono anche risorse non prelazionabili (quelle che non prevedono questa possibilita'). Il tipico OS prevede solo risorse non prelazionabili, solo il kernel ed abbattendo il processo possono portarmi via la risorsa. Le non prelazionabili hanno un ciclo di utilizzo:

- Vengono allocate (Request)

- Ne faccio uso (Use)
- vengono rilasciate (Release)

Tipicamente la funzione del semaforo e' mettersi davanti alla richiesta ed alla fine della release.

14.1.1 Request e release

A volte per intercettare una risorsa mi basta un semaforo (A) uso la risorsa uno e la rilascio, se pero' ho piu' di una risorsa ho bisogno di piu' semafori come nel caso B in cui ho scritto un codice che usa due risorse (ci ha gia' detto di fare gli up dei semafori in ordine inverso dei down), abbiamo gia' usato due semafori nel problema di produttore consumatore.

<pre>typedef int semaphore; semaphore resource_1; void process_A(void) { down(&resource_1); use_resource_1(); up(&resource_1); }</pre> (a)	<pre>typedef int semaphore; semaphore resource_1; semaphore resource_2; void process_A(void) { down(&resource_1); down(&resource_2); use_both_resources(); up(&resource_2); up(&resource_1); }</pre> (b)
---	---

Figure 14.1: Request Release per piu' di una risorsa

Mettiamo che qualcuno abbia fatto un errore (semplicemente perche' si e' sbagliato non perche' fosse un malintenzionato), due programmatori hanno scritto due programmi (che ovviamente poi si trasformano in processi) che operano sulla stessa risorsa condivisa, solo che il programmatore di sinistra nel processo B fa gli up in modo inverso dei down. Notiamo che il programmatore A nel processo B fa prima il down della risorsa 1 poi il down della risorsa due, l'up della risorsa 2 e l'up della risorsa 1 (che come abbiamo visto e' il modo giusto di trattare i semafori), mentre il programmatore di destra ha fatto prima il down della risorsa 2 e poi il down della risorsa 1, facendo poi correttamente l'up della 1 e l'up della 2. Cosa succede se il processo A viene interrotto prima di fare l'up della risorsa 2 e il B chiede un down del 2, il sistema non puo' evolvere e si ritrova in un deadlock. E' il os che deve dare metodologie che mi permettono di contrastare questi fenomeni.

<pre> typedef int semaphore; semaphore resource_1; semaphore resource_2; void process_A(void) { down(&resource_1); down(&resource_2); use_both_resources(); up(&resource_2); up(&resource_1); } void process_B(void) { down(&resource_1); down(&resource_2); use_both_resources(); up(&resource_2); up(&resource_1); } </pre>	<pre> semaphore resource_1; semaphore resource_2; void process_A(void) { down(&resource_1); down(&resource_2); use_both_resources(); up(&resource_2); up(&resource_1); } void process_B(void) { down(&resource_2); down(&resource_1); use_both_resources(); up(&resource_1); up(&resource_2); } </pre>
(a)	(b)

Figure 14.2: Request Release di piu' di una risorsa con errore

14.2 Deadlock

In letteratura la definizione di deadlock: Un insieme di processi e' in deadlock se ognuno di questi processi sta aspettando un evento che solo un altro processo dello stesso insieme puo' causare. Ho una situazione di questo tipo quando valgono 4 condizioni:

- Mutua esclusione: una risorsa puo' essere posseduta da solo un processo alla volta
- Hold and wait: un processo e' autorizzato a possedere una risorsa ed intanto aspetta che un'altra risorsa sia disponibile
- Non c'e' prelazione: e' tipico di molte situazioni
- Ho una condizione di attesa circolare: sto aspettando una risorsa di qualcuno che sta aspettando una risorsa di qualcun'altro che sta aspettando una risorsa che ho io

Per evitare che i deadlock non esistano nel sistema ci basta che venga eliminata una di queste condizioni.

14.2.1 Modello per il deadlock

Ho dei nodi del grafo che possono essere processi (cerchi) o delle risorse (quadrati), l'orientamento delle frecce mi dice se un processo ha acquisito una risorsa (B

sta attendendola risorsa S, A ha preso la risorsa R). Se con questo schema viene fuori un cerchio (come nella situazione C) vuol dire che c'è un deadlock. Se mi costruisco un modello fatto con un grado devo fare evolvere il sistema in modo che disegnando di volta in volta il grafo dei processi non mi trovo mai con un ciclo.

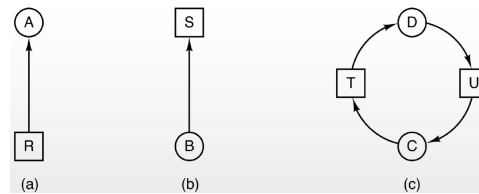


Figure 14.3: Modello per deadlock

14.2.2 Evitare un deadlock tramite scheduling

Nel momento in cui ho bisogno di risorse le metto nell'ordine giusto per non crearmi un ciclo. Primo esempio: Ci sono 6 fasi di questa schedulazione (non stiamo parlando di schedulazione di CPU, ma cosa avviene ogni volta che alloco una risorsa), la riga in alto sono la sequenza di operazioni che fanno i singoli processi (le sue intenzioni), processo B e C lavorano su S e T e T ed R rispettivamente, a sinistra (D) abbiamo la sequenza di operazioni che faccio nella schedulazione e traduco queste operazioni in disegni del grafo. Seguendo le varie istruzioni e traducendole in grafi trovo che nel momento J il sistema è in deadlock.

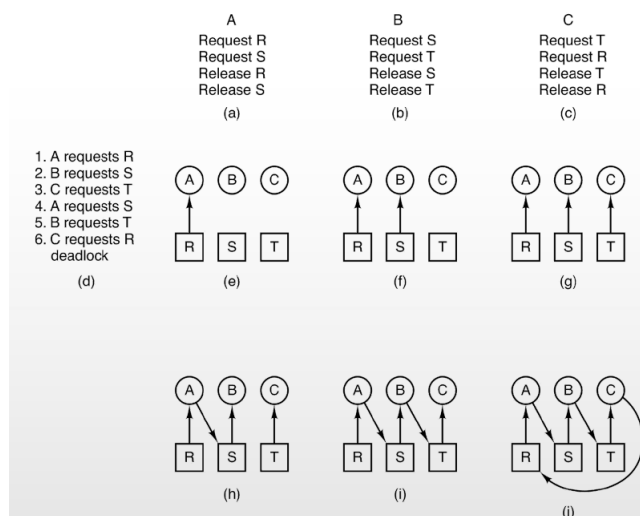


Figure 14.4: Situazione di deadlock

Per risolvere questo basta agire sullo scheduler, lasciamo liberi i processi di scegliere loro la schedulazione ma gli impediamo di andare in ordine (ABC). Può essere complicato se un processo ha tante sequenze e prevede che si sappia già l'ordine delle operazioni dei processi, inoltre in un sistema reale non sappiamo quanto tempo ci mettono i processi prima di rilasciare le risorse.

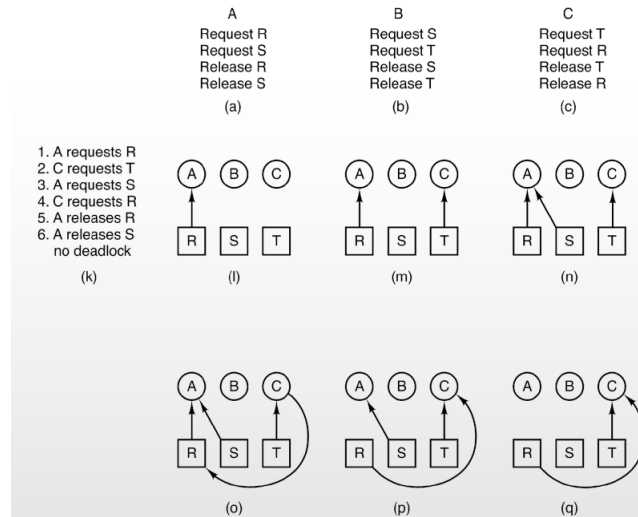


Figure 14.5: Soluzione tramite scheduling

14.3 Cosa fare

Rinuncio a risolverli con le schedulazioni, e devo adottare una policy (prendere una decisione):

- Ignorarli: algoritmo dello struzzo, posso ignorare i deadlock se non rappresentano tutto questo grosso problema per me, va contestualizzato molto bene, ad esempio se ho un sistema che per dei problemi crasha 4 volte al giorno e devo riavviarlo anche se un giorno dovrò farlo una volta in più per colpa di un deadlock non sarebbe un grosso problema. Ad oggi di sistemi che crashano molte volte al giorno non ne esistono tanti.
- Trovarli e risolverli: è la strada che prende Windows, in quanto ha dei suoi parametri di evoluzione dei processi e facendo monitoring continuo potrebbe rendersi conto che ci sono dei processi sempre bloccati e con delle statistiche parte un algoritmo che costruisce un grafo per come sono messi i processi in stato di bloccato per vedere se ci sono dei cicli (quindi scovo il deadlock) per risolverlo ci sono molte strategie potrei ad esempio prendere un processo a caso nel deadlock ed ucciderlo.

- Evitarli: cambiare il sistema in modo che una delle 4 condizioni viste prima non sia verificabile in nessun caso
- Prevenirli: quando qualcuno vuole allocare una risorsa decido se dargliela o no, perché se glie la dessi potrebbe cadere in un deadlock.

14.3.1 Come trovare un deadlock

E' semplice per noi umani, guardiamo A e noi vediamo subito il ciclo, ma come lo spieghiamo al calcolatore? Va usato un algoritmo (che chiede all'esame) che e' quello di Dijkstra (Daiestra) (applicato alle reti ad esempio, per trovare il percorso da far fare ad un pacchetto che deve andare da un punto all'altro della rete) e' un algoritmo che dato un grafo ed un nodo di partenza all'interno del grafo ci trova tutti i percorsi piu' brevi possibili per andare da quel nodo a tutti gli altri.

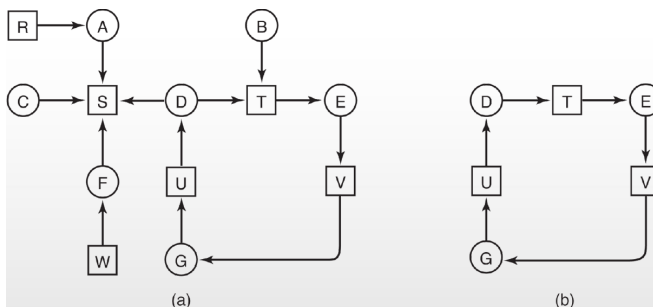


Figure 14.6: Trovare deadlock

Algoritmo di Dijkstra

E' diventato una sorta di coltellino svizzero con cui risolvere tanti problemi, noi facciamo partire l'algoritmo e gli facciamo esplorare il grafo e se nell'esplorazione torna al punto di partenza vuol dire che c'e' un ciclo ed il punto di partenza fa parte del ciclo. Partiamo cercando di capire come funziona l'algoritmo: Immaginiamo che il grafo sia un labirinto, parto da un punto del labirinto e devo esplorarlo tutto senza mai tornare al punto di partenza, dal mio punto ho dei corridoi che partono ed ho segnato quelli in cui sono gia' passato, ne prendo uno a caso di quelli in cui non sono gia' passato, mi porto al prossimo incrocio e qui ripeto la mia operazione: guardo i corridoi che ho a disposizione e ne prendo uno a caso di quelli non utilizzati, puo' capitare che ad un certo punto arrivo ad un vicolo cieco oppure arrivo ad un incrocio in cui tutti i corridoi di uscita sono gia' stati utilizzati quindi torno indietro di 1 (torno all'incrocio precedente) e rifaccio l'operazione. In pratica garantisce che se tornando indietro mano mano ritorno all'incrocio da cui sono partito ho esplorato tutto il labirinto. Ora trasportiamo questa intuizione nel nostro caso:

1. Per ogni nodo, N, nel grafo, faccio i prossimi 5 punti con N come nodo di partenza
2. Inizializzo una lista vuota L e segno tutti gli archi come non visitati
3. Aggiungo il nodo corrente alla fine di L e guardo se il nodo compare in L due volte. Se e' lo trovo due volte, significa che il grafo contiene un ciclo (di cui il nodo in cui sono adesso fa parte) e termino l'algoritmo. Se non lo trovo due volte passo al punto successivo.
4. Dal nodo in cui sono guardo se ci sono archi di uscita dallo stesso in cui non sono gia' passato, se ci sono vado al punto 5, se non ci sono vado al punto 6.
5. Prendo a caso un arco di uscita non attraversato e lo segno come attraversato. Al che arrivo ad un nuovo nodo e torno al punto 3.
6. Se il nodo in cui sono e' quello iniziale, significa che il grafo non contiene cicli e l'algoritmo termina. Se invece non e' il nodo iniziale significa che sono arrivato in un nodo senza uscite, quindi lo tolgo dalla lista L e torno al nodo precedente, rendo il nodo precedente il nodo corrente e torno al punto 3.

Parto da un nodo mi costruisco una lista L di nodi visitati e marco tutti gli archi del grafo come non utilizzati, il nodo dove sono adesso e' il nodo corrente, il nodo corrente lo aggiungo alla lista L dei nodi visitati e controllo se il nodo corrente compare due volte, se compare due volte vuol dire che sono partito da li' e quindi c'e' un ciclo, se non c'e' un ciclo passa 4 cerco un arco non marcato come gia' utilizzato uscente da quel nodo se esiste vado al punto 5 altrimenti vado a 6. 5: vado avanti, cambio il nodo corrente, marco l'arco come utilizzato ed aggiungo il nuovo punto corrente alla lista L, dopo di che torno a 3 chiedendomi se faccio un ciclo. 6: faccio il roll back, torno indietro di uno, tolgo il nodo corrente dalla lista e torno al passo numero 3. Se per caso a questo punto mi rendo conto che se facendo retromarcia sono tornato al punto iniziale il grafo e' esente da cicli, se per caso nel punto 3 trovo un punto messo due volte significa che c'e' un ciclo e questo punto fa parte del ciclo.

Questa operazione non va fatta una volta sola, perche' il sistema viene rappresentato da due grafi separati, quindi non basta prendere un nodo a caso e controllare perche' nel secondo grafo potrebbe esserci un ciclo, quindi devo ripetere l'operazione per tutti i nodi del sistema (punto 1 per ogni nodo che appartiene al grafo)

14.4 Come trovare un deadlock (risorse multiple) (Implementazione Dijkstra)

Per implementarlo prima devo creare un grafo per la rappresentazione del mio sistema, e non e' semplicissimo perche' devo fare un catalogo generale di risorse

e processi. C'è in realtà un modo più semplice che è però più astratto: immaginiamo di rappresentare il sistema con 4 oggetti matematici:

- l'array delle risorse disponibili (in alto a sinistra), quindi per ogni tipo di risorsa ho un valore numerico che mi dice quanti esemplari di questa risorsa ho.
- Dopo ci che di queste risorse un po' sono allocate ed un po' disponibili, mi segno quali sono quelle disponibili per ogni tipologia nell'array delle risorse non allocate A (alto a destra, per ogni di tipo 1 ne ho 3 da allocare).
- C'è una matrice C di n (numero processi) X m (numero tipologia risorse) che mi dice per ogni processo quante risorse ha allocato per ogni tipologia (C_{n2} mi dice che il processo C ha n risorse allocate di tipo 2) ed è la matrice dello stato del sistema
- C'è una matrice R delle risorse richieste, sono le risorse che ogni processo ha ancora bisogno di allocare (richieste future) per poter arrivare a termine, per cui il processo 1 ha allocate risorse che sono scritte nella prima riga della matrice C e per poter arrivare a termine ed essere tolto dal sistema deve ancora allocarsi le risorse elencate nella prima riga della matrice R, così via per i prossimi processi.

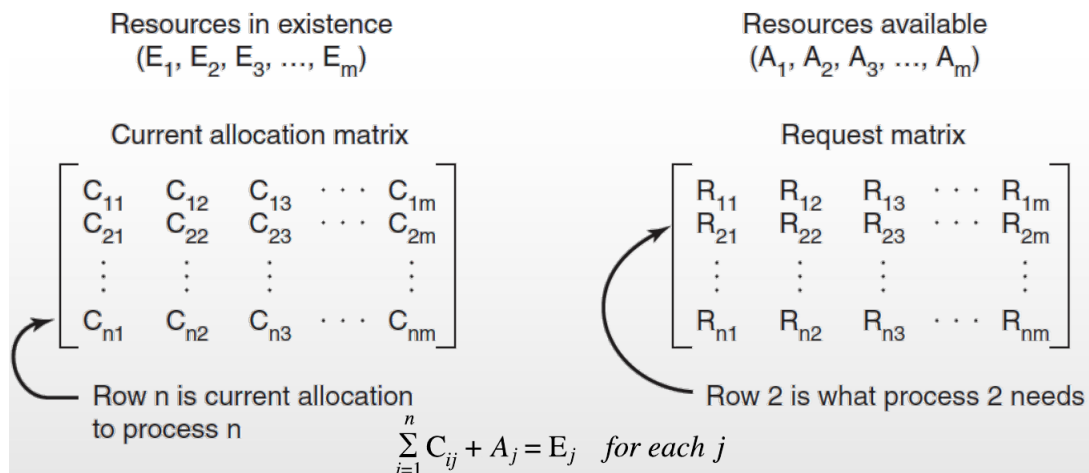


Figure 14.7: Risorse Multiple

Come faccio a capire se c'è un deadlock e simulare l'algoritmo con questa schifezza? faccio una simulazione, di questi n processi ce n'è almeno 1 per cui se gli do tutte le risorse disponibili questo riesce ad arrivare a termine? cerco una riga della matrice R che sia minore uguale del vettore A , tra vettori intende che elemento per elemento vale la disuguaglianza: $R_{11} < A_2$? e così via, se arrivo in fondo ed i due vettori sono capaci di rispettare questa disuguaglianza significa che questo processo arriva a termine se gli do tutte le risorse disponibili. Quindi facciamo finta di dargliele ed arrivando a termine libererà le risorse che ha allocate in questo momento. Quando questo processo termina indica le risorse indicate nella prima riga della matrice C , segno che il processo 1 è terminato, prendo la prima riga della C e sommo elemento per elemento alle risorse disponibili, quindi dovrei avere più risorse disponibili ed itero questa procedura per vedere se posso terminare tutti i processi. Se terminano tutti i processi posso stare tranquillo che non ho un deadlock ma se ho processi che avanzano e nonostante gli dia tutte le risorse non possono terminare vuol dire che quel processo è in deadlock.

Formalizziamo questa intuizione con l'algoritmo per risorse multiple:

1. Cerco un processo, P_i , non allocato per cui la i^{th} riga della matrice R è minore o uguale del vettore A
2. Se questo processo è trovato, aggiungo la i^{th} riga della matrice C al vettore A , segno il processo come allocato e torno al punto 1
3. Se non trovo nessun processo che rispetta le caratteristiche del punto 1, l'algoritmo termina

14.5 Risoluzione di un deadlock

Trovato un deadlock posso risolverlo in vari modi:

- Prelazionare una risorsa: prendo un nodo che è una risorsa particolarmente critica e la tolgo al processo che tanto è bloccato e la rialloco ad un altro processo che ne aveva fatto richiesta sperando che in questo modo si esca dal deadlock. Dipende dal tipo di risorsa perché non tutte possono essere prelazionate in questo modo, tipo una stampante non lo è semplicemente per la sua natura meccanica.
- Rollback: posso definire dei checkpoint del sistema in un momento in cui non c'è un deadlock e nel momento in cui ce l'ho faccio un passaggio indietro e porto l'allocazione delle risorse in una situazione stabile, si può fare ma è oneroso.
- Eliminazione di un processo: è il duale della prelazione risorse, stabilito che il nodo è critico ed è un processo posso dire al kernel di buttarlo via, una risorsa che va spesso in deadlock è la memoria. Non è un'operazione indolore perché se questo processo faceva qualcosa abbiamo perso il lavoro.

14.6 Evitare un deadlock

Non e' propriamente semplice, si fa con un approccio chiamato algoritmo del banchiere: un banchiere rischia di finire in deadlock nel momento in cui concede troppo credito ai suoi correntisti (perche' non ci sono soldi fisici per tutti), prima di concedere il prestito o ridare i soldi guardo lo stato della cassa, se lo stato di cassa e' tranquillo ti rido' i soldi, altrimenti no. Nel momento in cui una risorsa viene richiesta il sistema valuta se nel momento in cui viene rilasciata si trova in uno stato sicuro, se si, concede l'allocazione della risorsa altrimenti la rifiuta. Uno stato non sicuro non e' uno stato di deadlock, si definisce sicuro nel momento in cui esiste almeno una sequenza di schedulazione delle risorse per cui tutti i processi arrivano a termine, uno stato non sicuro e' se se non riesco a trovare neanche una sequenza (uno stato non sicuro e' uno stato in cui al momento non ho un deadlock, ma qualsiasi cosa faccia al passo successivo ci entrero').

Il grafico: sull'asse x ho il processo A, attraversa 4 istanti nella sua vita, da i1 a i3 ha bisogno di una risorsa (stampante), da i2 a i4 ha bisogno della risorsa plotter, sono non prelazionabili, soltanto tra i2 e i3 ha bisogno di entrambi. Anche B ha bisogno delle risorse. La linea tratteggiata rappresenta l'evoluzione del sistema, o metti in esecuzione A o B, se dovessi portare lo stato del sistema nelle zone tratteggiate mi troverei in deadlock, pero' io posso soltanto andare a dx se schedulo A e verso l'alto se schedulo B, per evitare di avere un deadlock devo girare intorno all'isola, ma per come si muove devo evitare anche i quadrati ai lati perche' le due zone bianche vicine sono zone non sicure, perche' posso evolvere, non sono in deadlock ma ci finiro' di sicuro.

Quindi definita un area di sicurezza mi voglio mantenere nell'area di sicurezza.

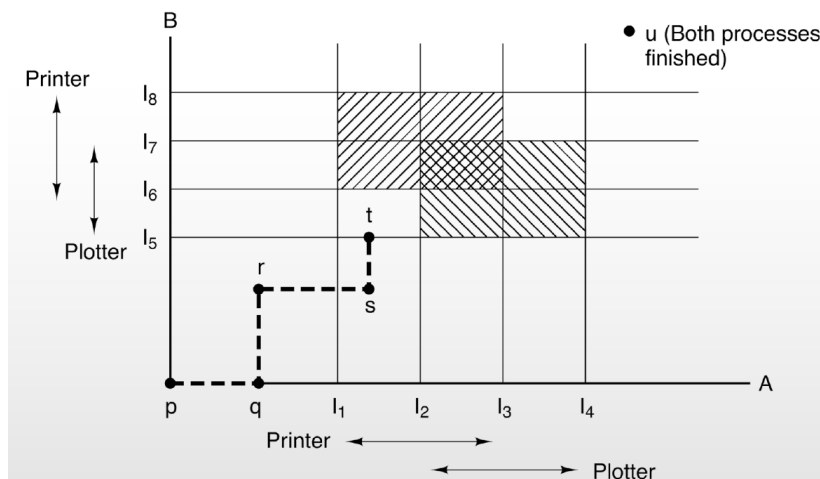


Figure 14.8: Evitare un deadlock: area sicura

Has	Max		Has	Max		Has	Max		Has	Max		Has	Max		Has	Max	
A	3	9	A	3	9	A	3	9	A	3	9	A	3	9	A	3	9
B	2	4	B	4	4	B	0	–	B	0	–	B	0	–	B	0	–
C	2	7	C	2	7	C	2	7	C	7	7	C	0	–	C	0	–
Free: 3			Free: 1			Free: 5			Free: 0			Free: 7					
(a)			(b)			(c)			(d)			(e)					

Has	Max		Has	Max		Has	Max		Has	Max	
A	3	9	A	4	9	A	4	9	A	4	9
B	2	4	B	2	4	B	4	4	B	0	0
C	2	7	C	2	7	C	2	7	C	2	7
Free: 3			Free: 2			Free: 0			Free: 4		
(a)			(b)			(c)			(d)		

Figure 14.9: Stati Sicuri

14.6.1 Algoritmo banchiere

(Questo e' l'algoritmo del banchiere, poi ci chiede qual'e' lo stato sicuro, come si ottiene e si continua) A fronte di una richiesta di allocazione di risorse, questa viene concessa se e solo se il sistema rimane in uno stato sicuro. Per rendermi conto se sono in uno stato sicuro faccio una simulazione con la risorsa che mi e' stata chiesta facendo tutto il discorso con le due matrici e se arrivo alla fine riuscendo a finire tutti i processi va bene altrimenti non alloco neanche la risorsa.

Has	Max		Has	Max		Has	Max	
A	0	6	A	1	6	A	1	6
B	0	5	B	1	5	B	2	5
C	0	4	C	2	4	C	2	4
D	0	7	D	4	7	D	4	7
Free: 10			Free: 2			Free: 1		
(a)			(b)			(c)		

Figure 14.10: Algoritmo del banchiere con una risorsa singola

	Process	Tape drives	Plotters	Printers	Blu-rays	
A	3	0	1	1		
B	0	1	0	0		
C	1	1	1	0		
D	1	1	0	1		
E	0	0	0	0		
Resources assigned						

	Process	Tape drives	Plotters	Printers	Blu-rays	
A	1	1	0	0		E = (6342)
B	0	1	1	2		P = (5322)
C	3	1	0	0		A = (1020)
D	0	0	1	0		
E	2	1	1	0		
Resources still assigned needed						

Figure 14.11: Algoritmo del banchiere con risorse multiple

14.6.2 Verifica se uno stato e' sicuro o meno

E' l'algoritmo delle due matrici e vettori:

1. Cerco una riga della matrice R che e' minore o uguale del vettore A. Se questa riga non esiste, il sistema potrebbe entrare nel deadlock finche' nessun processo puo' arrivare ad completamento (assumiamo che il processo prende tutte le risorse di cui ha bisogno fino a terminazione)
2. Assumiamo che il processo scelto prenda tutte le risorse di cui ha bisogno e le tenga finche' non termina. Segniamo questo processo come terminato e aggiungiamo tutte le sue risorse (che ha appena rilasciato terminando) al vettore A.
3. Ripetiamo i passaggi 1 e 2 fino a che tutti i processi sono segnati come terminati (nel tal caso lo stato da cui siamo partiti e stavamo valutando e' sicuro) o finche' un processo non riesce a terminare nonostante gli allorchiamo tutte le risorse di cui ha bisogno (in tal caso lo stato non e' sicuro).

14.7 Prevenire un deadlock

Voglio mettermi nelle condizioni di non dovermi neanche chiedere se ci sono deadlock, perche' so per certo che non ci saranno. Per farlo devo abbattere una delle condizioni viste prima:

- Mutual exclusion
- Hold and wait

- No preemption
- Circular wait

14.7.1 Mutua esclusione

Risorsa non condivisibile gestita da un processo e tutti gli altri chiedono a lui. Si fa per comodita' ma non risolve tanti problemi.

14.7.2 Hold and wait

Ho allocato una risorsa e la facolta' di richiederne un'altra mentre ho la risorsa, si risolve cambiando la semantica della richiesta di allocazione di una risorsa, non posso richiedere risorse se ne ho altre allocate, se ho gia' una risorsa devo mollarla per poi prendere tutte quelle che mi servono contemporaneamente

14.7.3 No prelazionabili

Ci sono delle risorse che non possono essere prelazionabili, tipo il sistema sonoro dei computer o la stampante, in generale le risorse che richiedono una componente meccanica pronunciata non possono essere prelazionate facilmente.

14.7.4 Attesa circolare

E' una cosa su cui posso veramente agire, e' una cosa estremamente semplice da risolvere, per non avere attese circolari basta dare un ordinamento globale alle risorse, non mi interessa capire se sono stesso tipo o diverse, il mio processo puo' allocarle secondo nell'ordine globale (posso solo salire), se ho bisogno di una risorsa precedente devo lasciare quello che ho e prendere quella prima. Se seguo questo schema non e' possibile entrare in attesa circolare.

Ricordiamo che quando l'interfaccia si incastra non e' per il deadlock ma c'e' un timer da qualche parte, inoltre l'interfaccia grafica non alloca mai risorse, e' passiva e se non succede niente non allcoa nulla, riceve messaggi dall'esterno ed implementa risposte per i messaggi che arrivano dall'esterno

14.8 Varianti sul tema deadlock

Il deadlock puo' assumere diverse sfaccettature:

- Two-phase locking: quando ho delle risorse che hanno bisogno di piu' di un'operazione quando devono essere allocate (il cd ha bisogno della chiusura del cassetto e l'innesto del motore)
- Communication deadlock: lui lo mette qui per dire che per lui non e' un deadlock. E' una situazione in cui ho mandato un messaggio che sintatticamente impegna una delle due parti ed aspetto una risposta che non puo' arrivare perche' ho mandato il messaggio sbagliato. Se abbiamo un

sistema distribuito (Os che si allarga su piu' nodi nella rete) allora il deadlock potrebbe coinvolgere processi che stanno su sistemi di calcolo diversi e distribuiti in rete. Questo deadlock e' difficile da trovare, quindi di solito si implementano dei timeout.

- Livelock: lo chiamano deadlock dinamico, c'e' una risorsa che prima e' presa da un processo A che non riesce ad usarla completamente poi presa da processo B che non riesce ad usarla completamente. Il trashing e' un livelock perche' tutti i processi funzionano ma la risorsa memoria non e' usata perche' se la passano continuamente.
- Starvation: non e' un deadlock, si ha quando un processo deve attendere un tempo troppo lungo per poter avere una risorsa. Quando abbiamo un processo a priorita' molto bassa ed il sistema di schedulazione a priorita' continua a dare la CPU ad altri processi, quindi come gia' abbiamo visto per evitare la starvation alziamo progressivamente la priorita' dei processi con priorita' bassa.

Chapter 15

Cloud

15.1 Virtualizzazione

Abbiamo già parlato di virtualizzazione ed è punto da cui siamo partiti per i sistemi in cloud. Abbiamo dell'hardware e sopra definiamo un hypervisor sul quale mettiamo i vari contenitori delle macchine virtuali, è bello perché in questi contenitori non possiamo fare danni, o meglio se si rovina lo buttiamo via. Questa roba è stata fatta per anni perché si usava la virtualizzazione, perché potevo avere 4 servizi (macchine dedicate) comprando un solo scatolone di metallo quindi risparmiavo soldi simulando i sistemi. L'obiettivo in pratica è ottimizzare l'uso delle risorse facendo collassare più server virtualizzati all'interno di una singola macchina fisica:

- Meno hardware da comprare
- Meno risorse sprecate

15.1.1 Caratteristiche della virtualizzazione

Riuscivo a spendere meno ma avevo una serie di vincoli:

- Comunque dovevo comprare un computer
- Quelli virtualizzati erano comunque sistemi operativi e come tali devono essere mantenuti, ad esempio grazie ad un sistemista
- Il sistema operativo ospite (ospitante) diventa critico per il funzionamento (in inglese si dice host e guest, in italiano è sempre ospite): perché se mi si blocca l'os che gestisce la scatola di metallo ho tutto fermo, si chiama single point of failure.

Da caratteristiche a problemi

Explicitiamo queste caratteristiche in problemi, perché con l'avanzamento della tecnologia abbiamo cominciato a non farci andare bene queste caratteristiche:

- Comunque devo comprare un calcolatore:
 - C'è crisi e si ha bisogno di risparmiare soldi
 - Il calcolatore consuma energia e deve essere raffreddato
 - Tra N anni devo cambiarlo (refresh tecnologico)
 - E poi devo pagare anche la manutenzione
- Quelli virtualizzati sono dei sistemi operativi, e come tali devono essere mantenuti:
 - Ogni macchina virtuale è lì solo e soltanto perché deve erogare un servizio
 - A che mi serve avere un altro sistema operativo

- E poi... chi lo paga lo stipendio di un sistemista in piu'?
- Il sistema operativo ospite diventa critico per il funzionamento: se si blocca quello ho una serie di sistemi e servizi che non funzionano piu': appunto, motivo in piu' per togliere pure quello

Soluzioni alla vecchia maniera

- Hosting:
 - Prendo in affitto il calcolatore (fisico) di qualcuno
 - Hardware e manutenzione sono un problema di chi ci affitta il calcolatore
 - se paghiamo un extra ci fanno il backup
- Housing:
 - compro un calcolatore e chiedo a qualcuno di dargli casa (pagandogli i costi)
 - ma elettricità e raffreddamento sono un problema suo

Ad oggi l'housing non lo fa piu' nessuno, l'hosting e' esattamente come prima solo che oggi ci danno un'istanza di una macchina virtuale e da qui comincia il cloud. Mentre la macchina fisica ha dei limiti, se abbiamo un problema dobbiamo avere dei giorni di fermo, mentre se abbiamo una macchina virtuale basta spegnere e riaccendere quindi risulta molto piu' flessibile.

15.1.2 Dinamica temporale di un servizio

Sull'asse delle x c'e' il tempo e sull'asse delle y c'e' l'uso del sistema, dove il 100% e' il massimo dell'utilizzo che faremo, quindi evinciamo che all'inizio avremo un picco di utilizzo del nostro servizio e successivamente calera' (si parla di un servizio popolato da persone).

Una cosa totalmente diversa e' parlare di quanto il nostro sistema di calcolo riesce a gestire, non puo' essere il 100%, la linea rossa e' quanto e' potente la mia macchina, tutto quello che c'e' sopra e' carico di lavoro che non riusciamo a gestire. Mentre tutto quello che sta sotto la linea rossa riusciamo a gestirlo, ma notiamo che andando avanti nel tempo stiamo buttando via un sacco di potenza di calcolo. Il cloud dice che tanto e' tutto erogato tramite sistemi virtualizzati che si possono cambiare di volta in volta, quindi la capacita' del sistema si adatta in maniera elastica in base a quello di cui abbiamo bisogno, e' scritto sul libro delle fiabe, idealmente sto sprecando meno risorse. In un'azienda vera non funziona cosi', aumento immediato mentre la riduzione e' alla fine del ciclo di fatturazione.

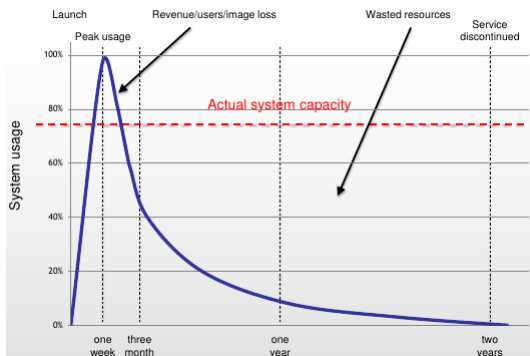


Figure 15.1: Dinamica temporale di un servizio

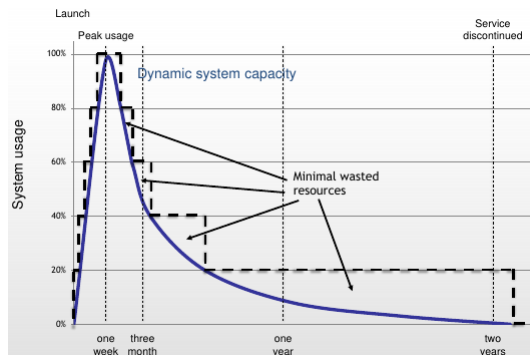


Figure 15.2: Dinamica temporale del cloud

15.2 Passaggio al cloud

- Ma il sistema di calcolo e' proprio necessario?
 - Tanto, la sua esistenza e' solo funzionale a eseguire un software
 - Non posso avere ilo software e basta?
- Ok, il calcolatore e' necessario ma non mi interessa averlo nel mio CED:
 - Pero', ne voglio uno configurato apposta per me e voglio poterlo allargare e restringere quando e come mi serve
- Non voglio avere risorse sprecate
 - Anzi, meglio ancora, e' possibile pagare le risorse solo se le sto usando?

15.3 Definizione di cloud secondo il NIST

Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g. networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. Esistono pochi servizi in cloud in relta' (tutti ne parlano ma pochi lo usano).

15.4 Google docs

Google docs e' un servizio in cloud che ci fornisce un servizio di composizione testi. Il passo successivo e' capire che di cloud ce n'e' piu' di uno posso fornire le risorse in cloud in tre modi diversi.

15.5 As A Service

In pratica, sto esternalizzando le risorse di calcolo e ne usufruiamo come se fossero un servizio attraverso la rete, si erogano tramite architettura SPI. Possono fornirmi un Software (tipo google docs, o un software di mail etc) in questo caso parlo di software as a service che non devo gestire io e mi viene erogato come servizio. Se a me il software non interessa posso scendere e prendere un PaaS (platforma as a service) e' un sistema che compro in cui mi viene dato un os virtualizzato gia' pronto (mi puo' servire se devo fare un sito web o implementare un mio servizio online, lo compro da Microsoft), va bene ma compro un sistema operativo. Se non mi va bene avere un sistema operativo gia' installato devo scegliere l'IaaS (Infrastructor as a Service) lo compro ad esempio da Amazon, ci danno solo le risorse di calcolo ed un terminale con cui gestire il sistema.

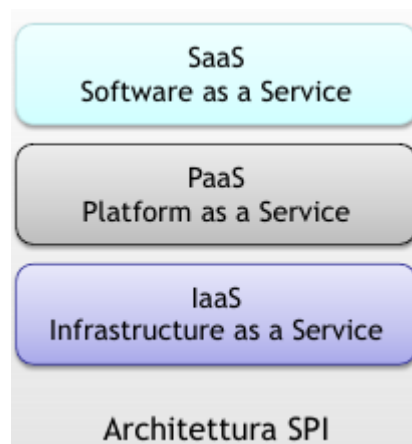


Figure 15.3: Architettura SPI

15.5.1 Prima e dopo

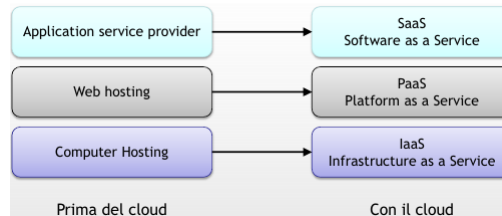


Figure 15.4: Prima e dopo

Ci sono aziende che vendono il cloud semplicemente virtualizzando un'istanza di linux con virtualbox, però c'è il problema che se non lo chiamano IaaS nessuno lo compra.

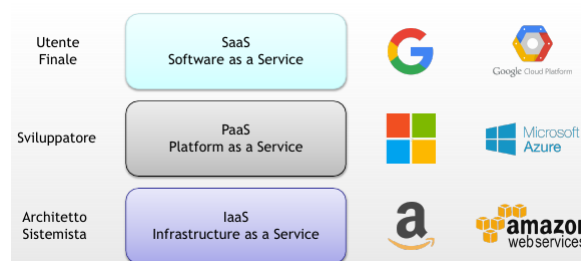


Figure 15.5: SPI esempi

La vera differenza che Google, Amazon e Microsoft si sono costruiti il cloud per loro, perché avevano bisogno di una enorme risorsa di calcolo quindi è fatto bene, gli altri il cloud lo fanno solo per venderlo (quindi è quello che è). Approfondire i container di Google per lo storage di dati, è chiaro che l'hardware si scassa, quando uno dei nodi all'interno del container si guasta sotto il 50% lo buttano via.

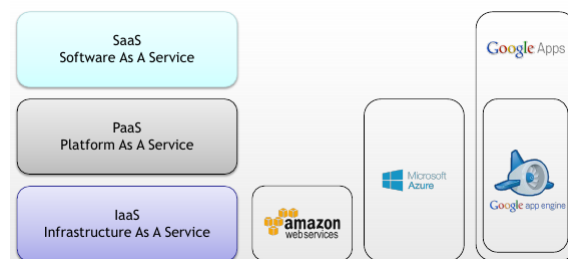


Figure 15.6: SPI realta'

Quando abbiamo a che fare con il cloud parliamo di utility computing.

15.6 Utility computing

Il cloud prende il nome di utility computing perche' i servizi mi vengono dati dal provider ed io prendo quello che mi viene dato pagando solo a consumo (pay per use).

Idealmente il cloud dovrebbe essere:

- Efficiente nell'uso delle risorse
- Scalabile: si adatta facilmente a condizioni di lavoro diverse
- Elastico: posso aggiungere e rimuovere risorse secondo bisogno
- Senza manutenzione
- Sempre disponibile: o meglio, quasi sempre disponibile
- Interoperabile e portabile: posso combinare servizi di cloud diversi e trasportare i miei dati da un cloud all'altro

15.7 Caratteristiche del cloud

- Efficienza nell'uso delle risorse:
 - Le risorse vengono aggregate e uniformate in maniera tale da fornire un servizio omogeneo a piu' utenti
 - L'allocazione delle risorse viene adattata dinamicamente in base alle richieste degli utenti
- Elasticita':
 - Le risorse di calcolo possono essere aggiunte e rimosse rapidamente in maniera tale da ottenere la scalabilita' necessaria alle richieste degli utenti
 - Scalabilita': la capacita' di un sistema di mantenere inalterata la sua efficienza anche di fronte ad un carico di lavoro variabile
 - * La domanda "quanto e' scalabile?" si traduce in "quanto lo posso fare lavorare prima che comincia a perdere colpi?"
- Senza manutenzione esterna:
 - I servizi del cloud vengono erogati (dinamicamente, adattandosi alle richieste degli utenti) senza richiedere l'intervento di un operatore umano
- Accessibile e sempre disponibile:
 - Le risorse devono (dovrebbero?) essere sempre disponibili, eventualmente prevedendo meccanismi di ridondanza (a carico dei provider)

- L'accesso alle risorse deve avvenire tramite meccanismi standard compatibili con piattaforme diverse (web 2.0). In questo modo ne si può usufruire non solo da PC ma anche, ad esempio, da una piattaforma mobile o un televisore

C'è una fregatura, perché l'elasticità dell'uso delle risorse è una convenienza del provider perché siccome c'è una dinamicità può piazzare sopra l'hardware 200 o 300 clienti che pagano. Le risorse dovrebbero essere sempre disponibili, sui contratti però non troveremo mai che la risorsa sarà disponibile al 100%.

15.8 Suddivisione per come rendo accessibile il cloud

Si può dividere in 4 tipologie a livello di accessibilità:

- Public: infrastruttura accessibile tramite internet libero o a pagamento
- Community (comunitario): infrastruttura condivisa tra entità con richieste infrastrutturali omogenee, ad esempio il cloud della pubblica amministrazione
- Privato: l'infrastruttura è strettamente interna alla mia azienda, e la uso per i miei servizi
- Ibrido: l'infrastruttura dei segmenti di cloud privato (mio) che sfruttano servizi di cloud pubblico (di miei fornitori)

15.9 Il lato oscuro del cloud

Stiamo usando male la tecnologia del cloud, come paradigma ce lo abbiamo già da decenni, una volta lo avevamo chiamato sistema distribuito, poi grid computing allora siamo tornati con i web service.

- Lo stiamo usando male, una generazione di manager lo sta percependo solo come un modo efficiente di tagliare i costi. Dobbiamo adattarci al cloud se non va bene per la mia azienda devo cambiare l'azienda non cercare di cambiare il cloud.
- Il cloud piace tanto alle banche (quindi c'è qualcosa che non va), perché se nel momento in cui ho un cliente che mi deve tanti soldi lo chiamo e gli faccio ridurre i servizi (il cloud) e la banca riprende i suoi soldi. Possedere un calcolatore è CAPEX mentre pagare un fornitore cloud è OPEX.
- non è vero che il cloud costa meno, bisogna fare bene i conti, smaltire i sistemi attuali abbiamo un costo, per migrare i sistemi abbiamo un costo, amazon ci dà dei dati da qui a 3 anni, ma dopo questi tre anni cosa

succede? perche' l'hardware dura 5 anni tendenzialmente. Aneddoto del canadese che ha trovato nel contratto che amazon protrebbe togliere i servizi con un preavviso di 40 sec.

- Ma la rete fa qualche differenza? Eccome! Dipende da quanti dati bisogna spostare e con che velocita' dovete accedervi.
- Non ci sono standard definiti: La modalita' di accesso e' standard (web 2.0) ma il modo in cui vi accedo dipende strettamente dal provider.

Chapter 16

Multimedia

16.1 Dato multimediale

E' praticamente un dato anomalo, siamo abituati a considerare i dati come sequenze di bit, sono sequenza di bit con caratteristiche strane. In realta' (non e' un film) ma ha queste caratteristiche:

- Va consumato al volo: quando guardiamo un filmato su youtube c'e' un web server che mi manda ed io lo consumo al volo
- Segue delle dinamiche tutte sue: si possono tollerare delle perdite, ma dipende dal tipo di dato. Le perdite video vengono tollerate molto meglio di quelle audio.
- E' analogico: per sua natura e' analogica, perche' raccogliamo informazioni con gli occhi e le orecchie (sono campionatori analogici) pero' dobbiamo metterla nel sistema di calcolo con dei bit in maniera digitale, e deve funzionare anche al contrario
- Occupa un sacco di spazio: se pero' accetto di vedere/sentire male allora posso trovare un compromesso (mpeg)

Il telefono e' stata il primo strumento di diffusione di dati multimediali.

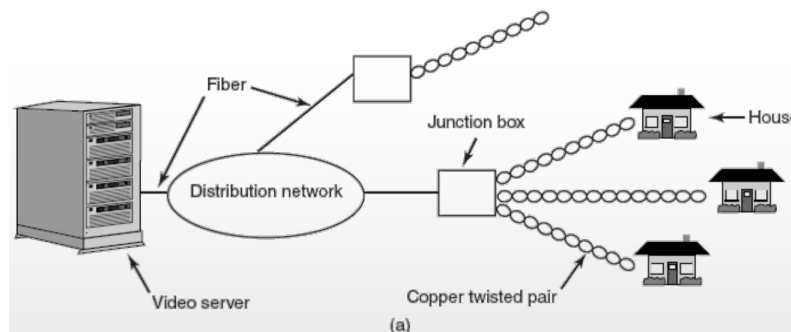


Figure 16.1: Rete telefonica

Quando abbiamo cominciato a parlare di telefonia e' nato in parallelo il concetto di rete, oggi facciamo fatica a disaccoppiare il concetto di informazioni multimediale dal concetto di distribuzione via rete. Una rete e' un sistema di trasferimento dati, un grafo, devo portare in bit da un punto A ad un punto B, se il bit rappresenta informazione multimediale ho un sistema di diffusione multimediale. La rete e' quella che mi condiziona cosa posso fare io sul dato multimediale, devo:

- Gestire un dato con quelle caratteristiche noiose
- Devo trattarlo in modo che non faccia schifo per chi lo riceve
- Deve essere compatibile con quello che la rete riesce a trattare

Nelle reti moderni abbiamo un sacco di complicazioni, c'è una rete di distribuzione (rete di provider), la nostra dsl ci arriva nel gabinato nella via e si trova a condividere aspetti tecnologici con quelli che ci stanno intorno. Ci sono due aspetti, queste stesse problematiche si spostano sul sistema di calcolo perché dobbiamo condividere risorse tra processi, il dato deve comunque arrivare in soft real time e dobbiamo avere abbastanza capacità del sistema di calcolo per vedere delle cose che non facciano schifo.

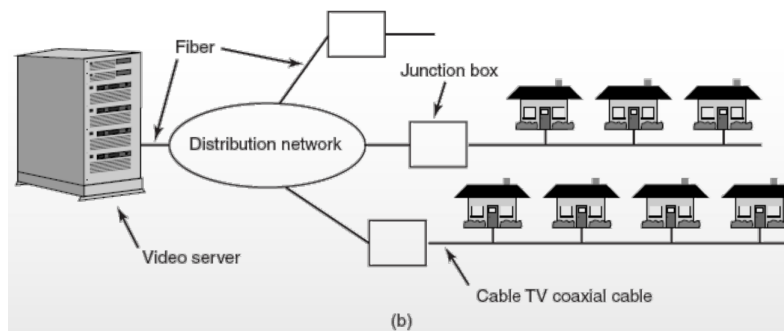


Figure 16.2: Rete Moderna

Negli anni abbiamo costruito reti progressivamente più veloci e di pari passo alla crescita della rete abbiamo prodotto dei sistemi di distribuzioni multimediale che usavano tutto quello che era disponibile. Nella tabella di sx abbiamo un'evoluzione di qualità dei contenuti, a dx abbiamo un'evoluzione dei sistemi di rappresentazione.

Source	Mbps	GB/hr	Device	Mbps
Telephone (PCM)	0.064	0.03	Fast Ethernet	100
MP3 music	0.14	0.06	EIDE disk	133
Audio CD	1.4	0.62	ATM OC-3 network	156
MPEG-2 movie (640 × 480)	4	1.76	IEEE 1394b (FireWire)	800
Digital camcorder (720 × 480)	25	11	Gigabit Ethernet	1000
Uncompressed TV (640 × 480)	221	97	SATA disk	3000
Uncompressed HDTV (1280 × 720)	648	288	Ultra-640 SCSI disk	5120

Figure 16.3: Velocità e disponibilità dei dati in rete

16.2 Caratteristiche delle informazioni multimediali (all'interno di un OS)

Cerchiamo di trasportare queste cose all'interno del os:

- Sono digitali (stanno in un file): ho quindi la necessita' di codificarle in qualche modo
- Occupano un sacco di spazio: quindi devo comprimerle (comprimere e' un modo di codificare), una codifica intelligente mi porta ad occupare meno spazio senza dare fastidio all'utente
- Devo poterne garantire la fruizione in real time: Mi crea dei problemi perche' devo rendere un os general purpose a fare delle cose che fa fatica a fare, se il video prevede 20 frame al secondo quelli devono essere
- Devono poter essere usate alla stregua di un media "classico": ovvero, mi aspetto di fare pause, play, fast forward etc etc (VCR functionality)

16.3 Codifica

La codifica significa stabilire un formato digitale. Vuol dire stabilire una corrispondenza tra informazione (analogica) e sequenze di bit per rappresentarla. Nell'immagine si e' provati a rappresentare i colori con uno o due bit, l'idea e' quella di dare all'informazione piu' frequente le stringhe di bit piu' corte. Piu' o meno la stessa cosa la facciamo nelle reti, il motivo per cui parliamo di capacita' e non di banda, la frequenza determina il numero di bit al secondo, quello che salta fuori e riesco a mandare dall'altra parte e' la frequenza di rete, se riesco ad avere 16 simboli ed ad ogni simbolo do 4 bit ho una banda di 1 boud e una capacita' di rete di 4 bit al secondo, aumentando la tecnologia riesco a far passare piu' dati.



Figure 16.4: Codifica di un dato multimediale

16.3.1 motivi per codificare

- Per dare sicurezza:
 - Proteggere il contenuto (cifatura)
 - Proteggere la proprietà intellettuale: watermarking, steganografia
- Per dare affidabilità: una giusta codifica può aiutare a correggere gli errori di trasmissione

Attenzione che NON vuol dire comprimere, una codifica che risparmia spazio siamo abituati a chiamarla compressione. La compressione di un contenuto è un effetto collaterale (piacevole) di una codifica ben strutturata.

16.3.2 dumb coding

Prendiamo un video, codifico il filmato come una sequenza di frame, codifico ogni frame come una matrice di pixel, codifico ogni pixel in formato RGB, e mi viene fuori che per un minuto di filmato mi servono 1.28 GB per riprodurlo, ovviamente non è sostenibile.

16.3.3 Ottimizzazione dello spazio

Esistono tecniche per creare delle associazioni tra un insieme di informazioni da rappresentare e stringhe binarie (bit) tali per cui il numero totale di bit utilizzati è minimo (codici a lunghezza media uniformemente minima, codifica di Huffman). Siamo in uno spazio di codifica *loseless*, per cui non accetto di perdere nessun tipo di informazione ma riduco solo lo spazio totale occupato.

16.3.4 Ridondanza

Si va ad aprire un mondo delle connessioni *lossy*, scartiamo quelle informazioni che non sono significative o possono essere ricostruite dal nostro cervello, parleremo di video ma si applicano pari pari all'audio.

- Ridondanza spaziale: se faccio una fotografia posso raggruppare informazioni di colore omogeneo, e posso buttare via delle cose, vedere tutte le sfumature non mi interessa
- Ridondanza temporale: riguarda i contenuti che si evolvono nel tempo, ad esempio il telegiornale, c'è pochissima differenza tra un frame e l'altro, quindi non ho bisogno di mandare sempre lo sfondo

Ridondanza spaziale

Il nostro cervello si è abituato a riconoscere perfettamente immagini orizzontali e meno a quelle verticali, posso costruire un diagramma per vedere come variano i pixel adiacenti e siccome siamo più sensibili orizzontalmente l'immagine ci sembra che abbia lo stesso tipo di conformazione, quando in realtà non è vero

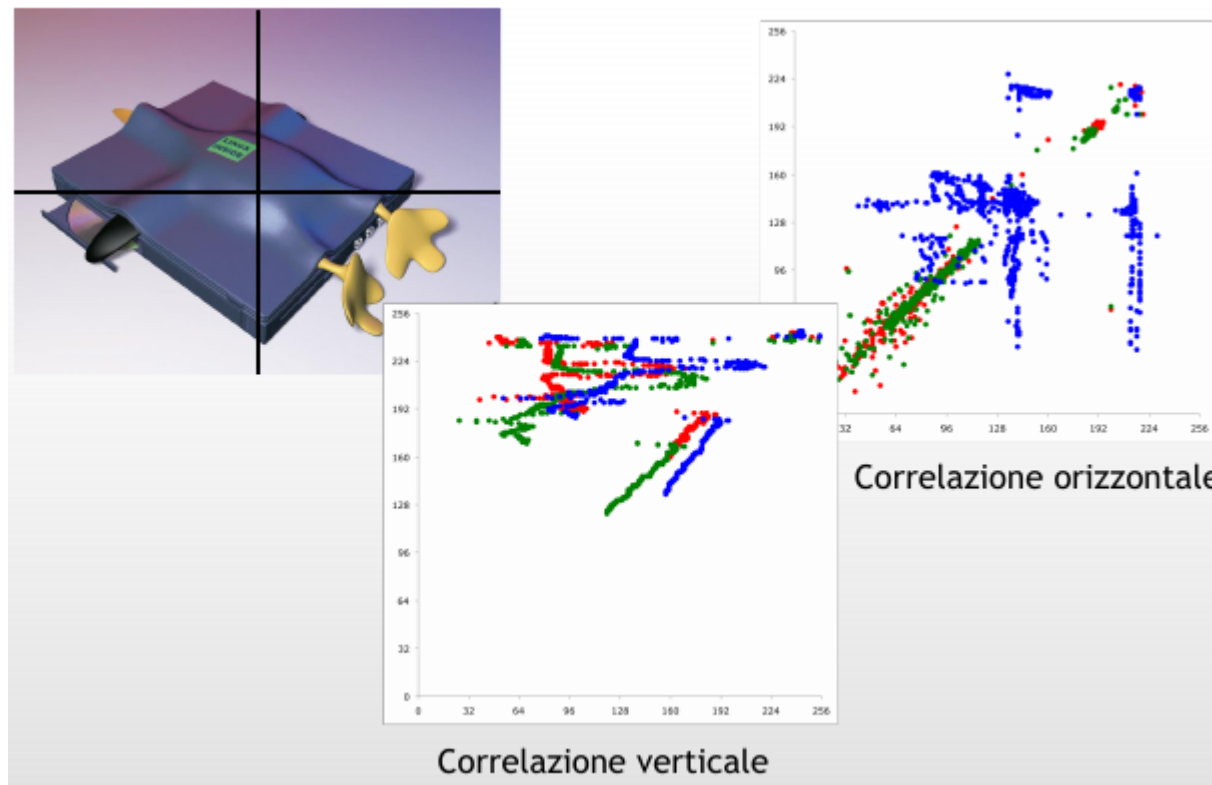


Figure 16.5: Ridondanza spaziale

Ridondanza temporale

Sottraggo due fotogrammi, tutto quello che e' ridondato e' nero, tutto il resto che rimane in bianco e' quello che cambia.

16.4 Compressione

Quello che ci da fastidio quando vediamo un immagine sgranata e' che dove mi aspetto una riga diagonale in realta' ho uno scalino. La compressione dei dati, o meglio la codifica, e' una cosa molto difficoltosa da trattare, perche' dipende da una serie di parametri difficili da controllare:

- Dal mezzo con cui si fruisce del contenuto: non puo' essere la stessa al variare della dimensione dello schermo
- Da agenti fisiologici: com'e' fatto l'occhio
- Da agenti psicologici: ci sono delle informazioni di cui ci fregiamo perche' siamo fatti cosi'

16.4.1 Mezzo di fruizione

Impone un limite tecnologico non evitabile, rischiamo di codificare/inviare delle informazioni che semplicemente non potranno essere fruite:

- Video a colori a una televisione in bianco e nero
- Video 4K a un cellulare
- Audio stereo a un cellulare GSM (nokia 3310)

16.4.2 Parametri fisiologici

Non tutte le informazioni hanno la stessa importanza per il nostro cervello, di una figura percepiamo piu' facilmente la forma, di un suono siamo piu' sensibili alle variazioni istantanee. Il cervello ricostruisce autonomamente parte dell'informazione, ma per ogni senso lo fa con soglie e parametri diversi.

16.4.3 Parametri psicologici

Se il nostro cervello dovesse recepire e catalogare tutte le informazioni impazziremmo, metto un filtro e tengo solo le informazioni essenziali, tipo il profilo di una persona, noi ricordiamo soltanto gli aspetti importanti di un oggetto e ricostruiamo in maniera psicologica tutto il resto L'essere umano ha una strana tendenza a riconoscere gli oggetti istintivamente:

- Non ci ragioniamo
- Lo facciamo a volte in base alla nostra esperienza
- Ci ricordiamo solo "gli aspetti importanti"

16.5 Audio e video

Sono tipi di dati multimediali profondamente diversi e vanno trattati in maniera disgiunta. Differenti vincoli:

- Tecnici: frequenza e campionamento, le due cose devono andare di pari passo (audio e video in sincrono)
- Fisiologici: sensibilita' non uniforme o distribuita diversamente
- Psicologici: diversa tolleranza alla perdita dei dati

16.6 MPEG

Si propone come soluzione per fare una codifica efficiente per ridurre lo spazio occupato. E' uno standard prodotto da telecom italia. Non e' la soluzione di tutti i mali. In pratica e' una serie di regole per la compressione spaziale temporale pensate per disturbare il minimo possibile la percezione dell'utente, hanno delle famiglie (audio e video), non e' tarato sull'esperto ma sull'utente medio.

E' stato raccolto da un gruppo di persone specializzate che ha fondato un organizzazione branca delle ISO, che ha cominciato a dire quali sono gli standard MPEG.

16.6.1 Vari tipi di MPEG

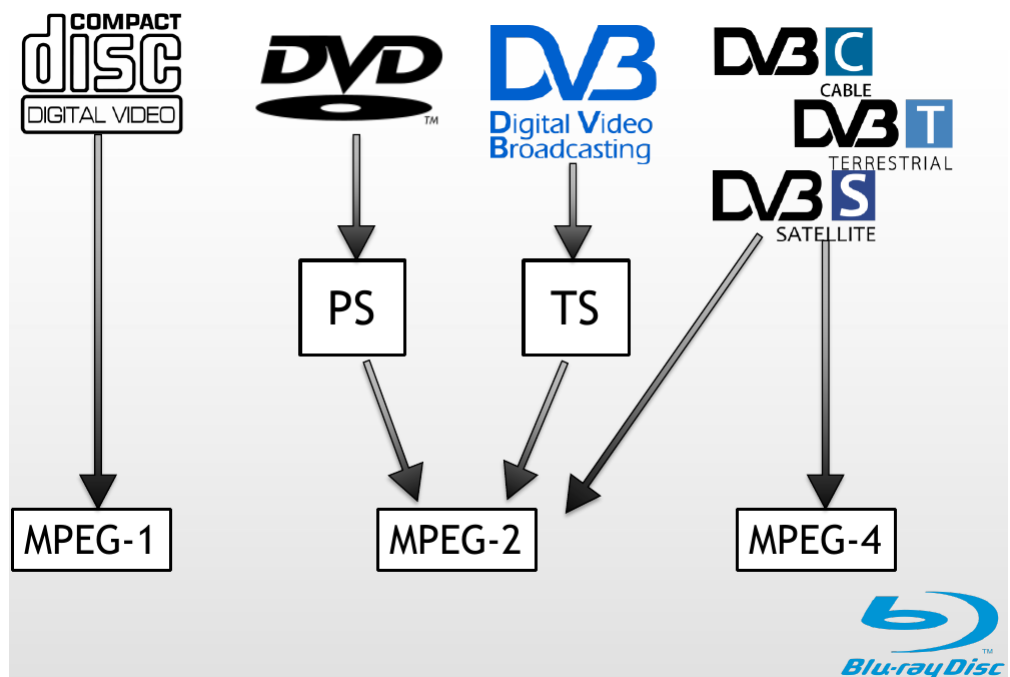


Figure 16.6: Vari tipi di MPEG

16.6.2 Codifica MPEG

Carrellata di quello che succede sullo standard. E' una codifica asimmetrica, ho delle linee guida e creo partendo da un contenuto una sequenza di byte (byte-stream) che me lo rappresentano, chi fa la codifica e' quello che deve fare il lavoro pesante, perche' codifico una volta e decodifichero' milioni di volte,

chi fa la decodificazione fa operazioni semplici deve essere poco oneroso per il calcolatore, in questo modo posso fare la codifica che voglio e la decodifica non cambia mai. Tenendo fisso lo standard, e' possibile fare evolvere parallelamente i due estremi:

- Posso implementare codificatori piu' efficienti senza modificare il software di visualizzazione
- Posso creare nuovi player (e nuovi dispositivi) senza codificare nuovamente i contenuti

Nel momento in cui applico la codifica MPEG smonto il mio video in una sequenza di fotogrammi, non lo tratto come una sequenza di immagini separate, attenzione che non sono una sequenza di immagini jpeg, quello si chiama MJPEG.

16.6.3 Ridondanza spaziale

Come faccio a combattere la ridondanza spaziale: separo i tre piani di colore secondo la YUV (un canale mi porta la luminanza e due la crominanza) per ciascuno di questi canali divido l'immagine in quadratini di 16x16 pixel, su ognuna di queste tessere vado a costruire una matrice numerica di quanto e' forte l'informazione in quel pixel (puo' essere colore o luce, da questo punto in poi sono numeri) avendo questi numeri applico una trasformata coseno discreta che mi porta l'informazione dal dominio dei valori al dominio delle frequenze, in pratica trasformo l'informazione da tipo numerico a tipo spettrale, per buttare via l'informazione che non mi serve dico alla trasformata quali armoniche tenere buone (in pratica ripulisco l'immagine tenendo solo i contorni degli oggetti), piu' ne tengo piu' sara' dettagliato il mio risultato, tanto le armoniche con frequenze piu' alte non interessano, a noi interessa la portante ed un numero di armoniche. Adesso abbiamo un sacco di 0. Abbiamo la portante in alto a sinistra e poi a destra ed in basso abbiamo le armoniche, la matrice invece che scandirla per colonne e per righe partiamo dall'alto a sinistra e scendiamo in diagonale a zig zag, in questo modo abbiamo un array con una coda di zeri che posso tagliare via. Mi sono costruito un sistema in cui ho buttato via informazioni che non mi servivano, ho una codifica con pochi byte.

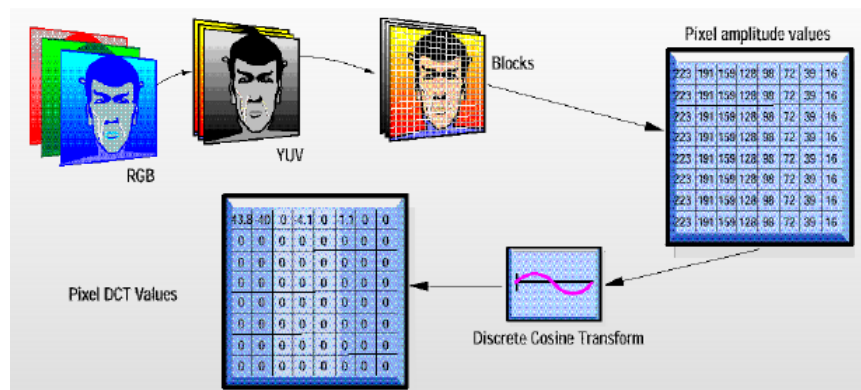


Figure 16.7: Ridondanza Spaziale

Linearizzazione

Un blocco, dopo essere stato trattato viene reso lineare (da matrice ad array) secondo questo schema:

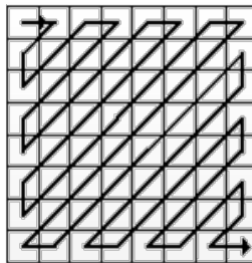


Figure 16.8: Linearizzazione

Ricordiamoci che in alto a sinistra si trova il valore piu' grande, mentre tutti gli altri contributi saranno spesso molto vicini a 0. Con queste condizioni di partenza la costruzione di un codice a lunghezza minima e' molto efficiente.

Questo pero' non basta, perche' un file cosi' trattato e' un immagine JPEG 2000. Devo ancora trattare la ridondanza temporale: Definiamo diversi tipi di frame e distinguiamo frame "indipendenti" (I, che codificano i frame jpeg2000) da altri che predicono (P) la variazione nel tempo. Per questo motivo ad esempio su youtube non ci fa mettere mai il video nel punto in cui vogliamo, perche' MPEG mette dei frame I e dei frame P, per far partire un video dobbiamo partire per forza da un frame I.

16.6.4 Ridondanza temporale

La predizione non e' in tutti i casi la soluzione migliore, pensate ad una automobile che scompare temporaneamente dietro un albero; perche' dovremmo dimenticare di averla vista per poi ricostruirla? Definiamo un terzo tipo di frame "bidirezionale" (B) che descrive una scena per differenza con quelle sia precedenti che successive.

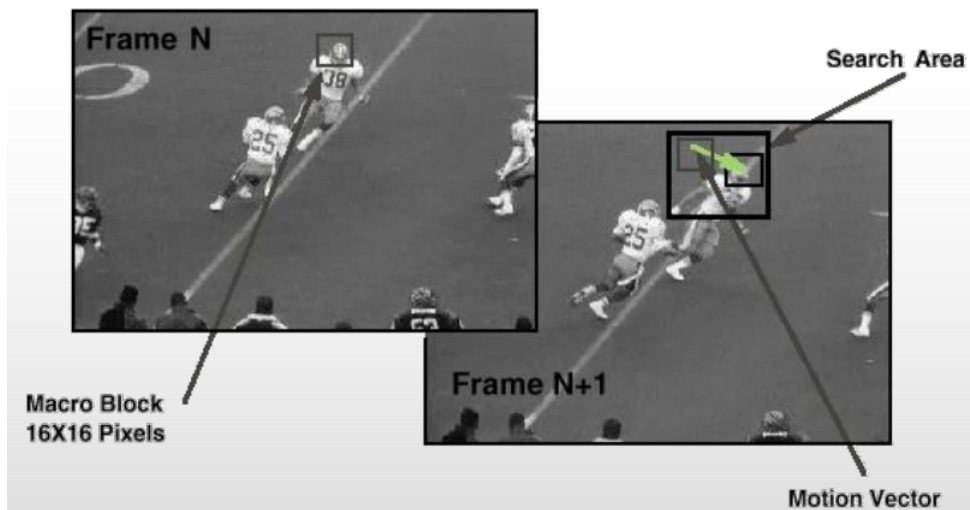


Figure 16.9: Ridondanza temporale

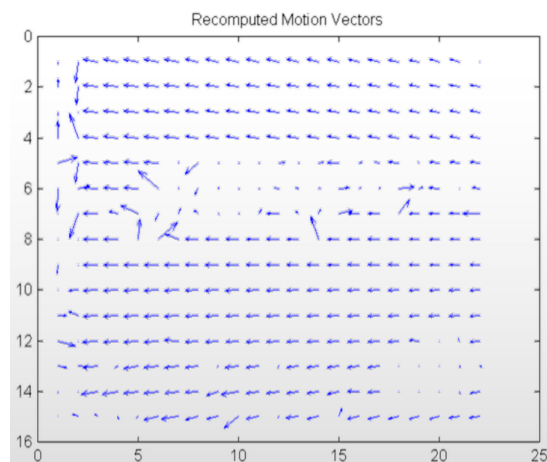


Figure 16.10: Motion Vector

16.6.5 B-Frame

In pratica prendiamo in prestito un macroblocco da un frame successivo (I o P) e ne diamo un motion vector rispetto al futuro.



Figure 16.11: B-Frame

16.6.6 GOP - Group Of Pictures

- Gli I-frame sono codificati indipendentemente: come immagini jpeg-2000
- I P-frame sono codificati per differenza rispetto all'I-frame che li precede:
 - Includono i vettori di movimento
 - Codificano i cambiamenti dell'immagine
- I B-frame sono codificati come differenza con gli I/P frame che li precedono/seguono (pensate ad una "predizione bidirezionale")

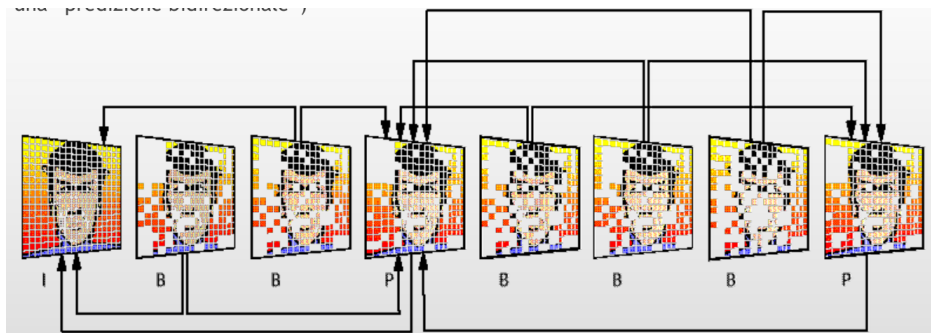


Figure 16.12: GOP

16.6.7 Per non confondersi

- Codifica e contenitore sono due cose profondamente diverse
- Una codifica esprime il modo in cui l'informazione audio o video viene trattata (MPEG piuttosto che Wavelet)

- Un container dichiara la sintassi con cui i bit codificati vengono scritti in un file (.AVI piuttosto che .MOV)
- Ovviamente, alcuni container sono progettati per ospitare in maniera piu' efficiente certi tipi di codifica
- E' perfettamente possibile (anche se non molto ottimizzato) avere un video codificato in MPEG dentro un file .AVI come avere un video codificato Wavlet dentro un file .MOV

16.7 Streaming

Non posso fare overprovisioning in un sistema di calcolo, ha ancora senso parlare di streaming ma in ambiti piu' ristretti. In un certo senso la webcam fa streaming verso la scheda grafica. Streaming = continuano ad arrivare dati senza pausa. Se in qualche modo dobbiamo parlare di streaming dobbiamo parlarne tutte le volte che:

- vi e' un contenuto che cominciamo ad utilizzare prima che sia arrivato del tutto, come youtube.
- L'invio del contenuto avviene in maniera continua e asincrona fintanto che non e' stato inviato tutto, il contenuto potrebbe essere infinito

La rete e' come se fosse sparita, perche' che ci sia di mezzo la rete o meno, i problemi sono sempre gli stessi.

Ho del contenuto che rappresento come quello che va sulla scheda video, ho un video che e' una sequenza di frame (25 al secondo) ogni frame e' una matrice di pixel tutti uguali quando li guardo, il totale dei byte che vado a gestire sono questa curva spezzata, questo video lo registro e lo metto su un disco. C'e' una seconda fase in cui vado a prendere il contenuto sul disco lo decodifico e lo sbatto nel sistema come una scalinata, dopo al punto 3 vedo sul monitor la sequenza di frame. La distanza tra il punto 2 e 3 e' il cosiddetto processing delay, guardando in un certo istante di tempo ho la distanza tra le due curve, per cui in parte ho gia' riprodotto il video e parte che ho ancora sul disco.

In realta' abbiamo due problemi principali:

- Il tempo di trasferimento dal nostro hard drive (file system) non e' regolare e non lo sara' mai, perche':
 - il sistema operativo sta facendo anche altro
 - Disco e video non sono sincronizzati
 - Gli interrupt non guardano in faccia a nessuno
- Neanche i frame sono tutti uguali
 - I, P e B hanno tempi di trasferimento diversi perche' sono di dimensione diverse

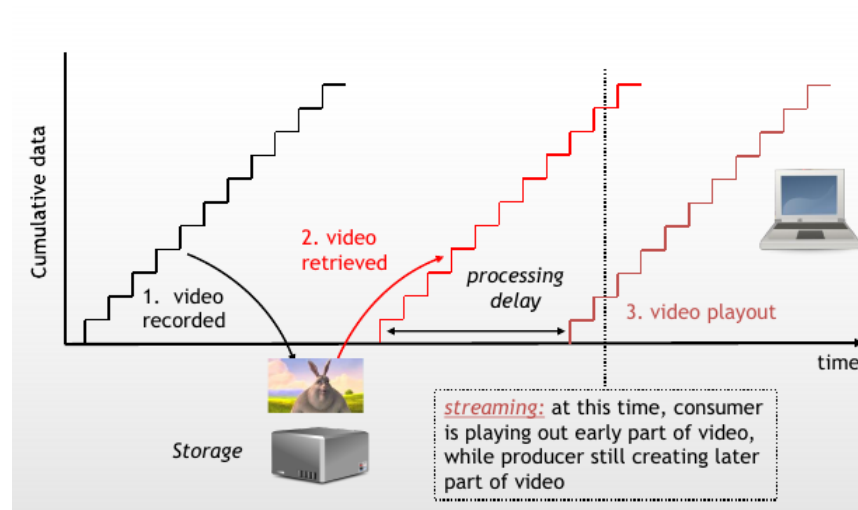


Figure 16.13: Come dovrebbe funzionare lo streaming idealmente

- I, P e B hanno tempi di elaborazione diversi: abbiamo bisogno di tutti i B che precedono un P per elaborare il prossimo frame mentre I e' indipendente

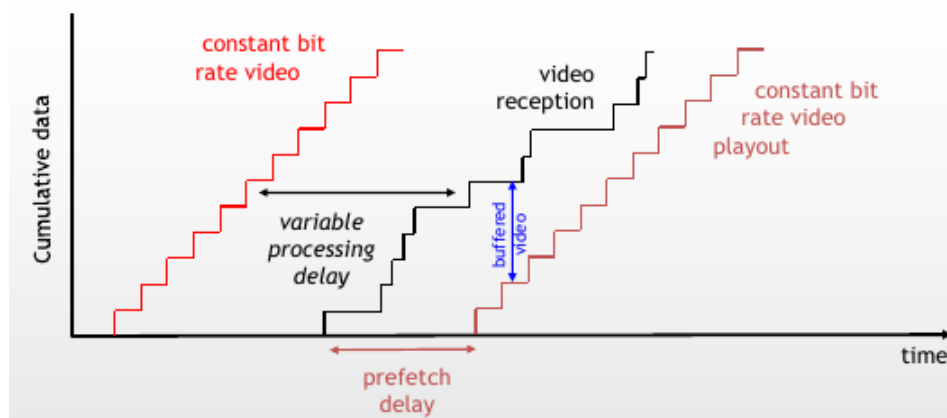


Figure 16.14: Streaming nella realta'

Il ritardo introdotto dall'accumulo prima del primo playout prende il nome di prefetch delay, il video che ho in pancia e aspetto venga riprodotto diventa una cosa variabile.

16.7.1 Buffering

L'unico modo che un'applicazione ha di rendere nuovamente regolare un contenuto multimediale e' quello di farlo passare per un buffer. I dati entrano come il sistema concede ed escono secondo il profilo richiesto.

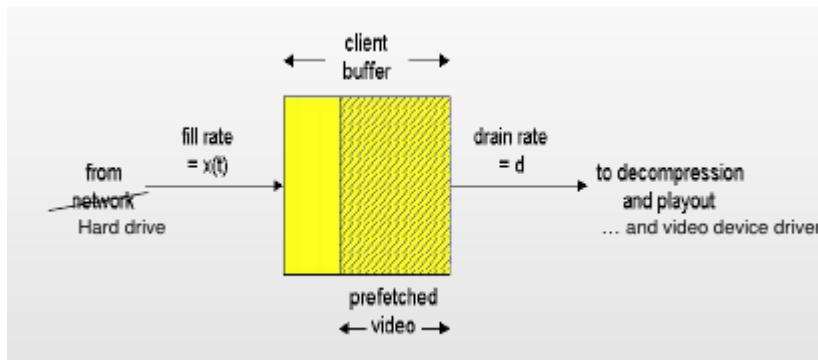


Figure 16.15: Buffering

Dimensionamento del buffer

- Troppo piccolo non va bene:
 - C'e' troppo poco gioco tra ricezione e smaltimento dei dati
 - Se non c'e' spazio, i dati in arrivo verranno scartati e persi
 - Si potrebbe andare in buffer overrun/underrun
- Troppo grande non va bene:
 - Sono risorse concesse dal sistema operativo
 - Il dispositivo potrebbe non averle fisicamente
 - Si introducono ritardi a volte inaccettabili

16.7.2 Buffer underrun

È quando si ricevono dei dati troppo lentamente e si arriva ad un punto nello streaming che non si hanno più dati da consumare. Nel grafico questo si nota in quanto i due segmenti si toccano.

16.7.3 Buffer overrun

È quando ricevo dati troppo velocemente tanto da riempire il buffer ed è necessario buttare via i dati che arrivano.

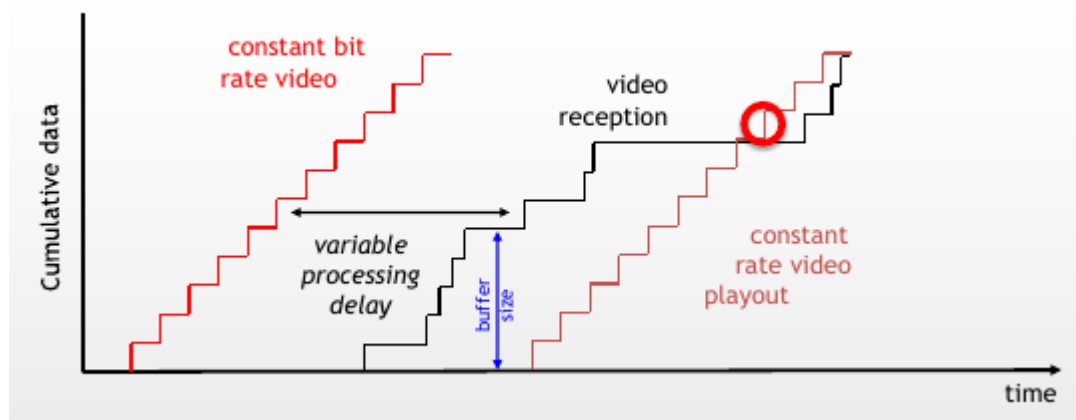


Figure 16.16: Buffer underrun

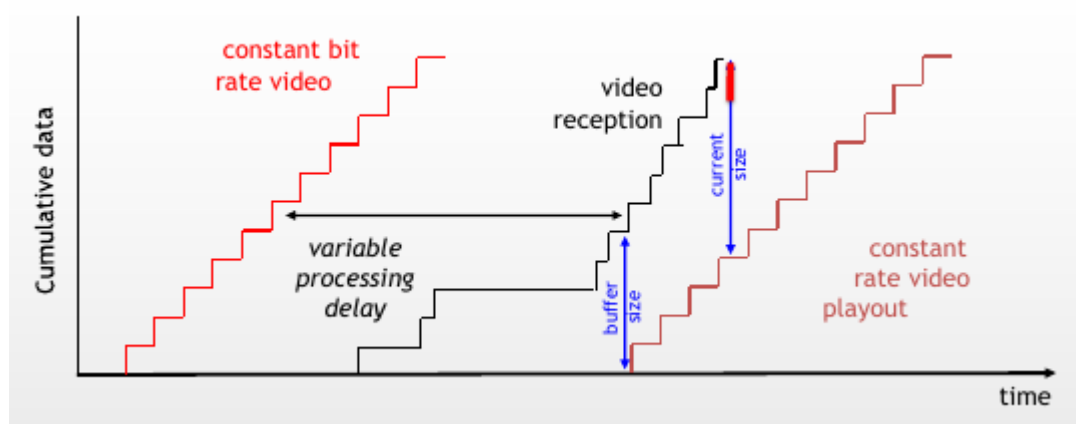


Figure 16.17: Buffer overrun

16.7.4 E se perdiamo dei dati?

Nessun problema, il dato multimediale e' strutturato appunto per tollerare questo, basta semplicemente fornire dei dati a caso (rumore) o degli zeri (buio) e il decoder, prima o poi ne uscirà'. Come quando guardiamo la televisione e vediamo le strisciate.

16.7.5 Le funzionalita' VCR

Sono tutte le funzionalita' tipiche di un sistema di riproduzione video (avanti ed indietro veloce, selezionare un punto di riproduzione). Abbiamo mai notato che quando saltiamo avanti ed indietro in un video ci mette sempre un po' a ripartire

e non parte quasi mai esattamente dal fotogramma che avevamo scelto? Questo perché il buffer viene svuotato e si ricostruisce quel minimo di informazioni che servono per una fruizione garantita. Solitamente il punto di partenza è sempre un I-frame. L'unico modo per saltare in modo efficiente da un frame all'altro è quello di garantire un modo efficiente tra un I-frame e l'altro. Per minimizzare i disturbi alla ricezione dei dati è necessario agire sul sistema operativo:

1. Tempo di elaborazione il più costante possibile, serve uno scheduler specializzato per il supporto ai sistemi multimediali
2. Tempo di estrazione più uniforme possibile: serve un file system specializzato al contenuto multimediale
3. Funzionalità VCR senza problemi: file system specializzato

16.8 Processi multimediali

I processi multimediali, per nostra fortuna, sono una categoria specifica di processi realtime, sono processi periodici:

- Se genero 20 frame al secondo, allora ho bisogno dello scheduler necessariamente ogni ventesimo di secondo
- In più, ogni volta richiedo sempre lo stesso burst di CPU

Come i processi real-time regionano in termini di scadenze temporali (deadline), solo che in questo caso sono ricorrenti e note a priori. Diciamo cioè che hanno un periodo noto. Un flusso multimediale è un insieme di più stream, cioè uno stream audio, video e sottotitoli. Ognuno ha i suoi tempi. Ci sono due algoritmi che cercano di trovare la sequenza giusta.

16.8.1 Processi periodici

Possiamo in questo caso esprimere una condizione di schedulabilità in funzione di periodo e burst:

$$\sum_{i=1}^m \frac{C_i}{P_j} \leq 1$$

16.8.2 RMS (Rate Monotonic Scheduling)

È una politica di scheduling pensata appositamente per i processi periodici:

1. Il burst di ogni processo deve essere terminato entro lo scadere del suo periodo
2. Non ci sono dipendenze tra processi
3. I burst richiesti sono sempre gli stessi ad ogni periodo

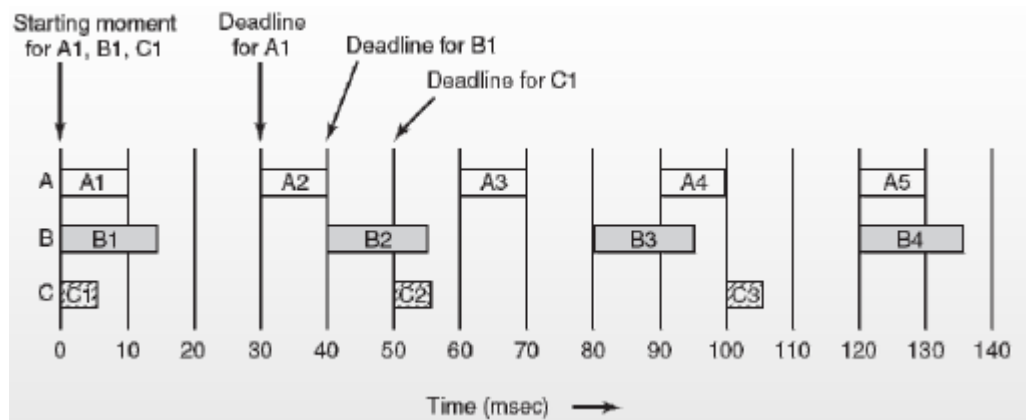


Figure 16.18: Processi periodici

4. La prelazione della CPU si presuppone abbia un overhead negligibile
5. I processi non periodici (se ce ne sono) vengono schedulati solo se la CPU non ha altro da fare (non hanno deadline)

RMS e' un algoritmo di scheduling a priorit  statica (il periodo e' da un blocco all'altro). Ogni processo riceve una priorit  inversamente proporzionale al periodo. Processi con periodi brevi (ovvero, frequenza di esecuzione alta) verranno schedulati con precedenza.

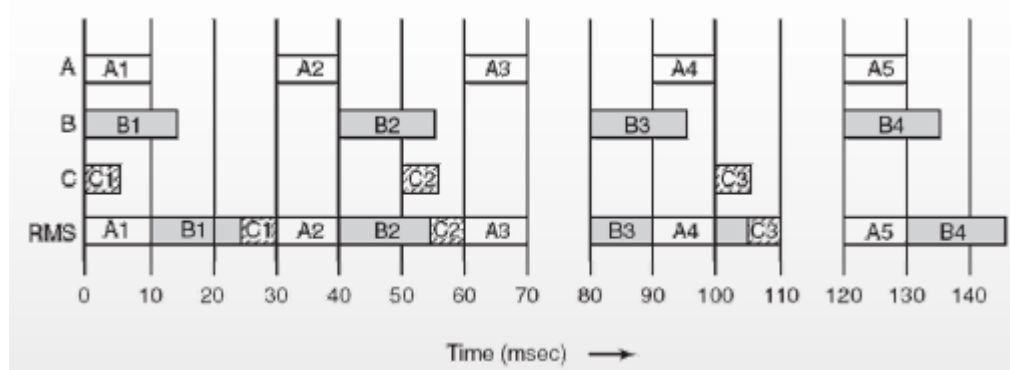


Figure 16.19: RMS

A=audio, B=video, C=sottotitoli. Nel momento in cui inizia la riproduzione multimediale con priorit  statica, assegno la priorit  ai processi. Ho A,B,C all'istante 0. Inizio con A perche' ha priorit  piu' alta. All'istante 10 A ha finito, tra B e C, B ha una priorit  piu' alta perche' ha un periodo piu' breve, ed infine si avra' C.

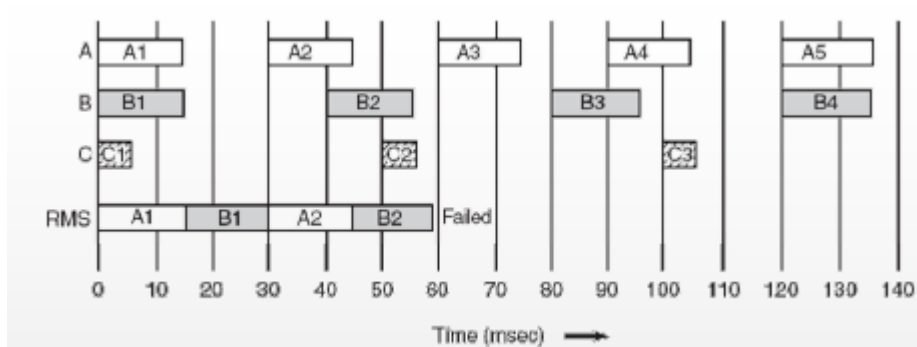
Fallimento RMS

Figure 16.20: Fallimento RMS

RMS non funziona sempre in quanto non sfrutta la CPU al 100%. Si e' dimostrato che RMS funziona se il tasso di utilizzo del sistema e' inferiore ad una certa soglia:

$$\sum_{i=1}^m \left(\frac{C_i}{P_j} \right) \leq (\sqrt[m]{2} - 1)$$

Dove m=numero dei processi e la sommatoria e' la condizione di schedulabilita'. Inoltre

$$\lim_{x \rightarrow \infty} m(\sqrt[m]{2} - 1) = 0.7$$

circa

E' possibile fare una schedulazione con RMS di processi periodici nel momento in cui l'uso della CPU non superi il 70%. Potrebbe essere o meno una soluzione dipende dall'hardware.

16.8.3 EDS (Earliest Deadline First Scheduling)

EDS e' una variante di RMS che prevede priorit  dinamiche. Ogni processo riceve una priorit  inversamente proporzionale al tempo rimanente alla sua deadline (cioe' la priorit  piu' alta viene assegnata al processo piu' vicino alla deadline). Con EDS non e' piu' strettamente necessario che i processi siano periodici e neppure che abbiamo un burst sempre uguale (e' un'approssimazione di SJF (Shortest Job First)), di fatto e' una politica di schedulazione real-time generica. Puo' raggiungere un rendimento prossimo al 100% a fronte pero' di un algoritmo molto piu' complesso.

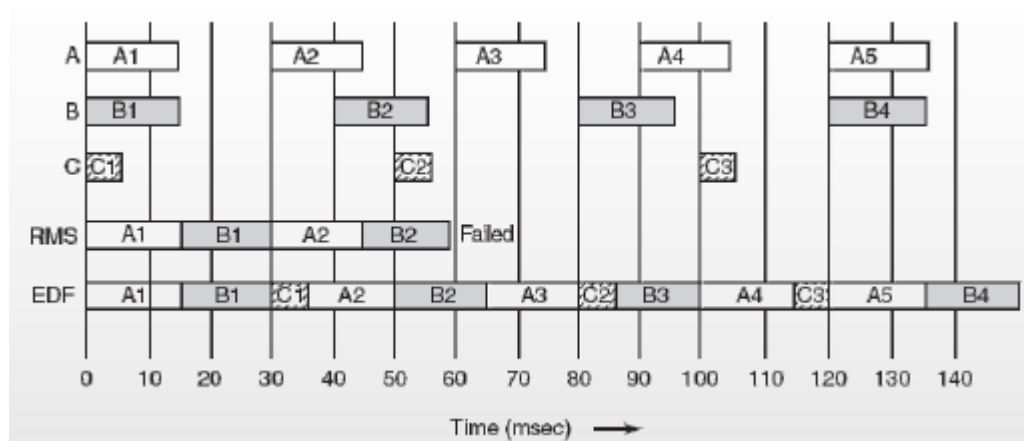


Figure 16.21: EDS

16.9 File System con supporto multimediale

I file system con supporto multimediale devono avere principalmente due caratteristiche:

- tempo di accesso ridotto al minimo
- facilita' di passare da un punto all'altro in riproduzione (sfruttiamo le caratteristiche di MPEG)

16.9.1 Minimo tempo di accesso

Basta memorizzare tutti i file sul disco in modalita' sequenziale, tenendo conto della codifica e del container. Ad ogni frame viene associato in modo contiguo la traccia video, audio e in fine i sottotitoli. Sapendo la dimensione massima di un frame, passare da un frame all'altro diventa molto facile.

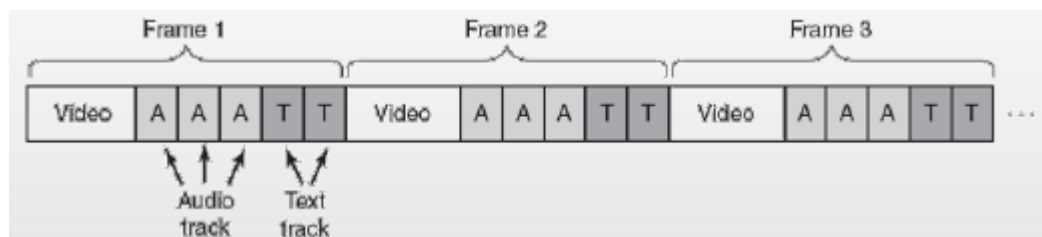


Figure 16.22: Minimo tempo di accesso

16.9.2 Manipolazione della riproduzione

Dobbiamo aggiungere al file dei metadati. All'interno dell'i-node vengono aggiunti delle informazioni sul GOP. Allora un datablock potrebbe essere più grande o più piccolo della dimensione di un GOP:

- Dimensione del blocco $\geq$ dimensione GOP:
 - Uso un "block index", per saltare all'interno del filmato
 - Ho una frammentazione interna
 - Non ho bisogno di gestire allocazioni sequenziali
- Dimensione blocco $<$ dimensione GOP:
 - Uso un frame index, questo è il caso medio. I frame index sono quei frame che fanno riferimento a dei I-Frame. Questi frame dovranno essere sempre i primi frame all'interno di un datablock. Se un datablock non è sufficiente a contenere tutto un GOP si continua nel datablock successivo, ma questo datablock non sarà elencato nel frame index. Tengo traccia di dove sono gli I-frame.
 - Devo gestirmi delle allocazioni sequenziali
 - Non ho spreco di spazio

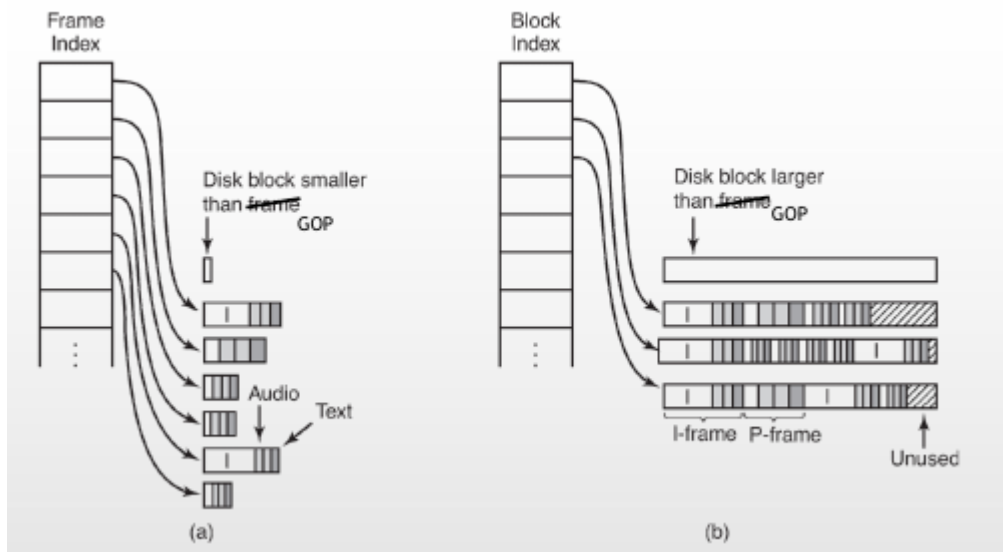


Figure 16.23: Allocazione indicizzata